TRUSTED® COMPUTING GROUP

SPECIFICATION

TCG PC Client Platform Firmware Profile Specification

Level 00 Version 1.06 Revision 52
Family “2.0”
December 4, 2023

Contact: admin@trustedcomputinggroup.org

PUBLISHED

DISCLAIMERS, NOTICES, AND LICENSE TERMS

THIS SPECIFICATION IS PROVIDED “AS IS” WITH NO WARRANTIES WHATSOEVER, INCLUDING ANY WARRANTY OF MERCHANTABILITY, NONINFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, OR ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

Without limitation, TCG disclaims all liability, including liability for infringement of any proprietary rights, relating to use of information in this specification and to the implementation of this specification, and TCG disclaims all liability for cost of procurement of substitute goods or services, lost profits, loss of use, loss of data or any incidental, consequential, direct, indirect, or special damages, whether under contract, tort, warranty or otherwise, arising in any way out of use or reliance upon this specification or any information herein.

Contact the Trusted Computing Group at www.trustedcomputinggroup.org for information on specification licensing through membership agreements.

Acknowledgements

The writing of a specification, particularly a security specification, takes many hours for both development and review. The TCG would like to acknowledge the contribution of those individuals (listed below) and the companies who allowed them to volunteer their time to the development of this specification. Special thanks are due to Amy C Nelson and Jiewen Yao, who served as Chairs of the PC Client Working Group during the development of this specification.

The TCG would also like to give special thanks to Amy C Nelson who served as the editor of this specification.

Contributors

Neo Hsueh, Advanced Micro Devices, Inc.
Dean Liberty, Advanced Micro Devices, Inc.
Thomas Peng, Advanced Micro Devices, Inc.
Robert Strong, Advanced Micro Devices, Inc.
Alex Tzonkov, Advanced Micro Devices, Inc.
Michael Zhang, Advanced Micro Devices, Inc
Gongyuan Zhuang, Advanced Micro Devices, Inc.
Jen Ye, Advanced Micro Devices, Inc.
Frederick Otumfuor, American Megatrends, Inc.
Yogesh Deshpande, ARM Ltd.
Thomas Fossati, ARM Ltd.
Abhimanyu Singh, ARM Ltd.
Stuart Yoder, ARM Ltd.
Monty Wiseman, Beyond Identity
Scott Phuong, Cisco Systems
Tom Brostrom, Cyber Pack Ventures
Dr. David Challener
Mark Beamon, Dell, Inc.
Travis Gilbert, Dell, Inc.
Jason Kolodziej, Dell, Inc.
Amy C Nelson, Dell, Inc.
Jason Young, Dell, Inc.
Chris Groff, DRS Technologies
Henk Birkholz, Fraunhofer Institute for Secure Information Technology
Michael Eckel, Fraunhofer Institute for Secure Information Technology
Jeff Anderson, Google Inc.
Chris Fenner, Google Inc.
Ferdinand Nolscher
Shiva Dasari, Hewlett Packard Enterprise
Theo Koulouris, Hewlett Packard Enterprise
Vali Ali, HP Inc.
Jeff Jeansonne, HP Inc.
David Roderick, HP Inc.
Joshua Schiffman, HP Inc.
Adrian Shaw, HP Inc.
Zhuo Liu, Huawei Technologies Co., Ltd.
Silvia Vlasceanu, Huawei Technologies, Co., Ltd.
Ken Goldman, IBM
Nayna Jain, IBM
Jeorg Borchert, Infineon Technologies
Ga-Wai Chin, Infineon Technologies
Andreas Fuchs, Infineon Technologies

Antonio Javier Cabrera Gutierrez, Infineon Technologies
Guillaume Raimbault, Infineon Technologies
Florian Schweiger, Infineon Technologies
Sven Schuch, Infineon Technologies
Thomas Bowen, Intel Corporation
Kirk Brannock, Intel Corporation
Eduardo Cabre, Intel Corporation
Olga Otte, Intel Corporation
Liran Perez, Intel Corporation
Ned Smith, Intel Corporation
Jiewen Yao, Intel Corporation
Robert Hart, Johns Hopkins University, Applied Physics Lab
Eric Sivertson, Lattice Semiconductor Corp.
Bill Keown, Lenovo (US) Inc.
Masoud Manoo, Lenovo (US) Inc.
David Rivera, Lenovo (US) Inc.
Jeannette Wilson, Microchip Technology
Ronald Aigner, Microsoft
Paul England, Microsoft
Lonnie Harrell, Microsoft
Erich McMillan, Microsoft
Brad Litterell, Microsoft
Dana Amsalem Cohen, Nuvoton Technology Corporation
Dan Morav Nuvoton Technology Corporation
Ludovic Jacquin, NVIDIA Corporation
Joe Pennisi, NVIDIA Corporation
Arnold Braine, NXP Semiconductors
Dick Wilkins, Phoenix Technologies
Olivier Collart, STMicroelectronics
Nourdine El Idrissi, STMicroelectronics
Yves Magnaud, STMicroelectronics
Charly Villette, STMicroelectronics
Zachary Blum, United States Government
Andrew Medak, United States Government
Lawrence Reinert, United States Government
Jean Fioretti, WISeKey Semiconductors

TPM DEPENDANCIES AND REQUIREMENTS

1. The TPM used for Host Platforms claiming adherence to this specification SHALL be compliant with the *TPM Library Specification; Family 2.0; Level 00; Revision 01.38* or later.
2. The TPM used for Host Platforms claiming adherence to this specification SHALL be compliant with the TCG PC Client Specific Platform TPM Profile for TPM 2.0 Version 1.00, Revision 1.05 or later.
3. The Platform Class for platforms claiming adherence to this specification SHALL be registered with the TCG administrator.
4. Host Platforms claiming adherence to this specification SHALL be compliant with the TCG ACPI Specification Family 1.2 and 2.0, Level 00, revision .37.

CONTENTS

LIST OF TABLES

LIST OF FIGURES

1 Scope

The PC Client Platform Firmware Profile Specification, “this Specification”, describes the behaviors and requirements of a PC Client system with a TPM 2.0 compliant with the Trusted Platform Module Library Specification Family 2.0 and the PC Client Platform TPM Profile 1.05, or later. The scope of the requirements includes the establishment of the chain of trust for measurement rooted in a Root of Trust for Measurement. This Specification describes what is and is not measured, how the data that is measured is captured in a log of events, and into which TPM PCR or NV Extend index the measurement is extended. It captures the requirements for initializing the TPM and the platform firmware interfaces to the TPM. This specification also includes requirements for platform firmware related to other TCG technologies, such as self-encrypting drives specified by the TCG Storage Working Group.

1.1 Key Words

The key words “MUST,” “MUST NOT,” “REQUIRED,” “SHALL,” “SHALL NOT,” “SHOULD,” “SHOULD NOT,” “RECOMMENDED,” “MAY,” and “OPTIONAL” in this document normative statements are to be interpreted as described in RFC-2119, Key words for use in RFCs to Indicate Requirement Levels.

1.2 Statement Type

Please note a very important distinction between different sections of text throughout this document. There are two distinctive kinds of text: informative comment and normative statements. Because most of the text in this specification will be of the kind normative statements, the authors have informally defined it as the default and, as such, have specifically called out text of the kind informative comment. They have done this by flagging the beginning and end of each informative comment and highlighting its text in gray. This means that unless text is specifically marked as of the kind informative comment, it can be considered a kind of normative statement.

EXAMPLE: Start of informative comment

This is the first paragraph of 1–n paragraphs containing text of the kind *informative comment* ...

This is the second paragraph of text of the kind *informative comment* ...

This is the nth paragraph of text of the kind *informative comment* ...

To understand the TCG specification the user must read the specification. (This use of MUST does not require any action).

End of informative comment

2 INTRODUCTION AND CONCEPTS

Start of informative comment

The Trusted Computing Group's architecture is platform-independent, intended to enhance trust in computing platforms. As such, the TPM Library Specification Family 2.0 is general in specifying both hardware and software requirements. The goal of the TCG member companies is to ensure compatibility among implementations within each type of computing architecture. It is anticipated that companion implementation documents will be created for each architecture.

This document serves as implementation reference documentation for PC Client firmware architectures. Specifically, this document defines:

1. Usage of PCR registers in the Pre-Operating System state through the transition to OS-Present state.

2. How Platform Firmware, or a component thereof, contributes to the Root of Trust for Measurement (RTM) of the platform, specifically through extending digests (measurement) of code to a TPM based Platform Configuration Register (PCR) and documentation of that measurement in a log (event log).
3. Behavior entering, during, and exiting power and initialization states.
4. Guidelines for Option ROMS.

This specification is based on the TPM Library Specification Family 2.0, Level 00 Revision 1.38. The reader is expected to have an understanding of the concepts, defined functionality, and terms expressed in that document. This specification will attempt to minimize the duplication of information from that document; therefore, concepts and terms defined in the TPM Library Specification will not be defined in this document. If there is a conflict in interpretation between this and the TPM Library Specification, the concept or functional description as defined in the TPM Library Specification takes precedence.

This specification also references other specifications as listed in Section 1.6 (External Specifications). The reader is expected to be familiar with the concepts and terminology contained in each where relevant.

It is important to understand that there are two uses for measurements: Attestation and Sealed Storage.

1. Attestation is used to provide information about the platform's state to a challenger. However, PCR contents are difficult to interpret; therefore, attestation is typically more useful when the PCR contents are accompanied by a measurement log. While not trusted on its own, the measurement log contains a richer set of information than do the PCR contents. The measurement log is not intended to be used to validate the PCR contents, rather, the PCR contents are used to provide the validation of the measurement log. Additional trust in the measurements is gained from comparing the PCR contents to a Reference Integrity Manifest (RIM), as specified in the TCG Reference Integrity Manifest Information Model. A RIM, provided by the Platform Manufacturer or Platform Firmware provider, is a signed representation of the measurement log.
2. Sealed Storage uses only the PCR contents to determine its action. Sealed Storage operations make no use of the measurement log and are simpler but provide only a success or fail for the validation of the platform's configuration.

If the only use of PCRs were attestation, there would be little reason to have more than just a few PCRs because they are only used for the validation of the measurement log. Challengers could validate the log, then parse through the measurement log for the information they need. Sealed Storage is best done when the types of measured objects are grouped, with objects of similar impact to the validation of the platform's trust grouped into the same PCR. This logic was chosen when arriving at the PCR mapping in this specification.

End of informative comment

2.1 PC Client Specific Architecture

Start of informative comment

The concepts and descriptions of the PC Client architecture are described in both Figure 1 and the descriptions. While the diagram in Figure 1 infers physical connections, the connections and associations between the components are logical.

End of informative comment

Figure 1 depicts the general architectural components of the PC Client platform as used in this specification. The components and their relationships within the diagram are meant to provide a reference for discussion and are not

meant to require or imply any particular design or implementation beyond what is stated in the normative text in this specification.

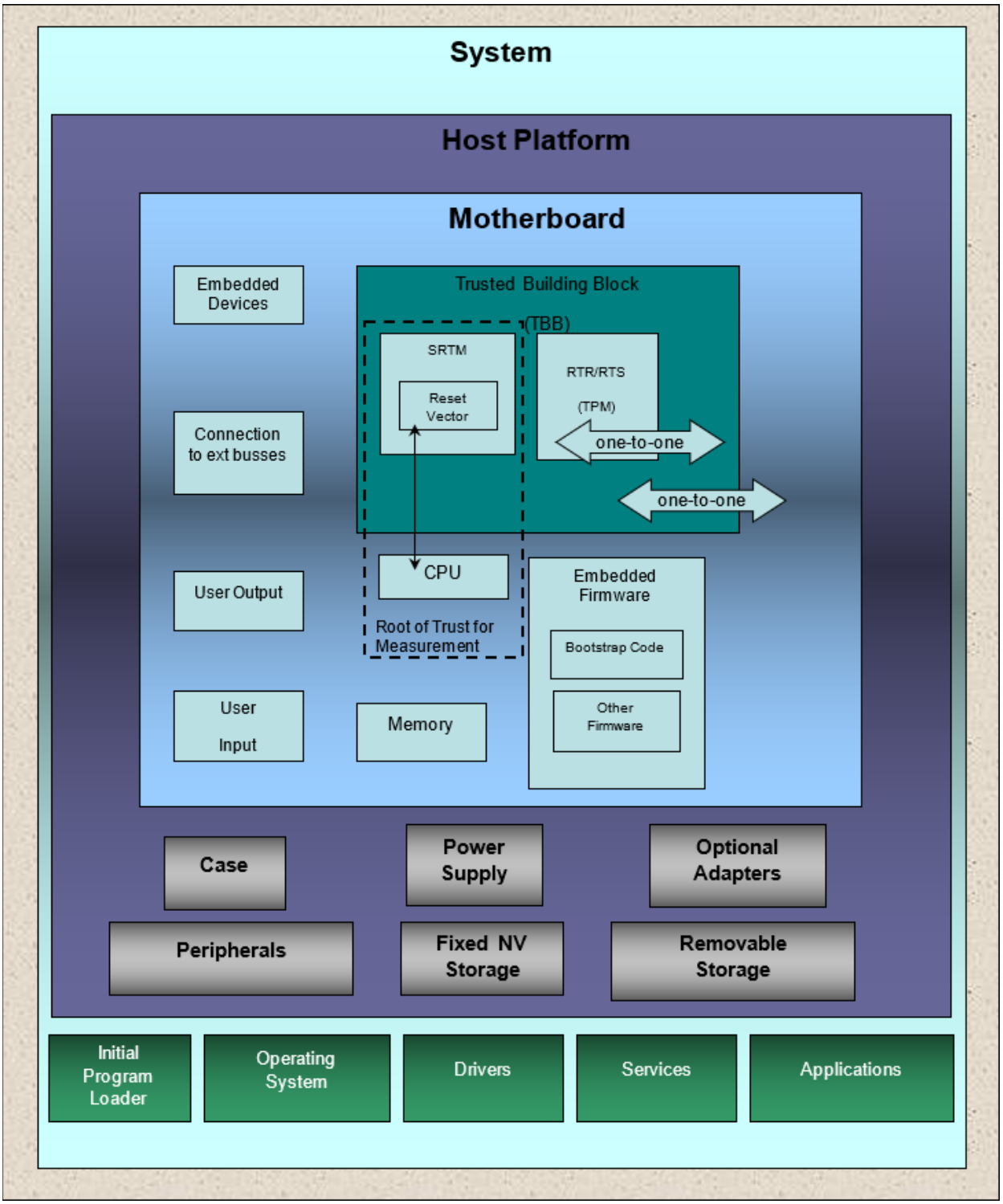

Figure 1 PC Client Platform Architectural Diagram

2.2 PC Client Concepts

2.2.1 Host Platform TPM

The term *TPM* within this specification SHALL refer to the TPM attached to the Host Platform for the purpose of providing protected capabilities for the Host Platform as defined by the TPM Library Specification identified in the TPM Dependency and Requirements section above.

2.2.2 Trusted Building Block (TBB)

A TBB consists of hardware and/or software that establishes a root of trust for measurement (provides an integrity measurement) and provides connectivity between an SRTM, the TPM, the PC Motherboard, the platform reset, and the TPM physical presence signal.

Because the SRTM and the TPM are the only implicitly trusted components of the PC Motherboard and since indication of physical presence requires a trusted mechanism to be enabled by the Host Platform owner, the indication of physical presence MUST be contained within the TBB.

The TBB provides functionality that permits an entity to believe in measurements that describe the current computing environment in the platform. An entity can assess those measurement results and compare them with values that are expected if the platform is operating as expected. If there is a match between the measurement results and the expected values, the entity can trust computations within the platform to execute as expected.

The SRTM initiates the measurement of the state of the hardware and software environment in a platform. Three data components are involved in creation of an integrity metric. The first component is the method used to gather the data. The second component is predicted values of measured data in a platform. The third component is the actual values of measured data in a platform. Any integrity challenger needs to know about all of these components in order to make a decision about the integrity of the platform.

A PC Client platform has a TBB consisting of the SRTM and the TPM. A PC Client has only one TPM, one SRTM, and one TBB. The TBB has a one-to-one binding with the PC Motherboard.

Figure 1 depicts the “one-to-one” logical relationships between the TBB, the TPM, and the PC Motherboard. The components within the box labeled “TBB” are within the TBB—for example, the SRTM and the connections to the TPM and the platform are part of the TBB. Note that the removable storage, input devices, output devices, CPU, remaining portion of Platform Firmware, and supporting hardware are not part of the TBB. The supporting hardware includes components to connect memory and I/O controllers to the PC Motherboard.

2.2.3 Roots of Trust

The terms Root of Trust for Measurement (RTM) and Core RTM (CRTM) within this specification refer to those entities that are associated with the Host Platform.

2.2.3.1 Root of Trust for Measurement (RTM)

The RTM is implicitly trusted. Trust in this component may be expressed in the Host Platform Certificate. The RTM is the point from which all trust in the measurement process is predicated. The RTM includes a core component (the CRTM), the computing engine to run the core component, and the physical connections of the core and the computing engine.

2.2.3.2 Core Root of Trust for Measurement (CRTM)

The component of the RTM from which the platform begins execution of one of its trusted states. Each transitive trust chain is rooted at this point.

2.2.3.3 Privacy Setting and the Scope of the RTM

Start of informative comment

If the Host Platform implements privacy settings using the command method (as defined in the TCG Physical Presence Interface Specification) for the indication of physical presence, those settings are under the control of a process within the SRTM and are measured as part of the SRTM. This is to provide verifiers (including the user or operator) with a method to validate that their privacy settings are respected and enforced. For example, if the platform uses Platform Firmware to detect the Operator by someone pressing a “function” key while the platform is under control of Platform Firmware, that detection code and the code that sends the physical presence commands must be measured unconditionally. Unconditionally measuring the physical presence command code on every boot allows a verifier to validate the code without needing to perform a physical presence command to obtain the code measurement. The code is not measured with a special event; it is measured as part of other SRTM measurements using events like the SRTM version identifier (EV_S_CRTM_VERSION) or a measurement like EV_POST_CODE.

End of informative comment

2.2.4 Host Platform

The Host Platform is the entity that executes the Host OS, which executes, presents output from, and receives input for the Host Applications for the users (remote or local). The Host Platform includes the PC Motherboard, Host Platform’s CPU, Host RTM, and Host TPM. The term “CPU” in this specification refers to the Host Platform’s CPU unless otherwise stated.

2.2.5 Non-Host Platforms

In the scope of this specification a Non-Host Platform is a self-contained execution environment within the system. These platforms execute in an environment separate from the Host Platform components. This is not to be confused with a peripheral, which, while potentially containing a powerful engine, only services and responds to requests from the Host Platform. For example, an Intelligent Platform Management Interface compliant management controller in servers would be considered a Non-Host Platform. The Non-Host Platform MUST NOT prevent the measurement, recording, and reporting of the true state of the platform. If a Platform Credential or Security Target is produced for this platform, they SHOULD provide an indication that a Non-Host Platform exists.

2.2.6 System

The system includes the Platform and all the Post-Boot components that compose the entire entity that performs actions for, or acts on behalf of, the user. The entity is the union of the Host Platform and the Non-Host Platforms. The Host Platform and the Non-Host Platforms may affect and influence each other.

2.2.7 Host Platform and TPM Reset

Start of informative comment

A Host Platform Reset is a hardware event that causes execution on the Host Platform’s CPU to permanently end its current instruction sequence and begin executing within the Core Root of Trust for Measurement (CRTM). This means that the CPU loses its entire context and begins execution at its reset vector. A Host Platform reset causes all Host Platform components to behave in their default, reset state.

Several events can cause Host Platform Reset, including those in the following non-exclusive list: initial Power-On; activation of a hardware reset line (i.e., PCI_Reset) including activation of the TPM_Init signal; initiated by the OS to begin a new boot session; and initiated by the CPU during certain unrecoverable fault conditions. The Host Platform has a consistent behavior, from a trust perspective, regardless of the cause of reset performed.

This section references only resets that apply to the Host Platform. Resets for Non-Host Platform and system are outside the scope of this specification.

Host Platform Reset only deals with establishing trust in the SRTM, not with other Host Platform components. Since all Host Platform components that are part of the transitive trust chain are measured, the action taken, or lack

thereof, by these components to a Host Platform Reset has no impact on the validity of the transitive trust chain. There may be an impact on the verifier's trust in the system but that is outside the scope of this specification.

The primary concern when establishing the transitive trust chain is that the reset of the Host Platform's CPU, which causes execution to begin within the SRTM, is "effectively simultaneous" with the Host TPM's reset.

TPM Device Reset is defined in the PC Client Specific Platform TPM Profile for TPM 2.0, Section 1.1 (Terminology).

End of informative comment

2.2.7.1 Reset Types

Start of informative comment

A Hardware Host Platform Reset occurs when a signal activates the reset signal of all Host Platform components. This may be a user-initiated or a software-initiated event triggered by a command to a hardware component that asserts the reset line. This includes resuming from S3.

A Cold Boot Host Platform Reset occurs when transitioning the Host Platform from a full Power-Off state in which no OS state is preserved on the Host Platform (except for that which is contained on any OS load device) to a Power-On state. This excludes returning from various power or suspend states that can occur after the Cold Boot Reset from an OS present state.

A Warm Boot Host Platform Reset occurs when software (often caused by a user keyboard input but may be software-induced) causes a Host Platform Reset. For this specification, a reboot is equivalent to transitioning through a Soft Off state.

A TPM_INIT occurs on a Cold Boot, a Warm Boot, and upon resuming from S3 (sleep). The SRTM issues the TPM2_Startup command, which tells the TPM whether to load saved state (for S3 resume) or not (for a boot).

Regardless of the types of Host Platform Reset described above, the normative text of this section applies to all of them.

End of informative comment

2.2.7.2 Host Platform Reset

1. Upon any Host Platform Reset, all execution cores within the Boot Strap Host Platform CPU MUST be reset and the Boot Strap Host Platform CPU MUST begin execution within the SRTM.
2. All remaining Host Platform CPU(s) MUST be reset.

2.2.7.3 TPM Reset

1. TPM Reset MUST NOT be executed without a Host Platform Reset.
2. TPM Reset MUST be executed (i.e., assertion of TPM_Init) when the Host Platform is Reset.

2.2.8 PCI Option ROM Request for Reset

Start of informative comment

The PCI Firmware Specification requires the actions in this section and, provided Platform Firmware performs this function, no further action is required. However, examples of non-compliant BIOS exist in the marketplace today. This section simply restates the requirement because it is important for security reasons.

End of informative comment

If BIOS supports Expansion ROM Configuration Utility Code Management, as described in Section 5.2.1.24 of the PCI Firmware Specification, Version 3, upon return from the Expansion ROM configuration code, Platform Firmware MUST check the return status from the configuration utility and if the configuration utility requests a system reset, Platform Firmware MUST perform a Host Platform Reset.

2.2.9 Trusted Process

A Trusted Process is either a hardware-based or a software-based process within the Host Platform that is trusted without the need for performing further inspection as expressed by the Host Platform Certificate. The Root of Trust for Measurement (RTM) is an example of a Trusted Process within its trust domain. (e.g., Static RTM or Dynamic RTM).

2.2.10 TPM Control Surface

2.2.10.1 TPM Visibility

Start of informative comment

Generally, OS loader components use UEFI Services to identify and interact with the TPM. A fully loaded OS uses the ACPI table entry (described in the *TCG ACPI Specification for TPM Family 1.2 and 2.0*) to identify the TPM and an OS resident TPM driver to interact with the TPM.

Platforms with a TPM intended for use by a fully loaded operating system populate the TPM device in the ACPI table of devices for the OS. Specifically, the TPM device will be represented by a device node in ACPI Source Language (ASL) code (see Section 9 ACPI) which will return a status (STA()) indicating the TPM is present. Additionally, the TCG2_EFI_PROTOCOL will be present and the EFI_TCG2_BOOT_SERVICE_CAPABILITY.TPMPresentFlag will be TRUE (see the TCG EFI Protocol Specification revision 13 or later).

Because some platform manufacturers may ship a platform with an operating system that does not have an OS TPM driver, the platform manufacturer may provide a mechanism to hide the existence of the TPM on the platform so users do not see the TPM device without an OS driver in an operating system device list. An alternative use case for this functionality would be an end user with a dual boot Operating System environment where one Operating System makes full use of the TPM and the second OS does not support TPM. The requirements for hiding the TPM are in Section 9.1.1 TPM Visible.

If the TPM is visible to the Operating System environment, a platform manufacturer may provide an option to disable the TPM or one or more of the hierarchies of the TPM. In this case, the TPM would still be visible to the OS which would have the means, through the Physical Presence Interface, to re-enable the disabled portions of the TPM on a subsequent boot. Providing this capability could cause OS or application features tied to the TPM to stop working, so care should be taken in presentation of these options to end-users.

Mechanisms to control visibility of the TPM to an operating system are platform manufacturer-specific. An example is a BIOS configuration option to hide or show the TPM to the OS.

End of informative comment

A TPM is visible if the Platform Firmware indicates the presence of the TPM device to the OS. A TPM is hidden if the Platform Firmware does not indicate the presence of the TPM device to the OS. See Section 9.1.2 TPM Hidden but Discoverable for more information.

2.2.10.2 TPM Discoverability

Start of informative comment

Because some platform manufacturers may provide a mechanism to prevent the discovery of a TPM by an Operating System which does not support it (thus leading to devices with no drivers loaded in the OS), an alternative mechanism is used to indicate whether a TPM is physically present on a platform. The Host Platform may be configured through some vendor proprietary mechanism to make the TPM visible to the OS and allow usage of the TPM.

As part of inventorying their computer system capabilities, some information technology staff in an enterprise may want to discover which hardware platforms have a TPM, whether the TPM is currently visible to the operating system or not. The existence of the TPM2 ACPI table means a mechanism exists on the platform for a platform owner to make the TPM visible to the operating system and available for use.

End of informative comment

A TPM is discoverable on a Host Platform if it is physically present, and the TPM could be made visible to the OS by defined platform owner action. A TPM is not discoverable on a Host Platform if it is not physically present on the platform or if the TPM is physically present, but system components prevent the platform owner from taking any platform manufacturer-defined action to enable the TPM. In the case that the TPM is not discoverable, the Platform Firmware disables the endorsement and storage hierarchies.

2.2.10.3 Enabling TPM

Start of informative comment

The TPM 2.0 device starts up with all of its functionality fully enabled. The Platform Firmware does not need to take any action, other than sending the TPM2_Startup and TPM2_SelfTest commands to make the TPM device itself ready for use. However, the Platform Firmware does need to make the TPM visible unless action is required to make it not discoverable. In the case where an end user would like to turn off TPM functions or prevent an Operating System environment from using it for one boot cycle or more, the Platform Firmware may support a mechanism to disable the TPM.

End of informative comment

A TPM is enabled on a Host Platform if it is visible to the Operating System and all available hierarchies are enabled. A TPM is disabled on a Host Platform if it is visible to the Operating System and at least the Storage and Endorsement Hierarchies are disabled. A disabled TPM may have the Platform Hierarchy disabled.

2.2.11 Boot State Transition

The transition between Pre-OS and OS-Present states is the first invocation of Ready to Boot or equivalent.

2.2.12 Establishing the Chain of Trust

2.2.12.1 Binding Between an Endorsement Key, a TPM, and a Host Platform

The relationship between the Endorsement Key, a TPM, and a Host Platform is described in the TPM Library Specification Family 2.0 Level 00 Revision 1.38, Part 1, Section 9.4.3.3 (RTR Binding to a Platform). A platform compliant with this specification SHALL ensure the TPM is powered (on/off) and reset at the same time as the host CPU.

2.2.12.2 Binding Methods

Start of informative comment

The method of binding the TPM to the PC Motherboard is an architectural and design decision made by the platform manufacturer and is not specified here. The two types of binding are: Physical and Logical. Physical binding relies on hardware techniques, while Logical binding relies on cryptographic techniques. The requirements for the strength of each binding are within the scope of a Protection Profile.

Example:

The TPM is a physical chip soldered to the Host Platform. Here the Endorsement Key is physically bound to the TPM (it's inside it) and the TPM is physically bound to the Host Platform by the solder.

End of informative comment

2.2.13 Locality

Start of informative comment

Hardware Proof of the Source of a Command

The TPM supports several authorization types to provide a trusted path between a user and a TPM's object (e.g., a TPM key or NV area). Common authorization methods are password, secret key, and public key. These methods rely on shared or provisioned secrets. Some technologies require a trusted path between hardware components. Methods involving shared or provisioned secrets are not practical for these hardware components. Locality uses hardware addressing to enforce a trusted path between hardware components. By restricting the addresses at which some hardware components can address the TPM, locality provides proof of the source component's identity. This allows TPM object policies to apply to specific hardware components enforced by the hardware.

Note that the enforcement agent for the trusted path (i.e., the authorization of the source of the command) is not in the TPM, rather it is the hardware controlling the channel to the TPM. The TPM will take commands sent to it at any locality and act on those commands, giving that command access to that locality's objects.

The PC Client defined TPM supports 5 Localities: 0-4. These are divided into five 4K Memory-Mapped IO (MMIO) ranges starting at the base address of the TPM's lowest address range. Localities are implemented by assigning each Locality to one of the 4K MMIO ranges. With the PC Client TPM mapped to the FED4_XXXX range, Locality 0 is mapped to FED4_0XXX; Locality 1 to FED4_1XXX; through Locality 4 to FED4_4XXX.

Locality 0 is designated as the default Locality. This is usually the platform's boot firmware, OS and applications. Locality 4 is designated for use by one of the CRTM's (Static or dynamic). Locality 1-3 are used by higher privileged components or software.

Note: the above description applies to hardware localities. There are also software localities used, for example, in virtualization, which are not associated with hardware addresses. These software localities are outside the scope of the PC Client TPM discussed here.

Sharing of the TPM

The TPM is a single-threaded, non-preemptive device and can therefore service only one request at a time. With localities, however, it is possible to have multiple domains sending commands to the TPM. The PC Client Platform TPM Profile provides a method to prevent one domain retaining control of the TPM until it has completed the current command.

To access the TPM, a driver within a domain (i.e., some entity operating at a specific locality) must request use of the TPM at that process's locality. The process then uses the TPM for one or more operations (essentially a session but without a session handle). When completed, and especially when a different domain is requesting use of the TPM, the software should then release the TPM. This allows the TPM to be shared between the various processes within the platform.

General Description of the CRTMs

Locality provides an expression of a CRTM called the Dynamic CRTM (DRTM) that is independent of the Host Platform Reset. As described in Figure 2: Relationship between Static and Dynamic RTMs, these two RTMs and their respective chains of trust have no relationship to each other except that they are both rooted at the same Root of Trust for Storage (RTS) and Root of Trust for Reporting (RTR) (i.e., the TPM). This is an important consideration in that the trust of each is dependent only on the trust in the common RTS/RTR and their own RTM.

End of informative comment

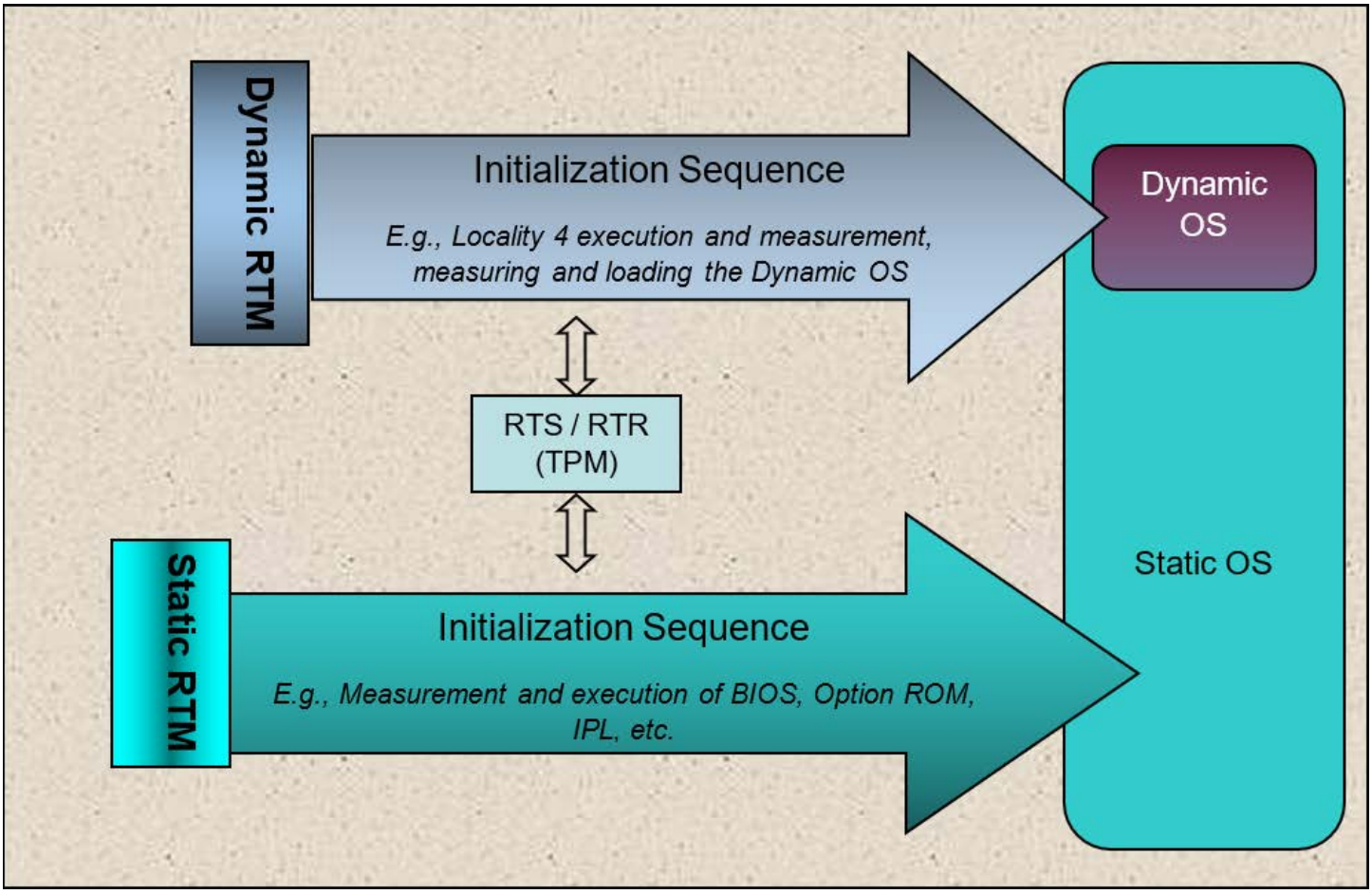

Figure 2 Example of SRTM and DRTM Initialization Sequence

2.2.13.1 Locality State Relationship

Start of informative comment

The expression of trust for each RTM is independent of the other. That is to say, each trust statement is verifiable within its own domain rooted at its own respective RTM. However, each chain of trust exists within an IT environment that is likely dependent on other components. For example, the trust in a Host Platform, which while verifiable within its own domain, is dependent on assumptions about its environment such as whether the routers are valid, the physical protections are in place, etc. It is possible and even conceivable that trust in the Host Platform as an entity will rely on the trust in the DRTM, SRTM or some subset of both.

An example: The Host Platform is used within a political region that requires certain privacy controls be respected and enforced prior to allowing the Host Platform to participate in using IT resources. The CRTM may not be able to verify that the user had proper use and control of the Host Platform's privacy settings. In this case, the IT infrastructure may require that the privacy setting be controlled and asserted by the processes within the SRTM's chain of trust. Therefore, when the Host Platform boots and attempts to join the IT environment, the verifier will first verify the chain of trust associated with the privacy setting (i.e., the SRTM chain of trust) before proceeding to verify the security assertions of the DRTM.

External entities and verifiers may associate the two chains of trust as being part of the same Host Platform where the two chains coexist within the Host Platform and therefore are treated at least in part as the union of their trust statements within a larger environment.

End of informative comment

1. Architecturally, the only commonality between the DRTM and the SRTM is the RTR/RTS (TPM).
2. There is no direct relationship between or dependence on the chain of trust established by the DRTM and the chain of trust established by the SRTM. From a trust perspective, these are architecturally distinct entities. Specific implementations MAY create a dependency between them. This dependency, if any, SHOULD be represented in the Host Platform Certificate.

2.2.14 System and TPM Power States

Start of informative comment

For PCs, a power management standard called ACPI defines a set of power states that each have different performance and power requirements. PC Client platforms that comply with this specification will support some or all of these power states. In general, the ACPI specification describes three types of power states: global states, system states, and device states. Refer to the ACPI specification for a full description.

System States

System states describe the power state of the entire system. Short descriptions of system states are:

G0 (or S0) – “On” or Working state (faster response times, high power usage)

G1 – Sleep states, which include several different system states:

S1 – Stand-by with low wakeup latency sleeping state, generally not supported by PC Client systems

S2 – Stand-by with CPU context lost sleeping state, generally not supported by PC Client systems

S3 – Suspend to RAM (system memory is preserved, other state is lost)

S4 (Hibernate) OS Initiated – The OS suspends system state to a persistent storage and restores it later

S4 (Hibernate) BIOS Initiated – Platform Firmware suspends system state to persistent storage and restores it later

G2 (or S5) – Soft Off (the system is restarted to return to the working state)

G3 – Mechanical Off (the system is restarted to return to the working state)

G0 and S0 are different names for the same state. Likewise, G2 and S5 are different names for the same state.

This specification uses the term S0 to mean both S0 and G0. Because G2, G3, S4, and S5 all refer to states where the system is not running, the term “Off” is used for all of them in this specification.

Device States

Device states describe the power use and context maintained for device on a system. Short descriptions of device states are:

D0 – Generally fully on (full functionality, probably full power use)

D1 – A lower power state with potentially longer latency

D2 – An even lower power state with potentially longer latency

D3 – Generally off (device context is lost)

The only transitions allowed for system states and devices states are those defined in the ACPI specification. There is not a direct correspondence between device states and system states; for example, a device may be state D3 while the system is in state S0.

TPM Device State

The TPM device per the PC Client Platform TPM Profile for TPM 2.0, Section 3.8 (Power Management) behaves identically in states D0 (fully-on), D1 (device-specific low power state), and D2 (another device-specific lower power state). Note: Implementing D1 and D2 for a TPM is not recommended. The TPM device is not allowed to exit state D3 without receiving a TPM_INIT.

The TPM has commands designed to ensure TPM state is preserved before placing the TPM in the D3 state and restore TPM state when leaving the D3 state. The OS issues a TPM2_Shutdown(STATE) command before placing the TPM in D3. The system may enter Hibernate or Sleep at this point. The platform firmware is unaware of the distinction between Hibernate (S4) and Off (S5). In both cases (Resume from S4 or S5), Platform firmware issues a TPM2_Startup (CLEAR). The TPM contains counters that track the number of Resets (cold boot), and the number of Restarts (resume from Hibernate). If, on an S4 resume, the platform firmware boots to a platform firmware-controlled environment, such as Setup, the platform firmware usually exits that environment by sending TPM2_Shutdown (CLEAR) and resets the platform. This causes the TPM to clear the Restart counter and increment the Reset counter, which can prevent an OS from resuming from S4. This is desirable behavior in the event of an attack, where the platform is booted to an environment not controlled by platform firmware (such as a USB key). However, if a legitimate user of the platform enters setup, the platform can preserve the state of TPM counters by sending a TPM2_Shutdown (STATE). Hybrid sleep scenarios occur when a laptop enters a S3, then due to a critically low battery, the OS transitions the system to S4. Such a scenario is invisible to platform firmware and thus platform firmware cannot take any action as described above to preserve the state of the TPM counters.

Other scenarios are possible when placing the TPM in D3, but they may result in the TPM not being usable later without a transition to S5 and are not discussed in this specification.

End of informative comment

2.2.15 General Host Platform Power Requirements

Start of informative comment

There is no required behavior during any power state as long as the Host Platform provides resources (e.g., power) to the TPM to perform its required functions during each state. For example, it is obvious that power must be applied during S0-S2. However, for example, the TPM may be implemented such that auxiliary power is required to maintain saved state (like PCR registers) during the system state S3 or the TPM device state D3. In this case, a Host Platform incorporating this type of TPM needs to provide necessary power to the TPM during D3 and S3, while Host Platforms incorporating TPMs using flash memory or other non-volatile (NV) storage technology will not require power during D3 or S3 for this purpose.

Another example is the clock. There is no requirement for the TPM to maintain this clock across any power state (besides S0-S2, of course). Some TPM manufacturers may choose to provide this feature as a “value add.” Those TPMs may require auxiliary power during non-S0 power states.

This specification does not define a mechanism to power down a TPM and restore its saved state while the system is in the ACPI Legacy state. The only mechanism defined in this specification to restore a TPM saved state is to

transition out of the ACPI S3 state to S0. While in ACPI Legacy mode, the system should keep the TPM powered so its state is preserved.

End of informative comment

2.3 Overview of the Measurement Process

Start of informative comment

This specification defines measurement as the process of hashing various components of the boot process, extending the results in the TPM and logging the component's measurement in the measurement log in the Host Platform memory. The specific components, order of measurements, and storage locations are specified in the normative text in subsequent sections.

As a TPM in a PC Client system is usually a discrete chip that runs at lower clock speeds than the CPU and is connected to the platform via a slow bus, interaction with the TPM can be a platform bottleneck during the boot process. Thus, this specification recommends minimizing the amount of data sent to the TPM for measurement and, once the initial measurement is made, making use of the CPU to perform hash computations of large amount of data.

End of informative comment

2.3.1 Usage and Optimization of Hash Functions

1. The Platform Firmware MAY use the TPM's Hash interface command but SHOULD do so for as little data and as few times as possible.
2. The Platform Firmware SHOULD use its own, Host Platform CPU-based hashing function as early as possible and SHOULD continue to use it.

2.4 Terminology

Start of informative comment

The following terms are used as defined below throughout the document. All other terms are defined in the PC Client Implementation Specification.

TPM Device Reset: the assertion of the __TPM_INIT hardware signal.

Platform Software: the source of the command, which may be an operating system driver or an application.

Platform Hardware: platform components including chipsets and associated microcode, and microprocessors and associated microcode.

SRTM: code supplied by the platform manufacturer, as a subset of Platform Firmware that initializes and configures platform components and is the portion of Platform Firmware that defines the initial trust boundary.

Operating System, or **OS:** generic term for the system software and its collection of drivers and services that manages computer hardware and software resources.

Static OS: the operating system that is loaded during the initial boot sequence of the platform from its platform reset.

Dynamic OS: an operating system that is loaded by the Static OS. There may be more than one Dynamic OS per Host Platform but only one can be loaded at a time. The Dynamic OS can be unloaded keeping the Static OS resident and operational.

Read: a transaction where the calling entity requests and receives data from a specified register or buffer in the TPM.

Write: a transaction where the calling entity sends data to a register or buffer in the TPM.

PC Client Platform Implementation Specification: the combination of the *PC Client Platform Firmware Profile Specification*, the *TCG EFI Protocol Specification,* the *PC Client ACPI Specification*, and the PC Client Physical Presence Interface Specification.

NUL: used in this specification to indicate a null character. For ASCII strings NUL is defined as 0x00. For UCS-2 strings, NUL is defined as 0x00 0x00.

Trust Anchor: a Root of Trust in the context of certificate validation. In the context of SPDM Certificate validation, the trust anchor can be either the root certificate, the hash of a root certificate, or an intermediate certificate received out of band. For the purpose of this specification, the Trust Anchor contains class information, not instance information.

EFI Variable name convention: All UEFI variable names prefaced with L"<VariableName>" are defined by the UEFI specification to be 16-bit Unicode variables.

End of informative comment

2.5 TCG Specification Dependency and Naming

2.5.1 Division of Documentation

Start of informative comment

The PC Client Specifications are divided into two documents:

1. The *PC Client Specific Platform TPM Profile for TPM 2.0* (PTP) discusses the specifics regarding the requirements of the TPM for PC Client but only the requirements for the TPM itself, not the requirements for a platform integrating the TPM. The PTP discusses the details of what interfaces and protocols are used to communicate with the TPM and the platform-specific set of requirements. The PTP includes the definitions of the items identified in the TPM Library specification as "Platform Specific" such as the minimum number of PCRs required and NV Storage available. The target audience for the PTP is the TPM manufacturers but platform manufacturers should review it as well.
2. This specification, the *PC Client Platform Firmware Profile,* specifies the requirements for the TPM as it is implemented on the platform. Issues such as TPM, platform and firmware provisioning, usage of TPM to record measurements of platform code, PCR mapping, functional interfaces, and interfaces are discussed. The target audience for this document is platform manufacturers.

The following TCG Specifications are referenced in this specification.

End of informative comment

2.5.2 "This" Specification

References to "this specification," unless contextually referencing a different antecedent, refer to the informative and normative comments contained in this document.

This specification defines only functional aspects of the concepts and implementation of a TPM, SRTM, and other support features for a PC Client Platform. The definition of the security mechanisms and the strength of those mechanisms are intentionally outside the scope of this specification.

2.5.3 Platform TPM Profile (PTP)

Start of informative comment

The PC Client Specific Platform TPM Profile for TPM 2.0 (PTP), Version 2.0, Revision 1.00 is the “companion” specification to this specification. The PTP defines the programmatic interface to the TPM and the method by which it is connected (i.e., which local Host Platform bus) to the Host Platform.

End of informative comment

2.5.4 TPM Library Specification

Start of informative comment

Refers to the TCG TPM Library Specification, Family 2.0 as released. The specification has four parts. Unless a specific part is referenced, this reference refers to all parts of the specification.

End of informative comment

2.5.5 TCG ACPI Specification

Start of informative comment

Refers to the TCG ACPI Specification for Family 1.2 and Family 2.0, Level 00 Revision .37, as released. The specification is managed by the server work group but is leveraged by PC Client platforms.

End of informative comment

2.5.6 TCG Physical Presence Specification

Start of informative comment

Refers to the TCG PC Client Physical Presence Interface Specification, Version 1.3, Revision 52 as released.

End of informative comment

2.5.7 TCG Platform Reset Attack Mitigation Specification

Start of informative comment

Refers to the TCG Platform Reset Attack Mitigation Specification, Version 1.0

End of informative comment

2.5.8 TCG EFI Protocol Specification

Start of informative comment

Refers to the TCG EFI Protocol Specification for TPM Family 2.0 version 1.0 revision 0.9 or later.

End of informative comment

2.5.9 TCG Reference Integrity Manifest Information Model

Start of informative comment

Refers to the TCG Reference Integrity Manifest Information Model version 1.0 or later.

End of informative comment

2.5.10 TCG Firmware Integrity Measurement Specification

Start of informative comment

Refers to the TCG Firmware Integrity Measurement Specification version 1.0 or later.

End of informative comment

2.6 External Specifications

This section lists links to the external non-TCG specifications referenced by this specification.

Advanced Configuration and Power Interface (ACPI), Version 6.0 or later, https://uefi.org/specifications

Unified Extensible Firmware Interface Specification (UEFI), Version 2.10 or later, https://uefi.org/specifications

Microsoft Portable Executable and Common Object File Format Specification, available at https://docs.microsoft.com/windows/win32/debug/pe-format

Microsoft Windows Authenticode Portable Executable Signature Format, Version 1.0, Microsoft Corporation, March 21, 2008, https://download.microsoft.com/download/9/c/5/9c5b2167-8017-4bae-9fde-d599bac8184a/Authenticode_PE.docx

U.S. National Institute of Standards and Technology (NIST) Special Publication 800-147 BIOS Protection Guidelines available at: https://csrc.nist.gov/pubs/sp/800/147/final

U.S. National Institute of Standards and Technology (NIST) Special Publication 800-147B, BIOS Protection Guidelines for Servers available at: https://csrc.nist.gov/pubs/sp/800/147/b/final

U.S. National Institute of Standards and Technology (NIST) Special Publication 800-155 BIOS Integrity Measurement Guidelines available at: https://csrc.nist.gov/pubs/sp/800/155/ipd

DMTF Security Protocol and Data Model (SPDM) Specification 1.0 or later available at: https://www.dmtf.org/dsp/DSP0274

2.7 Specification Conventions

Start of informative comment

The UEFI Specification defines all data structures in Little-Endian. PC Client processors represent data in Little Endian format as well, so all structures, constants and data created and defined by the PC Client PFP are in Little Endian format.

The justification for this is that when software deals with the Host Platform itself (e.g., the PC Client data, structures, etc.), it uses Little Endian—always. Software already deals with this bifurcation when communicating over a network. When getting packets from the network (e.g., FTP, HTTP, etc.) it deals with Big Endian; when it deals with data and structures from the Host Platform itself, it does so using Little Endian. Changing within the context or purview would be inconsistent.

End of informative comment

1. All constants and data SHALL be serialized as Little Endian format unless otherwise explicitly stated.
2. All strings SHALL be represented as an array of ASCII bytes with the left-most character placed in the lowest memory location, unless otherwise specified.
3. ASCII strings SHALL consist of 8-bit-wide characters.
4. ASCII strings SHALL NOT be NUL terminated, unless otherwise specified.
5. ASCII strings SHALL be printable.
6. Hexadecimal numbers are designated by ‘0x’ preceding the number or ‘h’ following the number. Decimal numbers do not have the preceding 0x.
7. In some memory layout descriptions, certain fields are marked reserved. Firmware MUST initialize such fields to zero and ignore them when read. On an update operation, firmware MUST preserve any reserved field.

3 Host Platform Roots of Trust Requirements

3.1 Locality Support Requirements

3.1.1 Pre-OS Environment

Start of informative comment

Before sending a command to the TPM, the software or other platform components must set the TPM to operate at the desired locality. The TPM can be set to only one locality at a time. The TPM may also have a state where there is no active locality set, called Locality None. The TPM can transition the active locality back and forth between different localities or Locality None. See the PC Client Platform TPM Profile for TPM 2.0 for details about how these transitions are performed.

This section specifies the TPM's locality for the pre-OS environment as it transitions to the Static OS.

Note: Per section 15.1.4, Platform Firmware uses Locality 0 to access the TPM.

End of informative comment

1. In Pre-OS, Platform Firmware MUST transfer control to UEFI Drivers and Applications with the TPM in Locality 0.
2. UEFI Drivers and Applications MUST transfer control back to Platform Firmware with the TPM in Locality 0.

3.1.2 OS Environment

Start of informative comment

Because a Static Root of Trust for Measurement environment may be hosted in a virtualized environment, the Static OS should not assume sole access to the TPM. This is one of many reasons why the TPM is allowed to switch localities. All environments that use the TPM should request use of the TPM; use it when granted; then release it when done using the TPM. This is similar to any non-preemptive environment. The request, granting and release of the TPM's locality is done at the TPM's hardware interface layer and is expected to be very fast so there is little, if any, perceivable latency.

Before sending a TPM command, an entity using the TPM first verifies that the TPM is in a locality the entity is authorized to access. If the TPM is not in the correct locality, the entity using the TPM requests use of the TPM's locality or attempts to use the seize protocol, defined in the PTP, to gain access to the TPM. Once the requesting entity is granted access, the entity sends commands. When done, the entity relinquishes locality, allowing other entities to make use of the TPM.

End of informative comment

3.2 SRTM

3.2.1 Introduction of Concepts

Start of informative comment

The SRTM environment provides the Root of Trust for the components of the Host Platform as they are initialized following a Host Platform Reset. The goal of the SRTM is to provide an unbroken chain of measurements for components that executed on the platform or security-relevant configuration data that may have influenced platform state. By assessing the chain of components, an entity may establish trust in the current state of the platform. The SRTM is the default RTM for all components because the SRTM measurements occur during boot by default. The SRTM uses Locality 0 and the memory addresses associated with Locality 0. See the PC Client Platform TPM Profile for TPM 2.0 for definitions of locality and memory addresses.

End of informative comment

3.2.2 Initial TBB Control and Host Platform Reset

Host Platform Reset MUST cause the Host Platform to begin execution within the SRTM.

3.2.3 Static Root of Trust for Measurement (SRTM)

The Static Root of Trust for Measurement (SRTM) MUST be an immutable portion of the Host Platform's initialization code that cannot be changed except by manufacturer-controlled means. See Sections 5.2 Flash Maintenance and 5.3 Firmware Compliance Requirements.

Note: The trust in the SRTM environment is based on the SRTM. The trust in all SRTM measurements is based on the integrity of the SRTM component.

Currently, in a PC, there are many types of SRTM architectures. An example is below.

3.2.3.1 Firmware Boot Block SRTM

Start of informative comment

In this architecture, the Platform Firmware is composed of a Boot Block (SEC/PEI/IBB) and a UEFI firmware. Each of these is an independent component and each can be updated independent of the other. In this architecture, the Boot Block is the SRTM while the UEFI Firmware is not but is a measured component of chain of trust.

End of informative comment

3.2.4 Transfer of Control from SRTM

Prior to transferring control to another entity within the SRTM environment, an executing entity MUST measure the entity to which it will transfer control.

3.3 Integrity Collection and Reporting

Start of informative comment

The Static PCRs, those that cannot be reset without a TPM_Init, are divided into two primary sets. The first set is designated for the Host Platform's pre-OS environment (PCR[0-7]) and the other designated for the Host Platform's Static OS (PCR[8-15]), see the PC Client Platform TPM Profile for TPM 2.0 for further information. PCR[6] may be used in either as determined by the platform manufacturer. The Static pre-OS PCRs provide the Host Platform's initial chain of trust starting from Host Platform Reset. These establish a chain of trust from the SRTM through the OS's Boot Manager Code. The definition of the Static OS PCRs is outside the scope of this specification and is the purview of the specific OS provider.

The platform may be designed to allow an operator to indicate to the platform that the platform is not to create measurements. For example, the platform may provide a "Firmware Setup" utility that allows the operator to disallow platform measurements. This setting may be stored into the platform's CMOS, for example. If the operator chooses this option, the platform must be designed to disable the TPM along with preventing the Platform Firmware from creating entries into the event log.

The property of the Extend operation makes the set of the integrity measurements taken into each PCR order-dependent. If two measurements, A and B, are extended into a single PCR, the PCR's resulting content will be different if the Extends are performed A then B vs. B then A. For this reason, the order in which the measurements are extended is important because platform applications may "seal" data to the pre-OS PCRs. A seal operation only functions properly with strict adherence to the measurement sequence.

This specification defines a list of measurements to be placed in each PCR. The order of the measurements is mostly advisory and some variance is both allowed and expected even with differing platform models from the same manufacturer. However, to make the seal (and other TPM operations) function properly, it is important that the

sequence of the measurements be consistent between each platform reset when there are no security-related changes. Platform Firmware vendors and other pre-OS software writers must be careful to design the components such that changes to the measurement sequence are not made inadvertently between each platform reset unless the execution or boot sequence actually changes.

With the introduction of UEFI systems, TCG produced the TCG PC Client EFI Protocol Specification. UEFI provides services which can be utilized by both the Platform Firmware and the Operating system. Two of these services which are critical to the correct operation of the integrity measurement and reporting system are the Get Event Log call (EFI_TCG2_GET_EVENT_LOG) and the Exit Boot Services call (EVT_SIGNAL_EXIT_BOOT_SERVICES). The Get Event Log call allows the OS to ask Platform Firmware to pass the platforms' TCG event log to the OS. The Exit Boot Services call (EBS) signals the end of Platform Firmware control of the boot process and the beginning of the OS control. Unfortunately, these two calls can be asynchronous, creating a situation where measurements can be made to the TPM and logged in the TCG Event Log after the OS has already received the Event Log. To address this, this specification adds a second instance of the event log, the Final Events Table, which is contained in an instance of a UEFI_CONFIGURATION_TABLE. This table will contain duplicate information for only those events which occur after the Get Event Log call.

End of informative comment

3.3.1 Collection and Reporting of Measurements

1. Platform Firmware MUST be designed to perform integrity measurements of the Host Platform's pre-OS environment per this specification.
2. Integrity collection and reporting MAY be available on the Host Platform upon delivery to the owner or MAY be made available to the owner using a method provided by the Host Platform Manufacturer.
3. The Host Platform MAY provide a method for the owner or operator to prevent the platform from making measurements. If the owner or operator indicates that the Host Platform is not to make measurements, then SRTM MUST disable the TPM as specified in Section 7.1 TPM Visibility to the OS.
4. Unless TPM is hidden, integrity measurements made by the Platform Firmware that are extended MUST be extended into all allocated banks for a PCR, see Section 10.1 Introduction.
5. Integrity measurements by Platform Firmware that are extended into PCR[0-7] MUST be done only while the Host Platform is in the Pre-OS environment after starting from an Off state.
6. Integrity measurements of the pre-OS environment MUST NOT be extended to the PCRs allocated to the operating system. Any Host Platform configuration change that occurs after the Host Platform transfers control to an operating system, or while resuming from other power states, is the purview of the operating system and measurements of those events are extended to the PCRs allocated to the operating system. PCR[0-7] are exclusively for the use of platform firmware. Operating system and OS present software should never use those PCRs.
7. When the TPM is visible, Platform Firmware MUST log all measurements made to the TPM in the TCG Event Log. See Section 10.1 Introduction:
    a. Prior to Exit Boot Services and the call by the Operating System to get the Event TCG Event Log.
    b. After the first call to get the Event Log from Platform Firmware, all measurements made by Platform firmware MUST be reported in the Event Log and in the EFI_TCG2_FINAL_EVENTS_TABLE, as defined in the TCG EFI Protocol Specification v1.0 revision 9 Section 7 (Log Entries after Get Event Log Service).
8. When the TPM is hidden, Platform Firmware MUST cap PCR[0-7] as defined in Section 3.3.4 PCR Usage, but SHALL NOT log measurements in the TCG Event Log. See Section 10.1 Introduction.
9. Integrity measurements made by the platform manufacturer for PCR[6] MAY occur at any time (Pre-OS or Post-Boot). See Section 3.3.4.7 PCR[6] – Host Platform Manufacturer Specific Measurements.
10. When the TPM is visible, but independent of TPM state, Platform Firmware MUST measure EV_SEPARATOR as defined in Section 10.4.1 Event Types. If the TPM is enabled, this event MUST be

entered into both copies of the event log. If the TPM is visible to the Operating System but the hierarchies are disabled, Platform Firmware MAY make other measurements as defined in this specification.

Note: EV_SEPARATOR delineates the point in the platform boot where Platform Firmware relinquishes control of the TPM measurement process. Measuring and extending EV_SEPARATOR enables an observer to know where Platform Firmware control ended. As an example, say that Platform Firmware controls a secret that is protected by the TPM. This secret must be protected from OS-resident agents. In this case, Platform Firmware would seal this secret to one or more TPM PCR such that only Platform Firmware could unseal it. A method to do this would be to calculate the value of the desired PCR prior to the EV_SEPARATOR event and seal to that value. Any attempt to unseal the PCR after this point would fail, as EV_SEPARATOR would alter the PCR value.

11. While not all events described in Section 10.4.3 EV_ACTION Event Types are produced by every platform on every boot, when one is produced, platform Firmware MUST create the event indicated in Section 10.4.3 EV_ACTION Event Types.
12. Platform firmware SHALL ensure the action of extending a measurement to a PCR and logging the associated event in the event log are atomic. See Section 10.3 Measurement Event Entries and Log.

3.3.2 Error Conditions

Start of informative comment

Some error conditions include failure of the TPM or its code on its power on initialization test, presentation by Platform Firmware of an incorrect Startup Type, or the TPM2_Startup command originating from an incorrect locality. Unlike with TPM 1.2, there are multiple ways to handle these errors.

If the SRTM is unable to successfully initialize the TPM using the TPM2_Startup command, it is possible that later code could successfully issue the command and record false integrity measurements. To prevent this, the SRTM takes steps to prevent use of the TPM or platform if it is unable to successfully initialize the TPM. A special case is when the attempt to initialize the TPM results in the TPM being in failure mode; later components will be unable to record false integrity measurements because for most commands the TPM will continue to return TPM_RC_FAILURE.

This section applies for resume from S3 or normal boot.

Figure 3 shows the remediation steps the SRTM takes when initializing the TPM fails.

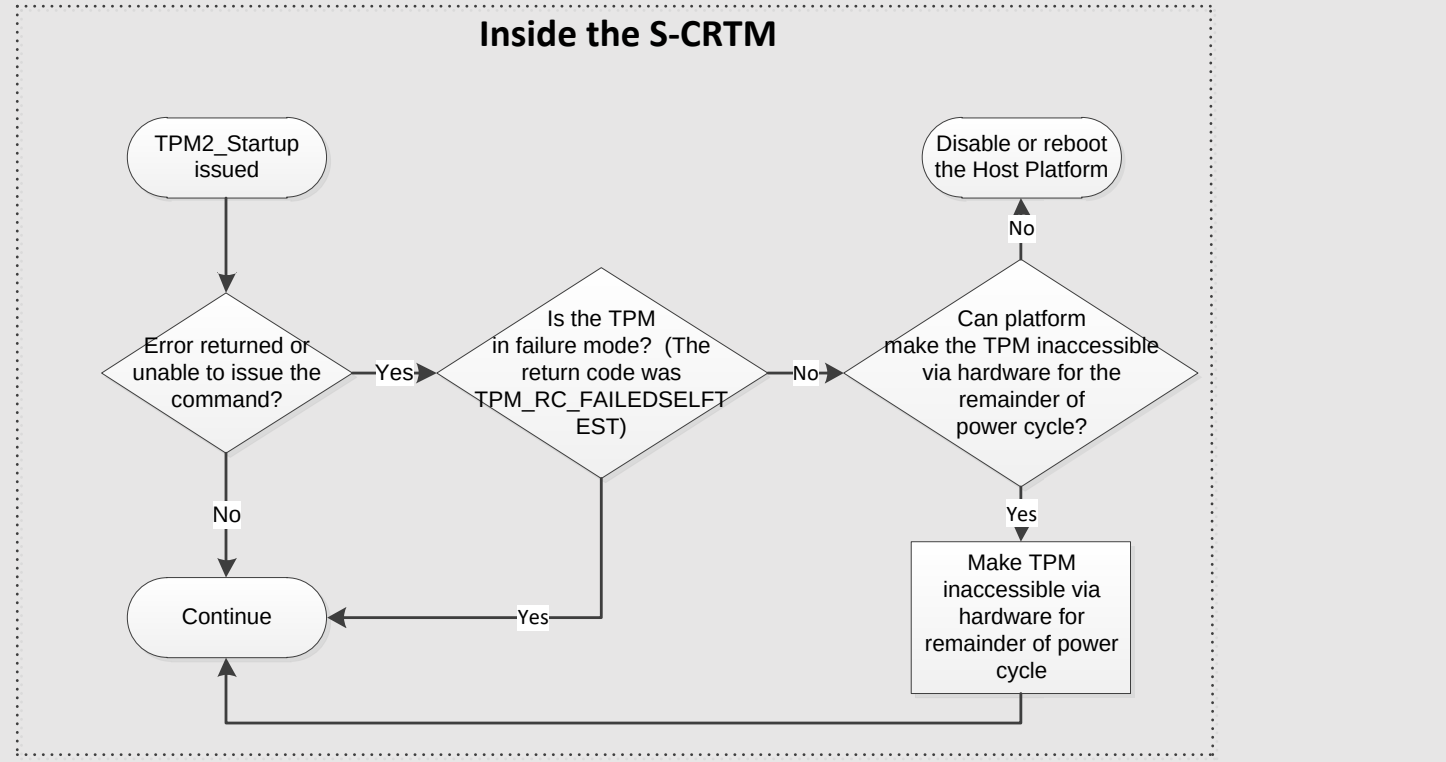

Figure 3 SRTM remediation steps when initializing the TPM

End of informative comment

3.3.2.1 Errors during TPM Initialization

1. If the TPM responds to a TPM2_Startup command with TPM_RC_INITIALIZE, Platform Firmware MAY ignore the return code and continue to boot.
2. If the TPM responds to a TPM2_Startup command with TPM_RC_FAILURE, Platform Firmware SHALL:
    a. Reboot the platform and perform a TPM2_Startup (CLEAR), or
    b. With the exception of sending the TPM2_HierarchyControl command, perform the actions defined in Section 7.1 TPM Visibility to the OS for hiding the TPM.
3. If the TPM responds to a TPM2_Startup command with TPM_RC_VALUE or TPM_RC_LOCALITY, the Platform Firmware SHALL:
    a. Reboot the platform, perform a TPM2_Startup (CLEAR) and make all required measurements.
4. If the TPM responds to a TPM2_Startup command with any of the following error codes: TPM_RC_COMMAND_CODE, TPM_RC_UPGRADE or TPM_RC_REBOOT, the TPM has attempted to perform a firmware field upgrade and encountered an error, e.g. power removed during the upgrade. Platform Firmware SHALL:
    a. Restart the field upgrade process, OR
    b. Notify the user and, with the exception of sending the TPM2_HierarchyControl command, perform the actions in 7.1 TPM Visibility to the OS for hiding the TPM.
5. If the TPM responds to a TPM2_Startup with TPM_RC_BAD_TAG, Platform Firmware SHALL:
    a. Send a TPM_Startup (TPM 1.2) command.
        i. If the TPM returns with TPM_RC_SUCCESS, then a TPM 1.2 is present and the behavior is not defined in this specification.

ii. If the TPM returns with TPM_RC_BAD_TAG, then the TPM is in Field Upgrade Mode. Platform Firmware SHALL perform the actions in normative 4.

3.3.2.2 Errors Recording Measurements

1. If the measurement of the SRTM, POST BIOS or Embedded UEFI Drivers cannot be made due to a failure of the Platform Firmware, the PCR's MUST be capped by extending the digest of the value 00000001h to each PCR[0-7], and an EV_SEPARATOR event SHOULD be recorded in the event log for each PCR, per Section 10.4.1 Event Types.
2. The SRTM SHOULD log the error event to the event log.
3. If each PCR[0-7] cannot be capped, the Host Platform SHOULD take any necessary action to notify the Host Platform's administrator, user, and operator and transition to a "fail-safe" mode by performing one of these actions:
    a. Make the TPM interface inaccessible via hardware for the remainder of the power cycle;
    b. Reboot the Host Platform;
    c. Disable the Host Platform; OR
    d. Perform a vendor-specific action that is equivalent to one of the options above.

3.3.3 Boot Event and PCR Usage Model

Start of informative comment

The purpose of this section is to provide the model for PCR usage and for adding events to the Event Log. This entire section is an Informative comment, including Figure 4, Figure 5, and Figure 6.

- Figure 4 is an architecture drawing that shows the principal firmware and software components defined in the UEFI Specification – UEFI Boot Services, UEFI Runtime Services, and the UEFI OS Loader – and their relationship to the platform hardware and the OS software.
- Figure 5 shows the sequence of behaviors for the UEFI components during the platform boot process and the OS boot process.

  Figure 6 maps, in general, the behavioral components on a UEFI platform to the PCRs into which each type of behavioral component is measured.

Together, these three diagrams form the boot event and PCR usage model upon which the requirements in subsequent sections of this specification are based.

Figure 4, immediately below, is part of this Informative comment. This diagram illustrates the relationships of various components of a UEFI-specification compliant system that accomplish platform boot and OS boot.

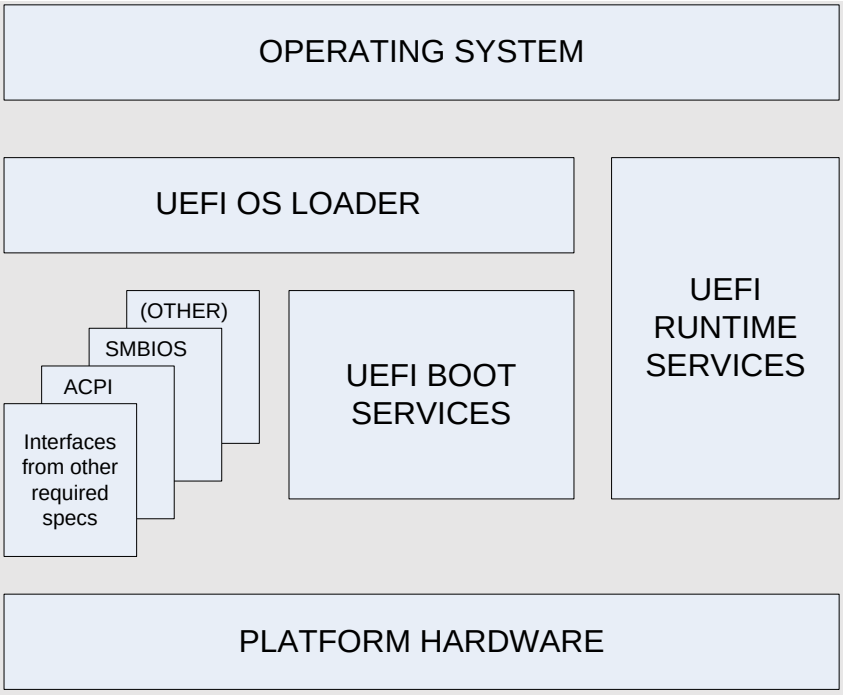

Figure 4 UEFI Architecture

Figure 5, immediately below, is part of this Informative comment and shows the sequence of behaviors for the UEFI components during the UEFI platform boot process and the UEFI OS boot process and, in general, the transitions where events are measured (labeled e0, e1, e2, and so on in the diagram).

In Figure 5, each one of the arrows with an “e” annotation represents a transition where a TCG Event is created; an un-annotated arrow does not represent a TCG event measurement. For example, the first measurement action, labeled “e0” is made by the platform initialization code.

Event “e0” describes the behavior of Platform Firmware from system board ROM that measures code contained in the rest of the platform UEFI firmware and that resides in the CRTM. This measuring agent may or may not contain the UEFI Boot and Runtime Services. In the case of a CRTM that does not contain UEFI Boot and Runtime Services, this measuring agent must then measure into PCR[0] the code that does contain UEFI Boot and Runtime Services. This initial code, if in CRTM, must measure its own version ID into PCR[0].

Event “e5” describes invocation of a UEFI application in the boot order list. This is typically an operating system loader. The event “e5” will have associated with it an event that measures the PE/COFF image into PCR[4] and the contents of the boot variable, namely the UEFI device path, into PCR[5]. For more information about this measurement action, see Section 8.2.2 Extending PCR[5].

This informative comment has given a couple of examples of measurement actions associated with particular platform boot transition events, but makes no attempt to describe the measurement events associated with all the transitions shown in Figure 5. To see all the measurement events associated with all the transitions shown in Figure 5, see Section 3.3.4 PCR Usage of this specification.

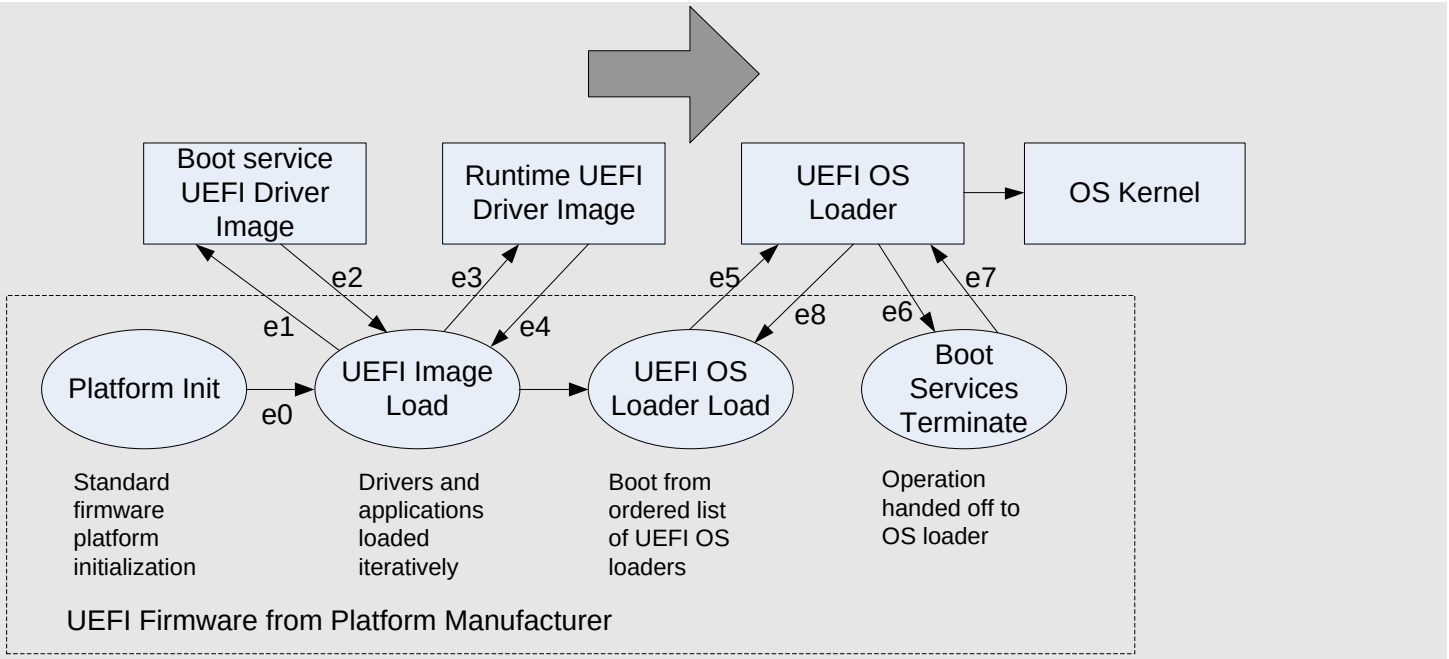

Figure 5 UEFI Platform Boot Process

Figure 6 PCR Mapping of UEFI Components, immediately below, shows a diagram illustrating the general scheme for PCR usage on a UEFI platform.

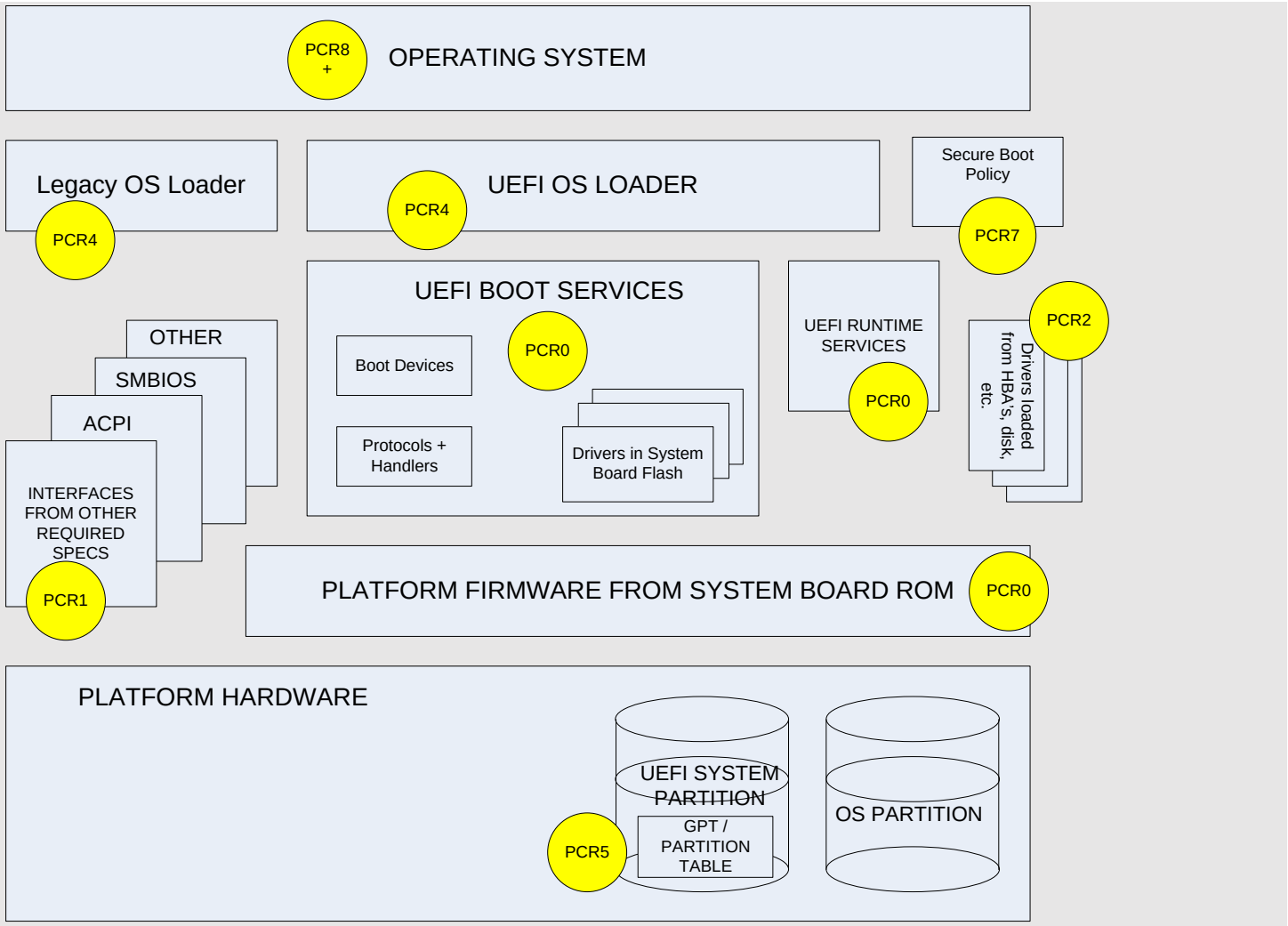

Figure 6 PCR Mapping of UEFI Components

End of informative comment

3.3.3.1 Measuring PE/COFF Image Files

Start of informative comment

PE/COFF images are measured in their memory-mapped store (ROM for execute-in-place and system memory for loaded images).

The measurement must be made prior to the application of any fix-ups or relocations.

An example of a PE/COFF image file is a UEFI OS loader, which would be an OS subsystem type of IMAGE_SUBSYSTEM_EFI_APPLICATION, IMAGE_SUBSYSTEM_EFI_BOOT_SERVICE_DRIVER, and IMAGE_SUBSYSTEM_EFI_RUNTIME_DRIVER (as per PE/COFF specification section 3.4.2).

For UEFI images, the data structure used in the TCG_PCR_EVENT2.event field will include but is not limited to “where” the image came from, namely its UEFI device path, and the address in memory for the shadowed PE image (see the definition of the UEFI_IMAGE_LOAD_EVENT structure, in the mandatory statements below).

The image in physical memory will ultimately have relocations applied (i.e., fix-ups). The event log will detail the load location of the image, so it will be possible to undo relocations by referring to the on-media image.

The Microsoft Authenticode Portable Executable Signature Format Specification identifies which PE/COFF image section types UEFI measurement agents must measure and which types of PE/COFF image sections are ignorable. What "measure before applying relocations" described above means in practice is that the UEFI implementation will perform "LoadImage()" actions (e.g., copying PE/COFF to memory, etc.), measurement, and then relocate the application, and finally, the UEFI service "StartImage()." As such, UEFI implementations of these services must punctuate their flow with this measurement action.

End of informative comment

1. When measuring a PE/COFF image, the TCG_PCR_EVENT2 structure Event field MUST contain the UEFI_IMAGE_LOAD_EVENT structure defined in Section 10.2.3 UEFI_IMAGE_LOAD_EVENT Structure.

3.3.4 PCR Usage

Start of informative comment

PCR[0-15] represent the Host Platform's static RTM and are associated with Locality 0. Therefore, all the PCRs associated with Locality 0 are resettable only upon Host Platform Reset.

This section defines the PCR assignments used for boot time integrity metrics and the methodology for collecting the metrics. The first seven PCRs are defined for use within the Pre-OS environment (PCR[4] being transition code, which is partially Pre-OS and partially OS-resident). Throughout the platform boot process, a log of all executable code and relevant configuration data is created and extended into PCRs as described below.

Each time a PCR is extended, a log entry is made in the TCG event log. This allows a challenger to see how the final PCR digests were built.

This section also describes the use of PCRs 16 and 23.

Note: PCR Indexes not defined in Table 1 PCR Usage are not allowed by this specification.

End of informative comment

Table 1 PCR Usage

PCR Index	PCR Usage
0	SRTM, BIOS, Host Platform Extensions, Embedded Option ROMs and PI Drivers
1	Host Platform Configuration
2	UEFI driver and application Code
3	UEFI driver and application Configuration and Data
4	UEFI Boot Manager Code (usually the MBR) and Boot Attempts
5	Boot Manager Code Configuration and Data (for use by the Boot Manager Code) and GPT/Partition Table
6	Host Platform Manufacturer Specific
7	Secure Boot Policy
8-15	Defined for use by the Static OS

PCR Index	PCR Usage
16	Debug
23	Application Support

1. Firmware on a PC Client Platform compliant with this specification SHALL measure all normative items using a tagged Hash structure for each PCR bank enabled in the platform's TPM. See Section 10.2 TCG Defined Structures.
2. Firmware on a PC Client Platform compliant with this specification SHALL measure all normative items into the PCR specified for the measurement.

3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers

Start of informative comment

The SRTM may measure itself or may be measured by an H-CRTM event extended to PCR[0]. The SRTM must measure to PCR[0] any portion of the PEI Code, including Manufacturer Controlled Embedded Drivers, Host Platform Firmware, etc., that are provided as part of the PC Motherboard. Primarily executable code, some fairly stable version identifiers, and configuration data are measured. Configuration data such as ESCD should not be measured as part of this PCR.

All these components are under the control of the manufacturer or its agent.

If for any reason a measurement cannot be made to PCR[0], none of the other SRTM PCR values can be trusted and therefore are outside the chain of trust. It is therefore necessary to invalidate all Host Platform SRTM PCRs as described in Section 3.3.2 Error Conditions.

Because the measurement of the early Platform Firmware is typically done within a resource-constrained environment (for example, the SRTM, likely the PEI Code) the event log corresponding to the Extend event may not be created at the time the TPM2_PCR_Extend is performed. It is acceptable to have the Platform Firmware generate the corresponding event log entry, reconstructing it from information (for example, the value that was extended) passed to it from PEI or earlier code. If this procedure is used, the Platform Firmware must be sure the entries within the event log are properly sequenced (for example, represent the same order as the sequence of TPM2_PCR_Extend operations).

Measurement of Non-Host Platforms

If a Non-Host Platform cannot be reliably detected by the Platform Firmware and measured into PCR[0], the existence of that Non-Host Platform should be indicated in the Host Platform Certificate. If a Non-Host Platform can be modified by an entity other than the Platform Firmware, it is measured in PCR[2].

Usage of the Event Type EV_POST_CODE

EV_POST_CODE is deprecated and no longer recommended as of PFP 1.06. Verifiers may find it in older implementations.

The intent of the event type EV_POST_CODE was to record measurements of components of the Host Platform's transitive trust chain. The imperative was to avoid omitting any portion of the transitive trust chain. The event was used as a catch-all for trust chain components provided by the platform manufacturer, even if the components are not technically "POST code" for the manufacturer's specific implementation.

Recommended values for the event field for the event type EV_POST_CODE of "POST CODE", "SMM CODE", "ACPI DATA", "BIS CODE", and "Embedded UEFI Driver" were defined to promote consistency across

implementations by different platform manufacturers. Because they were recommendations, other values may be observed.

The Event Data field for EV_POST_CODE in PFP 1.05 and earlier versions could be constructed with an EV_EFI_PLATFORM_FIRMWARE_BLOB structure (see Section 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition).

Usage of the Event Type EV_POST_CODE2

EV_POST_CODE2 has been introduced in PFP 1.06 to allow usage of the UEFI_PLATFORM_FIRMWARE_BLOB2 structure that provides a sized description field to hold the required strings defined below and in Table 13 EV_POST_CODE2 Strings.

The value placed in the UEFI_PLATFORM_FIRMWARE_BLOB2 blobDescription field is an 8-bit ASCII string without a NUL terminator. See Table 13 EV_POST_CODE2 Strings. The EV_EFI_PLATFORM_FIRMWARE_BLOB structure contains only an address and a length, resulting in difficulty for verifiers in understanding what code was measured. The structure was replaced in PFP 1.06 with EV_EFI_PLATFORM_FIRMWARE_BLOB2 structure.

Embedded UEFI Drivers and PCR[0], PCR[2] and PCR[3]

Manufacturer Controlled UEFI Drivers may be Drivers for devices that the platform manufacturer physically soldered to the motherboard or Drivers whose code is embedded within a firmware image. Embedded Drivers are under the control of the platform manufacturer and should not be intended for modification by the platform owner. Their code is measured in PCR[0] using the event EV_POST_CODE because their measurement may be combined with other firmware measurements.

In contrast, Non-Manufacturer Controlled Embedded Drivers, or UEFI Drivers, or Applications associated with devices in adapter slots may be added, removed, or updated by a platform owner and their code is measured in PCR[2].

Some UEFI Applications may use paging or other techniques to load and execute code that was hidden from the Platform Firmware when measuring the visible portion of the Application. It is the responsibility of the UEFI Application to measure this code prior to executing any portion of that hidden Option ROM code in PCR[2].

Hidden UEFI application code measurements are always placed in PCR[2]. UEFI Application configuration measurements are always placed in PCR[3]. Recording Manufacturer and Non-Manufacturer controlled hidden UEFI Driver and Application code consistently in PCR[2] and PCR[3], respectively, removes the need for a UEFI Application to know how it was packaged in a system. (For example, as a Manufacturer Controlled Embedded Driver versus as an Application in an adapter slot.)

Design Consideration and Distinctions Between PCR[0], PCR[2], and PCR[4]

PCR[0] typically represents a consistent view of the Host Platform between boot cycles. This allows Attestation and Sealed Storage policies to be defined using the less changeable platform manufacturer-provided components of the transitive trust chain. To allow correlation with RIMs, this specification requires measurement of the S-CRTM version. The S-CRTM version may be represented as either a NUL terminated, 16-bit WCHAR string or a GUID. This PCR contains the measurement of code of components provided by the Host Platform manufacturer. Therefore, the verifier or the entity performing the seal operation can choose to seal to only those components provided and updated by the Host Platform's manufacturer. This is also the reason embedded Driver binaries are measured into PCR[0] as well, thus providing this same consistent view of the platform regardless of user-selected options.

PCR[2] is intended to represent a more “user” configurable environment where the user has the ability to alter the set of installed components that are measured into PCR[2]. This is typically done by adding adapter cards, etc., into “user” accessible PCI or other slots.

PCR[4] is intended to represent the entity that manages the transition between the pre-OS and the OS-Present state of the platform. This PCR, along with PCR[5], identifies the initial OS loader.

In general, measure the following types of code into PCR[0] (as shown in Figure 6 PCR Mapping of UEFI Components):

- Platform Firmware from system board ROM that either contains or measures the UEFI Boot Services and UEFI Run Time Services that are loaded from firmware
- Embedded UEFI Drivers wholly contained within the Platform Firmware on the system board ROM
- EFI Boot Services in the UEFI System Table created in memory by Platform Firmware
- EFI Runtime Services in the UEFI System Table

A UEFI platform measures firmware integrated in the system board into PCR[0]. These are code modules known to be distributed by the Host Platform manufacturer. For a UEFI platform, this includes but is not limited to measuring:

- Version of the base firmware that either contains or measures the UEFI Boot Services (shown as the Platform Init code module of the UEFI Boot Manager in Figure 4 UEFI Architecture)

End of informative comment

Entities that MUST be logged in the Event Log if the TPM is enabled:

1. The event type EV_NO_ACTION Specification ID Event. See Section 10.4.5.1 Specification ID Version Event. Note: This event is not extended. This SHALL be the first event in the Event Log.
2. The event type EV_NO_ACTION Startup Locality Event, to record the locality from which the TPM2_Startup command was sent, if the Locality sending the TPM2_Startup command is Locality 3, or an H-CRTM event occurred. This event SHALL be recorded prior to any other extended event to PCR 0. See Section 10.4.5.3 Startup Locality Event.
3. The VendorID and ReferenceManifestGUID of the Platform Firmware, which either contains or measures the UEFI Boot Services and UEFI Run Time Services using EV_NO_ACTION event ReferenceManifestEvent. This event is only required if the platform supports NIST SP800-155. This event is not extended. See Section 10.4.5.2 Firmware Integrity Measurement Reference Manifest Event.

Entities that MAY be logged in the Event Log if the TPM is enabled:

1. If the H-CRTM measures the hashes or raw data of components, the platform firmware SHOULD use EV_NO_ACTION TCG_HCRTMComponentEvent to log the hashes or raw data of the components. If logged, the number and order of TCG_HCRTMComponentEvent events SHALL match the number and order of H-CRTM measurements. All TCG_HCRTMComponentEvent events SHALL immediately follow the EV_EFI_HCRTM_EVENT in the event log. See Section 10.4.5.5 H-CRTM Component Measurement Data for more information.

Entities that MUST be measured if the TPM is enabled:

1. If an H-CRTM event occurred, the H-CRTM event SHALL be measured using the event type EV_EFI_HCRTM_EVENT. The event data SHALL be the 8-bit non-NUL terminated ASCII string “HCRTM”. See Section 10.4.1 Event Types.
2. The SRTM’s version identifier, using the event type EV_S_CRTM_VERSION. See Section 10.4.1 Event Types.

3. A variety of entities listed below MUST be measured using the event type EV_POST_CODE2. This information MAY be measured as a single combined event or MAY be measured as multiple separate events. See Section 10.4.1 Event Types. The event field SHALL contain a UEFI_PLATFORM_FIRMWARE_BLOB2 Structure as defined in Section 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition. If a combined event is measured, the BlobDescription field SHALL be “POST CODE”. If separate events are measured, the BlobDescription field SHALL be the string specified below for the entity measured. The entities measured include but are not limited to the following:
    a. All Host Platform Firmware physically bound to the PC Motherboard that is executed by the Host Platform’s CPU(s) and is part of the Host Platform’s transitive trust chain. If the platform supports entering S2 or S3, this includes the S2 and/or S3 resume code.
        i. POST code (SHALL be the event field string of “POST CODE” in all capital letters).
        ii. Embedded SMM code and the code that sets it up (SHALL be the event field string of “SMM CODE” in all caps).
    b. BIS code (excluding the BIS Certificate) (SHALL be the event field string of “BIS CODE” in all caps).
    c. ACPI flash data prior to any modifications. This requirement may be met by either measuring the compressed image in flash or by measuring the in-memory expansion before fix-ups; however, the measurements MUST be made the same way across every boot cycle. (SHALL be the event field string of “ACPI DATA” in all caps.)
    d. Manufacturer Controlled Embedded UEFI modules/drivers as a binary image whose release and update is controlled by the Host Platform Manufacturer (SHALL be the event field string of “Embedded UEFI Driver”).
4. EFI drivers embedded in system ROM by the platform manufacturer using event type EV_EFI_RUNTIME_SERVICES_DRIVER.
5. The Platform Firmware that either contains or measures the UEFI Boot services and UEFI Run Time Services using event type EV_EFI_PLATFORM_FIRMWARE_BLOB2, see Section 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition.
6. Platform Firmware MUST attempt to detect and measure the presence of any Non-Host Platform. If Platform Firmware detects the presence of a Non-Host Platform, it MUST measure relevant information about its presence such as type, version, etc., using the event type EV_NONHOST_INFO. See Section 10.4.1 Event Types.
7. Per Section 3.3.2 Error Conditions, if an error occurs, the digest of the value 00000001h MUST be extended in PCR[0-7] and an EV_SEPARATOR event SHOULD be recorded in the event log for each PCR. See Section 10.4.1 Event Types.
8. Per Section 8.2.4 Measuring OS Boot Events, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call. See Section 10.4.1 Event Types.

Entities that MAY be measured if the TPM is enabled:

1. The SRTM MAY measure itself or MAY be measured by an H-CRTM. If the SRTM is measured by an H-CRTM sequence, the EV_EFI_HCRTM event type SHALL be used. Otherwise, the EV_S_CRTM_CONTENTS SHALL be used. See Method for Measurement subsection below and Section 10.4.1 Event Types.

Entities that SHOULD be measured if the TPM is enabled:

1. Components within Non-Host Platforms (e.g., firmware not intended to be executed by Host Platform’s CPU) that can only be updated by Platform Firmware and are not part of the Host Platform’s transitive trust chain

but may affect the trust of the Host Platform or system. If measured, this event MUST be recorded using the event type EV_NONHOST_CODE. See Section 10.4.1 Event Types.

2. Platform Firmware SHOULD measure the firmware of embedded components that support the SPDM "GET_MEASUREMENTS" functionality using event type EV_EFI_SPDM_FIRMWARE_BLOB. See Section 10.2.7 DEVICE_SECURITY_EVENT_DATA Structure and Section 10.4.1 Event Types. **Note:** This measurement may be made mandatory in future revisions of this specification.

Entities that MUST be measured if the TPM is disabled:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and the matching EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call.

TPM device is hidden:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] prior to the first invocation of the first Ready to Boot call. Platform Firmware SHALL NOT place an event in the event log.

Method for measurement:

The SRTM MUST record and log these measurements as follows:

1. Log the EV_NO_ACTION Specification Id Event
2. If an H-CRTM is supported:
    a. If an H-CRTM sequence occurred:
        1. Log the EV_NO_ACTION Startup Locality Event with Locality 4.
        2. Log the hash of the measured H-CRTM event using an EV_EFI_HCRTM event.
        3. Optionally log the EV_NO_ACTION TCG_HCRTMComponentEvent.
    b. If an H-CRTM sequence did not occur and the Startup Locality is 3, log the EV_NO_ACTION Startup Locality Event with Locality 3
    c. If an H-CRTM measured the SRTM version, log the SRTM version identifier measurement using an EV_S_CRTM_VERSION event.
    d. If an H-CRTM measured the SRTM contents, log the SRTM contents measurement(s) using an EV_S_CRTM_CONTENTS event.
    e. Platform Firmware may need to reconstruct events that could not be recorded in the event log due to the unavailability of memory. If it does so, it places this information into the event log and MUST NOT extend PCR[0] again with this reconstructed information, e.g. the Specification Version event, the Startup Locality event and the H-CRTM event.
3. If an H-CRTM is not supported:
    a. Measure and log the SRTM's version identifier using an EV_S_CRTM_VERSION.
    b. Measure and log the code to which the SCRTM is transferring control.
4. Any remaining measurements MAY be performed in any order, except for the EV_SEPARATOR prior to Ready to Boot invocation, which MUST be last. While these events may be measured in any order, the selected order MUST be maintained across boot cycles (resume from hibernate or cold boot).

3.3.4.2 PCR[1] – Host Platform Configuration

Start of informative comment

In general, the platform firmware measures into PCR[1] the configuration data that is associated with the code that measured into PCR[0].

For performance reasons, some implementations may combine CPU microcode updates with a Firmware POST code image. In this situation, it is acceptable to record CPU microcode updates in PCR[0] as part of an EV_POST_CODE event.

Because platforms may contain a variety of hardware that may or may not be security-relevant for a platform owner's use case, platform manufacturers may allow a platform owner to configure which optional PCR[1] measurements are recorded during the platform boot process. As a result, this section has a large number of entities that may be optionally measured. The platform manufacturer may provide a setup utility that allows the platform owner to choose measurements of interest to record during the boot process. The setup utility may allow a platform owner to select entities whose measurements are optional in this specification to be measured "a la carte" or "all or nothing." Which measurements are currently on or off are measured using the EV_PLATFORM_CONFIG_FLAGS event in a vendor-specific format. The EV_PLATFORM_CONFIG_FLAGS event is intended to be used to only describe events whose measurements in PCR[1] can be toggled on or off; the event data does not contain information about measurements not able to be configured to be measured in PCR[1]. If the platform manufacturer does not permit any configuration of measurements for optional PCR[1] measurements, it does not need to record the EV_PLATFORM_CONFIG_FLAGS event.

An example event data field for EV_PLATFORM_CONFIG_FLAGS for a platform that allows toggling of measurements for only the SMBIOS, BIS Certificate, and ESCD could be the ASCII string "SMBIOS=0;BIS=1;ESCD=0" when the platform is configured to record BIS Certificate information but not the SMBIOS nor the ESCD. If a vendor-specific setup tool was used to also measure ESCD, the event data for EV_PLATFORM_CONFIG_FLAGS would then contain the ASCII string "SMBIOS=0;BIS=1; ESCD=1".

Information about which optional entities are measured by the current boot process is represented in the required "Host Platform Configuration" measurement. Toggling which optional components are measured will always change the value for PCR[1] because the measurement for the EV_PLATFORM_CONFIG_FLAGS will reflect the change.

The Boot Integrity Services (BIS) Certificate may contain information that is privacy-sensitive; thus, recording a measurement of the BIS Certificate is optional. The BIS Certificate may be used by a different boot mechanism on the platform so it is measured in PCR[1] instead of being associated with initial program loader data in PCR[5]. The BIS Certificate may be maintained by Platform Firmware and built into an SMBIOS structure at boot time. Because other SMBIOS structures are measured in PCR[1], the BIS Certificate is, too.

SMBIOS structures are defined in the external SMBIOS specification. Some example SMBIOS structure values are how many memory slots exist on the platform and how many of these slots are currently populated with memory. Many SMBIOS structures exist, but the platform manufacturer should only record security-relevant SMBIOS structures. An example is the system-wide hardware security setting.

CMOS and NVRAM data measured into PCR[1] is placed in the event data field. The expected size of the data is very small. Any data that is security-sensitive, contains dynamic boot data or is dynamic (like the real-time clock) should be omitted. The NVRAM data is the host platform NVRAM data, not the TPM NVRAM.

EV_EFI_VARIABLE_DRIVER_CONFIG is used by OEMs to measure their Setup Variables. To verify this measurement, an IT administrator configures the system settings in setup, takes a snapshot of the EV_EFI_VARIABLE_DRIVER_CONFIG event(s) in PCR 1 in the event log and uses it as the "known good" baseline.

Note: EFI Variables are identified by both GUID and Variable Name. It is permissible in the EFI specification to have different variable names associated with a single GUID.

For example, for a UEFI server platform, this includes but is not limited to

- SAL system table

- SMBIOS Tables

PCR[1] also contains Boot Policy measurements. In order to record the alternate boot behaviors of a UEFI system, the UEFI Platform Firmware will measure the UEFI Boot#### variables and the BootOrder Variable into PCR[1]. These boot variables are just device paths. A description of the device paths and variables can be found in the UEFI Specification.

End of informative comment

Entities that MUST be measured if the TPM is enabled:

The following entities MUST always be measured. The platform manufacturer MUST NOT provide firmware configuration options that permit disabling the following measurements:

1. If Platform Firmware loads a CPU microcode update, it MUST be measured, using event type EV_CPU_MICROCODE. See Section 10.4.1 Event Types. Exception: Alternatively, CPU microcode updates MAY be measured in PCR[0] as part of a EV_POST_CODE2 event.
2. EFI Boot#### and UEFI BootOrder variables MUST be measured. These variables SHALL be measured using event type EV_EFI_VARIABLE_BOOT2. See Section 10.2.6 Measuring UEFI Variables.
3. If the Host Platform supports selection of optional PCR[1] measurements, it MUST measure which PCR[1] measurements are currently on or off using the event type EV_PLATFORM_CONFIG_FLAGS. The event data MUST not vary across boot cycles if the set of potential PCR[1] measurements measured does not vary. See the informative text for this section and Section 10.4.1 Event Types.
4. If the system supports UEFI 2.5 or later and DeployedMode is enabled, the following additional variables MUST be measured into PCR[1]:
   a. The DeployedMode variable value. The Event Type SHALL be EV_EFI_VARIABLE_DRIVER_CONFIG and the Event value shall be the value of the UEFI_VARIABLE data structure.
   b. The AuditMode variable value. The Event Type SHALL be EV_EFI_VARIABLE_DRIVER_CONFIG and the Event value shall be the value of the UEFI_VARIABLE data structure.
   c. If the platform supports changing either of these UEFI policy variables after they are initially measured in PCR[1],before ExitBootServices() has completed and the platform does not support UEFI 2.5 or later, the platform MUST be restarted. If the platform supports UEFI 2.5 or later and supports changing any of the UEFI Secure Boot policy variables after they are initially measured in PCR[1] and before ExitBootServices() has completed, the platform MAY be restarted OR the variables MUST be remeasured into PCR[1].
5. SMBIOS structures that contain static configuration information (e.g. Platform Manufacturer Enterprise Number assigned by IANA, platform model number, Vendor and Device IDs for each SMBIOS table) that is relevant to the security of the platform MUST be measured using the event type EV_EFI_HANDOFF_TABLES2 as described in Section 9.4.1 (Event Types).
6. Prior to entering a Platform Firmware Setup Utility, if the Host Platform does not unconditionally perform a Host Platform Reset upon exiting a Pre-OS Setup Utility, the platform MUST measure the EV_ACTION event “Entering ROM Based Setup”. See Section 10.4.3 EV_ACTION Event Types. (Note: This is only for Platform Firmware Setup Utilities as opposed to entering an Option ROM-Based Setup Utility that is recorded in PCR[3].)
7. Per Section 3.3.2 Error Conditions, if an error occurs, the digest of the value 00000001h MUST be extended in PCR[0-7] and an EV_SEPARATOR event SHOULD be recorded in the event log for each PCR. See Section 10.4.1 Event Types.
8. Per Section 8.2.4 Measuring OS Boot Events, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call. See Section 10.4.1 Event Types.

Entities that SHOULD be measured if the TPM is enabled:

1. EFI Setup Variables, which contain security-relevant configuration data, including Setup Admin authentication settings and Setup User authentication settings, that are not measured elsewhere SHOULD be measured. If measured, the event MUST be recorded in the event log using EV_PLATFORM_CONFIG_FLAGS or EV_EFI_VARIABLE_DRIVER_CONFIG. See Section 10.4.1 Event Types. **Note:** Verifiers of event logs should be aware that EFI variables are identified by both the GUID and the Variable Name. The EFI specification allows different Variable Names to be associated with 1 GUID.
2. Security-relevant CMOS data and security-relevant configuration data stored in Platform NVRAM and in effect when the platform boots SHOULD be measured. If measured, these events MUST use the event type EV_EFI_HANDOFF_TABLES2 as described in Section 10.4.1 Event Types. Sensitive data like passwords MUST be omitted from the NVRAM data because it could be placed unencrypted in the TCG event log.
3. Table of devices SHOULD be measured using event type EV_TABLE_OF_DEVICES described in Section 10.4.1 Event Types.

   This event allows Platform Firmware to measure the list of devices attached to the Host Platform. Examples of this include PCI devices, onboard video adapters, etc. Because of the wide variance of Host Platform architectures, the actual format of the data providing this information is left to the Host Platform Manufacturer. It is left to the challenger to discover the format of this data. It could, for example, reference the Host Platform Certificate, and then contact the Host Platform Manufacturer to obtain this information. Platform Firmware MAY create multiple entries of this event or MAY choose to encapsulate all the data into a single entry.

4. Non-Host Platform configuration information. If the system contains a Non-Host Platform that can only be updated by Platform Firmware, Platform Firmware SHOULD measure the configuration and MUST use the event type EV_NONHOST_CONFIG to record any security-relevant configuration information or data. See Section 10.4.1 Event Types.
5. Platform Firmware SHOULD measure the firmware configuration, if present, of embedded components that support the SPDM “GET_MEASUREMENTS” functionality using event type EV_EFI_SPDM_FIRMWARE_CONFIG. See Sections 10.2.7 DEVICE_SECURITY_EVENT_DATA Structure and 10.4.1 Event Types. **Note:** This measurement may be made mandatory in future revisions of this specification.

Entities that MAY be measured if the TPM is enabled:

The following entities MAY be measured. Even if measurement of these is provided by Platform Firmware, Platform Firmware MAY allow the owner to disable individual measurements or all of these measurements. (For example, disabling of individual measurements could be done using BIOS configuration settings.)

1. ESCD, using event type EV_EFI_HANDOFF_TABLES2 as described in Section 10.4.1 Event Types.
2. Other tables MAY be measured using event type EV_EFI_HANDOFF_TABLES2.
3. EFI Variables that impact system configuration MAY be measured using event type EV_EFI_VARIABLE_DRIVER_CONFIG.
4. If Platform Firmware has detected the platform case was opened or an analogous action occurred, the platform MAY record the EV_ACTION event “Chassis Intrusion”. The event MAY be persistently recorded across platform boots until Platform Firmware administrator clears the notification. See Section 10.4.3 EV_ACTION Event Types.

Entities that MUST be measured if the TPM is disabled:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call.

TPM device is hidden:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] prior to the first invocation of the first Ready to Boot call. Platform Firmware SHALL NOT place an event in the event log.

Entities that MUST NOT be measured as part of the above measurements:

1. Values and registers that are automatically updated (e.g., clocks).
2. System-unique information such as asset, serial numbers, etc, as they would prevent sealing to PCR[1] with a common configuration in a fleet of devices.
3. The BIS Certificate if it contains privacy-sensitive information.

Method for measurement:

Platform Firmware MUST perform these measurements as follows:

1. The entities specified in this PCR MAY be measured in any order deemed appropriate by the implementer, except for the EV_SEPARATOR prior to Ready to Boot invocation, which MUST be last. While these events may be measured in any order, the selected order MUST be maintained across boot cycles.
2. Where possible, these measurements SHOULD occur prior to measuring Option ROMs.

3.3.4.3 PCR[2]-UEFI Drivers and UEFI Applications

Start of informative comment

EFI Drivers and Applications contained on Host Platform adapters are measured by Platform Firmware to PCR[2].

For a UEFI platform, this includes but is not limited to

- EFI Drivers loaded from adapter cards
- EFI Applications loaded from adapter cards (e.g., setup utility for RAID controller)
- EFI Applications loaded from external storage media (system or peripheral Flash for example) via embedded option ROM contained within the UEFI firmware on the system board. An example of this would be an embedded UEFI driver that loads more code from an external location, such as a hidden partition on a drive. An example would be an embedded driver that pages the external code into memory, and thus the external code is not visible to LoadImage. If the embedded driver does not use the LoadImage service, it should perform all the actions that the LoadImage service would do.
- Drivers loaded from HBA's/disks and external storage media (as shown in Figure 6 PCR Mapping of UEFI Components).
- Non-Manufacturer Controlled Embedded UEFI Drivers and Applications that are physically contained on the PC Motherboard (as opposed to an add-in card), but the release and control of any update is not controlled by the Platform Manufacturer. These are measured in PCR[2].

Visible and Hidden Application Code

The PC Client Implementation provided a mechanism for adapters with Option ROMs to have hidden code that they were responsible for measuring. This specification does not support this behavior for UEFI adapters. All code should be measured by the LoadImage service.

Measuring Non-Host Platforms

If the platform contains a Non-Host Platform which can be updated by an entity other than the Platform Firmware, it is measured in PCR[2].

Interpreting UEFI Driver Measurements

The measuring agent is always the Platform Firmware (that is, code measured into PCR[0], even in the case where a measuring agent not measured into PCR[0] "requests" the program load, such as a UEFI Driver trying to load another UEFI Driver or Application). The UEFI Driver will use the UEFI LoadImage service. Since measurement occurs between shadowing of PE sections and the application of relocations, only the LoadImage service can call the TCG UEFI Protocol function TCG_HashLogExtendEvent at the appropriate time.

The UEFI Driver measurements in this specification do not correlate measurements of hidden UEFI Driver code with the UEFI Driver that measured the hidden code. An evaluator of the measurement log may need to have behavioral knowledge of the Adapters on the Host Platform to evaluate the measurements in the log and correlate which measurement is associated with a given UEFI Driver.

In general, the system measures into PCR[2] firmware code modules added to the Host Platform by a Value Added Retailer (VAR), the platform owner, or some unknown entity. The source of these firmware code modules measured into PCR[2] is not the Host Platform manufacturer.

Note: Components measured into PCR[2] and PCR[3] may be updatable during OS runtime. As such, measurements of these components made during the platform boot process, e.g SPDM measurements, may not reflect the firmware running after ExitBootServices(). If this occurs the measurements in PCR[2] and PCR[3] will be stale until the next boot. If there is a policy that depends on PCR[2] and PCR[3] values, the policy needs to account for this behavior.

End of informative comment

Entities that MUST be measured if the TPM is enabled:

1. EFI Boot Services drivers from add-in adapters or loaded by the driver in an add-in adapter MUST be measured using event type EV_EFI_BOOT_SERVICES_DRIVER. See Section 10.4.1 Event Types.
2. EFI Run Time drivers from add-in adapter or loaded by add-in adapter MUST be measured using event type EV_EFI_RUNTIME_SERVICES_DRIVER. See Section 10.4.1 Event Types.
3. Components within Non-Host Platforms (e.g., firmware not intended to be executed by the Host Platform's CPU) that can be updated by entities other than Platform Firmware and are not part of the Host Platform's transitive trust chain but may affect the trust of the Host Platform or system. If measured, this event MUST be recorded using the event type EV_NON_HOST_CODE. See Section 10.4.1 Event Types.
4. Per Section 3.3.2 Error Conditions, if an error occurs, the digest of the value 00000001h MUST be extended in PCR[0-7] and an EV_SEPARATOR event SHOULD be recorded in the event log for each PCR. See Section 10.4.1 Event Types.
5. Per Section 8.2.4 Measuring OS Boot Events, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call. See Section 10.4.1 Event Types.

Entities that SHOULD be measured if the TPM is enabled:

1. Platform Firmware SHOULD measure the firmware of add-in devices that support the SPDM "GET_MEASUREMENTS" functionality using event type EV_EFI_SPDM_FIRMWARE_BLOB. If Platform Firmware measures the SPDM firmware measurement and there are multiple blocks, Platform Firmware SHALL measure the blocks in the same order across boot cycles (resume from hibernate or cold boot). See Sections 10.2.7 DEVICE_SECURITY_EVENT_DATA Structure and Section 10.4.1 Event Types. **Note:** This measurement may be made mandatory in future revisions of this specification.

Entities that MUST be measured if the TPM is disabled:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call.

TPM device is hidden:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] prior to the first invocation of the first Ready to Boot call. Platform Firmware SHALL NOT place an event in the event log.

Method for measurement:

1. Platform Firmware MUST measure the event EV_EFI_RUNTIME_SERVICES_DRIVER or EV_EFI_BOOT_SERVICES_DRIVER for each UEFI Driver, non-bootable applications, Non-Manufacturer Controlled Embedded UEFI Driver, or Non-Manufacturer Controlled Embedded non-bootable applications prior to initializing the module.
2. Platform Firmware MUST measure the event type EV_POST_CODE2 for each Manufacturer Controlled Embedded UEFI Drivers (in PCR[0], see Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers prior to initializing the Manufacturer Controlled Embedded UEFI Driver.
3. The visible portion of UEFI Drivers and Applications measured in PCR[2] by Platform Firmware MAY be measured in any order deemed appropriate by the implementer. However, the order MUST be consistent across boot cycles if the physical location of the adapters containing the Driver or Application is unchanged. Platform Firmware MAY measure all UEFI Drivers and Applications and then initialize them all or MAY measure one and then initialize it before initializing the second, or MAY use some other variation per the discretion of the manufacturer.
   Note: Changing the adapter slot in which a UEFI Driver or Application is inserted may change the order of measurements.
4. Repeat until all UEFI Drivers and Applications are measured and executed.
5. When initialized, the UEFI Drivers and Applications MUST measure their “hidden” code prior to executing it.
6. The EV_SEPARATOR event prior to Ready to Boot invocation MUST be measured last.

3.3.4.4 PCR[3] – EFI Driver/Application Configuration

Start of informative comment

This section contains the PCR[3] measurement requirements for a UEFI platform.

In general, entities measure into PCR[3] data that is associated with code modules that are measured into PCR[2]. For example, on a UEFI platform, this includes but is not limited to UEFI variables defined and managed by drivers. See section 10.2.4 Measuring Industry Standard Tables and Data Structures.

An example of information measured into PCR[3] is a SCSI controller’s configuration of its hard disks, e.g., RAID type, drive assignments, etc.

If a UEFI Application has a Pre-OS Setup Utility, it needs to record the event type EV_ACTION event “Entering ROM Based Setup” in PCR[3] prior to entering the utility.

If a UEFI Application Pre-OS Setup Utility permits the user to change configuration data that is eventually recorded in PCR[3] and the setup utility does not depend on the configuration data recorded in PCR[3], the UEFI Application may be designed to permit the configuration changes to occur before the UEFI Application records the configuration data in PCR[3]. This may allow the Pre-OS Setup Utility to avoid needing to perform a Host Platform Reset if the Option ROM configuration is changed.

Note: Components measured into PCR[2] and PCR[3] may be updatable during OS runtime. As such, measurements of these components made during the platform boot process, e.g SPDM measurements, may not reflect the firmware running after ExitBootServices(). If this occurs the measurements in PCR[2] and PCR[3] will be stale until the next boot. If there is a policy that depends on PCR[2] and PCR[3] values, the policy needs to account for this behavior.

End of informative comment

Any pre-OS component that modifies the UEFI Driver or Application configuration MUST measure the new configuration into PCR[3] or cause a Host Platform Reset.

Entities that MUST be measured if the TPM is enabled:

1. If any configuration data exists in a UEFI Variable for a UEFI Driver or Application or the adapter that hosts the UEFI Driver, before that configuration data is used, it MUST measure the configuration data specific to the device using event EV_EFI_VARIABLE_DRIVER_CONFIG. See Section 10.4.1 Event Types.
2. Prior to entering an UEFI Application-Based Setup Utility, if the Host Platform does not unconditionally perform a Host Platform Reset upon exiting a Pre-OS ROM-Based Setup Utility, the platform MUST measure the EV_ACTION event “Entering ROM Based Setup”. See Section 10.4.3 EV_ACTION Event Types. (Note: This is only for UEFI Application-Based Setup Utilities versus entering a ROM-Based Setup Utility which is recorded in PCR[1].)
3. Non-Host Platform configuration information. If the system contains a Non-Host Platform which can be updated by entities other than Platform Firmware, Platform Firmware SHOULD measure the configuration and MUST use the event type EV_NONHOST_CONFIG to record any security-relevant configuration information or data. See Section 10.4.1 Event Types.
4. Per Section 3.3.2 Error Conditions, if an error occurs, the digest of the value 00000001h MUST be extended in PCR[0-7] and an EV_SEPARATOR event SHOULD be recorded in the event log for each PCR. See Section 10.4.1 Event Types.
5. Per Section 8.2.4 Measuring OS Boot Events, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call. See Section 10.4.1 Event Types.

Entities that SHOULD be measured if the TPM is enabled:

1. Platform Firmware SHOULD measure the firmware configuration, if present, of add-in devices that support the SPDM “GET_MEASUREMENTS” functionality using event type EV_EFI_SPDM_FIRMWARE_CONFIG. See Sections 10.2.7 DEVICE_SECURITY_EVENT_DATA Structure and 10.4.1 Event Types. **Note:** This measurement may be made mandatory in future revisions of this specification.

Entities that MUST be measured if the TPM is disabled:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call.

TPM device is hidden:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] prior to the first invocation of the first Ready to Boot call. Platform Firmware SHALL NOT place an event in the event log.

Entities that MUST NOT be measured as part of the above measurements:

1. Values and registers that are automatically updated (e.g., clocks).
2. System-unique information such as asset, serial numbers, etc., as they would prevent sealing to PCR[3] with a common configuration in a fleet of devices.

Method for measurement:

The UEFI Application MUST perform these measurements as follows:

1. Measures the event EV_EFI_VARIABLE_DRIVER_CONFIG, see Section 10.4.1 Event Types.
2. The remaining measurements MAY be performed in any order, except for the EV_SEPARATOR prior to Ready to Boot invocation, which MUST be last.

3.3.4.5 PCR[4] – Boot Manager Code and Boot Attempts

Start of informative comment

Overview

Systems often support the ability to boot from multiple boot devices in priority order. PCR[4] records the process of attempting to boot different hardware paths like from a DVD or a hard drive, what boot devices are attempted, and the Boot Manager Code that is loaded and executed from the device. If a boot from one device fails, measurements in PCR[4] record the attempt to boot the next device or boot path. If Boot Manager Code returns control back to Platform Firmware, each subsequent execution of Boot Manager Code is separately measured.

If boot fails for one device, it often returns execution control back to Platform Firmware so Platform Firmware may boot another device. Ultimately the boot process reflected in PCR[4] measurements may contain code from multiple boot devices. Platform Firmware records events in PCR[4] to demark different boot attempts. When interpreting measurements in PCR[4], verifiers should not disregard code that executed from failed boot attempts. Code that boots first has the ability to record events, and malicious code could record subsequent events, making it seem as if the malicious code failed to run.

If the boot device or the Boot Manager Code is not TCG-aware, it may have loaded additional unmeasured code. However, there is a record in PCR[4] showing entry to untrusted code.

The Boot Manager Code that BIOS decides to load and measure into PCR[4] may contain code that is part of the OS environment. Boot Manager should record code to which it transfers control into PCR[4] or PCRs allocated for the OS. Boot Manager Code should also record additional configuration information it uses into PCR[5] or PCRs allocated for the OS. While this specification does not place requirements on OS loader components or an OS, if the Boot Manager Code does not record measurements of later components, it will break the Static Root of Trust for Measurement.

Conditionally Measuring Which Device Attempts to Boot

Versions of this specification before version 1.21 required measuring which device attempted to Boot Manager Code even if the attempt did not execute any code not recorded previously in PCR[0] or PCR[2]. A drawback of this requirement was that inserting unbootable media into a boot device caused the boot measurements in PCR[4] to change even if code loaded from the media was never executed. Also, because the boot attempt event uniquely identified the device, replacement of a faulty device with a like device booting the exact same code would cause the PCR[4] measurement to change. A benefit of the requirement was that the log contained more information about which device booted when attempting to boot from a device that had an event defined in the specification for its device type.

For some hardware, it is possible for Platform Firmware to determine if a device is not bootable without executing code that had not been previously recorded in PCR[0] or PCR[2] during the boot attempt. An example is a DVD drive whose driver is measured in PCR[0]. The boot attempt may use a Platform Firmware driver to determine no DVD media is in the DVD drive and fail the boot attempt. An extended example is a DVD driver measured in PCR[0] or PCR[2] that prompts the user if they would like to boot from the DVD prior to loading code from DVD media; if the user does not press a confirmation key, the boot attempt fails, having run only code previously measured in PCR[0] or PCR[2]. Another extended example is a USB device with a driver already measured in PCR[0] or PCR[2] that determines the USB media does not contain boot code and aborts the boot attempt, only running previously measured code.

As of revision 1.21 of the TCG PC Client Implementation Specification for Conventional BIOS, not recording which device attempted to boot may be controlled via a BIOS configuration option or may be fixed by the Platform Manufacturer. If recording which device attempted to boot is disabled due to Platform Firmware configuration or design, Platform Firmware records the event type EV_OMIT_BOOT_DEVICE_EVENTS in PCR[4]; otherwise, the event is not recorded.

Boot Device Types

This section of the UEFI Platform Specification contains PCR usage requirements for the transition from the UEFI Boot Manager to the OS Loader, shown in Figure 5 UEFI Platform Boot Process, above. Note that the term ‘EFI OS Loader Load’ is a label for a behavior, not necessarily a label for a code module.

This section contains the PCR[4] measurement requirements for a UEFI platform.

In general, entities measure into PCR[4] code modules that could transition the platform from the pre-OS state to the OS-present state. Measure UEFI applications into PCR[4]; for a UEFI platform, UEFI applications include but are not limited to:

- Pre-OS diagnostics
- EFI OS Loader

The measurement values in PCR[4] must be consistent and repeatable across UEFI boots.

In the table below, ‘measuring entity’ means code in the CRTM or code measured into PCR[0] which provides image loading capability and invokes the measurement agent (the ‘platform code’).

The ‘measured object’ is a PE/COFF image; see Section 10.2.3 UEFI_IMAGE_LOAD_EVENT Structure, for a definition of the methodology for measuring PE/COFF images on a TCG UEFI platform.

The measuring agent is always the Platform Firmware (that is, code measured into PCR[0]). The measuring agent will typically measure the measurement object code as part of the UEFI LoadImage Service. Since measurement occurs between shadowing of PE sections and the application of relocations, only the UEFI LoadImage Service can call the TCG UEFI Protocol function TCG_HashLogExtendEvent at the appropriate time.

This model purposely precludes pre-OS agents from loading and invoking code outside the UEFI LoadImage Service. Only the PCR[4] application, such as the UEFI OS Loader, can use a different code load and measurement methodology, but the target PCR for this action will be in the range PCR[8] and above and outside the scope of this specification.

End of informative comment

Entities that MUST be measured if the TPM is enabled:

1. If the Platform Firmware is configured or designed to not record each attempt to boot a device, an EV_OMIT_BOOT_DEVICE_EVENTS event MUST be measured once. See Section 10.4.1 Event Types.

2. Per Section 8.2.4 Measuring OS Boot Events, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call. See Section 10.4.1 Event Types.
3. When Platform Firmware transfers control to the Boot Manager, firmware MUST record the EV_ACTION event “Returned Ready to Boot”. See Section 10.4.3 EV_ACTION Event Types.
4. For each boot attempt, if the EV_OMIT_BOOT_DEVICE_EVENTS event was not recorded, Platform Firmware MUST record the EV_EFI_ACTION event “Calling EFI Application from Boot Option” per Section 10.4.4 EV_EFI_ACTION Event Types when attempting to boot a device. See Section 8.2.4 Measuring OS Boot Events for specific details on how measurements are done for different boot devices and if applicable corresponding PCR[5] measurements.
5. Platform Firmware MUST record the EV_EFI_ACTION event “Booting to <Boot####> Option” to identify the specific Boot option to which the Platform Firmware passes control. The <Boot####> SHALL be the specific Boot option Variable Name. See Section 10.4.4 EV_EFI_ACTION Event Types. See Section 8.2.4 Measuring OS Boot Events for specific details on how measurements are done for different boot devices.
6. For the UEFI application code PE/COFF image described by the boot variable, Platform Firmware MUST record the EV_EFI_BOOT_SERVICES_APPLICATION into PCR[4]. See Section 8.2.4 Measuring OS Boot Events.
7. If a boot device returns control back to the Boot Manager, Platform Firmware MUST record the EV_EFI_ACTION event “Returning from EFI Application from Boot Option”. See Section 10.4.3 EV_ACTION Event Types.
8. Per Section 3.3.2 Error Conditions, if an error occurs, the digest of the value 00000001h MUST be extended in PCR[0-7] and an EV_SEPARATOR event SHOULD be recorded in the event log for each PCR. See Section 10.4.1 Event Types.

Entities that MAY be measured if the TPM is enabled:

1. Additional pre-OS environment code that is loaded by UEFI Applications using event EV_COMPACT_HASH. See Section 10.4.1 Event Types. Note this event type is deprecated for PCR 4.

Entities that MUST be measured if the TPM is disabled:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call.

Entities that MUST be measured if the TPM device is hidden:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] prior to the first invocation of the first Ready to Boot call. Platform Firmware SHALL NOT place an event in the event log.

Entities to exclude:

1. Portions of the Boot Manager pertaining to the specific configuration of the Host Platform. (e.g., disk geometry in the MBR).
2. For each boot attempt, if the EV_OMIT_BOOT_DEVICE_EVENTS event was recorded, Platform Firmware MUST NOT record the EV_EFI_ACTION event “Calling EFI Application from Boot Option” per Section 10.4.4 EV_EFI_ACTION Event Types when attempting to boot a device.

Method for measurement:

This section provides only a short overview of the actions required. See Section 8.2.4 Measuring OS Boot Events for specific details.

The measuring entity MUST measure all UEFI applications as defined in Section 3.3.3.1 Measuring PE/COFF Image Files into PCR[4]. Across boot cycles to the same boot device(s) connected in the same way, Platform Firmware measurements into PCR[4] MUST be measured in the same sequence every time.

Platform Firmware MUST perform these steps as follows:

1. Measure EV_OMIT_BOOT_DEVICE_EVENTS if Platform Firmware is designed or configured to not record which devices it attempts to boot.
2. Measure EV_EFI_ACTION “Calling EFI Application from Boot Option” and EV_EFI_ACTION “Booting to <Boot####> Option”. See Section 10.4.4 EV_EFI_ACTION Event Types.
3. Measure EV_SEPARATOR in PCR[0-7]. See Section 10.4.1 Event Types.
4. Measure the Boot Manager Code before transferring control to it. (Note: Some implementations may load Boot Manager Code but decide not to execute it. An example is Platform Firmware designed to read from a USB device to determine if it is bootable and conditionally run code from the device if it is. If the USB device is not bootable, Platform Firmware does not measure it because it never transfers control to it.)
5. Conditionally measure the Boot Manager configuration data in PCR[5] if appropriate for the class of device that is booted per Section 8.2.4 Measuring OS Boot Events.
6. Boot Manager Code SHOULD measure additional Boot Manager Code to which it transfers control into PCR[4] or into PCRs reserved for the OS.
7. Boot Manager Code SHOULD measure additional configuration data into PCR[5] or into PCRs reserved for the OS.
8. If control returns to Platform Firmware, measure the returning event as EV_EFI_ACTION event “Returning from EFI Application from Boot Option”. See Section 10.4.4 EV_EFI_ACTION Event Types.
9. For additional boot devices or paths, go to Step 5.

3.3.4.6 PCR[5] – Boot Manager Configuration and Data

Start of informative comment

The Boot Manager Code may have configuration or other data that is relevant to the trust properties of the Host Platform. For some specific situations documented in this specification, Platform Firmware will record the Boot Manager Data or configuration information. In other cases, Boot Manager Code or UEFI Application code itself will record its configuration or other data. In all cases, the configuration information is stable across multiple boot cycles if the boot proceeds in the same manner with the same trust properties. Transient information like time is not recorded in this PCR.

Information measured into this PCR by Platform Firmware is security-relevant data associated with booting the device. An example is data embedded within the Boot Manager Code such as the disk geometry within the GPT.

An example of Boot Manager Code recording configuration data is Boot Manager Code that allows the selection of alternate boot partitions. The partition selection information would be measured in this PCR by the Boot Manager Code.

Another example of UEFI Application recording configuration data is PXE code measuring partition data it loaded using the EV_EFI_ACTION event and encoding the data as an ASCII string.

This section contains the PCR[5] measurement requirements for a UEFI platform.

In general, entities measure into PCR[5] data associated with the code modules measured into PCR[4]. For a UEFI platform, this includes but is not limited to

- The GPT/Partition Table (as shown in Figure 6 PCR Mapping of UEFI Components)

Additional variables that may impact system behavior beyond the boot options that are measured into PCR[1] (the source of these additional variables may be the UEFI Specification or private variables) may be optionally measured by the UEFI boot application. These loader variables shall be measured into PCR[5]. The source of these additional variables may be the UEFI Specification or private variables. These variables can include, but are not limited to, language code, and so on.

End of informative comment

Entities that MUST be measured if the TPM is enabled:

1. All relevant Boot Manager configuration data, using the event EV_EFI_VARIABLE_DRIVER_CONFIG. See Section 10.4.1 Event Types.
2. Security-relevant data contained within the Boot Manager Code (e.g., disk geometry), using the event EV_EFI_GPT_EVENT. See Section 10.4.4 EV_EFI_ACTION Event Types for what to place in the event data field.
3. If applicable, when booting a UEFI Application or an adaptor that hosts an Application, it MUST measure other data, including comments, specific to the device using the event type EV_EFI_VARIABLE_DRIVER_CONFIG with an ASCII string “<vendor specific string>” value as described in Section 10.4.4 EV_EFI_ACTION Event Types as determined by the device manufacturer.
4. Platform Firmware MUST record the EV_EFI_ACTION event “Exit Boot Services Invocation”. See Section 10.4.4 EV_EFI_ACTION Event Types.
5. Per Section 8.2.4 Measuring OS Boot Events, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call. See Section 10.4.1 Event Types.
6. Per Section 3.3.2 Error Conditions, if an error occurs, the digest of the value 00000001h MUST be extended in PCR[0-7] and an EV_SEPARATOR event SHOULD be recorded in the event log for each PCR. See Section 10.4.1 Event Types.

Entities that MAY be measured if the TPM is enabled:

1. The GPT Table MAY be measured using the event type EV_EFI_GPT_EVENT.
2. EFI variables, either defined in the UEFI specification or private, that typically do not change from boot to boot and contain system configuration information MAY be measured using EV_EFI_VARIABLE_DRIVER_CONFIG.

Entities that MUST be measured if the TPM is disabled:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call.

Entities that MUST be measured if the TPM device is hidden:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] prior to the first invocation of the first Ready to Boot call. Platform Firmware SHALL NOT place an event in the event log.

Method for measurement:

1. The EV_SEPARATOR event prior to Ready to Boot invocation.
2. Platform Firmware measures the static data such as disk geometry.
3. The Boot Manager Code measures all relevant IPL configuration data per its defined events.

4. The sequence of events being measured MUST be maintained across boot cycles absent a change in the configuration of the platform.

3.3.4.7 PCR[6] – Host Platform Manufacturer Specific Measurements

Start of informative comment

This PCR is reserved for use by the Host Platform manufacturer. This allows for Host Platform Manufacturer-specific to use this PCR. The use of this PCR may not be common across different Host Platform manufacturers and even across different Host Platform models within the same Host Platform manufacturer.

It is anticipated the TCG event log used during Pre-OS environment may be copied and managed by TCG-capable operating systems. Additional entries made to the Pre-OS TCG event log after the OS is managing its own copy of the measurement log may be ignored by the OS. This specification does not define a mechanism for a platform manufacturer to add entries to an OS-managed copy of the TCG event log after the OS has started.

Operating systems have their own TPM driver. This specification does not define a mechanism for a platform manufacturer to send commands to the TPM after the firmware launches an operating system. A platform manufacturer may have difficulty extending PCR[6] without interfering with OS management of the TPM. In this case, a platform manufacturer would need to work with their OS vendor to implement a mechanism to record measurements in the OS-present event log.

Previously defined measurements for PCR[6] from the PC Client Implementation Specification for Conventional BIOS have been deprecated.

End of informative comment

1. The Host Platform Manufacturer MAY define the purpose of this PCR.
2. If the Platform Manufacturer extends PCR[6] during the Pre-OS environment, it MUST record a corresponding log entry.
3. If the Platform Manufacturer extends PCR[6] during the OS environment, it MUST NOT record a corresponding log entry in the Pre-OS TCG event log. However, it MAY record an event by collaborating with the OS to place an event in an event log managed by the OS.
4. The operating system and user applications SHOULD NOT use this PCR for sealing or attestation without knowing manufacturer-specific information about its usage.

Entities that MUST be measured if the TPM is enabled:

1. Per Section 8.2.4 Measuring OS Boot Events, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call. See Section 10.4.1 Event Types.
2. Per Section 3.3.2 Error Conditions, if an error occurs, the digest of the value 00000001h MUST be extended in PCR[0-7] and an EV_SEPARATOR event SHOULD be recorded in the event log for each PCR. See Section 10.4.1 Event Types.

Entities that MAY be measured if the TPM is enabled:

1. The Host Platform Manufacturer MAY measure platform specific events using the event type EV_COMPACT_HASH. See Section 10.4.6 Vendor Specific Events for more information.

Entities that MUST be measured if the TPM is disabled:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call.

Entities that MUST be measured if the TPM device is hidden:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] prior to the first invocation of the first Ready to Boot call. Platform Firmware SHALL NOT place an event in the event log.

Method for measurement:

1. The EV_SEPARATOR event prior to Ready to Boot invocation MUST be the last Platform Firmware initiated measurement in the PCR.

3.3.4.8 PCR[7] – Secure Boot Policy Measurements

Start of informative comment

This section contains the PCR[7] measurement requirements.

In addition, PCR[7] is used to record secure boot policy information as described below. See Chapter 27 of the UEFI Specification for more information on UEFI Secure Boot.

PCR[7] is used to reflect the UEFI 2.3.1 Secure Boot policy. This policy relies on the firmware authenticating all boot components launched prior to the UEFI environment and the UEFI platform initialization code (or earlier firmware code) invariantly recording the Secure Boot policy information into PCR[7].

End of informative comment

Secure Boot Variable Measurements and Event Types

Platform Firmware MUST measure the following values into PCR[7]:

1. The contents of the SecureBoot variable
2. The contents of the PK variable
3. The contents of the KEK variable
4. The contents of the UEFI_IMAGE_SECURITY_DATABASE_GUID /EFI_IMAGE_SECURITY_DATABASE variable (the DB)
5. The contents of the UEFI_IMAGE_SECURITY_DATABASE_GUID /EFI_IMAGE_SECURITY_DATABASE1 variable (the DBX)
6. The contents of the UEFI_IMAGE_SECURITY_DATABASE_GUID /EFI_IMAGE_SECURITY_DATABASE2 (the DBT) variable, if present and not empty
7. If the system supports UEFI 2.5 or later, the contents of the UEFI_IMAGE_SECURITY_DATABASE_GUID/EFI_IMAGE_SECURITY_DATABASE3 if present and not empty. (the DBR)
8. If the system supports UEFI 2.5 or later and DeployedMode is NOT enabled, the following additional variables MUST be measured into PCR[7]:
    a. The contents of the AuditMode variable
    b. The contents of the DeployedMode variable
9. In addition to the variables above, individual entries in the UEFI_IMAGE_SECURITY_DATABASE that are used to validate UEFI Drivers or UEFI Boot Applications in the boot path
10. If the system supports the UEFI device signature variable, the following additional variables:
    a. the contents of the EFI_DEVICE_SECURITY_DATABASE_GUID / EFI_DEVICE_SECURITY_DATABASE variable
    b. the individual entries in the UEFI Device Signature Variable are used to validate the SPDM device in the boot path.

The specific event types used to record these measurements are defined as follows:

1. For the contents of all UEFI variable value events, the eventType SHALL be EV_EFI_VARIABLE_DRIVER_CONFIG and the Event value SHALL be the value of the UEFI_VARIABLE_DATA structure (this structure SHALL be padded out to ensure the structure is byte-aligned).
    a. The measurement digest MUST be tagged Hash for each supported PCR bank of the event data which is the UEFI_VARIABLE_DATA structure.
    b. The UEFI_VARIABLE_DATA.UnicodeNameLength value is the number of CHAR16 characters (not the number of bytes).
    c. The UEFI_VARIABLE_DATA.UnicodeName contents MUST NOT include a NUL.
    d. If reading a UEFI variable returns UEFI_NOT_FOUND, platform firmware SHALL measure the absence of the variable. The UEFI_VARIABLE_DATA.VariableDataLength field MUST be set to zero and UEFI_VARIABLE_DATA.VariableData field will have a size of zero.
2. For entries in the UEFI_IMAGE_SECURITY_DATABASE, as defined in normative 9, the eventType SHALL be EV_EFI_VARIABLE_AUTHORITY.
    a. The event value SHALL be the value of the UEFI_VARIABLE_DATA structure.
    b. The UEFI_VARIABLE_DATA.VariableData value SHALL be the EFI_SIGNATURE_DATA value from the EFI_SIGNATURE_LIST that contained the authority that was used to validate the image.
    c. The UEFI_VARIABLE_DATA.VariableName SHALL be set to UEFI_IMAGE_SECURITY_DATABASE_GUID.
    d. The UEFI_VARIABLE_DATA.UnicodeName SHALL be set to the value of UEFI_IMAGE_SECURITY_DATABASE. The value MUST NOT include the NUL terminator.
3. Images that LoadImage fails to load due to (a) signature verification failure or (b) because the image does not comply with currently enforced UEFI 2.3.1 Secure Boot policy do not need to be measured in a PCR.
4. For the contents of device security variable value events, the eventType SHALL be EV_EFI_SPDM_DEVICE_POLICY.
    a. The event value SHALL be the value of the UEFI_VARIABLE_DATA structure (this structure SHALL be considered byte-aligned).
    b. The UEFI_VARIABLE_DATA.VariableData value SHALL be a list of EFI_SIGNATURE_LIST
    c. The UEFI_VARIABLE_DATA.UnicodeNameLength value is the number of CHAR16 characters (not the number of bytes).
    d. The UEFI_VARIABLE_DATA.UnicodeName contents MUST NOT include a NUL.
    e. If reading a UEFI variable returns UEFI_NOT_FOUND, platform firmware SHALL measure the absence of the variable.
        i. The UEFI_VARIABLE_DATA.VariableDataLength field MUST be set to zero.
        ii. UEFI_VARIABLE_DATA.VariableData field will have a size of zero.
5. For entries in the device security variable, as defined in normative 10.b, the eventType SHALL be EV_EFI_SPDM_DEVICE_AUTHORITY.
    e. The event value SHALL be the value of the UEFI_VARIABLE_DATA structure and SHALL be packed.
    f. The UEFI_VARIABLE_DATA.VariableData value SHALL be the EFI_SIGNATURE_DATA value from the EFI_SIGNATURE_LIST that contained the authority that was used to validate the SPDM device.
    g. The UEFI_VARIABLE_DATA.VariableName SHALL be set to the GUID of the device security variable.
    h. The UEFI_VARIABLE_DATA.UnicodeName SHALL be set to the value of device security variable name. The value MUST NOT include the NUL terminator.

Entities that MUST be measured if the TPM is enabled:

1. If the platform provides a firmware debugger mode which may be used prior to the UEFI environment or if the platform provides a debugger for the UEFI environment, then the platform SHALL extend an EV_EFI_ACTION event into PCR[7] before allowing use of the debugger. The event string SHALL be “UEFI Debug Mode”. The Platform Firmware MUST log this measurement in the event log using the string “UEFI Debug Mode” for the Event Data.
2. Before executing any code not cryptographically authenticated as being provided by the Platform Manufacturer, the Platform Manufacturer firmware MUST measure the Secure Boot Variables as defined above, in the order listed using the defined event types.
3. If the platform supports changing any of the following UEFI policy variables after they are initially measured in PCR[7] and before ExitBootServices() has completed and the platform does not support UEFI 2.5 or later, the platform MUST be restarted.
4. If the platform supports UEFI 2.5 or later and supports changing any of the defined Secure Boot variables after they are initially measured in PCR[7] and before ExitBootServices() has completed, the platform MAY be restarted OR the variables MUST be remeasured into PCR[7]. Additionally the normal update process for setting any of the defined Secure Boot variables SHOULD occur before the initial measurement in PCR[7] or after the call to ExitBootServices() has completed.
5. The system SHALL measure the EV_SEPARATOR event in PCR[7]. (This occurs at the same time the separator is measured to PCR[0-7].)
6. Before launching a UEFI Driver or a UEFI Boot Application (and regardless of whether the launch is due to the UEFI Boot Manager picking an image from the DriverOrder or BootOrder UEFI variables or an already launched image calling the UEFI LoadImage() function), the UEFI firmware SHALL determine if the entry in the UEFI_IMAGE_SECURITY_DATABASE_GUID/ EFI_IMAGE_SECURITY_DATABASE variable that was used to validate the UEFI image has previously been measured in PCR[7]. If it has not been, it MUST be measured into PCR[7]. If it has been measured previously, it MUST NOT be measured again. The measurement SHALL occur in conjunction with image load.
7. The EV_EFI_VARIABLE_AUTHORITY measurement in step 6 is not required if the value of the SecureBoot variable is 00h (off). In such cases, no validation occurs against the SecureBoot databases.

Entities that MUST be measured if the TPM is disabled:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] and an EV_SEPARATOR event MUST be recorded in the event log for PCR[0-7] prior to the first invocation of the first Ready to Boot call.

Entities that MUST be measured if the TPM device is hidden:

1. Per Section 3.3.1 Collection and Reporting of Measurements, the digest of the value 00000000h or FFFFFFFFh MUST be extended in PCR[0-7] prior to the first invocation of the first Ready to Boot call. Platform Firmware SHALL NOT place an event in the event log.

3.3.4.9 PCR[16] – Debug

Start of informative comment

This PCR is resettable from any locality and is for use by any entity on the Host Platform. It is intended, by convention, to be used as a debug PCR. Components should use this PCR for debugging purposes only (e.g., software development of components utilizing the PCR features of the TPM, e.g., TPM2_Create). Applications targeted for user, final, or production environments should not use this PCR in their final release.

Because the PCR is not intended for use with sealing or attestation, measurements for it should not be recorded in the event log.

End of informative comment

1. Any component on the Host Platform MAY use and reset PCR[16] at any time.
2. On platforms targeted for user, final or production environments, if the firmware does extend PCR[16], it MUST NOT record the event in the event log.

3.3.4.10 PCR[23] – Application Support

Start of informative comment

This PCR is resettable from any locality. It is to be used by the Static or Dynamic operating systems or its applications.

The PCR value is not preserved across S3/Resume cycles because it is a resettable PCR.

End of informative comment

The operating system defines the purpose of this PCR and MAY reset and use it at any time.

3.3.5 Localities assigned to RTM's

3.3.5.1 H-CRTM Localities and Concepts

Start of informative comment

The H-CRTM sequence is initiated at the earliest stage of platform boot by the CPU. The H-CRTM sequence is used to measure code that runs prior to platform firmware into the TPM. It is used to extend one event to PCR[0]. The H-CRTM sequence uses the interface commands _TPM_Hash_Start, _TPM_Hash_Data, and _TPM_Hash_End, prior to the TPM receiving a TPM2_Startup command. It is feasible to measure multiple components using _TPM_Hash_Data, but this set of measurements results in a single extend to PCR[0]. When an H-CRTM sequence occurs, the Platform Firmware has not started, and will not know that one occurred. Because of this, it is required that the code initiating an H-CRTM sequence leave some artifacts for Platform Firmware to construct the event it will record in the event log on behalf of the H-CRTM code. The artifacts left for Platform Firmware are not defined in this specification and are implementation specific. A platform firmware vendor needs to coordinate with their various component vendors.

When an H-CRTM sequence occurs, the code initiating the sequence will also send the TPM2_Startup command. To avoid the Platform Firmware sending an additional TPM2_Startup and receiving an error, an additional artifact will be created for Platform Firmware to understand the state of the TPM when it gets control of the boot process. Given that the TPM2_Startup can occur from locality 3, if an H-CRTM sequence occurs, Platform Firmware will construct the Startup Locality Event (See Section 10.4.5.3 Startup Locality Event). The artifacts left for Platform Firmware are not defined in this specification and are implementation specific. A platform firmware vendor needs to coordinate with their various component vendors.

End of informative comment

3.3.5.2 DRTM and Transitive Trust Chain

Start of informative comment

The Host Platform Manufacturer and Dynamic OS define the usage of Localities 1–4 and PCRs 17–22. While the usage of Locality 1–4 is beyond the scope of this specification, generally, the Host Platform is responsible for maintaining protections of the PCRs based on their associated Locality by controlling access to the _TPM_Hash_Start command memory address. The normative reference of the relationship between Localities and PCRs and their relative attributes is specified in the PC Client Platform TPM Profile for TPM 2.0. For more information please see the TCG D-RTM Architecture Specification Version 1.0.

The DRTM sequence utilizes the same _TPM_Hash_x interface commands. The sequence occurs after TPM2_Startup and extends to PCR 17.

End of informative comment

3.3.5.2.1 Localities 1-3

3.3.5.2.1.1 Integrity Collection and Reporting

The method of collecting and reporting PCRs[17-23] is beyond the scope of this specification and is the purview of the Host Platform's hardware and the Dynamic OS that supports the DRTM.

3.3.5.2.1.2 PCR Usage

The usage of PCRs[17-22] is beyond the scope of this specification and is the purview of the Dynamic OS.

3.3.5.2.1.3 Protection

Start of informative comment

How or if a Platform Manufacturer provides access to the address ranges for Localities 1 through 3 is out of scope for this specification.

End of informative comment

The Host Platform MAY provide protections to enforce the Locality messages from the source of the commands to the TPM.

3.3.5.2.2 Locality 4

3.3.5.2.2.1 Concepts

Start of informative comment

The use of Locality 4 is restricted to trusted hardware on the Host Platform. The Host Platform protects the address ranges for Locality 4. Even the Dynamic OS cannot access Locality 4.

The TPM2_PCR_Reset command for PCR[17] and the TPM_Hash_* commands require Locality 4. This protects PCR[17] from being reset except by trusted hardware associated with the DRTM.

Some platforms and some TPMs may not permit sending commands using the Locality 4 address range even by trusted hardware and may only permit use of the TPM_Hash_* commands for Locality 4.

End of informative comment

The assertion of Locality 4 MUST only be performed by the DRTM.

3.3.5.2.2.2 Integrity Collection and Reporting

How this is done is Host Platform implementation-specific.

3.3.5.2.2.3 PCR Usage

The PCR associated with this Locality is PCR[17]. This Locality MUST measure the first component executed after the DRTM and MAY measure other components as determined by Host Platform's implementation. This is analogous to PCR[0] in the SRTM.

3.3.5.2.2.4 Protection

The Host Platform MUST provide protections to ensure that the entire Locality 4 address range is accessible only by the DRTM.

3.3.6 NV Index Usage

Start of informative comment

By convention, this specification assigns TPM PCRs to align with different parts of the Host Platform's static RTM and chain of trust. In some cases, the PCRs cannot be used to measure the device instance specific data or boot dynamic data. In this case a TPM NV Index can be used to record the information instead of PCR.

The platform firmware uses TPM2_NV_Extend() to extend the digest, similar to TPM2_Extend(). The verifier can use TPM2_NV_Certify() to certify the NV index. The process is similar to TPM2_Quote(). The NV_Extend indexes used for this purpose are defined in the TCG PC Client Platform TPM Profile for TPM 2.0 Version 1.06. The NV Indexes defined for this purpose have reserved handles from 0x01C40200 - 0x01C402FF.

Table 2 NV Extend Indexes for Instance Measurements

NV Index	Name	Usage
0x01C40200	**NV_EXTEND_INDEX_FOR_INSTANCE**	**NV Extend Record for instance data.** See Section 3.3.6.2 NV Extend Record for Instance Data
0x01C40201	**NV_EXTEND_INDEX_FOR_DYNAMIC**	**NV Extend Record for dynamic data.** See Section 3.3.6.3 NV Extend Record for Dynamic Data
0x01C40202	**EVENT_LOG_INTEGRITY_ EXIT_PM_AUTH**	**Event Log Integrity for ExitPmAuth.** See Section 3.3.6.1 NV Index for Event Log Integrity
0x01C40203	**EVENT_LOG_INTEGRITY_ READY_TO_BOOT**	**Event Log Integrity for ReadyToBoot.** See Section 3.3.6.1 NV Index for Event Log Integrity
0x01C40204 - 0x01C402FF	**N/A**	**Reserved by TCG**

End of informative comment

3.3.6.1 NV Index for Event Log Integrity

Start of informative comment

The TCG event log is used to convey valuable but untrusted information to a verifier. A platform OEM may provision a NV index to record a hash of the event log. The definition of the NV Index is contained in the TCG PC Client Platform TPM Profile for TPM 2.0 (insert revision and reference). The Event Log Integrity index is provided to enable verifiers to detect modification of Event meta data and informational events. The structure written to this index is defined in Section 10.5.1 .

Platform OEMs should write the Version and remaining data area to the index. The EVENT_LOG_INTEGRITY hash algorithm is the same as the TPM NV Index nameAlg. The OEM should select the largest size supported by the firmware and the TPM. See Section 10.5.1 for structure definition.

Platform OEMs should provision two EVENT_LOG_INTEGRITY NV Indexes, because these two NV Indexes have different security properties. There are also 2 corresponding EFI variables. The first index,

EVENT_LOG_INTEGRITY_ EXIT_PM_AUTH, is used to record the event log integrity prior to the platform firmware invoking the first component that is not from the platform manufacturer, such as option ROMs. The second index, **EVENT_LOG_INTEGRITY_ READY_TO_BOOT,** is used to record the event log integrity prior to the platform firmware transferring to control to an OS loader. This specification defines two EFI variables, L"EventLogIntegrityDescExitPmAuth" and L"EventLogIntegrityDescReadytoBoot". The variables L"EventLogIntegrityDescExitPmAuth" and L"EventLogIntegrityDescReadytoBoot" contain the vendor-specific descriptions of changes to the event log that correspond to the hash recorded in the corresponding NV Indexes. The nomenclature used for these variables follows the UEFI Specification convention to indicate that these variables use the UEFI-defined encoding for characters and strings as UCS-2 values defined by Unicode 2.1. To prevent changes by third party components, the platform firmware should lock the **EVENT_LOG_INTEGRITY_ EXIT_PM_AUTH** NV Index using TPM2_NV_WriteLock, prior to loading any component not controlled by platform firmware. At the same time, platform firmware should apply a Read-Only attribute to the EFI variable L"EventLogIntegrityDescExitPmAuth". Because the platform is likely already running third-party components, there is no need to adopt same protection mechanisms used for the **EVENT_LOG_INTEGRITY_EXIT_PM_AUTH** index or the EFI variable L"EventLogIntegrityDescExitPmAuth"..

End of informative comment

If the Event Log Integrity Feature is supported, platform firmware SHALL follow the steps during provisioning defined as follows:

If the EVENT_LOG_INTEGRITY NV Indexes need to be provisioned (the NV Index is defined and the TPMA_NV_WRITTEN bit is CLEAR):

1. Platform firmware SHALL write the EVENT_LOG_INTEGRITY_DATA Version per Table 48 EVENT_LOG_INTEGRITY_DATA definition. Platform Firmware SHALL write the CurrentHash and PreviousHash fields with all zeros.

The **EVENT_LOG_INTEGRITY_ EXIT_PM_AUTH** Index SHALL be used to record configuration changes that are detected by platform firmware prior to invoking any 3rd party code, such as before the UEFI platform initialization specification defined **`EFI_END_OF_DXE_EVENT_GUID.`**

The **EVENT_LOG_INTEGRITY_ READY_TO_BOOT** Index SHALL be used to record Event Log changes that are detected by platform firmware prior to passing control to the OS Loader, such as before the UEFI defined **`EFI_EVENT_GROUP_READY_TO_BOOT`** event.

If the EVENT_LOG_INTEGRITY NV Indexes are provisioned, then the platform firmware SHALL perform the following procedure:

1. The platform firmware SHALL read the EVENT_LOG_INTEGRITY NV Index nameAlg**.**
2. When the system boots to an event integrity collection point, the platform firmware SHALL calculate the digest of the event log – HASH(EventLog) based upon the nameAlg.
3. Platform firmware shall follow the procedure below to update the Event Log Integrity Indexes and EFI Variables.

The platform firmware SHALL perform the following steps to update both Event Log Integrity NV Indexes using TPM2_NV_Write:

1. If the **CurrentHash** is different from the HASH(EventLog), record **CurrentHash** to **PreviousHash**. Otherwise, record all-0 to the **PreviousHash**,
2. Record HASH(EventLog) to the **CurrentHash**

3. Write **CurrentHash** and **PreviousHash** to the NV Index

The platform firmware SHALL perform the following steps to update the EFI Variables – L"EventLogIntegrityDescExitPmAuth" and L"EventLogIntegrityDescReadytoBoot" using the UEFI runtime service SetVarible() API, defined in the UEFI Specification. See Section 2.6 External Specifications

1. If the **CurrentHash** is different from the HASH(EventLog), record the configuration change to the corresponding EFI Variable. Otherwise, delete the corresponding EFI Variables.

3.3.6.2 NV Extend Record for Instance Data

Start of informative comment

By default, Platform Firmware only measures class (non-identifying) information to the TPM PCRs, but not device instance or system unique information, such as asset, serial numbers, or DICE TCB information. Platform firmware may provision a NV index to record the measurement of instance information. This NV index can be used to verify the integrity of the instance information in the TCG event log.

End of informative comment

Entities that SHOULD be measured in the NV_EXTEND_INDEX_FOR_INSTANCE

1. Platform Firmware SHOULD extend the device certificate from devices that support the SPDM "GET_CERTIFICATE" functionality. See Sections 10.4.5.4.1 NV_INDEX_INSTANCE_EVENT_LOG and 10.2.7 DEVICE_SECURITY_EVENT_DATA Structure. **Note:** This measurement may be made mandatory in a future revision of this specification.
    a. If measured, platform Firmware SHALL set the TCG_PCR_EVENT2.pcrIndex to NV_EXTEND_INDEX_FOR_INSTANCE handle.
    b. Platform firmware SHALL set the TCG_PCR_EVENT2.eventType to EV_NO_ACTION.
    c. Platform firmware SHALL set the TCG_PCR_EVENT2.digests to the digest of TCG_PCR_EVENT2.event according to the TPM PCR banks HashAlg instead of NV Index nameAlg. The whole digests SHALL be extended to NV_EXTEND_INDEX_FOR_INSTANCE as data blob.
    d. Platform Firmware SHALL set the TCG_PCR_EVENT2.event to NV_INDEX_INSTANCE_EVENT_LOG_DATA with signature "NvIndexInstance".
2. SMBIOS structures that contain static class and instance information (e.g. componentManufacturer, componentModel, componentSerial, componentRevision, fieldReplaceable described in SMBIOS based component class registry specification) SHOULD be measured using the event type EV_EFI_HANDOFF_TABLES2 as described in Sections 10.2.4 Measuring Industry Standard Tables and Data Structures and 10.4.1 (Event Types), with the exception of zeroizing instance data. **Note:** The class data is included in both measurements to allow correlation with the information contained in a TCG Platform Certificate.

Entities that SHALL NOT be measured in the NV_EXTEND_INDEX_FOR_INSTANCE

1. Any dynamically updatable information SHALL NOT be included in the instance data.

3.3.6.3 NV Extend Record for Dynamic Data

Start of informative comment

By default, the platform firmware SHALL NOT measure any dynamic / ephemeral / changeable / automatically updated data to the PCR. In SPDM measurement use case, the verifier may want to know the NONCE value in the challenge / response process and track NONCEs to ensure there is no duplication. Platform firmware may provision a NV index to record the measurement of this dynamic information. This NV index can be used to verify the integrity of the dynamic information in the TCG event log.

Requirements for the NV Index are defined in the TCG PC Client Platform TPM Profile for TPM 2.0 Systems Version 1.06.

End of informative comment

Entities that SHOULD be measured in the NV_EXTEND_INDEX_FOR_DYNAMIC.

1. Platform Firmware SHOULD extend the nonces used by platform firmware (SPDM challenger) and by the device (SPDM responder) that support SPDM “CHALLENGE” and “GET_MEASUREMENT” functionality if platform firmware sends these commands. See Sections 10.4.5.4.3 NV_INDEX_DYNAMIC_EVENT_LOG_DATA and 10.2.7 DEVICE_SECURITY_EVENT_DATA Structure.
   Note: This measurement may be made mandatory in a future revision of this specification.
   a. If measured, platform firmware SHALL set the TCG_PCR_EVENT2.pcrIndex to NV_EXTEND_INDEX_FOR_DYNAMIC handle.
   b. Platform firmware SHALL set the TCG_PCR_EVENT2.eventType to EV_NO_ACTION.
   c. Platform firmware SHALL set the TCG_PCR_EVENT2.digests to the digest of TCG_PCR_EVENT2.event according to the TPM PCR banks HashAlg instead of NV Index nameAlg. The whole digests SHALL be extended to NV_EXTEND_INDEX_FOR_DYNAMIC as data blob.Platform Firmware SHALL set the TCG_PCR_EVENT2.event to NV_INDEX_DYNAMIC_EVENT_LOG_DATA with signature “NvIndexDynamic ”.

Entities that SHALL NOT be measured in the NV_EXTEND_INDEX_FOR_DYNAMIC for dynamic data.

1. Any privacy-sensitive information SHALL NOT be included in the dynamic instance data.

3.3.7 SPDM Device Authentication and Measurement Summary

Start of informative comment

Ideally, platform firmware records the device certificate and device measurement for each SPDM responder. In reality, a platform may include a mix of SPDM devices and non-SPDM devices. SPDM devices may have different capabilities, such as SPDM CERT_CAP, CHAL_CAP, signed MEAS_CAP and unsigned MEAS_CAP, which translate to variations in the availability of SPDM certificates. Based upon different SPDM capabilities and information returned by the SPDM protocol, platform firmware needs flexible policy to record the data and enable Verifiers to decide what is acceptable. To simplify the discussion, we summarize the policy in Table X.

Table 3 SPDM Device Measurement Policy

Category	Example Conditions	AuthState and Data to be measured
Trusted Information	• SPDM device returns a valid certificate chain, verified with SPDM CHALLENGE_AUTH response, and • The SPDM MEASUREMENTS response has a valid signature, and	AUTH_SUCCESS + Data

	• The SPDM certificate chain has a Trust Anchor that matches the UEFI device signature variable.	
Untrusted verifiable Information without trust anchor	• SPDM device returns a valid certificate chain, verified with SPDM CHALLENGE_AUTH response, and • The SPDM MEASUREMENTS response has a valid signature. • However, the trust anchor does not match the UEFI device signature variable.	AUTH_NO_AUTHORITY + Data Data is still returned because the verifier can check the information using its own trust anchors.
Untrusted verifiable Information without binding	• SPDM device returns a valid certificate chain, but it does not support validation of the leaf certificate via SPDM CHALLENGE_AUTH, and • The SPDM MEASUREMENTS response has a valid signature.	AUTH_NO_BINDING + Data Data is still returned because the verifier can check the information binding.
Untrusted unverifiable Information	• SPDM device does not have any certificate chain, or • The SPDM device cannot return signed SPDM MEASUREMENTS.	AUTH_FAIL_NO_SIG + No Data No data is returned because there is no way to validate the authenticity of the data.
Invalid Information	• SPDM device has an invalid certificate chain, e.g. the integrity of the certificate chain is broken, or • The signature of SPDM CHALLENGE_AUTH is invalid, or • The signature of SPDM MEASUREMENTS is invalid.	AUTH_FAIL_INVALID + No Data No data is returned because the data from device is invalid.
No SPDM capability	The device does not support SPDM, but the platform firmware has a vendor specific method to request and receive measurement data from the device.	AUTH_NO_SPDM + Data (or) AUTH_NO_SPDM + No Data Platform firmware may or may not return the data. If data is returned, it is recorded using the OEM SubHeaderType. See Section 10.2.7 DEVICE_SECURITY_EVENT_DATA Structures

The high-level flow is shown in Figure 7 High Level SPDM Device Measurement Flow. The basic concept of “Get & Record Certificate” flow is shown in Figure 8 SPDM Device Measurement Flow – Get and Record Certificate. The basic concept of “Get & Record Measurement” flow is shown in Figure 9 SPDM Device Measurement Flow – Get and Record Measurement.

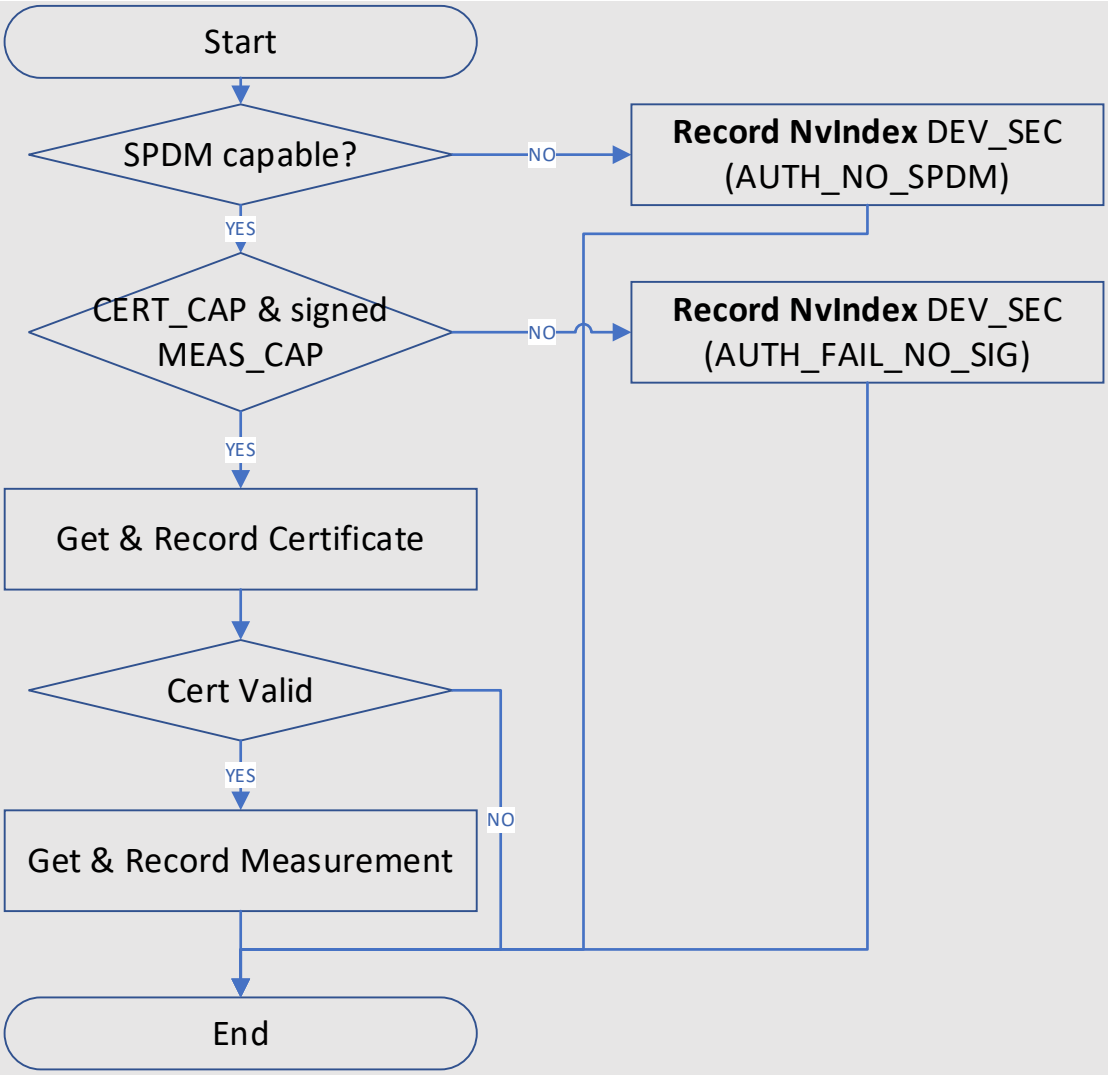

Figure 7 High Level SPDM Device Measurement Flow

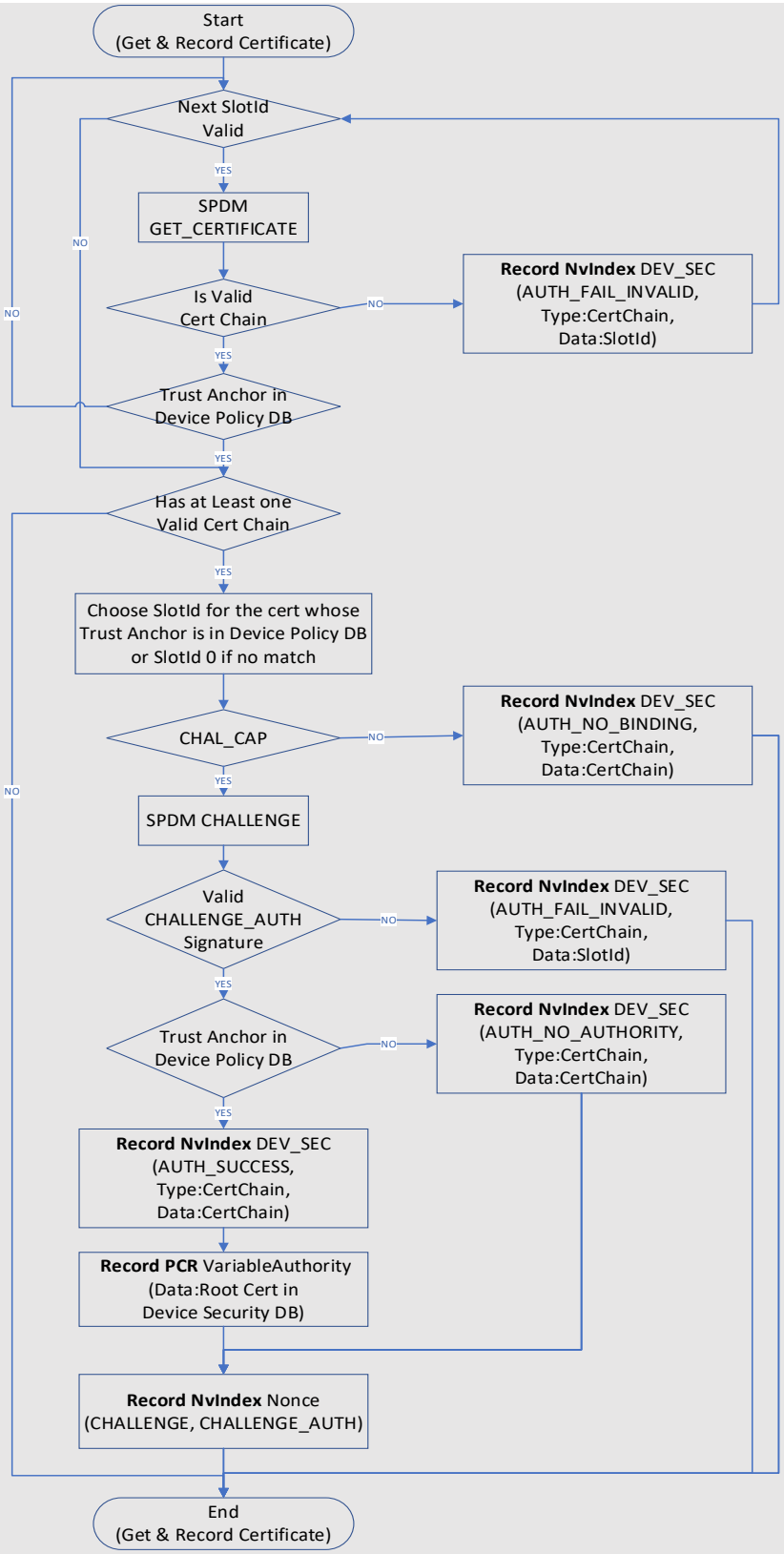

Figure 8 SPDM Device Measurement Flow – Get and Record Certificate

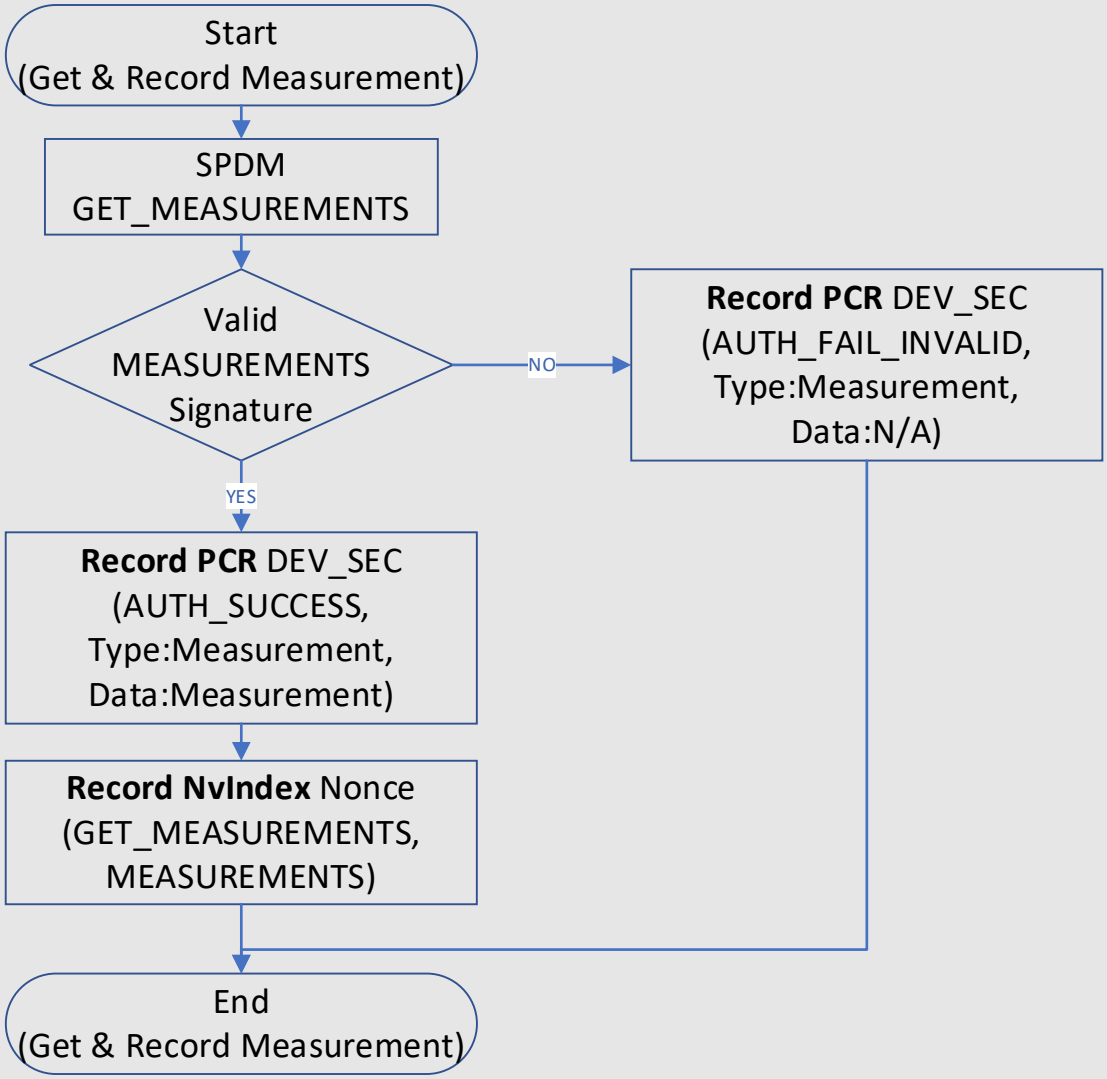

Figure 9 SPDM Device Measurement Flow – Get and Record Measurement

The purpose of the actions is summarized in below tables:

Table 4 SPDM Certificate Verification Action Summary

Certificate Verification Actions	Purpose	Failure Result
• Verify trust anchor is in UEFI device signature variable	Authority	AUTH_NO_AUTHORITY + Data
• Verify certificate chain in CHALLENGE_AUTH	Identity Binding	AUTH_NO_BINDING + Data
• Verify root cert hash with root cert, and	Integrity	AUTH_FAIL_INVALID + No Data

• For each intermediate certificate (i), verify the cert i is signed by cert i-1, and • Verify digital signature in CHALLENGE_AUTH with public key in leaf cert		
• Use a fresh nonce to perform CHALLENGE	Freshness	AUTH_FAIL_INVALID + No Data

Note that the actions in Table 4 SPDM Certificate Verification Action Summary and Table 5 SPDM Signature Verification Action for Measurements Summary are not presented in the order that platform firmware might take. The order is described in the normative text below Table 5 SPDM Signature Verification Action for Measurements Summary.

Table 5 SPDM Signature Verification Action for Measurements Summary

Signature Verification Actions for Measurements	Purpose	Failure Result
• Verify digital signature in MEASUREMENTS with public key in leaf certificate	Integrity	AUTH_FAIL_INVALID + No Data
• Get all measurements at one time, or • Get individual measurements and check “content changed” field in last MEASUREMENTS	Atomicity	AUTH_FAIL_INVALID + No Data
• Use a fresh nonce to perform GET_MEASUREMENTS	Freshness	AUTH_FAIL_INVALID + No Data

If the platform firmware supports SPDM device authentication and measurement as defined in this specification, the below flow is used.

End of informative comment

1. The platform firmware SHALL measure the UEFI device signature variable into PCR[7] with eventType being EV_EFI_SPDM_DEVICE_POLICY. See Section 3.3.4.8.

Start of informative comment

This database includes a trust anchor. A trust anchor is either a root CA certificate or an intermediate CA certificate. This specification does not support revoked or denied CA certificates. It is up to a verifier to evaluate the certificates used and make a trust decision.

End of informative comment

2. The platform firmware SHALL enumerate system buses and locate SPDM devices.
    a. If a platform expects an SPDM device but finds the device is not SPDM capable and cannot retrieve measurement data in a vendor specific manner, then platform firmware SHALL measure NV_INDEX_INSTANCE_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_INSTANCE with AuthState being AUTH_NO_SPDM, SubHeaderType being SPDM Cert Chain and SubHeaderLength being 0. See Sections 10.4.5.4.1 NV_INDEX_INSTANCE_EVENT_LOG and 10.2.7.3 Authenticated SPDM Structures.

b. If a platform expects an SPDM device but finds the device is not SPDM capable but can retrieve measurement data in a vendor specific manner, then platform firmware SHALL measure NV_INDEX_INSTANCE_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_INSTANCE with AuthState being AUTH_NO_SPDM, SubHeaderType being OEM Type and SubHeader being DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_OEM_MEASUREMENT. See Sections 10.4.5.4.1 NV_INDEX_INSTANCE_EVENT_LOG and 10.2.7.3 Authenticated SPDM Structures
c. If a device supports SPDM CERT_CAP and MEAS_CAP with signature, then platform firmware continues the remaining steps in this flow. Otherwise, the platform firmware SHALL measure NV_INDEX_INSTANCE_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_INSTANCE with AuthState being AUTH_FAIL_NO_SIG, SubHeaderType being SPDM Cert Chain and SubHeaderLength being 0. See Sections 10.4.5.4.1 NV_INDEX_INSTANCE_EVENT_LOG and 10.2.7.3 Authenticated SPDM Structures.

Start of informative comment

The platform firmware follows the SPDM specification to create a connection with the SPDM device, such as by sending SPDM GET_VERSION, GET_CAPABILITIES, NEGOTIATE_ALGORITHM commands.

End of informative comment

3. The platform firmware SHOULD send SPDM GET_CERTIFICATE command to the SPDM device to get the device certificate chain(s), inclusive of SPDM slot 0 to slot 7. See Sections 10.4.5.4.1 NV_INDEX_INSTANCE_EVENT_LOG and 10.2.7.3 Authenticated SPDM Structures for definitions of the SPDM Event Types. The platform firmware SHALL validate the device certificate chains:
    a. If the device certificate chain integrity of the SPDM responder is invalid (e.g. the integrity is broken):
        i. Platform firmware SHALL measure NV_INDEX_INSTANCE_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_INSTANCE with AuthState being AUTH_FAIL_INVALID, SubHeaderType being SPDM Cert Chain and SubHeader being DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_CERT_CHAIN without SPDM_CERT_CHAIN,
        ii. This device certificate SHALL NOT be recorded or used for digital signature verification, and
        iii. If there is another certificate slot, Platform firmware SHALL go to 3.
        iv. If there is no valid certificate slot, Platform firmware SHALL stop measuring this device.
    b. If the device certificate chain integrity of the SPDM responder is valid:
        i. The platform firmware SHALL check if the trust anchor matches any entry in the UEFI device signature variable, and
            1. If the trust anchor does not match any entry in the UEFI device signature variable, platform firmware SHALL enumerate the next slot (Go back to step 3.a.). If no certificate chain in any populated slot has a trust anchor that matches any of the entries in the UEFI device signature variable, the platform SHALL measure the certificate chain in step 4.b or 5.b.
            2. If the trust anchor matches an entry in the UEFI device signature variable, platform firmware MAY stop the Certificate slot enumeration and go to step 4

Start of informative comment

Because issuing GET_CERTIFICATE for all populated slots of a device might be time consuming, the platform firmware may choose various optimizations.

For example, during first boot, the platform firmware may issue GET_DIGESTS and GET_CERTIFICATE for every slot, then cache the certificate digest and corresponding device certificate chain. In the next boot, the platform

firmware may issue GET_DIGESTS only and compare the response to the cached digests. If they match, then the platform firmware may skip issuing GET_CERTIFICATE and use the cached device certificate chain in 3.a.

For example, the platform firmware may use a pre-defined policy to choose a subset of slots, if the platform firmware knows the device and expects the certificate chain with the trust anchor from the specific slots.

No matter what optimization flow the platform firmware does, the order of measurement must be maintained to avoid PCR value change or NV Index value change in different boots.

End of informative comment

4. The platform firmware SHALL send SPDM CHALLENGE to the SPDM device to authenticate the device if the device supports SPDM CHAL_CAP. The slot ID SHOULD match the certificate chain whose trust anchor is in the UEFI device signature variable. If no certificate chain in any populated slot has a trust anchor that matches any of the entries in the security policy database, then slot ID 0 SHOULD be used. See Sections 10.4.5.4.1 NV_INDEX_INSTANCE_EVENT_LOG and 10.2.7.3 Authenticated SPDM Structures for definitions of the SPDM Event Types.
    a. The platform firmware SHALL verify the returned digital signature in CHALLENGE_AUTH message.
        i. If the digital signature verification fails, the platform firmware SHALL measure NV_INDEX_INSTANCE_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_INSTANCE with AuthState being AUTH_FAIL_INVALID, SubHeaderType being SPDM Cert Chain and SubHeader being DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_CERT_CHAIN without SPDM_CERT_CHAIN. The corresponding device certificate SHALL NOT be used any more.
    b. If the device does not support SPDM CHAL_CAP,
        i. The platform firmware SHALL measure NV_INDEX_INSTANCE_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_INSTANCE with AuthState being AUTH_NO_BINDING, SubHeaderType being SPDM Cert Chain and SubHeader being SPDM certificate chain.
5. If the certificate and transcript signature of SPDM CHALLENGE_AUTH are valid, platform firmware SHALL measure the device certificate. See Sections 3.3.6.3 NV Extend Record for Dynamic Data, 10.2.7.3 Authenticated SPDM Structures and 10.4.5.4.3 NV_INDEX_DYNAMIC_EVENT_LOG.
    a. If the trust anchor is present in the UEFI device signature variable,
        i. The platform firmware SHALL measure NV_INDEX_INSTANCE_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_INSTANCE with AuthState being AUTH_SUCCESS, SubHeaderType being SPDM Cert Chain and SubHeader being SPDM certificate chain.
        ii. The platform firmware SHALL measure the matching entry in the security policy database into PCR[7] with eventType being EV_EFI_SPDM_DEVICE_AUTHORITY, if the authority entry has not been measured previously. See Section 3.3.4.8 PCR[7] – Secure Boot Policy Measurements.
    b. If no certificate chain in any populated slot has a trust anchor that matches any of the entries in the security policy database,
        i. The platform firmware SHALL measure NV_INDEX_INSTANCE_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_INSTANCE with AuthState being AUTH_NO_AUTHORITY, SubHeaderType being SPDM Cert Chain and SubHeader being SPDM certificate chain.
    c. The platform firmware SHALL measure NV_INDEX_DYNAMIC_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_DYNAMIC with data being the NONCE value in SPDM CHALLENGE and CHALLENGE_AUTH. See section 3.3.6.3 NV Extend Record for Dynamic Data.
6. The platform firmware SHALL send SPDM GET_MEASUREMENTS to the SPDM device to collect measurements of the device if the device supports SPDM MEAS_CAP with digital signature. The platform firmware may collect all available device measurements with one command or collect individual device measurements one by one. See Sections 10.2.7.1 Common Structures.

a. The platform firmware SHALL require digital signature for SPDM MEASUREMENTS. If the device does not support signed measurements, the device SHALL be treated as supporting no SPDM measurements.
b. If the platform firmware collects single measurements from the device, then it SHALL guarantee the atomicity of the device measurements. The platform firmware SHALL only request digital signature for the last measurement index and verify the content changed field returned. If the device reports that the measurement content has been modified, then the platform firmware SHALL collect the measurement again. If the content is reported to have been modified again, then the platform firmware SHALL treat the measurement as invalid and measure the DEVICE_SECURITY_EVENT_DATA2 with AuthType being AUTH_FAIL_INVALID, SubHeaderType being SPDM Measurement Block and SubHeaderLength being 0.
c. The platform firmware SHALL verify the digital signature of the MEASUREMENTS messages.
    i. If the digital signature verification fails, the platform firmware SHALL measure DEVICE_SECURITY_EVENT_DATA2 with AuthType being AUTH_FAIL_INVALID, SubHeaderType being SPDM Measurement Block and SubHeaderLength being 0. This device measurement SHALL NOT be used any more.

7. The platform firmware SHALL measure the SPDM device measurement if the digital signature of the measurement is valid. See Sections 10.2.7.1 Common Structures and 10.4.5.4.3 NV_INDEX_DYNAMIC_EVENT_LOG.
    a. The platform firmware SHALL measure DEVICE_SECURITY_EVENT_DATA2 with AuthType being AUTH_SUCCESS, SubHeaderType being SPDM Measurement Block and SubHeader being device measurement. All the device measurement entry should be measured separately. See Section 10.2.7.
    b. The platform firmware SHALL measure NV_INDEX_DYNAMIC_EVENT_LOG_DATA to NV_EXTEND_INDEX_FOR_DYNAMIC with data being the NONCE value in SPDM GET_MEASUREMENTS and MEASUREMENTS. See section 3.3.6.3 NV Extend Record for Dynamic Data.

4 Non-Volatile Storage

Start of informative comment

Non-Volatile Storage (NV RAM) is made available by the TPM for use by various processes on the Host Platform. A limited amount of resources is required to be provided by the TPM.

End of informative comment

4.1 NV RAM Size and Allocation

Start of informative comment

The PC Client Platform TPM Profile for TPM 2.0 specifies a minimum amount of NV Storage the TPM must provide. This value is based upon the anticipated usages of this area at the time the specifications were written.

End of informative comment

5 Maintenance

Start of informative comment

Maintenance for the PC Specific Implementation Specification refers to the processes surrounding upgrade or replacement of the Platform Firmware ROM / Flash. All requirements for TPM maintenance are manufacturer-defined per the TPM Library Specification for Family 2.0.

End of informative comment

Implementation of maintenance is optional. If it is implemented, it MUST be implemented as defined in this section.

1. Firmware meeting the requirements of the US National Institute of Standards and Technology (NIST) Special Publication 800-155 BIOS Integrity Measurement Guidelines SHALL produce one or more identifiers used to associate the platform with its reference manifest (RM). These identifiers are EV_NO_ACTION log events containing an event of type TCG_Sp800_155_PlatformId_Event. See Section 10.4.5.2 Firmware Integrity Measurement Reference Manifest Event.

5.1 Platform Firmware Recovery Mode

Start of informative comment

This is a failure-recovery mode of Platform Firmware that is invoked by the Boot Block typically when the UEFI Firmware is corrupt. The Platform Firmware Recovery Mode will perform a minimal initialization of the system and then attempt to boot a recovery program from some type of external media. The Platform Firmware Recovery Mode may not have the capability to continue the SRTM measurement chain.

A couple of attack scenarios have been identified due to the “Firmware Recovery” feature implemented in many instances of Platform Firmware. A method to counter these attacks could implement:

1. Use of hardware to disable the TPM until the next _TPM_Init; or,
2. Disabling the TPM by calling TPM2_Startup (CLEAR) followed by TPM2_HierarchyControl (EH Disable, SH Disable); or,
3. Measuring of components to maintain the SRTM for BIOS Recovery Mode components; or,

Unconditionally performing a Host Platform Reset upon completion of BIOS Recovery Code.

End of informative comment

1. It MUST NOT be possible for a BIOS Recovery Mode to allow impersonation of another boot state as represented by the SRTM. This applies to the values in the PCR[0-7].
2. If Recovery requires a boot in Recovery Mode, the Firmware SHALL:
    a. Initialize the TPM in a disabled state by sending a TPM2_Startup (CLEAR) followed by a TPM2_HierarchyControl (EH disable, SH disable) to disable each of the hierarchies.
    b. Cap PCR[0-7] in all active PCR banks with the digest of the value 00000000h or FFFFFFFFh and record an EV_SEPARATOR Event in the event log. See Section 10.4.1 Event Types.

5.2 Flash Maintenance

Start of informative comment

There are two scenarios: A Manufacturer Approved Environment (MAE), and a Non-MAE (NMAE). The MAE may update any portion of Platform Firmware while the NMAE must not update the CRTM.

End of informative comment

1. The Platform Manufacturer MUST control the update, modification and maintenance of the CRTM within the MAE.
2. The MAE is vendor-specific and MUST provide protections for the TBB and SRTM.
3. At the completion of the update process, the reset vector MUST point to the one and only one SRTM that is controlled by the MAE.
4. At the completion of the update process, the execution MUST begin at the SRTM designated by the platform manufacturer.
5. If a BIOS update is installed that modifies the SRTM, Platform Firmware update process MUST unconditionally transition the platform to the Off state or reboot.

5.3 Firmware Compliance Requirements

The UEFI Firmware SHOULD meet the requirements of the U.S. National Institute of Standards and Technology *(NIST) Special Publication 800-147 Bios Protection Guidelines* or *Special Publication 800-147B, BIOS Protection Guidelines for Servers*. See Section 2.6 External Specifications.

6 TCG Certificates and Verification of a Platform for SP800-155 Compliance

Start of informative comment

The TCG PC Client Firmware Integrity Measurement Specification defines the requirements for platform firmware vendors to provide Attestation Evidence and Endorsements of reference values. It also describes the end to end process of attesting and verifying platform firmware. The Platform Firmware Profile defines the evidence. The TCG Reference Integrity Manifest Information Model Specification and TCG PC Client Reference Integrity Manifest Specification define the format and encoding of the endorsed reference values. A Verifier needs the means to discover and locate the various endorsed artifacts. To facilitate that, the PC Client PFP defines an informational event to convey the existence and location of these artifacts. As of PFP 1.06, the informational event includes the existence and location of a platform certificate in addition to the RIM. PFP 1.06 also defines UEFI variables to facilitate discovery of platform certificates and RIMs, see Section 10.4.5.2.1 UEFI Variables for Platform RIM and Certificate Discovery.

An event log may contain a TCG_Sp800_155_PlatformId_Event2 structure that the Verifier uses to identify an endorsed RIM that contains the same VendorId and ReferenceManifestGuid components as included in the TCG_Sp800_155_PlatformId_Event2 structure. The Verifier obtains a RIM by a mechanism outside the scope of this specification.

See Section 10.4.5.2 Firmware Integrity Measurement Reference Manifest Event (Sp800_155 Event) and Section 10.4.5.2.1 UEFI Variables for Platform RIM and Certificate Discovery for more information.

End of informative comment

The UEFI Firmware SHOULD meet the requirements of the TCG PC Client Firmware Integrity Measurement Specification.

7 TPM Discoverability

Start of informative comment

The TPM powers up with all hierarchies enabled and all functions available by default. This is different than TPM 1.2, which followed a gradual opt-in model. There are cases where a Platform OEM may decide to configure a TPM so that it is hidden from an end-user, an operating system, or OS present applications. An example of such a case may be where the TPM is restricted by regulation from being imported into a specific country or geography. There are other cases where an end-user might decide that they want to disable part or all of the TPM to ensure it is not usable by OS present applications.

This section describes the requirements for Platform OEMs when they provide the ability for an end-user or Platform Firmware to selectively or completely disable the TPM. Additional requirements may be detailed in other sections.

The PC Client Implementation Specification for Conventional BIOS used the terms “enumeration” and “discoverable” to describe these states of the TPM where the TPM was visible or not to the OS and the end-user. These terms were generally confusing and so are not used in this specification. The requirements are defined in relation to visibility of the TPM to the OS present interface and to the end-user through BIOS Setup.

Note: When hiding the TPM from the OS, it is important that the platform firmware not leave the TPM in state that prevents the TPM or the platform from entering low power mode. As such, firmware drivers should be designed to relinquish locality or transition the TPM interface into Idle, even if the TPM responds with an error, as defined in Section 3.3.2.1 Errors during TPM Initialization.

End of informative comment

7.1 TPM Visibility to the OS

Start of informative comment

The default state of the TPM makes it ready and available to the OS and OS present applications.

Note: Disabling the platform hierarchy will hide all NV Indices in the TPM with the attributes platformCreate SET and phEnableNV CLEAR. Hiding the TPM but leaving the platform hierarchy enabled may allow a dictionary attack to force the TPM into Lockout.

End of informative comment

If the Platform OEM implements a capability to prevent the OS from discovering the TPM, or hiding it from the OS and OS-present applications, the firmware SHALL implement all of the following requirements:

1. When the TPM is visible to the OS and enabled:
    a. The TPM2 ACPI Table SHALL be listed in the RSDT and XSDT tables as described in Section 9.1 ACPI Device Object for TPM.
    b. The TPM device object SHALL be present in the ACPI namespace and _STA() SHALL return 0xF. See Section 9.1 ACPI Device Object for TPM.
    c. The Platform OEM SHALL make all measurements in compliance with the requirements of this specification.
    d. The Platform OEM-provided mechanism to make the TPM hidden from the OS SHALL perform a Host Platform Reset.
    e. The TPM SHALL remain visible to the OS on resume from sleep or hibernate.
    f. The Firmware SHALL support the EFI_TCG2_PROTOCOL as specified in the TCG EFI Protocol Specification.
    g. The EFI_TCG2_BOOT_SERVICE_CAPABILITY.TPMPresentFlag SHALL return TRUE.

h. The Firmware SHALL support the Physical Presence Interface as specified in the TCG Physical Presence Interface Specification for TPM Family 1.2 and 2.0 revision 1.3 version 52, or later.
i. The TPM control surface as defined in Section 7.3 Platform Firmware Setup TPM Control Surface SHALL be present in BIOS Setup.
j. All TPM Hierarchies MUST be enabled, without separate user action to disable one or more hierarchies.
k. Platform Firmware SHALL populate the TCG Event Log, see Section 10.1 Introduction.
l. The TPM state SHALL NOT change on resume from sleep or hibernate.

2. When the TPM is visible to the OS but disabled:
   a. The TPM2 ACPI Table SHALL be listed in the RSDT and XSDT tables.
   b. The TPM device object SHALL be present in the ACPI namespace and STA() SHALL return 0xF.
   c. The EFI_TCG2_PROTOCOL SHALL be present.
   d. The EFI_TCG2_BOOT_SERVICE_CAPABILITY.TPMPresentFlag SHALL return TRUE.
   e. The Platform Firmware SHALL disable the Storage and Endorsement Hierarchies by use of the command TPM2_Hierarchy_Control (ehDisable and shDisable).
   f. Platform Firmware MAY disable the Platform Hierarchy.
   g. Platform Firmware SHALL cap PCR[0-7] as described in Section 3.3.4 PCR Usage.
   h. Platform Firmware SHALL create the TCG Event Log as described in Section 10.1 Introduction.
   i. The TPM state SHALL NOT change on resume from sleep or hibernate
3. When the TPM is discoverable but hidden from the OS:
   a. The TPM2 ACPI Table SHALL be listed in RSDT and XDST tables.
   b. The TPM device object SHALL be present in the ACPI namespace and STA() SHALL return 0x0.
   c. The EFI_TCG2_PROTOCOL SHALL be present.
   d. The EFI_TCG_BOOT_SERVICE_CAPABLITY.TPMPresentFlag SHALL return FALSE.
   e. Platform Firmware SHALL disable the Storage and Endorsement Hierarchies by use of the command TPM2_Hierarchy_Control (ehDisable and shDisable).
   f. Platform Firmware MAY disable the Platform Hierarchy.
   g. Platform Firmware SHALL cap PCR[0-7] as described in Section 3.3.4 PCR Usage.
   h. Platform Firmware MAY create the TCG Event Log.
   i. The Platform OEM-provided mechanism to make the TPM visible to the OS SHALL perform a Host Platform Reset.
   j. The Platform Firmware driver SHALL ensure the TPM can go to the Idle state, e.g. relinquish Locality (FIFO) or write the TPM_CRB_CTRL_REQ_x.goIdle to allow the platform to enter low power mode.
   k. The TPM state SHALL NOT change on resume from sleep or hibernate.
4. When the TPM is not discoverable and is hidden from the OS:
   a. The TPM2 ACPI Table SHALL NOT be listed in the RSDT or XSDT tables.
   b. The TPM device object SHALL NOT be present in the ACPI namespace.
   c. The EFI_TCG2_EFI_PROTOCOL SHALL NOT be present.
   d. The TPM SHALL NOT be usable by OS-present applications.
   e. The TPM Storage and Endorsement Hierarchies SHALL be disabled by Firmware on every boot using a TPM2_HierarchyControl command for both hierarchies.
   f. The Platform Firmware driver SHALL ensure the TPM can go to the Idle state, e.g. relinquish Locality (FIFO) or write the TPM_CRB_CTRL_REQ_x.goIdle to allow the platform to enter low power mode.
   g. The TPM Platform Hierarchy MAY be disabled by Firmware on every boot using a TPM2_HierarchyControl command.
   h. Firmware SHALL cap PCR[0-7] as described in Section 3.3.4 PCR Usage.
   i. Platform Firmware SHALL NOT create the TCG Event Log.
   j. The Platform OEM-provided mechanism to make the TPM visible to the OS SHALL perform a Host Platform Reset.
   k. The TPM SHALL remain hidden from the OS on resume from sleep or hibernate.

7.2 TPM Visibility to End-Users through BIOS Setup

Start of informative comment

PC Client OEMs will provide the control surface for TPM through Platform Firmware Setup menu. In the event that the TPM is completely hidden and disabled, the user interface for TPM will also be absent from BIOS Setup.

Note: Disabling the platform hierarchy will hide all NV Indices in the TPM with the attributes platformCreate SET and phEnableNV CLEAR. Hiding the TPM but leaving the platform hierarchy enabled may allow a dictionary attack to force the TPM into Lockout.

End of informative comment

If the Platform OEM implements a capability to hide the TPM from the end-user, the firmware SHALL implement all of the requirements documented in normative 4 of Section 7.1 TPM Visibility to the OS plus the following requirement:

1. The TPM control surface as defined in Section 7.3 Platform Firmware Setup TPM Control Surface SHALL NOT be present in BIOS Setup.

7.3 Platform Firmware Setup TPM Control Surface

Start of informative comment

PC OEMs present a view of the TPM setup to end users through their Setup utilities. PFP 1.06 removed the control surface definition for TPM 1.2, as TPM 1.2 is not available in new PC Client Platforms. For Verifiers and platform owners with an install base of systems that include TPM 1.2, the normative requirements for the 1.2 Control surface are documented in PFP 1.05.

End of informative comment

Table 6 Example of TPM 2.0 Firmware User Interface

User Interface Menu Labels for TPM 2.0	User Interface Help Text
"On/Off"	"On" – TPM fully operational "Off" – See Section 7.1 TPM Visibility to the OS **NOTE:** This state is OEM specific, User Interface is not defined by TCG only TPM and Event Log behavior is defined by TCG.
Enabled/Disabled"	"Enabled" Default state of the TPM, defined by TCG. "Disabled" See Section 7.1 TPM Visibility to the OS
"Clear"	"Clear" TPM2_Clear command sent to TPM by platform firmware. Requires Platform Authorization or Physical Presence. Clears Storage Hierarchy and ehProof. TPM remains Enabled.

Measurement Hash Algorithm Selection	<Algorithm Selection> TPM2_PCR_Allocate command sent to the TPM by FW. Requires Platform Authorization or Physical Presence. Sets the Hash Algorithm of the specified PCR bank to the selected algorithm. Requires a reboot.

1. If the Platform Firmware supports the ability to disable the Storage and Endorsement Hierarchies:
    a. The Platform Firmware SHALL support a mechanism to enable and disable them in Setup
    b. The Platform Firmware MAY support the Physical Presence Interface mechanism to enable and disable them.
    c. Platform Firmware MAY support a mechanism to enable/disable both hierarchies independently.
    d. Platform Firmware SHALL provide descriptive text describing the implications of disabling the hierarchies.
2. Platform Firmware SHALL provide a mechanism to Clear the TPM from Setup.
3. If the Platform supports changing the algorithm used for measuring events, Platform Firmware MAY support a mechanism to do so through Setup.

8 State Transitions

8.1 Architecture and Definitions

Start of informative comment

A handoff to an operating system generally occurs after Platform Firmware has completed its initialization and testing of the Host Platform hardware. As defined in Section 2.2.11 Boot State Transition, the event that marks the transition from the Pre-OS state to the OS-Present state on the Host Platform is the first invocation of Ready to Boot or an equivalent event.

As stated in Section 3.1 of the UEFI Specification, by the time the Boot Manager code gets invoked, the UEFI firmware has found and initialized all the bootable devices on the Host Platform for this boot cycle and has built a priority-ordered list of those devices, created as Boot### and BootVariable### UEFI variables. Each entry in the list includes a pointer to a UEFI application that is capable of booting an operating system from that device.

The Boot Manager starts at the top of the prioritized list and simply attempts to boot a UEFI application from each bootable device, in turn, by loading and executing the application on that device.

If the UEFI application on a device fails to boot an operating system, it returns control to Boot Manager.

Measuring and Logging Events in the Pre-OS to Post-Boot Transition

TCG-enabled firmware must measure and log the events described in the previous section of this Informative comment. LoadImage() must, without exception, measure into PCR[4] the event of loading and executing UEFI application code from a bootable device; this measurement will automatically result in the proper entry being made into the Event Log. Platform Firmware must not hash any data areas when it measures this event. If secure boot is enabled, the LoadImage() service will measure the authority, if it was included in the UEFI_IMAGE_SECURITY_DATABASE_GUID / EFI_IMAGE_SECURITY_DATABASE variable, that was used to verify the OS Loader into PCR[7]. See Section 3.3.4.8 PCR[7] – Secure Boot Policy Measurements.

If the UEFI application on a bootable device returns back to the Boot manager, that event must be measured.

And, as stated in Section 3.3.4.5 PCR[4] – Boot Manager Code and Boot Attempts of this specification, if Boot Manager Code returns control back to Platform Firmware, each subsequent execution of Boot Manager Code must be separately measured.

End of informative comment

8.2 Procedure for Pre-OS to OS-Present Transition

Start of informative comment

In order to measure the transition from the Pre-OS state to the OS-Present state, a number of steps need to be performed. This section of the specification will outline and describe these steps.

Note that the term ‘EFI OS Loader Load’ is a label for a behavior, not necessarily a label for a code module.

Measure code that boots an operating system on a UEFI platform into PCR[4] and measure data associated with that code into PCR[5].

End of informative comment

8.2.1 Extending PCR[4]

Start of informative comment

Just before passing control of the Host Platform to the operating system, the UEFI Firmware needs to perform several actions in order to assure that the chain of measurements of the Host Platform boot process is contiguous. PCR[4] will contain the measurements describing the code which attempts to boot an OS.

End of informative comment

8.2.2 Extending PCR[5]

Start of informative comment

PCR[5] is reserved for any configuration data associated with code modules measured into PCR[4]. Information such as GPT/Partition tables is measured into PCR[5]. PCR[5] may be utilized and extended by any boot loader for variable data.

End of informative comment

8.2.3 Extending PCR[7]

Start of informative comment

Measurements in PCR[7] reflect the UEFI Secure Boot policy, introduced in UEFI 2.3.1.

End of informative comment

8.2.4 Measuring OS Boot Events

Start of informative comment

An Event Log enables a challenger to determine the state of trust of the UEFI platform and enables software on the UEFI platform to reconstruct boot events. The Operating System handoff code needs to fill the Event Log with information about the boot devices used to get to the Operating System. The UEFI platform functions designed for computing hash values and extending PCRs should automatically log the extended events.

However, there are events that must be added to the Event Log that are not the result of a PCR extend operation.

The purpose of this section is to specify all the required events added to the Event Log for OS boot, including events that do not result from a PCR extend operation.

End of informative comment

1. The Platform Firmware MUST measure the event EV_SEPARATOR into PCR[0-7] once for each platform boot cycle and the measurement of that event MUST draw the line between leaving the pre-OS environment and entering the OS-Present environment. The data within the event field of the EV_SEPARATOR event MUST be 32 bits (a double-word) of 0's or FF's (that is, 00000000h or FFFFFFFFh) unless an error condition occurs. See Section 3.3.2.2 Errors Recording Measurements.
2. The Platform Firmware code that loads the OS Loader MUST measure, into PCR[4], every attempt to load and execute a OS Loader (a UEFI application).
3. A boot device has a UEFI application. The boot variable describes the location of the application. The Platform Firmware launches the application. If an OS or application that was launched by the Boot Option returns back to UEFI firmware, this affects the transitive trust chain and the return action MUST be measured into PCR[4].
4. Sequence of measuring OS boot events MUST proceed as specified below.
    a. Upon selecting a boot device, the UEFI firmware measures the event type EV_EFI_ACTION "Calling EFI Application from Boot Option" into PCR[4] and measures the event type EV_EFI_ACTION "Booting to <Boot####> Option" where Boot#### is the Boot Option Variable Name.
    b. Measure EV_SEPARATOR into PCR[0-7]. This occurs only once in the flow.
    c. If UEFI Secure Boot is enabled, measure the entry in the UEFI_IMAGE_SECURITY_DATABASE_GUID /EFI_IMAGE_SECURITY_DATABASE that was used to validate the UEFI image into PCR[7] as described in section 3.3.4.8 PCR[7] – Secure Boot Policy Measurements steps 5 and 6.

d. In this optional step, measure the GPT with event type EV_EFI_GPT_EVENT2 and event data set to the UEFI_GPT_DATA structure into PCR[5] or measure the GPT with event type EV_EFI_GPT_EVENT and event data set to the UEFI_GPT_DATA structure into PCR[5].
e. Measure the selected UEFI application code PE/COFF image described by the boot variable with event type EV_EFI_BOOT_SERVICES_APPLICATION into PCR[4].
f. Execute UEFI application from boot variable.
    i. In this optional step, if the executing code from step e loads additional applications or drivers prior to successive steps (i.e., return or exit boot services), the loaded applications or drivers MUST be measured into PCR[4].
g. Measure EV_EFI_ACTION event “Returning from EFI Application from Boot Option” into PCR[4].
h. If the firmware needs to select a next boot device, the UEFI firmware MUST jump to step a.
i. If ExitBootServices() is invoked, then an EV_EFI_ACTION event “Exit Boot Services Invocation” MUST be measured. The return value of ExitBootServices() MUST be reflected in a measured event, into PCR[5], as either “Exit Boot Services Returned with Failure” or “Exit Boot Services Returned with Success”, depending upon the return code from the ExitBootServices() call.

8.2.5 Passing Control of the TPM from Pre-OS to OS-Present Environments

Start of informative comment

Once Platform Firmware has turned control over to an operating system, the Post-Boot environment will load its own set of drivers and code to access the TPM. This could cause a potential conflict since there may be contention between the Pre-OS and OS-Present environments for use and access of the TPM.

TPM runtime services protocol provides for a solution to this problem through Exit Boot Services (EBS). Once the OS-Present environment has loaded its driver support, it will call this function to disable the UEFI pre-OS services. Boot Manager should not return control back to Platform Firmware after calling Exit Boot Services, because Platform Firmware may lose its ability to measure additional events once the method completes.

If the operating system is to use the transitive trust chain beyond the measurements indicated in this specification, it is expected that the OS Boot Loader continues the measurement of the boot process.

End of informative comment

8.3 Power States, Transitions, and TPM Initialization

Start of informative comment

This section describes which Host Platform and OS actions are expected within power states and during transitions between power states. Careful power management needs to occur for the TPM device so its state accurately reflects the measurements of the current platform state. When the system is in the working state (S0) or a powered standby state (S0ix, S1, or S2), the TPM state needs to persist, thus the TPM will also be in the working state (D0). Some transitions, like turning off the machine, are expected to cause the TPM to lose its state. Other transitions, like entering and resuming from sleep, should save and restore TPM state if the OS and platform cooperate correctly; otherwise, the TPM will return an error. Transitioning the TPM between the various power states involves use of the TPM2_Shutdown and TPM2_Startup commands. See Section 2.2.14 System and TPM Power States for more information.

End of informative comment

1. If the TPM is visible to the OS as described in Section 7.1 TPM Visibility to the OS, the Host Platform MUST:
    a. Provide whatever resources (e.g., power) are needed for the TPM to behave as though it is in the D0 state while:
        i. The TPM device is in any device power state except for D3.

ii. The TPM device is in D0 and the Host Platform is in any of the following system power states:
    1. S0, S0ix
    2. S1 OR
    3. S2

b. Provide whatever resources (e.g., power) are needed for the TPM to maintain its saved state when the TPM enters D3 and the Host Platform enters S3. (For example, if the TPM places its saved state in NV Storage, it may not need power when in the D3 state. However, if the TPM does not use NV Storage to maintain its state when in the D3 state, the Host Platform needs to provide power for the device when the TPM is in the D3 state, and the Host Platform is in S3.)

2. When the Host Platform is in the S3 state, it SHOULD prohibit all TPM functions. (If the TPM does receive commands during S3, they may invalidate the saved TPM state.)

8.3.1 General Host Platform and OS Power Requirements

Start of informative comment

Because power management of the TPM is under the control of the TBB, an OS cannot place the TPM device in D3 or transition the TPM out of D3 directly; however, an OS may initiate a transition to S3 that may cause the TBB to place the TPM in D3. If the system resumes from S3, the TBB will transition the TPM out of D3 and if the TPM state was saved properly, the TPM state is restored by the SRTM in the S3 resume path.

By implementing the requirements below, an OS can use a process for transitioning the TPM into and out of D3 that preserves the TPM's state saved with the TPM2_Shutdown (STATE) command and restores it.

End of informative comment

1. The TBB MUST be the only entity that can change the TPM device power state.
2. The TBB MUST ensure the TPM is only in the D0, D1, or D2 state when the platform is not under the control of the TBB.
3. The OS booted by Platform Firmware SHALL support ACPI.
4. After issuing a TPM2_Shutdown (STATE) command, the OS SHOULD NOT issue TPM commands before transitioning to S3 without issuing another TPM2_Shutdown (STATE) command.

8.3.2 Power State Transitions

Start of informative comment

Each section below describes the behavior and requirements for entering or exiting each system power state. The only transitions allowed are those defined in the ACPI specification.

In general, the power state transition requirements for the Firmware can be summarized as:

1. The SRTM prepares the TPM for use when booting or resuming from S3.
2. If the SRTM is modified, the Firmware needs to reboot instead of resuming from S2 or S3.
3. TPM_INIT is only issued by the SRTM during boot or during S3 resume.
4. During boot and resuming from S3, Platform Firmware issues a TPM2_SelfTest command.

End of informative comment

8.3.3 Off to S0 (Working)

Start of informative comment

This transition is from an Off state to the platform power state that supports ACPI operating systems. This is a normal boot process that initializes the TPM and begins the Static Root of Trust for Measurement. The Host Platform may later transition in either direction between the Legacy and the S0 state.

If the TPM is disabled or the platform owner has disabled measurements, the SRTM may issue the TPM2_Startup and TPM2_HierarchyControl commands such that the TPM is disabled. The SRTM would further cap PCR[0-7] with an EV_SEPARATOR Event. See Section 10.4.1 Event Types.

Transitioning out of S4 is like transitioning out of S5 except the FW or OS loader restore a memory image from persistent storage. The pre-OS components are measured normally for the current platform state, including the components that restore the memory image. It is not permitted for the Platform Firmware to process a PPI request following a resume from hibernate without initiating a host platform reset. This is not reflected in Figure 7, which only describes the transition from S5 to S0.

The FW is responsible for issuing the TPM2_SelfTest command, immediately following TPM2_Startup (Clear).

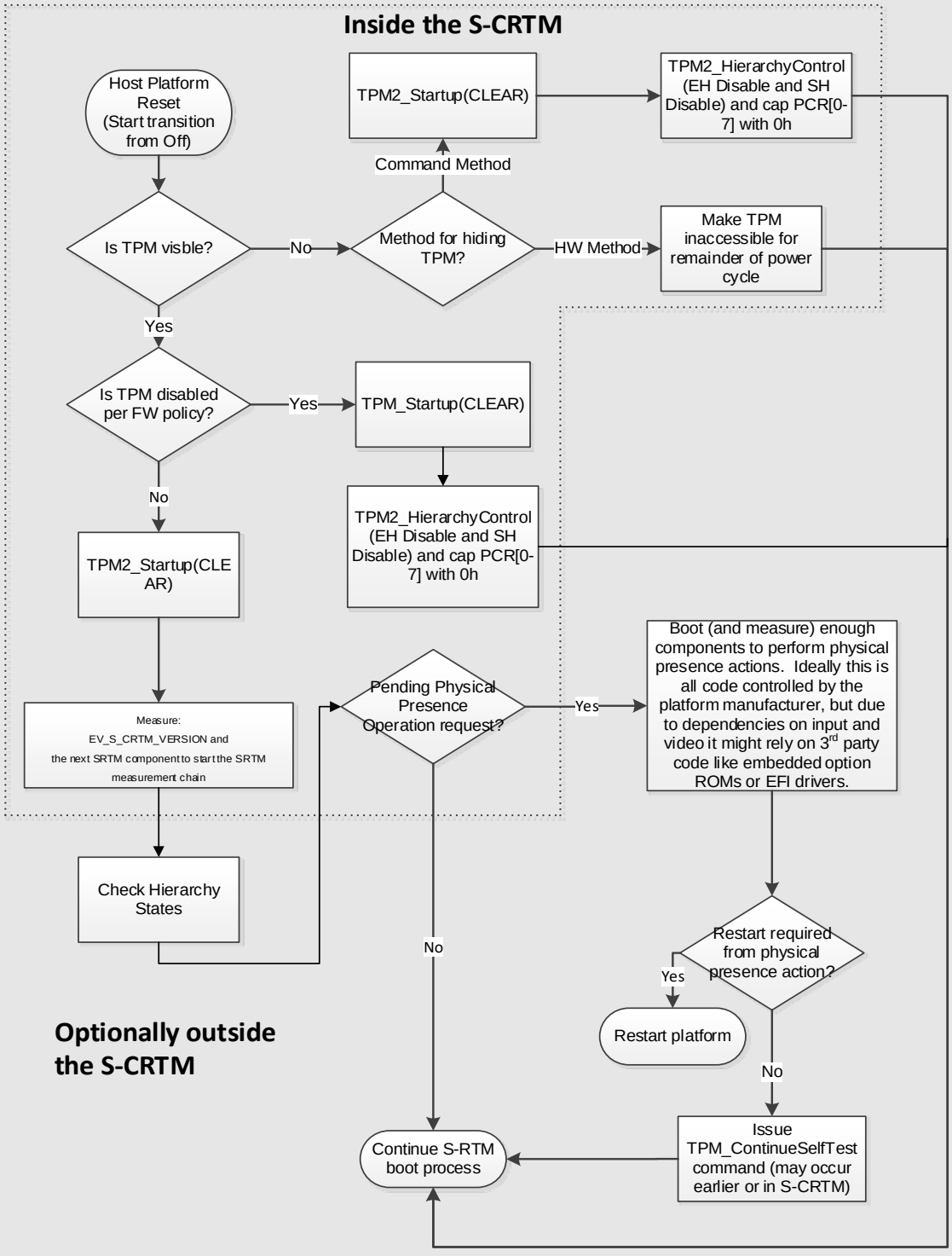

Figure 10 Firmware Actions during transitions from Off

End of informative comment

When transitioning from the Off state to the S0 states:

1. The Host Platform MUST issue a _TPM_INIT.
2. If the TPM interface is accessible and the TPM is hidden from the OS, the SRTM MUST
   a. Issue a TPM2_Startup (CLEAR) command,
   b. Issue a TPM2_HierarchyControl (EH Disable and SH Disable) command, and
   c. Cap PCR[0-7] by extending in all active PCR banks the digest of the value 00000000h or FFFFFFFFh and recording an EV_SEPARATOR Event in the event log for each PCR, see Section 10.4.1 Event Types.
3. If the TPM interface is accessible and the TPM is enumerated, the SRTM MUST issue a TPM2_SelfTest (FULLTEST = NO) command prior to the first invocation of the first Ready to Boot call.
4. If the TPM is visible to the OS and the TPM interface is accessible, the platform manufacturer MUST set platformAuth to a high entropy value and MAY set platformPolicy during execution of the SRTM such that later software is unable change objects in the Platform Hierarchy or operations that require platform authorization.
5. If the TPM is enabled and visible, the SRTM MUST start the SRTM measurement chain by recording integrity measurements during the boot process per Section 3.3.1 Collection and Reporting of Measurements.
6. Boot Managers SHOULD be prepared for the TPM's Self-Test actions associated with the TPM2_SelfTest command to be in progress when the Boot Manager is launched.

8.3.4 S0 (Working) to Off

Start of informative comment

This transition is from the running OS to the Off state. In contrast to power loss, this is an orderly shutdown. Because TPM state is expected to be discarded after entering S5 (Off), there are no required actions.

The TCG Platform Reset Attack Mitigation Specification permits configuring the platform so memory is erased during boot under certain conditions. One variable condition is whether the platform performed an orderly shutdown or unexpectedly lost power. If this transition is an orderly shutdown, the OS should be able to purge secrets from the Host Platform memory if appropriate. Host Platforms implementing the TCG Platform Reset Attack Mitigation Specification may perform additional actions to avoid clearing memory on the next boot according to the TCG Platform Reset Attack Mitigation Specification.

End of informative comment

8.3.5 S4 (Hibernate) to S0 (Working)

Start of informative comment

This transition is from an S4 (Hibernate) state. From the perspective of platform firmware, this is a normal boot process that initializes the TPM and begins the Static Root of Trust for Measurement. If, on an S4 resume, the platform firmware boots to a platform firmware-controlled environment, such as Setup, the platform firmware usually exits that environment by sending TPM2_Shutdown (CLEAR) and resets the platform. This causes the TPM to clear the Restart counter and increment the Reset counter which can prevent an OS from resuming from S4. This is desirable behavior in the event of an attack, where the platform is booted to an environment not controlled by platform firmware (such as a USB key). However, if a legitimate user of the platform enters setup, the platform can preserve the state of TPM counters by sending a TPM2_Shutdown (STATE). Hybrid sleep scenarios occur when a laptop enters a S3, then, later, due to a critically low battery, the OS transitions the system to S4. Such a scenario is invisible to platform firmware and thus platform firmware cannot take any action as described above to preserve the state of the TPM counters.

End of informative comment

1. On resume from S4 (Hibernate), if platform firmware boots to Setup or an EFI application controlled by platform firmware (e.g. a battery charging application), Platform firmware SHALL send a TPM2_Shutdown (STATE) command on exit from the environment.
2. If platform firmware boots to a platform firmware-controlled environment more than once, following a resume from S4, platform firmware SHALL send TPM2_Shutdown (CLEAR) on subsequent ReadyToBoot events.
3. If platform firmware tracks the S4 entry event using a platform-firmware-maintained indicator, platform firmware SHALL clear the indicator immediately prior to sending the TPM2_Shutdown (STATE) command or at invocation of an EfiReadyToBoot event.

8.3.6 S0 (Working) to S4 (Hibernate)

Start of informative comment

This transition is from the running OS to the Hibernate state. The OS distinguishes between S4 and S5 by saving a file on the disk that is used by the OS to resume its state. Because certain TPM state is needed by the OS to resume from S4, it sends a TPM2_Shutdown (STATE) command. The platform firmware path has not varied between resume from S4 and S5. However, as TPM capabilities (such as Reset and Restart counters) that are preserved in the S4 path have been adopted by OS's, it is important that platform firmware handle corner cases that impact the OS in the resume from S4 path. Hybrid sleep scenarios occur when a laptop enters a S3, then, later, due to a critically low battery, the OS transitions the system to S4. Such a scenario is invisible to platform firmware and thus platform firmware cannot take any action as described above to preserve the state of the TPM counters.

End of informative comment

1. Platform firmware SHALL register for OS notifications of S4 entry events.
2. Platform firmware MAY use a platform-firmware-maintained indicator to track the S4 entry state.
    a. If Platform firmware uses an indicator, platform firmware SHALL maintain the indicator across the S4 entry and resume cycle.

8.3.7 S1 (Sleep) to S0 Working, S0 to S1

Start of informative comment

These transitions are between power states that all fully preserve the state of the TPM.

The TPM will not be in the D3 state when this transition occurs.

The S1 resume process does not jump to a resume vector. Instead, the processors continue execution where they stopped when entering S1.

If there are any changes to the Host Platform's components or configuration, measuring these changes is the responsibility of the OS.

End of informative comment

The Host Platform MUST NOT issue a TPM_INIT.

8.3.6 S0 (Working) to S2 (Sleep)

Start of informative comment

This transition is between power states that all fully preserve the state of the TPM.

The TPM will not be in the D3 state when this transition occurs.

The S2 resume process does jump to a reset vector because CPU context is lost while in S2.

A special case occurs when FW updates are installed and Host Platform code that executes on S2 or S3 resume is modified so the measurement in PCR[0] is no longer accurate. If a FW update has occurred since the last transition from S5 to S0, the OS should reboot the Host Platform instead of allowing a transition to S2 or S3. If the OS does allow a transition to S2 or S3, as a failsafe, the SRTM is responsible for detecting modified FW code during the S2 or S3 resume and forcing a reboot. The net result is upon a successful resume to the OS, the SRTM measurements in PCR[0-7] contain the correct measurements for the actual S2 or S3 resume code that executed or PCR[0] has been invalidated.

If there are any other changes to the Host Platform's components or configuration, measuring these changes is the responsibility of the OS.

End of informative comment

4. The Host Platform MUST NOT issue a TPM_INIT.
5. The OS SHOULD NOT transition from S0 to S2 if a FW update has occurred since the last transition from an Off state to S0.

8.3.7 S2 (Sleep) to S0 (Working)

Start of informative comment

This transition is between power states that fully preserve the state of the TPM.

The TPM will not be in the D3 state when this transition occurs.

Note: See the informative text in Section 8.3.6 S0 (Working) to S2 (Sleep).

Figure 11 Firmware Actions for S2 Resume illustrates the SRTM actions when resuming from S2.

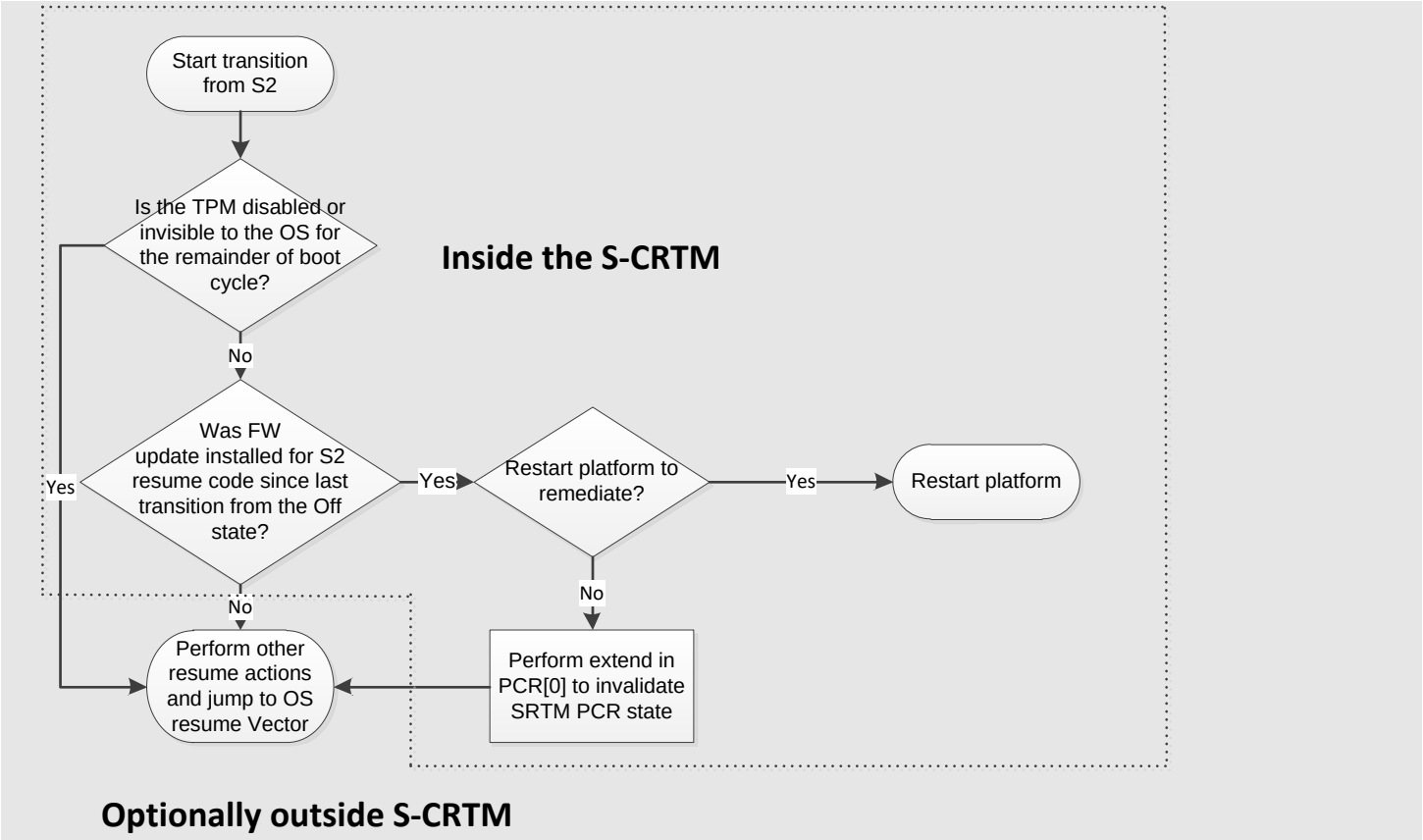

Figure 11 Firmware Actions for S2 Resume

End of informative comment

1. The Host Platform MUST NOT issue a TPM_INIT.
2. If the TPM is visible and enabled:
   a. The SRTM MUST be able to determine whether there has been an update to any S2 resume portion of the Platform FW since the previous transition from an Off state. The SRTM SHOULD use a method that does not significantly add to the time it takes to resume from S2; in particular, it is not necessary for SRTM to re-measure Platform FW code and compare this measurement to the measurement of FW code performed at the previous transition from S4 or S5. The determination of whether Platform Firmware changed MAY be done using a simple flag.
   b. If the SRTM detects a modification to the S2 resume portion of the Platform Firmware since the last transition from an Off state, the SRTM MUST either:
      i. Force the Host Platform to reboot, or
   c. Make the contents of PCR[0] invalid by extending the digest of the value 00000001h in PCR[0]. Note: No corresponding event is logged in the event log because the OS may be managing the log.

8.3.8 S0 (Working) to S3 (Sleep)

Start of informative comment

This transition is from a working state to a sleep state that only needs to provide the necessary power to the TPM to Maintain its saved state, not the full TPM context. Upon resume from S3, the saved TPM state is generally restored.

See the informative text in Section 8.3.6 S0 (Working) to S2 (Sleep) for a special case regarding FW updates.

Entering into and exiting from the S3 state is a coordinated effort between the OS and the FW. Since there is no mandated behavior for the TPM during the various power states, this design and protocol assumes the TPM has only two power states: D0 and D3. There is no requirement for the TPM to sense any of the normal Host Platform alerts indicating a transition into the S3 power state. Nor is there a requirement for the TPM to sense the normal Host Platform alerts indicating a transition from the S3 power stating into the S0 power state. It is therefore a requirement that the Host Platform FW and OS participate in notifying the TPM of these transitions.

The OS driver notifies the TPM that it is about to transition to the D3 state and the system is about to transition from the S0 to the S3 power states by sending the TPM2_Shutdown(STATE) command. This notifies the TPM that it is to save the required states into non-volatile memory. Upon resume from the S3 power state, the TPM must be notified whether to restore a previously saved state or perform normal initialization. This is done using the TPM2_Startup command. The initial state of the TPM can only be determined by the components of the RTM. Therefore, the SRTM makes the determination as to whether the Host Platform is resuming from an Off state or the S3 power state. The SRTM must issue the TPM2_Startup command with the appropriate parameter, indicating to the TPM whether to initialize or restore the previously saved state of the TPM.

If TPM2_Startup(STATE) is called when there is no saved state to restore, the TPM returns TPM_RC_VALUE. The SRTM should perform a host platform reset and send a TPM2_Startup(CLEAR) before passing control to the Operating System

Note: See the requirement in Section 8.3.1 General Host Platform and OS Power Requirements for OS actions when placing the TPM in D3.

The OS should issue a TPM2_Shutdown (STATE) command prior to transitioning the platform to S3. The OS should not transition from S0 to S3 if a platform firmware update has occurred since the last transition from an Off state to S0.

End of informative comment

8.3.9 S3 (Sleep) to S0 (Working)

Start of informative comment

This transition is a resume from an S3 suspend state. Host Platform Reset and TPM_INIT are asserted. The SRTM issues the TPM2_Startup(STATE) command, loading the previously saved state, without re-measuring Pre-OS components. The SRTM passes control to the OS. If there are any changes to the Host Platform's components or configuration, measuring these changes is the responsibility of the OS.

See Section 8.3.8 S0 (Working) to S3 (Sleep) for more informative text regarding this transition.

The Platform Firmware resume code won't jump to the OS Resume Vector while a command is executing. If it did, the OS driver would be unsure if it should allow the command to complete or abort it.

Figure 12 Firmware Actions for Resume from S3 below illustrates the logic for the SRTM actions when resuming from S3.

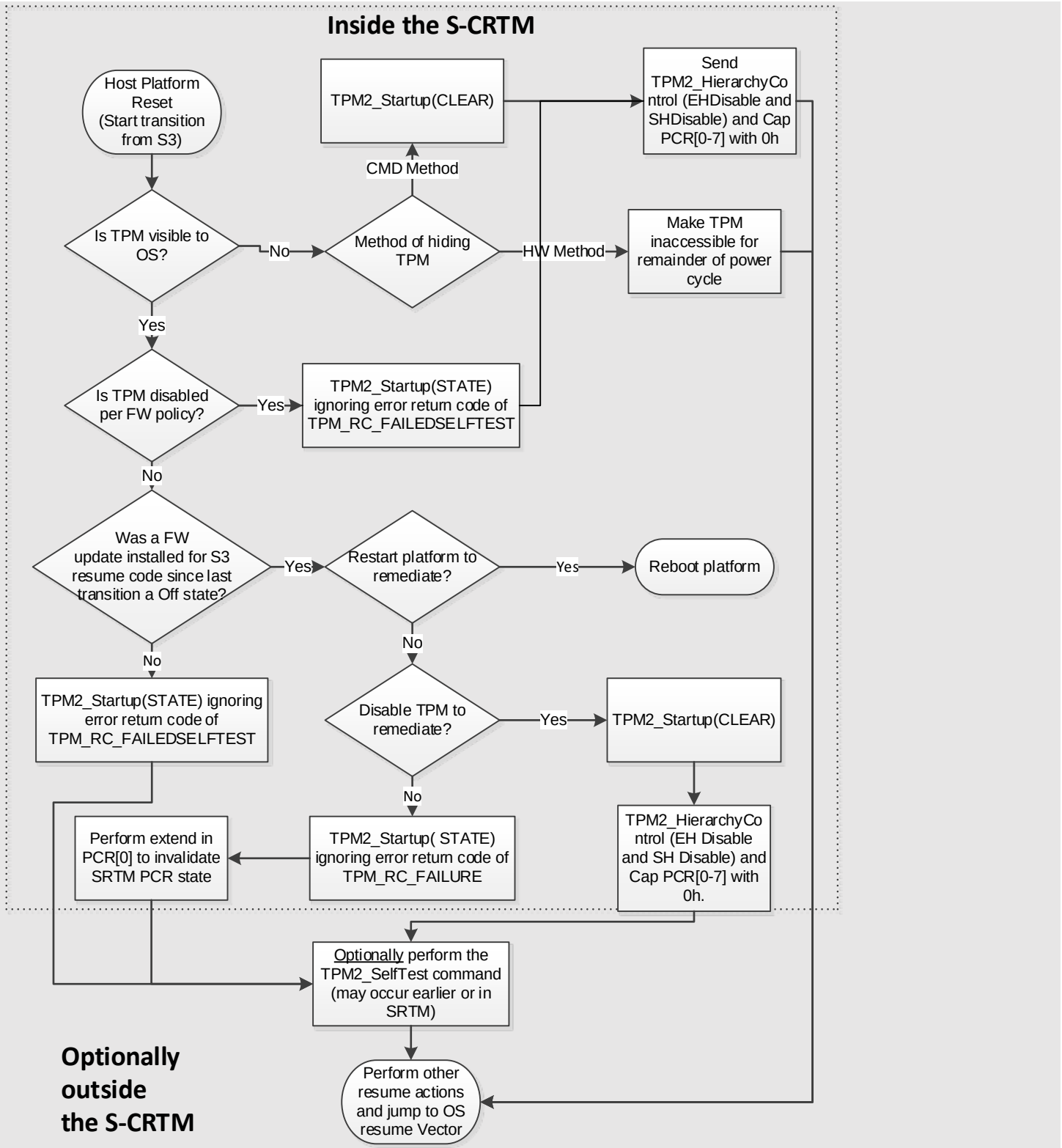

Figure 12 Firmware Actions for Resume from S3

Note: In Figure 12 Firmware Actions for Resume from S3, if the TPM2_Startup(STATE) returns TPM_RC_FAILURE, it isn't necessary to extend an error in PCR[0] to invalidate the SRTM PCR state because the TPM is in failure mode

End of informative comment

1. The Host Platform MUST issue a TPM_INIT.
2. The SRTM MUST issue a TPM2_Startup command.
    a. Under these conditions the SRTM MAY issue a TPM2_Startup(CLEAR) command if:
        i. The TPM is hidden from the OS,
        ii. The TPM is disabled per FW policy, or
        iii. A FW update has occurred for the S2 or S3 portion of the Platform FW since the last transition from S5.
    b. Otherwise, the SRTM MUST issue the TPM2_Startup(STATE) command.
    c. When issuing the TPM2_Startup(STATE), the SRTM SHOULD ignore an error resulting from the TPM entering failure mode (TPM_RC_FAILURE).
3. If the TPM is visible and enabled:
    a. The SRTM MUST be able to determine whether there has been an update to any S3 resume portion of the FW since the previous transition from an Off state. **Note:** The SRTM SHOULD use a method that does not significantly add to the time it takes to resume from S3; in particular, it is not necessary for SRTM to re-measure FW and compare this measurement to the measurement of FW performed at the previous transition from an Off state. The determination of whether FW changed MAY be done using a simple flag.
    b. If the SRTM detects a modification to the S3 resume portion of the FW since the last transition from an Off state, the SRTM MUST either:
        i. Force the Host Platform to reboot, or
        ii. The SRTM MUST:
            1. Issue a TPM2_Startup(CLEAR) command,
            2. Issue a TPM2_HierarchyControl (EH Disable, SH Disable) command, and
            3. Make the contents of PCR[0] invalid by extending the digest of the value 00000001h in PCR[0]. Note: No corresponding event is logged in the event log because the OS may be managing the log.
4. The Firmware MAY issue a TPM2_SelfTest command.
5. If the TPM2_SelfTest command immediately returns success and the TPM performs the Self-Test actions associated with the command asynchronously, the FW MAY launch the OS Resume Vector while the TPM2_SelfTest Self-Test actions are still in progress.

9 ACPI

Start of informative comment

In order to facilitate device discovery and OS driver loading, the platform's ACPI name space contains an appropriate device object for the TPM. This device scope appears in the appropriate bus hierarchy, i.e., within the bus the TPM is on. The TPM device also contains at a minimum, an _HID object, and resource descriptors to claim all hardware resources consumed by the TPM. Note the TPM device for TPM 2.0 is different than 1.2. According to the ACPI Specification (version 5, Errata A, Section 6.1.5 and 6.1.3), a hardware ID or compatibility ID is either a PNP ID with format "AAA####" or ACPI ID with format "NNNN####". The remaining four hexadecimal digits should be set to a value that allows software to differentiate different device classes built by the same manufacturer.

The example below shows a minimal snippet of typical ASL, with a TPM allocated Interrupt 3.

```
Interrupt 3.

Device (TPM) {

      Name (_HID, "MSFT0101")

      Name (_CRS, ResourceTemplate() {

                  Memory32Fixed (ReadWrite, 0xFED40000, 0x5000,)

                  IRQ(Level, ActiveLow, Shared) {3}

      })

}
```

If legacy IO ports or any other hardware resources are decoded by the TPM, they are declared here also. Additionally, an _CID may be included. Per the ACPI specification, an _CID may be a single ID, or may be a package of IDs listed in order of preference.

The example device object below shows a vendor-specific _HID to facilitate loading of a vendor specific driver. The generic TPM 1.2 PNP ID is given in the _CID object. There are also legacy IO ports declared in this example device object. This example TPM is allocated Interrupt 5.

```
Device (TPM) {

       Name (_HID, EISAID("IFX0101"))

       Name (_CID, "MSFT0101")

     Name (_CRS, ResourceTemplate() {

          Memory32Fixed (ReadWrite, 0xFED40000, 0x5000,)

          IRQ(Level, ActiveLow, Shared) {5}

          IO (Decode16, 0x4700, 0x4700, 0x01, 0x0C) //IO runtime 4700h-470Ch

     })

}
```

Some operating systems require all device resources to be claimed in ACPI. When the TPM is hidden, the TPM device object will not be present in ACPI. However, if appropriate, the platform manufacturer may claim the resources used by the TPM through other ACPI descriptions of the platform.

End of informative comment

9.1 ACPI Device Object for TPM

1. A TPM device object MUST contain a _HID object with a value that is formatted according to the ACPI specification:
    a. If the TPM manufacturer has an ACPI-assigned manufacturer ID:
        i. The _HID object SHALL contain the manufacturer ID of the TPM vendor and a unique identifier and is in the format “AAA####” or “NNNN####”, and
        ii. a _CID object with the value of “MSFT0101” or evaluate to a package where the value “MSFT0101” is one of the IDs within the package.
    b. If the TPM manufacturer does not have an ACPI-assigned manufacturer ID, then the _HID object SHALL contain the value of “MSFT0101”.

9.1.1 TPM Visible

While the TPM device is visible:

1. The platform ACPI namespace MUST contain an ACPI device object in an appropriate scope for the TPM according to Section 8.1.
2. When present
    a. The ACPI device object representing the TPM MUST claim all hardware resources consumed by the TPM. **Note:** This includes any legacy IO ports and other hardware resources.
    b. If there are configurable resource options, then the ACPI device object representing the TPM MUST also contain _PRS and _SRS control methods as required by the ACPI specification.

9.1.2 TPM Hidden but Discoverable

1. The platform ACPI namespace MUST contain an ACPI device object in an appropriate scope for the TPM according to Section 8.1.
2. When present
    a. The ACPI device object representing the TPM MUST claim all hardware resources consumed by the TPM. **Note:** This includes any legacy IO ports and other hardware resources.
    b. If there are configurable resource options, then the ACPI device object representing the TPM MUST also contain _PRS and _SRS control methods as required by the ACPI specification.

9.1.3 TPM Hidden and Not Discoverable

While the TPM device is hidden, the platform ACPI namespace MUST NOT contain an ACPI device object for the TPM or MUST return _STA “Not Present”.

9.2 ACPI Table Usage

Start of informative comment

A system’s firmware uses an ACPI table to identify the system’s TCG capabilities to the OS-present environment. The information in this table is not guaranteed to be valid until the Platform Firmware performs the transition from the pre-OS state to OS-Present state.

The presence of the TPM2 ACPI table together with the ACPI device object is the standard mechanism that this specification provides for an operating system to discover if a TPM may be enumerated on the Host Platform. Operating systems should not expect to be able to use a TPM if it is not enumerated in the ACPI device list.

If the TPM device implements an interface that corresponds to the TPM 2.0 Command Response Buffer Interface Specification, the ACPI TPM2 table points to the Control Area memory region.

If the ACPI TPM2 table contains the address and size of the Platform Firmware TCG log, firmware "pins" the memory associated with the Platform Firmware TCG log and reports this memory as "Reserved" memory via the INT 15h/E820 interface. This is done to ensure that the log area contains the TPM2_PCR_Extend operations performed by the most recent system transition from an Off state. If the log were in reclaimable memory, the Platform Firmware would not be able to report the system configuration on the return from hibernation (S4) since the memory would have been reclaimed for other use by the operation system on its last boot from S5.

End of informative comment

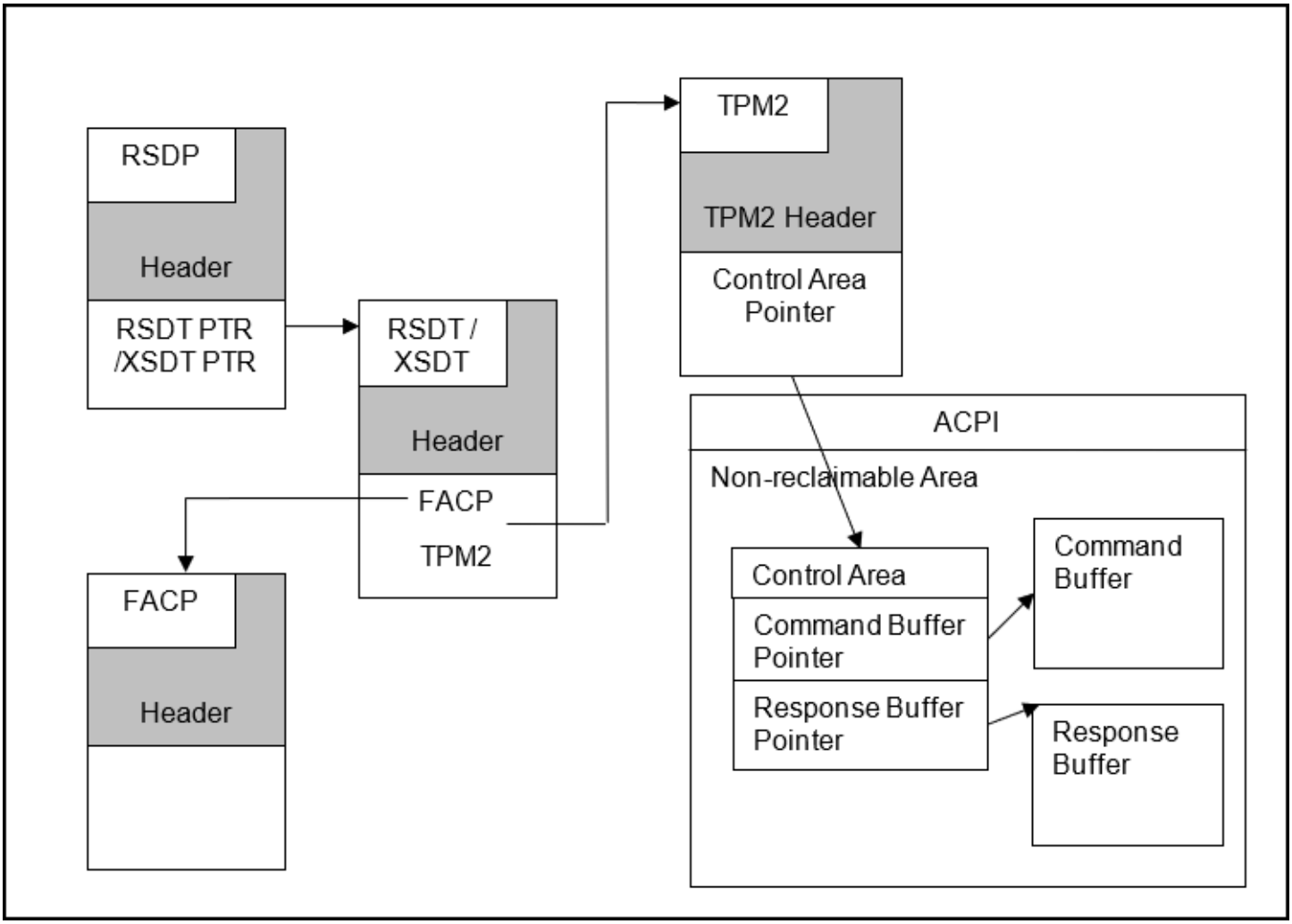

Figure 13 ACPI Table points to CRB Control Area

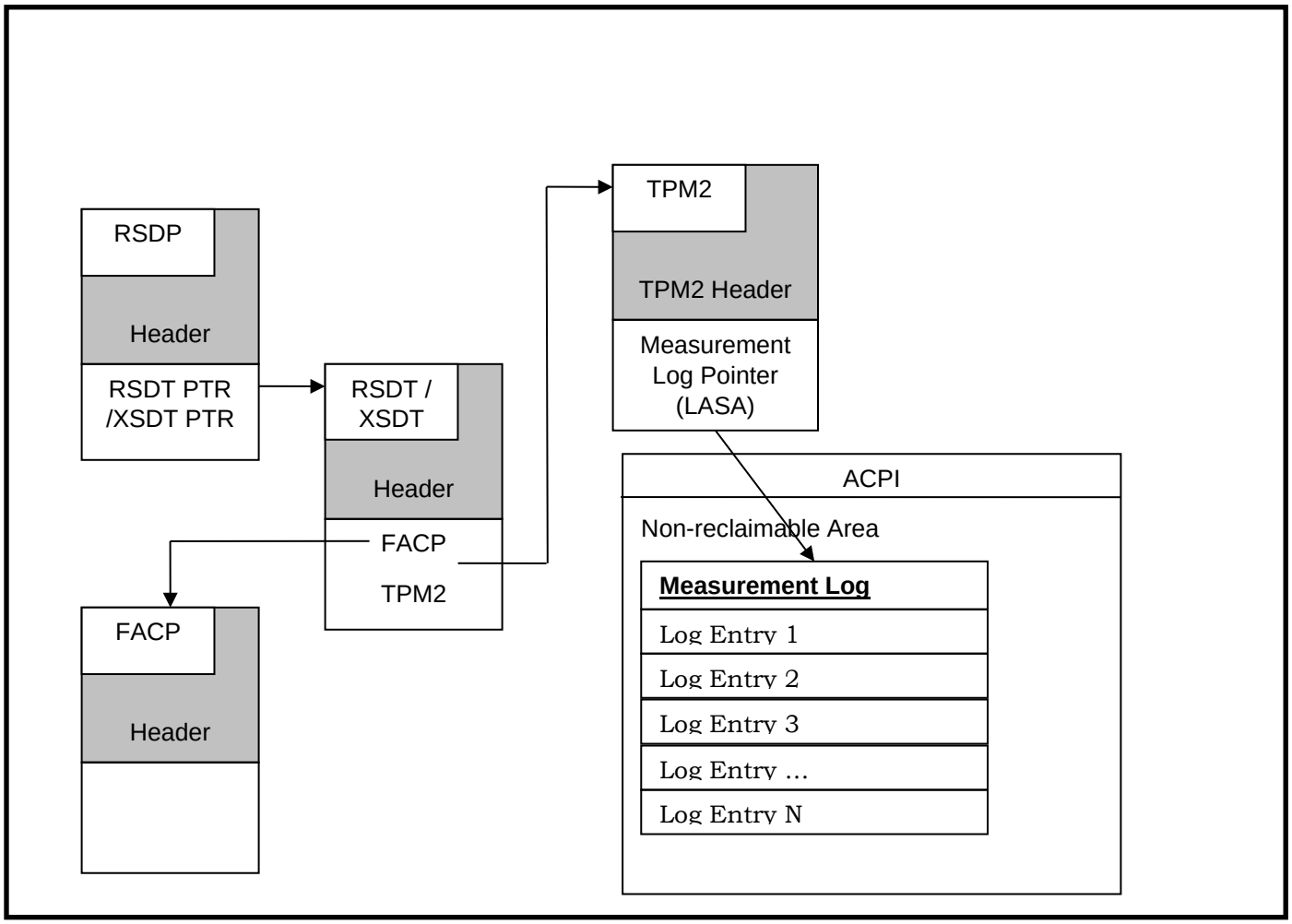

Figure 14 ACPI Table with pointer to log area

1. The ACPI table indicated above as “TPM2” is defined in the TCG ACPI Specification version 1.1 revision 37.
2. The ACPI TPM2 table MUST be listed in the RSDT or XSDT ACPI table if the TPM “is discoverable” per the definition above in Section 7 TPM Discoverability, regardless of whether the TPM is currently visible or not. The pointer to the TPM2 table must be correct and the TPM2 table MUST be populated.
3. The ACPI TPM2 table MUST NOT be listed in the RSDT or XSDT ACPI table if the TPM is “not discoverable” per the definition above in Section 7 TPM Discoverability.
4. If the ACPI TPM2 table contains the address and size of the Platform Firmware TCG log:
    a. The Log Area Minimum Length for the TCG event log MUST be at least 64KB.
    Note: The TCG ACPI specification uses the field name “Log Area Minimum Length”, but the field value is the actual log area length reserved by Platform Firmware, not a lower bound.
    b. The whole memory range of Log Area Start Address (LASA) through LASA + (LAML – 1) MUST be used exclusively to store event log event data. (For example, firmware storage of the location of the last event in the event log must be stored elsewhere.)
    c. The whole memory range of LASA through LASA + (LAML – 1) MUST be initialized to zero by Platform Firmware.

d. The OS MAY use the buffer (LASA through LASA + LAML – 1) to store additional event data or other purposes after calling ExitBootServices, even though the TCG EFI Protocol for adding additional log entries is not available.

9.3 TPM Interrupt Support

Start of informative comment

As usage of the TPM by the OS and by OS-present applications becomes more prevalent, methods of improving TPM performance are increasingly needed. Discrete TPMs have supported interrupts as defined in the PC Client Specific Platform TPM Profile for TPM 2.0, but Platform Firmware has rarely enabled interrupts. Drivers didn't utilize interrupts because the interrupts were not enabled and defined in ACPI by Platform Firmware. This specification requires Platform Firmware to enable interrupts if the TPM supports them. As there is no programmatic method to determine TPM interrupt support, Platform Firmware needs to be preconfigured based on the TPM implemented on the platform. The justification is largely to enhance performance. A TPM driver can, but is not required to, register for an interrupt for command completion rather than polling the TPM interface. The usage is intended for the OS and OS present applications, not for Platform Firmware. An informative example of ASL code, post fix-ups, is contained in the Appendix.

Note: FW- and SOC-based TPM's do not have physical interrupt pins, and thus cannot support interrupts.

End of informative comment

1. If the TPM supports interrupts, Platform Firmware SHALL declare them in the ACPI table and allow configuration.

10 Event Logging

10.1 Introduction

Start of informative comment

The Event Log is the informational record of measurements made to PCRs by the Platform Firmware, with some events not extended to PCRs. The informational events are used to convey valuable but untrusted information to an evaluator of the log. Each measurement made, as well as the information events, are recorded in the event log as individual entries with specified fields (see Section 10.2 TCG Defined Structures). The first event in the event log is the informational event TCG_EfiSpecIdEvent (See Section 10.4.5.1 Specification ID Version Event). As parsers need a way to determine the format of the events in the log, the first event identifies the version of the log, which implies the format of the events, and the number and size of the recorded digests. The layout of the TCG_PCR_EVENT2 structure that is used for the rest of the event log depends on the information in the first event. For that reason, the first event is formatted as TCG_PCClientPCREvent structure, which is inherited from the Conventional Spec (defined in Section 10.2.1 TCG_PCClientPCREvent Structure) and is independent of digest sizes. Previously, the TCG PC Client Implementation Specification for Conventional BIOS and the TCG PC Client EFI Platform Specification mandated the TCG_EfiSpecIdEvent to be the first event in the event log, so this specification provides the version and digest information in the TCG_EfiSpecIdEvent.

PC Client firmware specifications for TPM 1.2 enabled platforms defined an event log that mandated the use of SHA1 to hash event data and extend the 20-byte digest into PCRs. This specification refers to this log format as the SHA1 log format. With TPM 2.0, PCRs may support other hashing algorithms besides SHA1. If Platform Firmware extends digests to PCRs using other hashing algorithms, an event in the event log has to contain all of the recorded digests. This specification refers to this log format as a crypto agile log format or just as the event log.

A single event in the event log always starts with a header that consists of the PCR index, an event type, the recorded digest(s), and a size of the event data. The PCR index identifies the PCR into which one or more digest of the event data has been extended. The event type identifies the type of event that is recorded in the event log entry. The digest(s) field contains the digest values of the hashed information that is extended into the PCR, note that the digests values may occur in any order within an event. Finally, the event data size defines the size of additional event data. For the SHA1 log format, the digest is a fixed size field of 20 bytes. For the crypto agile log format, the digest consists of a TPML_DIGEST_VALUES (defined in the TPM2 Library Specification Part 2) structure. TCG_EfiSpecIDEvent is formatted as a TCG_PCClientPCREvent structure which is inherited from the Conventional Spec (defined in Section 10.2.1 TCG_PCClientPCREvent Structure). Since this event contains the information needed for parsers to deal with the remaining events in the event log, it is placed first.

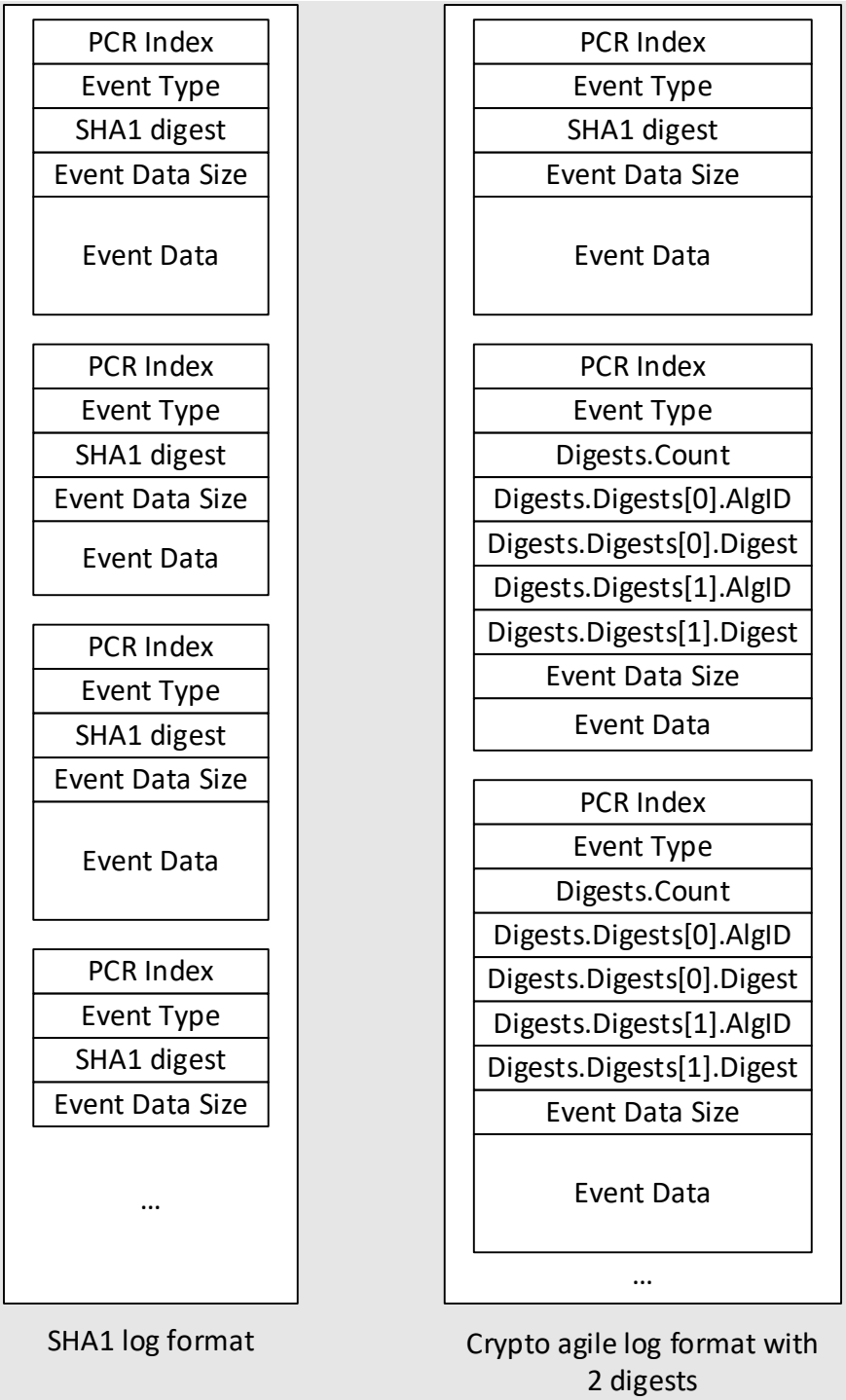

Figure 15 Depiction of Log Formats

Note that for the digests in the crypto agile log format, although the type names from the TPM 2.0 Library Specification are used (e.g., TPML_DIGEST_VALUES), the storage of Integers in the event log is little-endian. In this case, the Count and the AlgID fields, leveraged from the TPM 2.0 Library specification, are stored as little-endian. If Platform Firmware utilizes the TPM2_PCR_Event command to hash the data, this command returns a TPML_DIGEST_VALUES structure encoded in big-endian. Platform Firmware would need to modify the

TPML_DIGEST_VALUES structure so that count and AlgID fields are little-endian before adding the event to the event log. Likewise, if Platform Firmware is preparing the TPML_DIGEST_VALUES structure to send to the TPM using a TPM2_PCR_Extend command, the values must be big-endian for the TPM structure. There may be other instances where this type of re-encoding is required. This specification does not produce an exhaustive list.

An event will be densely packed. That is, even though there can be multiple digests in an event, the algorithm ID of the next digest follows immediately after the last byte of the previous digest, without padding. See the example in Table 7 below for an illustration of the offsets in the event log.

To illustrate the crypto agile log event format, here is an EV_SEPARATOR event as example:

Table 7 Crypto Agile Event Log Event Example 1

Field Name	Offset	Size (in bytes)	Content
PCRIndex	0x00	4	2
eventType	0x04	4	0x00 00 00 04 (EV_SEPARATOR)
Digests	0x08		
Digests.Count	0x08	4	1
Digests.Digests	0x0C		
Digests.Digests[0].AlgID	0x0C	2	0x00 04 (SHA1)
Digests.Digests[0].Digest	0x0E	20	0x90, 0x69 0xca, 0x78, 0xe7, 0x45, 0x0a, 0x28, 0x51, 0x73, 0x43, 0x1b, 0x3e, 0x52, 0xc5, 0xc2, 0x52, 0x99, 0xe4, 0x73
EventSize	0x22	4	4
Event	0x26	4	0x00, 0x00, 0x00, 0x00

The encoding as a byte stream would look as follows. The start of the line describes the offset for the first byte in the line. (All values in hexadecimal.)

0000: 02 00 00 00 04 00 00 00 – 01 00 00 00 04 00 90 69

0010: ca 78 e7 45 0a 28 51 73 – 43 1b 3e 52 c5 c2 52 99

0020: e4 73 04 00 00 00 00 00 – 00 00

Table 8 is the same separator event using two PCR banks:

Table 8 Crypto Agile Event Log Event Example

Field Name	Offset	Size (in bytes)	Content
PCRIndex	0x00	4	2
eventType	0x04	4	0x00 00 00 04 (EV_SEPARATOR)
Digests	0x08		
Digests.Count	0x08	4	2
Digests.Digests	0x0C		
Digests.Digests[0].AlgID	0x0C	2	0x00 04 (SHA1)
Digests.Digests[0].Digest	0x0E	20	0x90, 0x69 0xca, 0x78, 0xe7, 0x45, 0x0a, 0x28, 0x51, 0x73, 0x43, 0x1b, 0x3e, 0x52, 0xc5, 0xc2, 0x52, 0x99, 0xe4, 0x73
Digests.Digests[1].AlgID	0x22	2	0x00 0b (SHA-256)
Digests.Digests[1].Digest	0x24	32	0xdf, 0x3f, 0x61, 0x98, 0x04, 0xa9, 0x2f, 0xdb, 0x40, 0x57, 0x19, 0x2d, 0xc4, 0x3d, 0xd7, 0x48, 0xea, 0x77, 0x8a, 0xdc, 0x52, 0xbc, 0x49, 0x8c, 0xe8, 0x05, 0x24, 0xc0, 0x14, 0xb8, 0x11, 0x19
EventSize	0x44	4	4
Event	0x48	4	0x00, 0x00, 0x00, 0x00

The second example as byte stream looks like this:

0000: 02 00 00 00 04 00 00 00 – 02 00 00 00 04 00 90 69

0010: ca 78 e7 45 0a 28 51 73 – 43 1b 3e 52 c5 c2 52 99

0020: e4 73 0b 00 df 3f 61 98 – 04 a9 2f db 40 57 19 2d

0030: c4 3d d7 48 ea 77 8a dc – 52 bc 49 8c e8 05 24 c0

0040: 14 b8 11 19 04 00 00 00 – 00 00 00 00

Note that the algorithm ID of the SHA-256 digest at offset 34 follows directly after the last meaningful byte of the SHA-1 digest. Also, the event data size field (EventSize) at offset 68 follows directly after the last meaningful byte of the SHA-256 digest.

Information events have an event type of value EV_NO_ACTION and PCR index of zero (See Section 10.4.5 EV_NO_ACTION Event Types). The event data field of an EV_NO_ACTION event may contain different data. To distinguish between these different no action events, each EV_NO_ACTION event data starts with a 16 bytes Signature that defines the format of the event data. The TCG_EfiSpecIdEvent event in the crypto agile event log has a SHA1 log format digest field of 20 bytes of zero (See Section 10.4.5.1 Specification ID Version Event). All other EV_NO_ACTION events in a crypto agile log have digest entries for each recorded hashing algorithm and the digests are set to be all zeros.

The 16 bytes Signature for the TCG_EfiSpecIdEvent as defined in this specification is the NUL-terminated ASCII string “Spec ID Event03”. Based on this string an evaluator can determine that this log is formatted as crypto agile log and that the TCG_EfiSpecIdEvent contains information about the recorded digests. Table 9 TCG_EfiSpecIdEvent Example is an informative example and is not updated with every revision of this specification. If two PCR banks are enabled and platform firmware extends SHA1 and SHA256 digests, the event may look like this (the vendor specific fields may vary):

Table 9 TCG_EfiSpecIdEvent Example

Field Name	Offset	Size(in bytes)	Content
PCRIndex	0x00	4	0x00 00 00 00
eventType	0x04	4	0x00 00 00 03 (EV_NO_ACTION)
Digest	0x08	20	20 bytes of zeros
EventSize	0x1C	4	0x00 00 00 25
Event	0x20	37	TCG_EfiSpecIdEvent
TCG_EfiSpecIdEvent .Signature	0x20	16	“Spec ID Event03”
TCG_EfiSpecIdEvent .platformClass	0x30	4	0x00 00 00 00
TCG_EfiSpecIdEvent .familyVersionMinor	0x34	1	0x00
TCG_EfiSpecIdEvent .familyVersionMinor	0x35	1	0x02
TCG_EfiSpecIdEvent .specRevision	0x36	1	0x6A (106 decimal)

TCG_EfiSpecIdEvent .uintnSize	0x37	1	0x02 (UINT64)
TCG_EfiSpecIdEvent .NumberOfAlgorithms	0x38	4	0x00 00 00 02 (note, this is the Count field)
TCG_EfiSpecIdEvent .digestSizes[0].AlgId	0x3C	2	0x00 04 (SHA1)
TCG_EfiSpecIdEvent .digestSizes[0].DigestSize	0x3E	2	20
TCG_EfiSpecIdEvent .digestSizes[1].AlgId	0x40	2	0x00 0b (SHA256)
TCG_EfiSpecIdEvent .digestSizes[1].DigestSize	0x42	2	32
TCG_EfiSpecIdEvent .vendorInfoSize	0x44	1	0x00

End of informative comment

1. Platform Firmware SHALL create an event log.
    a. This event log SHALL contain all events measured by Platform Firmware.
2. Platform Firmware SHALL create a secondary event log of type EFI_TCG2_FINAL_EVENTS_TABLE, as defined in the TCG EFI Protocol Specification v1.0 revision 9 Section 7 (Log Entries after Get Event Log Service).
    a. This event log SHALL contain all events measured as defined in the TCG EFI Protocol Specification.
3. The first event log entry SHALL be a TCG_PCClientPCREvent structure. See Section 10.2.1 TCG_PCClientPCREvent Structure.
4. The first log entry in the measurement log MUST start at the Measurement Log Start Address. Subsequent measurements MUST be contiguous.
5. Event log entries after the first entry SHALL be TCG_PCR_EVENT2 structures. See Section 10.2.2 TCG_PCR_EVENT2 Structure.
6. For each Hash algorithm enumerated in the TCG_PCClientPCREvent entry, there SHALL be a corresponding digest in all TCG_PCR_EVENT2 structures. **Note:** This includes EV_NO_ACTION events which do not extend the PCR.
7. Sizes for all Hash algorithms included in a TCG_PCR_EVENT2 structure SHALL be enumerated in the TCG_PCClientPCREvent structure.
8. There SHALL be a Hash algorithm in the TCG PCClientPCREvent structure for all allocated PCR banks.
9. There MAY be Hash algorithms in the TCG_PCClientPCREvent structure that do not correspond to allocated PCR banks. See Section 12 Predictive Event Logs.
10. There SHALL NOT be padding between event log entries.
11. All integer structure members SHALL be marshaled in little-endian format.

10.2 TCG Defined Structures

Start of informative comment

Some of the data structures used here are derived from UEFI data structures. To simplify parsing of the UEFI data structures, members of type UINTN have been replaced by members of type UINT64.

End of informative comment

10.2.1 TCG_PCClientPCREvent Structure

Start of informative comment

This structure is inherited from the TCG PC Client Implementation Specification for Conventional BIOS. It is only used for the Specification Id Event structure defined in Section 10.4.5.1 Specification ID Version Event.

End of informative comment

```
typedef tdTCG_PCClientPCREvent {
      UINT32                pcrIndex;
      UINT32                eventType;
      BYTE                  digest[20];
      UINT32                eventDataSize;
      BYTE                  event[eventDataSize];
} TCG_PCClientPCREvent;
```

Table 10 TCG_PCClientPCREvent Structure

Type	Name	Description
UINT32	pcrIndex	PCR 0
UINT32	eventType	SHALL be an EV_NO_ACTION event, see Section 10.4.1 Event Types.
BYTE[20]	digest	SHALL be 20 Bytes of 0x00
UINT32	eventDataSize	The size of the event
BYTE[eventDataSize]	event	SHALL be a TCG_EfiSpecIdEvent, see Section 10.4.5.1 Specification ID Version Event

10.2.2 TCG_PCR_EVENT2 Structure

Start of informative comment

Each “Measurement Log” entry shown in Figure 13 ACPI Table points to CRB Control Area is an instance of this TCG_PCR_EVENT2 structure, except for the Specification ID Version Event, which uses a TCG_PCClientPCREvent{} structure (see Section 10.2.1 TCG_PCClientPCREvent Structure) to allow software that is not TPM 2.0 aware to parse the log. For software parsing the event log, the parser can choose an arbitrary maximum size, but this specification recommends a maximum value for the TCG_PCR_EVENT2.eventSize field of 1MB.

Structures defined in the remaining section of Section 10.2 are used to populate the TCG_PCR_EVENT2 event field.

End of informative comment

Firmware SHALL construct a list of digests for all enabled PCR banks using the following structure:

```
typedef tdTPML_DIGEST_VALUES {
      UINT32      count;
      TPMT_HA     digests[count];
} TPML_DIGEST_VALUES;
```

Table 11 TPML_DIGEST_VALUES Structure

Type	Name	Description
UINT32	count	The number of digests in the list
TPMT_HA	digests	The list of tagged digests, as sent to the TPM as part of a TPM2_PCR_Extend or as received from a TPM2_PCR_Event command

Each entry in the Event Log, except the TCG_EfiSpecIdEvent{} structure which is the first event (see Section 10.4.5.1 Specification ID Version Event), SHALL use the following structure:

```
typedef struct tdTCG_PCR_EVENT2{
      UINT32                      pcrIndex;
      UINT32                      eventType;
      TPML_DIGEST_VALUES          digests;
      UINT32                      eventSize;
      BYTE                        event[eventSize];
} TCG_PCR_EVENT2;
```

Table 12 TCG_PCR_EVENT2 Structure

Type	Name	Description
UINT32	pcrIndex	The PCR Index to which this event was extended.
UINT32	eventType	Defined in Section 10.4.1 Event Types.
TPML_DIGEST_VALUES	digests	A counted list of tagged digests, which contain the digest of the event data (or external data) for all active PCR banks.
UINT32	eventSize	The size of the event data.

BYTE	event	The data of the event.

10.2.3 UEFI_IMAGE_LOAD_EVENT Structure

Start of informative comment

This structure is used in measuring a PE/COFF image. This structure is used to construct the event data for the events EV_EFI_BOOT_SERVICES_APPLICATION and EV_EFI_BOOT_SERVICES_DRIVER. The structure members are defined in the UEFI Specification. The typographic convention for structure member names follows the convention defined in the UEFI specification, not the TCG specification. See Section 8.2.4 Measuring OS Boot Events for the procedure for measuring Boot events. If LoadImage is called to load an image that platform firmware has already decompressed into memory, the DevicePath field in this structure is empty.

The ImageLocationInMemory reflects the location in memory at the time the platform firmware loaded the Image. Once the OS assumes control of the platform, the Image location may be modified and thus the event would not reflect the correct location. If that happens, platform firmware reports zeros for ImageLocationInMemory or any other structure member that is inconsistent.

End of informative comment

```
typedef struct tdUEFI_IMAGE_LOAD_EVENT {
      EFI_PHYSICAL_ADDRESS   ImageLocationInMemory; // PE/COFF image
      UINT64                 ImageLengthInMemory;
      UINT64                 ImageLinkTimeAddress;
      UINT64                 LengthOfDevicePath;
      BYTE                   DevicePath[LengthOfDevicePath];
                                   // See UEFI spec Section EFI Device Path Protocol
                                   // for the encodings for DevicePath.
} UEFI_IMAGE_LOAD_EVENT;
```

If platform firmware cannot guarantee consistency of `ImageLocationInMemory` of the `UEFI_IMAGE_LOAD_EVENT` structure across boot cycles, platform firmware SHALL replace the field with zeros.

10.2.4 Measuring Industry Standard Tables and Data Structures

Start of informative comment

A UEFI platform may support several industry-standard tables and data structures. These include, but are not limited to, ACPI, SMBIOS, and so on. To enable Verifiers to understand what table is measured per event, a new structure has been defined in this revision of the specification. Platform Firmware implementers are recommended to use the new structure. The UEFI_HANDOFF_TABLE_POINTERS structure has been deprecated. Platform Firmware should not mix the structures in the same event log.

The UEFI Specification states the following: “UEFI_CONFIGURATION_TABLE must be:

```
typedef struct {
```

```
            EFI_GUID          VendorGuid;
            VOID              *VendorTable;
}UEFI_CONFIGURATION_TABLE;”
```

The ***VendorTable pointer*** may not reflect the correct location in memory once the OS assumes control of the boot process. In this case, platform firmware should populate the ***VendorTable pointer*** field with zeros.

End of informative comment

All industry standard tables measured by platform firmware SHALL use UEFI_HANDOFF_TABLE_POINTERS2.

```
typedef struct tdUEFI_HANDOFF_TABLE_POINTERS2 {
UINT8                              TableDescriptionSize;
BYTE[TableDescriptionSize]         TableDescription;
UINT64                             NumberOfTables;
UEFI_CONFIGURATION_TABLE           TableEntry[NumberOfTables];
} UEFI_HANDOFF_TABLE_POINTERS2;
```

Where TableDescription is a Platform-Manufacturer-defined printable, 8-bit ASCII string.

If platform firmware cannot guarantee consistency of `*VendorTable` of the `UEFI_HANDOFF_TABLE_POINTERS2` structure across boot cycles, platform firmware SHALL replace the field with zeros.

Start of informative comment:

This structure has been deprecated.

typedef struct tdUEFI_HANDOFF_TABLE_POINTERS {

UINT64 NumberOfTables;

UEFI_CONFIGURATION_TABLE TableEntry[NumberOfTables];

} UEFI_HANDOFF_TABLE_POINTERS;

End of informative comment

10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definitions

Start of informative comment

The original UEFI_PLATFORM_FIRMWARE_BLOB structure as defined does not provide information enabling a Verifier to understand the measurement and has been deprecated. This revision of the PFP defines a new structure: UEFI_PLATFORM_FIRMWARE_BLOB2 which adds a Platform-Manufacturer-defined string to describe the measurement. The UEFI_PLATFORM_FIRMWARE_BLOB structure may be deprecated in a future revision. The EFI_PHYSICAL_ADDRESS may represent a physical address in a non-volatile memory location, or it may represent a memory mapped address. For example, if the code to be measured is present in a SPI flash device which permits execution, then the address would be the physical address in the SPI flash device where execution occurs. If the code is present in an NVME device which does not permit execution, then the address would be the memory location where the code is loaded. **Note**: This structure can be used for non-EFI code.

End of informative comment

1. When the CRTM measures the Platform Firmware from system board ROM that loads and launches UEFI Boot Services and UEFI Run Time Services, the CRTM MUST measure the code contained in the Host Platform system board firmware.
2. The CRTM MUST also add an entry of event type EV_S_CRTM_CONTENTS to the Event Log (see Table 7-1) to record the measurement from step 1.
3. Other system board code beyond CRTM content MUST also be measured using event type EV_POST_CODE or EV_POST_CODE2;
    a. If the event type is EV_POST_CODE, the event field SHALL contain a UEFI_PLATFORM_FIRMWARE_BLOB structure or ASCII strings, see Table 13 EV_POST_CODE2 Strings and Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers.
    b. If the event type is EV_POST_CODE2, the event field SHALL contain a UEFI_PLATFORM_FIRMWARE_BLOB2 structure.
4. Below is the definition of the UEFI_PLATFORM_FIRMWARE_BLOB2 structure that the CRTM MUST put into the Event Log entry TCG_PCR_EVENT2.event field for event types EV_POST_CODE2, EV_S_CRTM_CONTENTS, and EV_EFI_PLATFORM_FIRMWARE_BLOB2.
    a. For event type EV_POST_CODE2, the strings defined in Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers and Table 13 EV_POST_CODE2 Strings SHALL be used. The string SHALL be encoded in printable, 8-bit ASCII in the BlobDescription field and SHALL NOT be NUL Terminated.
    b. For event type EV_EFI_PLATFORM_FIRMWARE_BLOB2, the BlobDescription SHALL be encoded in printable, 8-bit ASCII. The strings are vendor defined.

```
typedef struct tdUEFI_PLATFORM_FIRMWARE_BLOB2 {
UINT8                          BlobDescriptionSize;
BYTE[BlobDescriptionSize]      BlobDescription;
EFI_PHYSICAL_ADDRESS           BlobBase;
UINT64                         BlobLength;
} UEFI_PLATFORM_FIRMWARE_BLOB2;
```

If Platform Firmware cannot guarantee that BlobBase will be the same after the OS loads as it is in pre-OS, Platform Firmware SHALL zeroize BlobBase. **Note:** This situation may occur for addresses that are mapped to the file system and not to the SPI flash, or that are not memory mapped.

Start of Informative comment:

This structure is used only for EV_POST_CODE

End of informative comment

```
typedef struct tdUEFI_PLATFORM_FIRMWARE_BLOB {
EFI_PHYSICAL_ADDRESS     BlobBase;
UINT64                   BlobLength;
} UEFI_PLATFORM_FIRMWARE_BLOB;
```

Table 13 EV_POST_CODE2 Strings

EV_POST_CODE2 Strings	Used for
POST CODE	BIOS code
SMM CODE	SMM code
ACPI DATA	ACPI data prior to fix ups
BIS CODE	BIS Code
Embedded UEFI Driver	UEFI Drivers embedded in platform firmware

10.2.6 Measuring UEFI Variables and UEFI GPT Data

Start of informative comment

This section defines the event data structures associated with the measurement of UEFI variables.

EFI variables are key/value pairs that consist of identifying information with attributes (the key) and arbitrary data (the value) used to store information passed between the platform and UEFI applications such as OS loaders. See section 7.2 of the UEFI Specification for more information.

The only UEFI variables requiring measurement are those that effect boot policy (that is, where to get the OS Loader), and if UEFI Secure Boot is enabled those that affect the UEFI Secure Boot policy as described in Section 3.3.4.8 PCR[7] – Secure Boot Policy Measurements. Other UEFI variables, such as language, or private variables (that is, GUIDs not defined in the UEFI Specification) are measured into the appropriate PCR, depending upon usage (that is, into PCR[1], PCR[3], or PCR[5]); for more information, see Sections 3.3.4.2 PCR[1] – Host Platform Configuration, 3.3.4.4 PCR[3] – EFI Driver/Application Configuration, and 3.3.4.6 PCR[5] – Boot Manager Configuration and Data of this specification. Even for boot variables that point to the same UEFI application but with different optional data, the measurements are distinct as the measurement will cover the entire UEFI_LOAD_OPTION.

In PFP version 1.05 and earlier versions, the definition of the UEFI_GPT_DATA structure did not account for instance specific GUIDs. This oversight made PCR 5 difficult to verify unless a Verifier performs an event-by-event verification or the platform OEM provides a RIM. In PFP 1.06, the GUIDs are zeroed to ensure PCR 5 does not contain instance specific information.

EFI, unlike conventional BIOS, does not need active partition table flags to dictate which OS loader to choose. The OS loader choice is mediated by the UEFI boot options in variables. But the disk partition topology is still important to reflect the system configuration. This configuration information is contained in a UEFI_GPT_DATA structure. The event type EV_EFI_GPT_EVENT designates the measurement of this on-disk geometry, and the event log data structure is described below.

An example of an EFI Variable, UEFI_IMAGE_SECURITY_DATABASE_GUID /EFI_IMAGE_SECURITY_DATABASE1 variable (the DBX), that would be captured by this event type is available at https://uefi.org/revocationlistfile. For an informative example demonstrating how an EFI Variable would be formatted in a TCG_PCR_EVENT2 structure that contains the UEFI_VARIABLE_DATA structure, see Annex B: UEFI Variable Example.

EV_EFI_VARIABLE_DRIVER_CONFIG may be used to record measurements of vendor-specific data, in which case verification of these events is difficult without published information by the vendor or a RIM.

End of informative comment

For Event types = EV_EFI_VARIABLE_DRIVER_CONFIG, EV_EFI_VARIABLE_BOOT2, and EV_EFI_VARIABLE_BOOT, the event log entries share a common data structure, namely UEFI_VARIABLE_DATA. EV_EFI_VARIABLE_DRIVER_CONFIG is used to designate the measurement of any UEFI variable.

For Event type EV_EFI_VARIABLE_AUTHORITY, the event log entry uses the UEFI_VARIABLE_DATA structure below, but the specific values placed in each structure member are defined in Section 3.3.4.8 PCR[7] – Secure Boot Policy Measurements.

```
typedef struct tdUEFI_VARIABLE_DATA {
  UEFI_GUID   VariableName;
  UINT64      UnicodeNameLength;
  UINT64      VariableDataLength;
  CHAR16      UnicodeName[];
  INT8        VariableData[];   // Driver or platform-specific data
} UEFI_VARIABLE_DATA;
```

Table 14 UEFI_VARIABLE_DATA

Type	Name	Description
UEFI_GUID	VariableName	The VendorGUID parameter in the GetVariable() API. See the UEFI Specification.
UINT64	UnicodeNameLength	The Length in CHAR16 of the Unicode name of the variable.
UINT64	VariableDataLength	The size in bytes of the variable data. See the UEFI specification, GetVariable() API – DataSize.
CHAR16[UnicodeNameLength]	UnicodeName	The CHAR16 Unicode name of the variable without NULL-terminator. The VariableName parameter in the GetVariable() API. See the UEFI Specification
INT8[VariableDataLength]	VariableData	The Data parameter of the EFI variable in the GetVariable() API. See the UEFI specification.

For the boot variable measurement, the data to be recorded are precisely described in the UEFI specification. Specifically, there are event types EV_EFI_VARIABLE_BOOT and EV_EFI_VARIABLE_BOOT2. Platform firmware MUST measure BootOrder and UEFI Boot#### variables. The boot variable measurement uses the UEFI_VARIABLE_DATA event structure.

The disk partition topology MUST be measured using the UEFI_GPT_DATA structure below and recorded in event type EV_EFI_GPT_EVENT2. EV_EFI_GPT_EVENT may contain instance specific data and SHOULD NOT be used.

```
typedef struct tdUEFI_GPT_DATA {
  UEFI_PARTITION_TABLE_HEADER UEFIPartitionHeader;
  UINT64                      NumberOfPartitions;
  UEFI_PARTITION_ENTRY        Partitions [NumberOfPartitions];
} UEFI_GPT_DATA;
```

Table 15 UEFI_GPT_DATA

Type	Name	Description
UEFI_PARTITION_TABLE_HEADER	UEFIPartitionHeader	The GPT Partition Header See the UEFI specification, GPT Header.
UINT64	NumberOfPartitions	The number of the GPT partitions in the partition array. This corresponds to the GPT Header NumberOfPartitonsEntires. See the UEFI Specification GPT Header.
UEFI_PARTITION_ENTRY [NumberOfPartitions]	Partitions	The array of the GPT Partition Entry. See the UEFI specification, GPT Partition Entry Array.

10.2.7 DEVICE_SECURITY_EVENT_DATA Structures

Start of informative comment

This section defines the event structure for measurements associated with an add-in device, e.g. a PCIE adapter or NVME storage device, that supports the "GET_MEASUREMENTS" functionality defined in the DMTF SPDM Specification. This functionality enables a caller to query a device for the measurements of embedded firmware code and configuration.

PFP 1.06 adds new structures that support verification by platform firmware of the SPDM responder certificates and message signatures. The new structures are defined in Section 10.2.7.3 Authenticated SPDM Structures. The structures originally defined in PFP 1.05 are present in Section 10.2.7.2 Unauthenticated SPDM Structures. The common structures that are used for both the authenticated and unauthenticated measurements are defined in Section 10.2.7.1 Common Structures.

The DEVICE_SECURITY_EVENT_DATA_HEADER structure contains the measurement(s) and hash algorithm (SpdmHashAlg) identifier returned by the SPDM "GET_MEASUREMENTS" function without modification. The DeviceType field is populated based on which device layer is used to exchange the SPDM message. If the SPDM message is exchanged via the PCI protocol defined in the PCI specification, the PCI device context is used. If the SPDM message is exchanged via a USB protocol as defined in the USB specifications, the USB device context is used. If a PCI-USB card is present in the system, the card device path is presented as /PciHostBridge/PciBridge/PCIUsbController. Assuming the system uses the PCI protocol to exchange the SPDM messages with this card, the PCI device context should be used.

The SPDM responder may have multiple measurement blocks. The BIOS may use one SPDM_GET_MEASUREMENT command to retrieve all measurement blocks. The BIOS needs to extend each measurement block separately and create an event for each measurement block, because the different measurement blocks may go into different PCRs. See Sections 3.3.4.3 PCR[2]-UEFI Drivers and UEFI Applications,

3.3.4.4 PCR[3] – EFI Driver/Application Configuration, and 10.4.1 Event Types for a description of which types of SPDM measurements map to which PCRs. For example, if the device returns SHA384(DeviceFirmware) and SHA384(DeviceHardwareConfiguration), the BIOS extends SHA384(DeviceFirmware) to PCR2 and creates the event. Then the BIOS needs to extend SHA384(DeviceHardwareConfiguration) to PCR3 and create another event. BIOS needs to ensure that events are measured in the same order across boot cycles.

End of informative comment

10.2.7.1 Common Structures

```
typedef struct tdDEVICE_SECURITY_EVENT_DATA_PCI_CONTEXT {
  UINT16  Version;
  UINT16  Length;
  UINT16  VendorId;
  UINT16  DeviceId;
  UINT8   RevisionID;
  UINT8   ClassCode[3];
  UINT16  SubsystemVendorID;
  UINT16  SubsystemID;
} DEVICE_SECURITY_EVENT_DATA_PCI_CONTEXT;
```

Table 16 DEVICE_SECURITY_EVENT_DATA_PCI_CONTEXT Definition

Type	Name	Description
UINT16	Version	Version of this structure, SHALL be “0”
UINT16	Length	Length of this structure
UINT16	VendorId	PCI VendorId
UINT16	DeviceId	PCI DeviceId
UINT8	RevisionId	PCI RevisionId
UINT8[3]	ClassCode	PCI ClassCode
UINT16	SubsystemVendorId	PCI SubsystemVendorId
UINT16	SubsystemId	PCI SubsystemId

```
Typedef struct tdDEVICE_SECURITY_EVENT_DATA_USB_CONTEXT {
UINT16        Version;
UINT16        Length;
USB_DEVICE_DESCRIPTOR    DeviceDescriptor;
USB_BOS_DESCRIPTOR       BosDescriptor;
```

```
USB_CONFIG_DESCRIPTOR    Configuration1Descriptor;
USB_CONFIG_DESCRIPTOR    Configuration2Descriptor;
…                    ...;
USB_CONFIG_DESCRIPTOR    ConfigurationNDescription;
} DEVICE_SECURITY_EVENT_DATA_USB_CONTEXT;
```

Table 17 DEVICE_SECURITY_EVENT_DATA_USB_CONTEXT Description

Type	Name	Description
UINT16	Version	Version of this structure, SHALL be “0”
UINT16	Length	Length of this structure
USB_DEVICE_DESCRIPTOR	DeviceDescriptor	USB Device Descriptor, as defined by USB specification. The number of the configuration is included in the Device Descriptor.
USB_BOS_DESCRIPTOR	BosDescriptor	Complete BOS Descriptor (if present), as defined by USB specification.
USB_CONFIG_DESCRIPTOR	Configuration1Descriptor	Complete Configuration 1 Descriptor, as defined by USB specification.
USB_CONFIG_DESCRIPTOR	Configuration2Descriptor	Complete Configuration 2 Descriptor (if present)
…	…	…
USB_CONFIG_DESCRIPTOR	ConfigurationNDescriptor	Complete Configuration n Descriptor (if present)

```
typedef union tdDEVICE_SECURITY_EVENT_DATA {
  DEVICE_SECURITY_EVENT_DATA_PCI_CONTEXT         PciContext;
  DEVICE_SECURITY_EVENT_DATA_USB_CONTEXT         UsbContext;
} DEVICE_SECURITY_EVENT_DATA_DEVICE_CONTEXT;
```

Table 18 DEVICE_SECURITY_EVENT_DATA_DEVICE_CONTEXT Definition

Type	Name	Description
DEVICE_SECURITY_EVENT_DATA_PCI_CONTEXT	PciContext	PCI Context.
DEVICE_SECURITY_EVENT_DATA_USB_CONTEXT	UsbContext	USB Context.

10.2.7.2 Unauthenticated SPDM Structures

Start of informative comment

If the SPDM device lacks a Certificate, Platform Firmware uses these structures to record the SPDM supplied measurements. If the SPDM device has a Certificate, Platform Firmware uses the Authenticated SPDM structures.

End of informative comment

```
typedef struct tdDEVICE_SECURITY_EVENT_DATA_HEADER {
  UINT8                          Signature[16];
  UINT16                         Version;
  UINT16                         Length;
  UINT32                         SpdmHashAlg;
  UINT32                         DeviceType;
  SPDM_MEASUREMENT_BLOCK         SpdmMeasurementBlock;
  UINT64                         DevicePathLength;
  UINT8                          DevicePath[DevicePathLength];
                                // See UEFI spec for
                                // the encodings.

} DEVICE_SECURITY_EVENT_DATA_HEADER;
```

Table 19 DEVICE_SECURITY_EVENT_DATA_HEADER Definition

Type	Name	Description
UINT8 [16]	Signature	The NUL-terminated ASCII String “SPDM Device Sec” SHALL be: {0x53 0x50 0x44 0x4d 0x20 0x44 0x65 0x76 0x69 0x63 0x65 0x20 0x53 0x65 0x63 0x00}
UINT16	Version	Version of this structure, SHALL be “1”
UINT16	Length	Length of the entire DEVICE_SECURITY_EVENT_DATA structure
UINT32	SpdmMeasurementHashAlg	The SpdmMeasurementHashAlg returned from the device, as defined in the SPDM 1.1 Specification – 10.4 NEGOTIATE_ALGORITHMS request and ALGORITHMS response messages, table Successful ALGORITHMS response message format, field “MeasurementHashAlgo”

UINT32	DeviceType	0 – No Device Context 1 – PCI Device Context 2 – USB Device Context 3-0x7FFFFFFF – reserved for TCG future use 0x80000000 – 0xFFFFFFFF – reserved for OEM
SPDM_MEASUREMENT_BLOCK	SpdmMeasurementBlock	The SPDM measurement block header returned from the device, as defined by the DMTF SPDM Specification
UINT64	DevicePathLength	Length of the DevicePathField.
UINT8 [DevicePathLength]	DevicePath	The UEFI DevicePath of the device.

```
typedef struct tdDEVICE_SECURITY_EVENT_DATA {
  DEVICE_SECURITY_EVENT_DATA_HEADER         EventDataHeader;
  DEVICE_SECURITY_EVENT_DATA_DEVICE_CONTEXT DeviceContext;
} DEVICE_SECURITY_EVENT_DATA;
```

Table 20 DEVICE_SECURITY_EVENT_DATA Definition

Type	Name	Description
DEVICE_SECURITY_EVENT_DATA_HEADER	EventDataHeader	The header of the event data.
DEVICE_SECURITY_EVENT_DATA_DEVICE_CONTEXT	DeviceContext	The device context of the event data.

10.2.7.3 Authenticated SPDM Structures

```
typedef struct tdDEVICE_SECURITY_EVENT_DATA_HEADER2{
  UINT8                         Signature[16];
  UINT16                        Version;
  UINT8                         AuthState;
  UINT8                         Reserved;
  UINT32                        Length;
  UINT32                        DeviceType;
```

```
  UINT32                          SubHeaderType;
  UINT32                          SubHeaderLength;
  UINT64                          SubHeaderUID;
  UINT64                          DevicePathLength;
  UINT8                           DevicePath[DevicePathLength];
} DEVICE_SECURITY_EVENT_DATA_HEADER2;
```

Table 21 DEVICE_SECURITY_EVENT_DATA_HEADER2 Definition

Type	Name	Description
UINT8 [16]	Signature	The NUL-terminated ASCII String “SPDM Device Sec2” SHALL be: {0x53 0x50 0x44 0x4d 0x20 0x44 0x65 0x76 0x69 0x63 0x65 0x20 0x53 0x65 0x63 0x32}
UINT16	Version	Version of this structure, SHALL be “2”
UINT8	AuthState	0: AUTH_SUCCESS. The digital signature of the data is valid. The public key certificate chain is validated with the entry in in the UEFI device signature variable. The data from the device is recorded in this structure. 1: AUTH_NO_AUTHORITY. The digital signature of the data is valid, but the public key certificate chain is not validated with the entry in in the UEFI device signature variable. The data from the device is recorded in this structure. 2: AUTH_NO_BINDING. The digital signature of the measurement data is valid, but the reported device capabilities, negotiated parameters or certificate chains were not validated by a transcript. For example, the device does not support CHALLENGE_AUTH but can support MEASUREMENT with signature. The data from the device is recorded in this structure. 3: AUTH_FAIL_NO_SIG. The data has no digital signature. For example, the device has no signing capability. The data from the device is not recorded in this structure because it is not verifiable. SubHeaderLength SHALL be zero.

		4: AUTH_FAIL_INVALID. The data is invalid. For example, the certificate chain is invalid, or the digital signature of the data is invalid. The data from the device is not recorded in this structure because it is not valid. SubHeaderLength SHALL be zero. 5-0xFE: reserved for TCG future use 0xFF: AUTH_NO_SPDM. Device is not an SPDM-capable device. The data from the device may be returned. If the data is returned, the data SHALL use OEM SubHeaderType.
UINT8	Reserved	SHALL be zero.
UINT32	Length	Length in bytes of the entire DEVICE_SECURITY_EVENT_DATA2 structure, including device path, sub header and device context.
UINT32	DeviceType	0: No Device Context 1: PCI Device Context 2: USB Device Context 3-0x7FFFFFFF: reserved for TCG future use 0x80000000-0xFFFFFFFF: reserved for OEM
UINT32	SubHeaderType	0: SPDM Measurement Block 1: SPDM Cert Chain 2-0x7FFFFFFF: Reserved for TCG future use 0x80000000-0xFFFFFFFF: Available for OEM Use
UINT32	SubHeaderLength	Length in bytes of the sub header structure. If the SubHeaderType is unknown to the verifier, the verifier can use this to skip the sub header and parse the device context.
UINT64	SubHeaderUID	A universal identifier (UID) assigned by the event log creator. The UID can be used to bind two sub header structures together. For example, instance specific data extended into an NV Extend index and class data extended into a PCR that

		are for the same SPDM responder should be linked by the same UID.
UINT64	DevicePathLength	Length in bytes of the DevicePathField.
UINT8 [DevicePathLength]	DevicePath	The UEFI DevicePath of the device.

```
typedef union tdDEVICE_SECURITY_EVENT_DATA_SUB_HEADER {
  DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_MEASUREMENT_BLOCK
   SpdmMeasurementBlock;
  DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_CERT_CHAIN SpdmCertChain;
  DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_OEM_MEASUREMENT OemMeasurement;
} DEVICE_SECURITY_EVENT_DATA_SUB_HEADER;
```

Table 22 DEVICE_SECURITY_EVENT_DATA_SUB_HEADER Definition

Type	Name	Description
DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_MEASUREMENT_BLOCK	SpdmMeasurementBlock	SPDM Measurement Block
DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_CERT_CHAIN	SpdmCertChain	SPDM Certificate Chain
DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_OEM_MEASUREMENT	OemMeasurement	OEM specific Measurement data

```
typedef struct tdDEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_MEASUREMENT_BLOCK {
  UINT16                        SpdmVersion;
  UINT8                         SpdmMeasurementBlockCount;
  UINT8                         Reserved;
  UINT32                        SpdmMeasurementHashAlgo;
  SPDM_MEASUREMENT_BLOCK        SpdmMeasurementBlock[SpdmMeasurementBlockCount];
} DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_MEASUREMENT_BLOCK;
```

Table 23 DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_MEASUREMENT_BLOCK Definition

Type	Name	Description
UINT16	SpdmVersion	SPDM version used in the SPDM GET_MEASUREMENT command. It is the defined in SPDM 1.1 specification – 10.2 GET_VERSION request and VERSION response messages, table VersionNumberEntry definition
UINT8	SpdmMeasurementBlockCount	The count of SPDM Measurement Blocks. See Section 3.3.4.3 PCR[2]-UEFI Drivers and UEFI Applications for the method for measurement.
UINT8	Reserved	SHALL be zero.
UINT32	SpdmMeasurementHashAlgo	SPDM Measurement Hash Algorithm. It is defined in SPDM 1.1 specification – 10.4 NEGOTIATE_ALGORITHMS request and ALGORITHMS response messages, table Successful ALGORITHMS response message format, field “MeasurementHashAlgo”
SPDM_MEASUREMENT_BLOCK	SpdmMeasurementBlock	Multiple SPDM Measurement Blocks. Each SPDM Measurement Block is defined in SPDM 1.1 specification – 10.11.1 Measurement Block – table Measurement block format.

```
typedef struct tdDEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_CERT_CHAIN {
  UINT16                          SpdmVersion;
  UINT8                           SpdmSlotId;
  UINT8                           Reserved;
  UINT32                          SpdmBaseHashAlgo;
  SPDM_CERT_CHAIN                 SpdmCertChain;
} DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_CERT_CHAIN;
```

Table 24 DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_SPDM_CERT_CHAIN Definition

Type	Name	Description
UINT16	SpdmVersion	SPDM version used in the SPDM GET_CERTIFICATE command. It is the defined in SPDM 1.1 specification – 10.2 GET_VERSION request and VERSION response messages, table VersionNumberEntry definition
UINT8	SpdmSlotId	The SlotId associated with this SPDM Certificate Chain. It is defined in SPDM 1.1 specification – 10.8 GET_CERTIFICATE request and CERTIFICATE response message, table GET_CERTIFICATE request message format, field “param1”.
UINT8	Reserved	SHALL be zero.
UINT32	SpdmBaseHashAlgo	SPDM Base Hash Algorithm for the root certificate in the SPDM Certificate chain. It is defined in SPDM 1.1 specification – 10.4 NEGOTIATE_ALGORITHMS request and ALGORITHMS response messages, table Successful ALGORITHMS response message format, field “BaseHashSel”
SPDM_CERT_CHAIN	SpdmCertChain	SPDM Certificate Chain. It is defined in SPDM 1.1 specification – 10.6.1 Certificates and certificate chains – table Certificate chain format.

```
typedef struct tdDEVICE_SECURITY_EVENT_DATA_SUB_HEADER_OEM_MEASUREMENT {
  UINT32                        Type;
```

```
  UINT32                          Length;
  UINT8                           Value[Length];
} DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_OEM_MEASUREMENT
```

Table 25 DEVICE_SECURITY_EVENT_DATA_SUB_HEADER_OEM_MEASUREMENT Definition

Type	Name	Description
UINT32	Type	The OEM defined data type.
UINT32	Length	Length in bytes of the value.
UINT8[Length]	Value	The OEM defined measurement data value.

```
typedef struct tdDEVICE_SECURITY_EVENT_DATA2 {
  DEVICE_SECURITY_EVENT_DATA_HEADER2         EventDataHeader;
  DEVICE_SECURITY_EVENT_DATA_SUB_HEADER      EventDataSubHeader;
  DEVICE_SECURITY_EVENT_DATA_DEVICE_CONTEXT  DeviceContext;
} DEVICE_SECURITY_EVENT_DATA2;
```

Table 26 DEVICE_SECURITY_EVENT_DATA2 Definition

Type	Name	Description
DEVICE_SECURITY_EVENT_DATA_HEADER2	EventDataHeader	The header of the event data.
DEVICE_SECURITY_EVENT_DATA_SUB_HEADER	EventDataSubHeader	The sub-header of the event data.
DEVICE_SECURITY_EVENT_DATA_DEVICE_CONTEXT	DeviceContext	The device context of the event data.

10.3 Measurement Event Entries and Log

Start of informative comment

The value within a PCR is used both for Sealed Storage and for attestation. When used for attestation, the raw hash value carries little meaning. Therefore, more meaningful structures are used that carry with them information. This information is contained within the structure TCG_PCR_EVENT2.event as described in Section 10.2.2 TCG_PCR_EVENT2 Structure.

The procedure for creating the measurement is to perform a hash of the data contained within the TCG_PCR_EVENT2.event field for each supported hash algorithm and construct a TPMT_HA structure. The

resulting tagged hashes are placed into the TCG_PCR_EVENT2.digests field and used as the data in the TPM2_PCR_Extend operation.

These structures are stored as an unstructured array within the Measured Boot Log. None of the Pre-OS entities, including ACPI, are required to interpret this data. A Boot Manager, a Pre-OS Authentication Application or an OS can obtain a copy of the log using the UEFI Get_Event_Log API defined in the EFI Protocol Specification. Once the OS controls the Host Platform, it is expected to read this data and transfer it to its own event log.

Due to the characteristics of the hashing operation, the verification of the Measurement Log entries is order-dependent. This, by the way, is a beneficial characteristic by adding trust in the order of the events. That is, if measurement A is taken and Measurement Log entry A is created, followed by a measurement B and creation of a Measurement Log entry B, it is important to a verifier that the sequence of Measurement Log entry A and Measurement Log entry B be deterministic.

The event ordering within the event log is sequential. This convention provides a consistent view of when events occurred globally rather than relative to each PCR. For example, if the following sequence occurred, the event log would appear as (assuming the event just prior to event A was event #12):

Measure event A into PCR[2] -> Event 13

Measure event B into PCR[2] -> Event 14

Measure event C into PCR[3] -> Event 15

Measure event D into PCR[2] -> Event 16

And if someone parsed the events by PCRs, the events would appear as:

For PCR[2]:

Event 13, Event 14, Event 16

For PCR[3]:

Event 15

Note: Events are not numbered in the event log because the event structure does not contain an event number. Furthermore, from a security perspective, there is no way to detect if events in the log were re-ordered as long as the events for each individual PCR are sequential.

No log of events is recorded by the firmware if the TPM is hidden.

After the invocation of UEFI Get_Event_Log the system may generate additional events, including the results of the EVT_SIGNAL_EXIT_BOOT_SERVICES (EBS). EBS terminates all boot services so that the UEFI Get Event Log function is no longer usable, thus the need to have a declarative variant of the events in a data structure that persists into OS runtime.

For the purposes of attestation, a post-firmware process extending events into PCRs and logging them is recommended to always record the event in the event log prior to extending the event to the PCR. As the process of recording events and extending is not atomic, it is possible for an attestation process to occur between these steps.

Consumers of the event log seeking to understand the length of the event log should consult Section 9.2 ACPI Table Usage and the TCG ACPI Specification.

End of informative comment

1. Platform Firmware SHALL use the hash identified by the algorithmID field recorded by Platform Firmware in the TCG_EfiSpecIdEvent Structure as specified in Section 10.4.5.1 Specification ID Version Event. There MAY be more than one.
2. The procedure for forming a TCG_PCR_EVENT2 structure SHALL be:
    a. Set A to an instantiation of a TCG_PCR_EVENT2 structure.
    b. Set A.pcrIndex to the index of the PCR to be extended.
    c. Set A.eventType to the specified Event Type defined in Section 10.4.1 Event Types.
    d. Fill A.event with either the data to be measured or the description of data to be measured and set A.eventSize to the size of A.event.
    e. Set B to A.digests, a TPML_DIGEST_VALUES structure:
        i. Set B.count to the number of digests in A.digests
        ii. Create C, a TPMT_HA structure, = B.digests[index] where index starts at 0 and continues to count - 1:
            1. Set C.hashAlg to algorithmID of B.digests[index].
            2. Create C.digest according to A.eventType field as defined in Table 27
            3. Repeat for each implemented hash algorithm.
        iii. Create B.digests by appending each instance of C. Similar to the TPMT_HA structure used in TPM commands, the fields are densely packet, i.e., if the result of the hashing algorithm is 20 bytes, the C.digest field is only 20 bytes.

Note for steps d and e: The description used here is not to be taken literally for all event types. Where the event field will contain data or structures, this statement is to be taken literally and the event field is to be filled in. However, when measuring code, it is not practical to place the entire code area into the event field just to take the hash of it. Also, it is not expected for the event field to contain the entire code area in the event log for these event types. For precise contents of this field, refer to the description of the event type.

3. The procedure for extending the PCR and recording the event SHALL be:
    f. Extend A.pcrIndex using A.digests as the value for each implemented PCR bank. Note: All integers should be encoded in Big Endian, before being sent to the TPM.
    g. Append A to the event log. Note: All integers should be encoded in Little Endian before appending to the event log.
    h. The sequence of the Measurement Log entries MUST be in the same sequence that the events were extended into the TPM PCRs.
4. If the TPM is hidden, as defined in Section 7 TPM Discoverability, Platform Firmware MUST not create an event log nor record any log entries.

10.4 Event Descriptions

Start of informative comment

Each event recorded in the Event Log is tagged as a particular event type. This section specifies all the event type tags that must or may be added to the Event Log on a PC client platform compliant to this specification.

The value within a PCR is used for sealed storage, attestation, and re-construction of the boot flow. The raw hash value in a PCR is sufficient for sealed storage, but not for attestation or replay.

Event Log entries add value to the raw hash values in PCRs for attestation as well as for re-constructing the events that triggered the measurements into the PCRs.

End of informative comment

10.4.1 Event Types

Start of informative comment

One element in an Event Log entry is TCG_PCR_EVENT2.eventType (see section 10.2.2 TCG_PCR_EVENT2 Structure). Table 27, below, is normative and specifies all the event types that can be used in an Event Log entry for the measurement events specified in this document for a UEFI platform.

The value associated with these UEFI specific platform event types must not overlap with the event type values already defined for other TCG platform architecture specifications.

EFI platforms may also use non-EFI specific events from the PC Client Implementation Specification for Conventional BIOS, including but not limited to, EV_POST_CODE, EV_SEPARATOR, EV_S_CRTM_VERSION, EV_S_CRTM_CONTENTS and EV_NO_ACTION. This specification does not speak to measurement of optional tagged events, as defined in the TCG v1.21 PC Client Implementation Specification for Conventional BIOS, section 10.4.2. A composite platform that supports UEFI and conventional BIOS may have such entries in the Event Log.

The value associated with a UEFI specific platform event type is located in the range between 0x80000000 and 0x800000FF, inclusive.

End of informative comment

The event types in Table 27 are defined for the field TCG_PCR_EVENT2.eventType. Any event type defined in this table, unless otherwise specified, is extended into the PCR(s) identified in the Description column. Any event type value not defined in this table SHALL NOT be used by PC Client Platform Firmware and is reserved for application and OS usage. Unless otherwise stated, the Events defined in Table 27 Events are specified only for platform firmware. Usage by the OS is not defined by this specification.

Table 27 Events

Value	Label	Description
0x00000000	EV_PREBOOT_CERT	Reserved for TCG future use

Value	Label	Description
0x00000001	EV_POST_CODE This event MUST extend the PCR	This event type is deprecated as of PFP 1.06. Used for PCR[0] only to record POST code, embedded SMM code, ACPI flash data, BIS code or manufacturer-controlled embedded option ROMs as a binary image. The digest field contains the tagged hash of the code or data to be measured (e.g., POST portion of Platform Firmware) for each PCR bank. The event field SHOULD contain a UEFI_PLATFORM_FIRMWARE_BLOB structure (see Section 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition. For POST code, the event data SHOULD be “POST CODE”). If the UEFI_PLATFORM_FIRMWARE_BLOB2 structure is used, the string SHALL be placed in the BlobDescription field of the BLOB structure. For embedded SMM code, the event data SHOULD be “SMM CODE”. For ACPI flash data, the event data SHOULD be “ACPI DATA”. For BIS code, the event data SHOULD be “BIS CODE”. For embedded option ROMs, the event data SHOULD be “Embedded UEFI Driver”. See Sections 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers and 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition.
0x00000002	EV_UNUSED	This event type is deprecated and MUST NOT be used. This event is not extended.

Value	Label	Description
0x00000003	EV_NO_ACTION This event MUST NOT extend any PCR	Used for PCRs[0,6], NV_EXTEND_INDEX_FOR_INSTANCE, NV_EXTEND_INDEX_FOR_DYNAMIC The digests field MUST contain all 0x00's for each allocated Hash algorithm. The event field contains informative data that was not extended into any PCR. This event is not extended. See section 10.4.5.3 Startup Locality Event. If the PCRIndex is an NVIndex, then the digests field MUST be the data extended to the NVIndex. The digests calculation MUST be based upon the TPM PCR Bank HashAlg instead of NV Index nameAlg. The event field contains informative data. See Sections 3.3.6.2 and 3.3.6.3.
0x00000004	EV_SEPARATOR This event MUST extend the PCRs 0 through 7 inclusive.	Used for PCRs[0, 1, 2, 3, 4, 5, 6 and 7]. Per Section 3.3.2 Error Conditions, used to indicate an error occurred recording the CRTM, POST BIOS, or Embedded Option ROMs. For this situation, the digest field MUST contain the tagged digest of the value 00000001h. The event field is arbitrary to allow platform manufacturers to include an indication of what error occurred or other useful troubleshooting information. Per Sections 3.3.4 PCR Usage and 8.2 Procedure for Pre-OS to OS-Present Transition, used to delimit actions taken during the pre-OS and OS environments. For this situation, the digests field MUST contain the tagged hash of the event data for each PCR bank. The event data size MUST be 4. The event field MUST contain the hex value 00000000h or FFFFFFFFh.
0x00000005	EV_ACTION This event MUST extend the PCR	Used for PCRs [1, 2, 3, 4, 5, and 6]. The digests field contains the tagged hash of the event field for each PCR bank. A specific action measured as a string defined in Section 10.4.3 EV_ACTION Event Types.

Value	Label	Description
0x00000006	EV_EVENT_TAG	Used for PCRs defined for OS and application usage. Defined for use by Host Platform Operating System or Software. May be used by Platform Firmware to measure Legacy Option ROMS using a TCG_PCClientTaggedEvent structure. The event field contains the structure defined in Section 10.4.2 Tagged Event Log Structure. The digests field MUST contain the tagged hash of the event data for each bank. Note: The event data is arbitrary to allow host platform software a standard event type to report events of their choosing using a vendor defined format. See Section 10.4.2 Tagged Event Log Structure.
0x00000007	EV_S_CRTM_CONTENTS This event MUST extend the PCR	Used for PCR[0] only. The digests field contains the tagged hash of the SRTM for each PCR bank. The event field SHOULD contain a UEFI_PLATFORM_FIRMWARE_BLOB2 structure. See Section 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition. When used by an H-CRTM to measure the SRTM, the event field MAY contain a NUL-terminated ASCII String. See Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers.

Value	Label	Description
0x00000008	EV_S_CRTM_VERSION This event MUST extend the PCR	Used for PCR[0] only. The digests field contains the tagged hash of the event field for each PCR bank. The event field contains the version of the SRTM as either a NUL-terminated UCS-2 string or a 16-byte GUID. The event field SHALL NOT be all zeros or NUL. **Note:** GUIDs and non-NUL strings were added as of PFP 1.06. Verifiers might observe GUIDs and NUL strings in firmware compliant with earlier versions of the PFP. See Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers.
0x00000009	EV_CPU_MICROCODE This event MUST extend the PCR	Used for PCR[1] only. The digests field contains the tagged hash of the microcode patch applied for each PCR bank. The event field contains a descriptor of the microcode patch. See Section 3.3.4.2 PCR[1] – Host Platform Configuration.
0x0000000A	EV_PLATFORM_CONFIG_FLAGS This event MUST extend the PCR	Used in PCR[1] only. The digests field contains the tagged hash of the event field for each PCR bank. The event field contents are manufacturer implementation-specific but MUST represent whether each optional PCR[1] measurement is measured or not for the set of measurements that can be toggled by the platform owner. See Section 3.3.4.2 PCR[1] – Host Platform Configuration.

Value	Label	Description
0x0000000B	EV_TABLE_OF_DEVICES This event MUST extend the PCR	Used in PCR[1] only. The digests field contains the tagged hash of the event field for each PCR bank. The event field contents are manufacturer implementation-specific. The event field contains the Platform Manufacturer-provided Table of Devices or other Platform Manufacturer-defined information. The Platform Manufacturer defines the content and format of the Table of Devices. The Host Platform Certificate may provide a reference to the meaning of these structures and data. See Section 3.3.4.2 PCR[1] – Host Platform Configuration.

Value	Label	Description
0x0000000C	EV_COMPACT_HASH This event MUST extend the PCR	Used for PCR 4 (Deprecated) and PCR 6 The event field MAY be informative, or MAY be hashed to generate the digests field, depending on the component recording the event. Deprecated Usage for PCR 4: In PFP 1.05, this event was used to measure the Boot Loader prior to wide deployment of EFI implementations. It used the TCG_CompactHashLogExtendEvent function. It was typically used by Boot Manager Code to measure events. The contents of the event field were specified by the caller. See Section 3.3.4.5 PCR[4] – Boot Manager Code and Boot Attempts Usage for PCR 6: This event may also be used by Platform Firmware vendors to measure events into PCR[6]. The event field SHALL be an 8-bit ASCII, non-NUL terminated string <Vendor Name> <Unique Identifier> where Vendor Name and Unique Identifier are vendor defined. See Sections 3.3.4.7 PCR[6] – Host Platform Manufacturer Specific Measurements and 10.4.6 Vendor Specific Events
0x0000000D	EV_IPL	Used for PCR[4] – deprecated, SHALL NOT be used for PCR [0:3, 5:7] This event is deprecated for platform firmware. It may be used by Boot Manager Code to measure events. The contents of the event field are specified by the caller.
0x0000000E	EV_IPL_PARTITION_DATA	Used for PCR[5] - deprecated This event is deprecated

Value	Label	Description
0x0000000F	EV_NONHOST_CODE This event MUST extend the PCR	Used for PCR[0,2] only. The executable component of any Non-Host Platform. The digests field contains the tagged hash of the Non-Host Platform code for each PCR bank. The event field may be determined by the platform manufacturer. See Sections 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers and 3.3.4.3 PCR[2]-UEFI Drivers and UEFI Applications.
0x00000010	EV_NONHOST_CONFIG This event MUST extend the PCR	Used for PCR[1,3] only. Configuration information or data associated with a Non-Host Platform. The digests field contains the tagged hash of the Non-Host Platform configuration information for each PCR bank. The event field is defined by the platform manufacturer. See Sections 3.3.4.2 PCR[1] – Host Platform Configuration and 3.3.4.4 PCR[3] – EFI Driver/Application Configuration.
0x00000011	EV_NONHOST_INFO This event MUST extend the PCR	Used for PCR[0] only. Information about the presence of a Non-Host Platform. The digests field contains the tagged hash of the event field for each PCR bank. The event field could be, but is not required to be, information such as the Non-Host Platform manufacturer, model, type, version, etc. The event field is defined by the platform manufacturer. See Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers.

Value	Label	Description
0x00000012	EV_OMIT_BOOT_DEVICE_EVENTS	Used for PCR[4] only. The digests field contains the tagged hash of the event field for each PCR bank. The event field MUST be “BOOT ATTEMPTS OMITTED” in all caps. See Section 3.3.4.5 PCR[4] – Boot Manager Code and Boot Attempts.
0x00000013	EV_POST_CODE2	Used for PCR[0] only to record POST code, embedded SMM code, ACPI flash data, BIS code or manufacturer-controlled embedded option ROMs as a binary image. The digest field contains the tagged hash of the code or data to be measured (e.g., POST portion of Platform Firmware) for each PCR bank. The event field SHALL contain a UEFI_PLATFORM_FIRMWARE_BLOB2 structure, see Section 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition. For POST code, the event data SHALL be “POST CODE”. If the UEFI_PLATFORM_FIRMWARE_BLOB2 structure is used, the string SHALL be placed in the BlobDescription field of the BLOB structure. For embedded SMM code, the event data SHALL be “SMM CODE”. For ACPI flash data, the event data SHALL be “ACPI DATA”. For BIS code, the event data SHALL be “BIS CODE”. For embedded option ROMs, the event data SHALL be “Embedded UEFI Driver”. See Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers and 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition.

Value	Label	Description
0x00000014- 0x0000FFFF	Reserved by TCG	Reserved for TCG future use
0x80000000	EV_EFI_EVENT_BASE	Base value for all UEFI platform event type values below Not a valid event. This event is not extended.
0x80000001	EV_EFI_VARIABLE_DRIVER_CONFIG	May be used for PCR[1, 3, 5 or 7] This event is used to measure configuration for EFI Variables. The digests field contains the tagged hash of the event field for each PCR bank. The event field MUST contain a UEFI_VARIABLE_DATA structure, including the variable data, the GUID and the Unicode string. See Section 10.2.6 Measuring UEFI Variables
0x80000002	EV_EFI_VARIABLE_BOOT	This event has been deprecated as of Specification 1.05 and has been replaced by EV_EFI_VARIABLE_BOOT2. MAY be used for PCR[1] for backwards compatibility. This event may be used to measure boot variables. It is recommended for implementations to use EV_EFI_VARIABLE_BOOT2. The digests field contains the tagged Hash of the variable data (not the UEFI_VARIABLE_DATA structure) for each PCR bank. This was clarified in PFP 1.05. The event field MUST contain a UEFI_VARIABLE_DATA structure See Section 10.2.6 Measuring UEFI Variables

Value	Label	Description
0x80000003	EV_EFI_BOOT_SERVICES_APPLICATION	Used for PCR[2,4] only This event measures information about the specific application loaded from the Boot Device. The digests field contains the tagged Hash of the PE/COFF image, as defined in Section 3.3.3.1 Measuring PE/COFF Image Files, from the loaded UEFI Boot Services application for each PCR bank. The event field MUST contain a UEFI_IMAGE_LOAD_EVENT structure. See Section 10.2.3 UEFI_IMAGE_LOAD_EVENT Structure
0x80000004	EV_EFI_BOOT_SERVICES_DRIVER	Used for PCR[0,2] only The digests field contains the tagged hash of the image, as defined in Section 3.3.3.1 Measuring PE/COFF Image Files, from the loaded UEFI Boot Services driver for each PCR bank. The event field MUST contain a UEFI_IMAGE_LOAD_EVENT structure See Section 10.2.3 UEFI_IMAGE_LOAD_EVENT Structure
0x80000005	EV_EFI_RUNTIME_SERVICES_DRIVER	Used for PCR[0,2] only This event measures information about embedded and add-in adapters with UEFI drivers. The digests field contains the tagged hash of the image, as defined in Section 3.3.3.1 Measuring PE/COFF Image Files, from the loaded UEFI Runtime Services driver for each PCR bank. The event field MUST contain a UEFI_IMAGE_LOAD_EVENT structure. See Section 10.2.3 UEFI_IMAGE_LOAD_EVENT Structure

Value	Label	Description
0x80000006	EV_EFI_GPT_EVENT	Used for PCR[5] only This event measures the UEFI GPT Table. The digests field contains the tagged hash of the event data for each PCR bank. The event data MUST contain a UEFI_GPT_DATA structure. See Section 10.2.6 Measuring UEFI Variables.
0x80000007	EV_EFI_ACTION	Used for PCR[1,2,3,4,5,6,7] The digests field contains the tagged hash of Event field for each PCR bank. A specific action measured as an EFI string defined in Section 10.4.4 EV_EFI_ACTION Event Types.
0x80000008	EV_EFI_PLATFORM_FIRMWARE_BLOB	This event has been deprecated as of PFP 1.05 and has been replaced by EV_EFI_PLATFORM_FIRMWARE_BLOB2 See Section 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition
0x80000009	EV_EFI_HANDOFF_TABLES	This event has been deprecated as of PFP 1.05 and has been replaced by EV_EFI_HANDOFF_TABLES2 The digests field contains the tagged hash of the system configuration tables that are referenced by entries in UEFI_HANDOFF_TABLE_POINTERS for each PCR bank. The event field MUST contain the UEFI table UEFI_HANDOFF_TABLE_POINTERS. See Section 10.2.4 Measuring Industry Standard Tables and Data Structures.

Value	Label	Description
0x8000000A	EV_EFI_PLATFORM_FIRMWARE_BLOB2	Used for PCR[0, 2, 4] only This event measures information about non-PE/COFF images. This allows for non-PE/COFF images in PCR[2] or PCR[4]. The digests field contains the tagged hash of all the code (PE/COFF .text sections or other) for each PCR bank. The event MUST contain a UEFI_PLATFORM_FIRMWARE_BLOB2 structure. See Section 10.2.5 UEFI_PLATFORM_FIRMWARE_BLOB2 and UEFI_PLATFORM_FIRMWARE_BLOB Structure Definition
0x8000000B	EV_EFI_HANDOFF_TABLES2	Used for PCR[1] only The digests field contains the tagged hash of the system configuration tables which are referenced by entries in UEFI_HANDOFF_TABLE_POINTERS2 for each PCR bank. The event field MUST contain the UEFI table UEFI_HANDOFF_TABLE_POINTERS2. See Section 10.2.4 Measuring Industry Standard Tables and Data Structures.
0x8000000C	EV_EFI_VARIABLE_BOOT2	Used for PCR[1] only This event is used to measure configuration for EFI Variables including UEFI Boot##### variables and the BootOrder variable. The digests field contains the tagged hash of the Event field for each PCR bank. The event field MUST contain a UEFI_VARIABLE_DATA structure. The contents of the UEFI_VARIABLE_DATA structure include the variable data, GUID, and unicode string. See Section 10.2.6 Measuring UEFI Variables

Value	Label	Description
0x8000000D	EV_EFI_GPT_EVENT2	Used for PCR[5] only This event measures the UEFI GPT Table. PFP 1.06 added this event to ensure that dynamic and instance specific data is zeroed out in this structure. The digests field contains the tagged hash of the event data for each PCR bank. The event data MUST contain a UEFI_GPT_DATA structure. Platform firmware SHALL replace `DiskGUID` in the `UEFI_PARTITION_TABLE_HEADER` and `UniquePartitionGUID` in the `UEFI_PARTITION_ENTRY` structures with zeros. See Section 10.2.6 Measuring UEFI Variables
0x8000000E- 0x8000000F	Reserved by TCG	Reserved for TCG future use
0x80000010	EV_EFI_HCRTM_EVENT	Used for PCR[0] only This event is used to record an event for the digest extended to PCR[0] as part of an H-CRTM event. The digests field contains the tagged hash of the measured H-CRTM component for each PCR bank The Event Data MUST be the 8-bit ASCII non-NUL terminated string: “HCRTM”. See Section 3.3.5.1 H-CRTM Localities and Concepts
0x80000011- 0x800000DF	Reserved by TCG	Reserved for TCG future use.

Value	Label	Description
0x800000E0	EV_EFI_VARIABLE_AUTHORITY	Used for PCR[7] only This event describes the Secure Boot variables. The digests field contains the tagged hash of the Event. The event field MUST contain a UEFI_VARIABLE_DATA structure where the VariableData field contains the EFI_SIGNATURE_DATA value from the EFI_SIGNATURE_LIST used to validate the loaded image. See Section 10.2.6 Measuring UEFI Variables and Section 3.3.4.8 PCR[7] – Secure Boot Policy Measurements. Note: this Event was modified from PFP 1.05 to align with the EDKII reference code implementation. PFP 1.05 stated that the digest field contained tagged hash of the EFI_SIGNATURE DATA field.
0x800000E1	EV_EFI_SPDM_FIRMWARE_BLOB	Used for PCR[0, 2] only. This event is used to record an extended digest for the firmware of an embedded component or an add-in device that supports SPDM “GET_MEASUREMENTS” functionality. This event records extended digests of SPDM GET_MEASUREMENT responses that correspond to firmware, such as immutable ROM, mutable firmware, firmware version, firmware secure version number, etc. The digests field contains the tagged hash of the Event. The event field MUST be a DEVICE_SECURITY_EVENT_DATA or DEVICE_SECURITY_EVENT_DATA2 structure. See Section 10.2.7 DEVICE_SECURITY_EVENT_DATA Structure.

Value	Label	Description
0x800000E2	EV_EFI_SPDM_FIRMWARE_CONFIG	Used for PCR[1, 3] only. This event is used to record an extended digest of the configuration of firmware for an embedded component or an add-in device that supports SPDM "GET_MEASUREMENTS" functionality. This event records extended digests of SPDM GET_MEASUREMENT responses that correspond to configuration, such as hardware configuration, firmware configuration, debug and device mode, etc. The digests field contains the tagged hash of the Event. The event field MUST be a DEVICE_SECURITY_EVENT_DATA or DEVICE_SECURITY_EVENT_DATA2 structure. See Section 10.2.7 DEVICE_SECURITY_EVENT_DATA Structure.
0x800000E3	EV_EFI_SPDM_DEVICE_POLICY	Used for PCR[7] only This event describes the UEFI device signature variable. The digests field contains the tagged hash of the Event. The Event field MUST contain a UEFI_VARIABLE_DATA structure where the VariableData field contains a list with elements of the type EFI_SIGNATURE_LIST as the UEFI device signature variable. See Section 10.2.7 DEVICE_SECURITY_EVENT_DATA Structures and Section 3.3.7 SPDM Device Authentication and Measurement Summary

Value	Label	Description
0x800000E4	EV_EFI_SPDM_DEVICE_AUTHORITY	Used for PCR[7] only This event describes the UEFI device signature variable. The digests field contains the tagged hash of the Event. The Event field MUST contain a UEFI_VARIABLE_DATA structure where the VariableData field contains the EFI_SIGNATURE_DATA value from the EFI_SIGNATURE_LIST used to validate the SPDM device. See Section 10.2.7 DEVICE_SECURITY_EVENT_DATA Structures and Section 3.3.7 SPDM Device Authentication and Measurement Summary
0x800000E5- 0x8000FFFF	Reserved by TCG	Reserved for TCG future use

10.4.2 Tagged Event Log Structure

Start of informative comment

This event type was originally specified in the TCG PC Client Implementation Specification for Conventional BIOS. It was deprecated in the original version of the PC Client Specific Platform Firmware Profile for TPM 2.0 systems. As a result of more interest in platform attestation services, which are not just confined to the results of the Platform Firmware measurements, but also the host platform OS and software environment, this event type has been redefined in this specification for use by OS and software on a PC Client platform. Other than the Event Type and structure, it is not specified. Verifiers of the event log should consult their OS and software vendors for instructions on interpretation of these events. This structure can be nested or concatenated in a single TCG_PCR_EVENT2 structure. A Tagged Event Structure can be parsed because it is formatted as a “TLV (Tag Length Value)”. The over all size of the event is contained in the eventSize field.

End of informative comment

1. Tagged Event Data MUST be measured and logged using the TCG_PCR_EVENT2 structure.
2. The TCG_PCR_EVENT2.eventType SHALL be EV_EVENT_TAG.
3. The TCG_PCR_EVENT2.event SHALL contain one or more of the TCG_PCClientTaggedEvent structures:
4. TCG_PCR_EVENT2.eventSize contains the size of all the TCG_PCClientTaggedEvent structures stored in the TCG_PCR_EVENT2.event field.

```
typedef struct tdTCG_PCClientTaggedEvent{
     UINT32     taggedEventID;
     UINT32     taggedEventDataSize;
```

```
    BYTE        taggedEventData[taggedEventDataSize];
} TCG_PCClientTaggedEvent;
```

Table 28 TCG PC Client Tagged Event Structure

Type	Name	Description
UINT32	taggedEventID	This is a unique identifier defined by the measuring OS or application. Note: This is Host OS or software defined data and not defined by this specification.
UINT32	taggedEventDataSize	Size of taggedEventData
BYTE	taggedEventData	The data of the event. Note: This is Host OS or software defined data and not defined by this specification.

10.4.3 EV_ACTION Event Types

For each EV_ACTION event produced by the platform, Platform Firmware MUST create the event indicated below in the following actions strings. The strings below are enclosed in quotes for clarity; the actual log entries SHALL NOT include the quote characters, nor a NUL terminator. They SHALL be logged using the following:

TCG_PCR_EVENT2.eventType = EV_ACTION. See Section 10.4.1 Event Types

TCG_PCR_EVENT2.digests = the tagged hash (for each PCR bank) of the Event[] field

TCG_PCR_EVENT2.event[] = ASCII string from Table 29 EV_ACTION Event types:

Table 29 EV_ACTION Event types

Action Index[1]	String	Purpose and Comments	PCR
0	“Calling Ready to Boot”	BIOS is calling Ready to Boot. If no additional strings are posted in log, that means the Boot Manager did not return control to Platform Firmware.	4

[1] This is only used for reference within this document.

Action Index[1]	String	Purpose and Comments	PCR
1	“Returned Ready to Boot”	Entering the Ready to Boot handler. This means either Platform Firmware has transferred control to the Ready to Boot handler; or Platform Firmware received control back from a failed IPL. NOTE: The term “returned” is an anachronism originating from a previously misdefined usage of this event; however, there is no reason to change the string.	4
2	“Return via INT 18h”	Deprecated	n/a
3	“Booting BCV Device s”	BIOS is IPL/Booting a BCV Device. The value ‘s’ is an ASCII string that unambiguously describes the boot device. This SHOULD include an indication of logical or physical device location and any ID string returned by the device. May be deprecated in a future version of this specification	4
4	“Booting BEV Device s”	BIOS is IPL/Booting a BEV Device. The value ‘s’ is an ASCII string supplied by the BEV device. May be deprecated in a future version of this specification	4

Action Index[1]	String	Purpose and Comments	PCR
5	"Entering ROM Based Setup"	BIOS is entering ROM-Based Setup or Option ROM-Based Setup during pre-OS environment. If the Host Platform is designed to always perform a Host Platform Reset upon exit from the ROM-Based Setup Utility, then this measurement does not have to be made. See sections 3.3.4.2 PCR[1] – Host Platform Configuration and 3.3.4.4 PCR[3] – EFI Driver/Application Configuration.	1 or 3
6	"Booting to Parties n"	BIOS is IPL/Booting from a PARTIES Partition #n. The value 'n' is the actual numeric value of the partition number represented as a printable ASCII hex value (e.g., partition zero would get the string value "0"), where N is the index into the BEER table. May be deprecated in a future version of this specification.	4
7	"User Password Entered"	Deprecated	1
8	"Administrator Password Entered"	Deprecated	1
9	"Password Failure"	Deprecated	1
10	"Wake Event n"	Deprecated	n/a
11	"Boot Sequence User Intervention"	User altered the boot sequence.	1
12	"Chassis Intrusion"	The case was opened.	1
13	"Non Fatal Error"	Deprecated	n/a
14	"Start Option ROM Scan"	Deprecated	n/a
15	"Unhiding Option ROM Code"	Deprecated	n/a
16	"<OpRom Specific non-IPL String>"	Deprecated	n/a

Action Index[1]	String	Purpose and Comments	PCR
17	"<OpRom Specific IPL String>"	Deprecated	n/a
18	"HDD Password Entered"	Deprecated	1

10.4.4 EV_EFI_ACTION Event Types

Start of informative comment

The EV_EFI_ACTION event type defined in Table 30, below extends into a specific PCR the measurement of a specific string value that indicates a specific event occurred during the platform or OS boot process.

Note: The opening and closing quote characters shown in the String Value column of Table 30 must not be included in the TCG_PCR_EVENT2.event field and must not be included in the input to the measurement function.

End of informative comment

Table 30, below, specifies all the specific string values that can be used for EV_EFI_ACTION on a UEFI platform. The strings below are enclosed in quotes for clarity; the actual log entries SHALL NOT include the quote characters or a NUL terminator.

Table 30 EV_EFI_ACTION Strings

Action Index[2]	String	Purpose and Comments	PCR
1	"Calling EFI Application from Boot Option"	Boot Manager attempting to execute code from a Boot Option See Section 8.2.4 Measuring OS Boot Events	4
2	"Returning from EFI Application from Boot Option"	Attempt to execute code from Boot Option was unsuccessful See Section 8.2.4 Measuring OS Boot Events	4
3	"Exit Boot Services Invocation"	Boot Manager has sent the call to UEFI to end Boot Services. See Section 8.2.4 Measuring OS Boot Events	5
4	"Exit Boot Services Returned with Failure"	UEFI encountered an error closing Boot Services See Section 8.2.4 Measuring OS Boot Events	5

[2] This is only used for reference in this document.

Action Index[2]	String	Purpose and Comments	PCR
5	"Exit Boot Services Returned with Success"	UEFI successfully existed Boot Services and pre-OS environment has been terminated See Section 8.2.4 Measuring OS Boot Events	5
6	"UEFI Debug Mode"	If a debugger is launched, this string is used to record the entry into debug mode. See Section 8.2.4 Measuring OS Boot Events	7
7	"Booting to <Boot####> Option"	The specific Boot Option the Boot Manager passes control to. See Section 8.2.4 Measuring OS Boot Events	4

10.4.5 EV_NO_ACTION Event Types

Start of informative comment

An EV_NO_ACTION event will not result in a digest being extended into a PCR, but has value to a Verifier of the log, because one of its purposes is to communicate the Specification Version with which the platform complies. For EV_NO_ACTION events other than the EFI Specification ID event (see Section 10.4.5.1 Specification ID Version Event) the log will contain EV_NO_ACTION events in the TCG_PCR_EVENT2 format, so a parser does not have to switch between event formats. To eliminate further conditional treatment of the TCG_PCR_EVENT2 structure in the parser, the TCG_PCR_EVENT2 event for EV_NO_ACTION events will contain a digest entry for each algorithm that is used in other TCG_PCR_EVENT2 events, but the digest values will be all zeros. For example, if only the SHA256 PCR bank is used, the TCG_PCR_EVENT2.digests.count value is 1, TCG_PCR_EVENT2.digests.digests[0].algID is TPM_ALG_SHA256, and TCG_PCR_EVENT2.digests.digests[0].digest is 32 bytes of zeros. In the normative text the short notation of TCG_PCR_EVENT2.digests = 0 is used to express this.

EV_NO_ACTION events in the TCG_PCR_EVENT2 format are used for the TCG_Sp800_155_PlatformId_Event2 event and the TCG_EfiStartupLocalityEvent. See Sections 10.4.5.2 Firmware Integrity Measurement Reference Manifest Event and 10.4.5.3 Startup Locality Event.

End of informative comment

EV_NO_ACTION Structure for Informational Events (Not extended to a PCR)

1. All EV_NO_ACTION events SHALL set TCG_PCR_EVENT2.pcrIndex = 0, unless otherwise specified.
2. All EV_NO_ACTION events SHALL set TCG_PCR_EVENT2.eventType = EV_NO_ACTION. See Section 10.4.1 Event Types.
3. All EV_NO_ACTION events SHALL set TCG_PCR_EVENT2.digests to all 0x00's for each allocated Hash algorithm. See Section 10.2.2 TCG_PCR_EVENT2 Structure.
4. All EV_NO_ACTION events SHALL set TCG_PCR_EVENT2.event to one of the events described in the following sections.

5. For each EV_NO_ACTION event produced by the platform, Platform Firmware MUST create and record the event indicated in this section.
6. EV_NO_ACTION events MUST NOT extend the PCR, but they are recorded in the event log.

EV_NO_ACTION Structure for NV_Extend Events

1. All EV_NO_ACTION events SHALL set TCG_PCR_EVENT2.pcrIndex to the specified NV_Extend Index Handle
2. All EV_NO_ACTION events SHALL set TCG_PCR_EVENT2.eventType = EV_NO_ACTION. See Section 10.4.1 Event Types).
3. All EV_NO_ACTION events SHALL set TCG_PCR_EVENT2.digests to the hash of the NV_INDEX_EVENT_LOG_DATA. The digest field SHALL be extended to the NV_EXTEND_INDEX_FOR_INSTANCE. See Section 3.3.6.2 NV Extend Record for Instance Data and Section 10.4.5.4 SPDM Structures
4. All EV_NO_ACTION events SHALL set TCG_PCR_EVENT2.event to the structure specified in Section 3.3.6 NV Index Usage. See also Section 10.4.5.4 SPDM Structures.
5. For each EV_NO_ACTION event extended to an NV_Extend Index produced by the platform, Platform Firmware MUST create and record the event indicated in Sections 3.3.6 NV Index Usage and 10.4.5.4 SPDM Structures.

10.4.5.1 Specification ID Version Event

Start of informative comment

The first event in the event log is the Specification ID version. This event is not extended to a PCR. This event provides information to a consumer of the event log about what version of the specification is implemented by firmware. Because the TCG_PCR_EVENT2 structure contains digest size information which cannot be interpreted by the consumer, the TCG_EfiSpecIdEvent{} structure will be the event field of a TCG_PCClientPCREvent{} (see Section 10.2.1 TCG_PCClientPCREvent Structure) structure with a 20 byte digest field of all zeros. It is an EV_NO_ACTION event type. The familyVersionMinor, familyVersionMajor, and specRevision fields should be modified to match the version of the PFP with which the Platform Firmware complies.

The TCG_EfiSpecIDEvent{} structure contains one or more TCG_Efi_SpecIdEventAlgorithmSize structures. These contain the algorithmID and digestSize for the measurement algorithms used by firmware. As consumers of the event log may not recognize the algorithmID, the digestSize field allows a parser to consume the events in the log without knowing the implicit size of the algorithm defined by the algorithmID. The information contained in this structure will vary based on the capabilities of the TPM and the platform. If platform firmware does not support the same number of hash algorithms as the TPM has enabled, i.e. TPM enables SHA-1 and SHA-256 but platform firmware only supports SHA-256, platform firmware has two options. For the first option, platform firmware would inform the user and disable the unsupported bank. For the second option, the platform would use the TPM2_Event command (or TPM2_EventSequence, TPM2_SequenceData, and TPM2_SequenceEnd) to send data to the TPM for the TPM to hash and extend to all enabled banks. Because the second option adds significant time to the platform boot process, it is not anticipated to be used, but is offered as there are circumstances where it might be needed. The Table 31 EfiSpecIdEventAlgorithmSize Examples provides an example of what might be observed in the structure based on platform support.

NOTE: There are no UINTN fields defined in this specification.

Table 31 EfiSpecIdEventAlgorithmSize Examples

TPM Algorithm Support	Platform Algorithm Support	EfiSpecIdEventAlgorithmSize
Single PCR Bank, Single Algorithm	Single Algorithm	1 EfiSpecIdEventAlgorithmSize structure
Single PCR Bank, Multiple Algorithms	Single Algorithm	1 EfiSpecIdEventAlgorithmSize structure
Single PCR Bank, Multiple Algorithms	Multiple Algorithms	1 EfiSpecIdEventAlgorithmSize structure
2 PCR Banks, 2 Algorithms	Single Algorithm	1 EfiSpecIdEventAlgorithmSize structure
2 PCR Banks, 2 Algorithms	Multiple Algorithms	2 EfiSpecIdEventAlgorithmSize structures

End of informative comment

```
typedef struct tdTCG_EfiSpecIdEventAlgorithmSize {
   UINT16      algorithmId;
   UINT16      digestSize;
} TCG_EfiSpecIdEventAlgorithmSize;
```

Table 32 TCG_EfiSpecIdEventAlgorithmSize

Type	Name	Description
UINT16	algorithmID	This value of this field SHALL be the algorithm ID (hashAlg) of the Hash used by BIOS Note: Systems that support multiple active PCR banks may report more than one algorithm ID. This field contains the TPMI_ALG_HASH identifier for the implemented Hash algorithm.
UINT16	digestSize	The size of the digest produced by the implemented Hash algorithm.

```
typedef struct tdTCG_EfiSpecIdEvent {
   BYTE                    Signature[16];
   UINT32                  platformClass;
   UINT8                   familyVersionMinor;
   UINT8                   familyVersionMajor;
   UINT8                   specRevision;
   UINT8                   uintnSize;
   UINT32                  numberOfAlgorithms;
```

```
    TCG_EfiSpecIdEventAlgorithmSize
                                digestSizes[numberOfAlgorithms];
    UINT8                       vendorInfoSize;
    BYTE                        vendorInfo[VendorInfoSize];
} TCG_EfiSpecIDEvent;
```

Table 33 TCG_EfiSpecIdEvent

Type	Name	Description
BYTE[16]	Signature	The NUL-terminated ASCII string ”Spec ID Event03”. SHALL be set to {0x53, 0x70, 0x65, 0x63, 0x20, 0x49, 0x44, 0x20, 0x45, 0x76, 0x65, 0x6e, 0x74, 0x30, 0x33, 0x00}.
UINT32	platformClass	The value for the Platform Class. The enumeration is defined in the TCG ACPI Specification Client Common Header.
UINT8	familyVersionMinor	This field combined with the familyVersionMajor field indicates that the platform firmware supports TPM Library Specification 2.0. Any BIOS supporting this PFP Family, Version and Revision (2.0) MUST set this value to 0x00.
UINT8	familyVersionMajor	This field combined with the familyVersionMinor field indicates that the platform firmware supports TPM Library Specification 2.0. Any BIOS supporting this PFP Family, Version and Revision (2.0) MUST set this value to 0x02.
UINT8	specRevision	Any BIOS supporting this revision (1.06) MUST set this value to 106 decimal. This value changes when the PFP revision changes.
UINT8	uintnSize	Specifies the size of the UINTN fields used in various data structures used in this specification. 0x01 indicates UINT32 and 0x02 indicates UINT64.
UINT32	numberOfAlgorithms	The number of Hash algorithms in the digestSizes field. This field MUST be set to a value of 0x01 or greater.
TCG_EfiSpecIdEvent AlgorithmSize [numberOfAlgorithms]	digestSizes	Each TCG_EfiSpecIdEventAlgorithmSize SHALL contain an algorithmId and digestSize for each hash algorithm used in the TCG_PCR_EVENT2 structure, the first of which is a Hash algorithmID and the second is the size of the respective digest.
UINT8	vendorInfoSize	Size in bytes of the VendorInfo field. Maximum value MUST be FFh bytes.

BYTE[VendorInfoSize]	vendorInfo	Provided for use by Platform Firmware implementer. The value might be used, for example, to provide more detailed information about the specific BIOS such as BIOS revision numbers, etc. The values within this field are not standardized and their interpretation requires input from an OEM. Platform-specific or -unique information MUST NOT be provided in this field.

Start of informative comment

The TCG_EfiSpecIdEvent contains a few fields that have changed across revisions of the Platform Firmware Profile. Table 34 TCG_EfiSpecIdEvent Change History provides a change history across revisions.

End of informative comment

Table 34 TCG_EfiSpecIdEvent Change History

Specification Revision	Defined Table Values/Changes	Notes
PFP Revision 21	specVersionMinor: 0x00 specVersionMajor: 0x02 specErrata: 0x02	Initial Revision
PFP Version 1.03 Revision 51		No change
PFP Version 1.04		No Change
PFP Version 1.05	Fields Name Changes: From: specVersionMinor To: familyVersionMinor: 0x00 From: specVersionMajor To: familyVersionMajor: 0x02 From: specErrata To: specRevision: 105 decimal	Names changed to synchronize with Cover Page
PFP Version 1.06	specRevision 106 decimal	

End of informative comment

10.4.5.2 Firmware Integrity Measurement Reference Manifest Event (Sp800_155 Event)

Start of informative comment

The TCG_Sp800_155_PlatformID_Event2 structure is used to convey information about the Reference Integrity Manifest (RIM) to a verifier of the Event Log.

The TCG_Sp800_155_PlatformId_Event structure is deprecated with this revision of the specification. There is no known use of this structure, but to ensure parsers can correctly handle the modifications made in this revision of the PFP, a new structure has been created. The ReferenceManifestGuid field should meet the requirements in RFC

4122. See the TCG Reference Integrity Manifest Information Model Specification and the TCG PC Client Reference Integrity Manifest Information Model Specification. See Section 2.5.9 TCG Reference Integrity Manifest Information Model.

In PFP 1.06, the structure was updated to include pointers to the location of the Platform RIM and the signature was updated to “SP800-155 Event3”.

End of informative comment

```
typedef struct tdTCG_Sp800_155_PlatformId_Event3 {
  BYTE            Signature[16];
  UINT32          PlatformManufacturerId;
  UEFI_GUID       ReferenceManifestGuid;
  UINT8           PlatformManufacturerStrSize;
  BYTE            PlatformManufacturerStr [PlatformManufacturerStrSize];
  UINT8           PlatformModelSize;
  BYTE            PlatformModel[PlatformModelSize];
  UINT8           PlatformVersionSize;
  BYTE            PlatformVersion[PlatformVersionSize];
  UINT8           FirmwareManufacturerStrSize;
  BYTE            FirmwareManufacturerStr[FirmwareManufacturerStrSize];
  UINT32          FirmwareManufacturerId;
  UINT8           FirmwareVersionSize;
  BYTE            FirmwareVersion[FirmwareVersionSize];
  UINT32          RimLocatorType;
  UINT32          RimLocatorLength;
  BYTE            RimLocator[RimLocatorLength];
  UINT32          PlatformCertLocatorType;
  UINT32          PlatformCertLocatorLength;
  BYTE            PlatformCertLocator[PlatformCertLocatorLength];
} TCG_Sp800_155_PlatformId_Event3;
```

Table 35 TCG_SP800_155_PlatformID (RIM) Event

Type	Name	Description
BYTE[16]	Signature	The non-NUL-terminated ASCII String “SP800-155 Event3”. SHALL be set to {0x53 0x50 0x38 0x30 0x30 0x2d 0x31 0x35 0x35 0x20 0x45 0x76 0x65 0x6e 0x74 0x33}
UINT32	PlatformManufacturerId	Platform Manufacturer ID is an integer defined

		at http://www.iana.org/assignments/enterprise-numbers
UEFI_GUID	ReferenceManifestGuid	ReferenceManifestGuid is the 16-byte identifier of a given platform's static configuration of code.
UINT8	PlatformManufacturerSize	Size in bytes of the PlatformManufacturer field. Maximum value MUST be FFh bytes.
BYTE[Platform ManufacturerSize]	PlatformManufacturer	The Platform Manufacturer name as a NUL-terminated ASCII string.
UINT8	PlatformModelSize	Size in bytes of the PlatformModel field. Maximum value MUST be FFh bytes.
BYTE[PlatformModelSize]	PlatformModel	The platform PlatformModel Name or Number as a NUL-terminated ASCII string.
UINT8	PlatformVersionSize	Size in bytes of the PlatformVersion field. Maximum value MUST be FFh bytes.
BYTE[PlatformVersionSize]	PlatformVersion	The PlatformVersion Name or Number as a NUL-terminated ASCII string.
UINT8	FirmwareManufacturerSize	Size in bytes of the FirmwareManufacturer field. Maximum value MUST be FFh.
BYTE[FirmwareManufacturerSize]	FirmwareManufacturer	The Firmware Manufacturer name as a NUL-terminated ASCII string.
UINT32	FirmwareManufacturerID	Firmware Manufacturer ID is an integer defined at http://www.iana.org/assignments/enterprise-numbers
UINT8	FirmwareVersionSize	Size in bytes of the FirmwareVersion field. Maximum value MUST be FFh.
BYTE[FirmwareVersionSize]	FirmwareVersion	The Firmware Version Name or Number as a NUL-terminated ASCII string.
UINT32	RimLocatorType	Type = URI, Local UEFI Device Path, or UEFI Variable
UINT32	RimLocatorLength	Length of the RIM Locator
BYTE[RimLocatorLength]	RimLocator	Location for the RIM

UINT32	PlatformCertLocatorType	Type = URI, Local UEFI Device Path, or UEFI Variable
UINT32	PlatformCertLocatorLength	Length of the Platform Cert Locator, may be zero if a platform certificate is not provided
BYTE[PlatformCertLocatorLength]	PlatformCertLocator	Location for the Platform Certificate

Start of informative comment

The SP800_155_PlatformID Event was first defined in the TCG PC Client Platform Firmware Profile Specification Family 2.0 Level 00 Revision 21 as a very simple structure, with only 2 elements: a Vendor ID and a GUID for the RIM. Over several revisions of the Platform Firmware Profile Specification, additional changes were made to the event. Table 36 Change History for Sp800_155_PlatformId_Event summarizes the changes.

Table 36 Change History for Sp800_155_PlatformId_Event

Specification Revision	Structure Change	Notes
PFP Revision 21		Initial Revision
PFP Version 1.03 Revision 51	Signature Added	
PFP Version 1.04	No Change	
PFP Version 1.05	• Event Type changed to Sp800_155_PlatformId_Event2 • Additional elements added to structure: Platform Manufacturer String, Platform Model, Platform Version, Firmware Manufacturer string, Firmware Manufacturer ID, and Firmware Version	Aligned with the PC Client Firmware Integrity Measurement Specification
PFP Version 1.06	• Event Type changed to Sp800_155_PlatformId_Event3 • Additional elements added: RIM Locator TLV, Platform Certificate Locator TLV	

End of informative comment

10.4.5.2.1 UEFI Variables for Platform RIM and Certificate Discovery

Start of informative comment

The UEFI Variables defined in this section are provided as a convenience for platform firmware implementers to programmatically communicate the presence and location to Verifiers of a Platform RIM or Platform Certificate. UEFI variables are also defined for VARs to communicate the presence and location of those artifacts. The variables are uniquely identified by the GUID and NAME elements.

End of informative comment

Table 37 Base/OEM Platform RIM EFI Variable definition

Variable	Description
GUID	**{865DF6ED-29F8-439A-927A-7542CEB4A012}**
NAME	**L"PlatformRIM"**
Attribute	**EFI_VARIABLE_BOOTSERVICE_ACCESS + EFI_VARIABLE_RUNTIME_ACCESS** Platform Firmware SHALL ensure the variable is read-only from the OS perspective. NOTE: **EFI_VARIABLE_NON_VOLATILE** is not required if the OEM wants to create on boot every time.
Data	Contains a list of Platform Attestation Variable Data Structure defined in Table 41 Platform Attestation Variable Data Structure.
Security Property	Integrity

Table 38 VAR Platform RIM EFI Variable definition

Variable	Description
GUID	**{865DF6ED-29F8-439A-927A-7542CEB4A012}**
NAME	**L"PlatformRIM####"** *####* is replaced by a unique option number in printable hexadecimal representation using the digits 0–9, and the upper case versions of the characters A–F (0000–FFFF).
Attribute	**EFI_VARIABLE_BOOTSERVICE_ACCESS + EFI_VARIABLE_RUNTIME_ACCESS + EFI_VARIABLE_NON_VOLATILE** This variable can be created at any time and should be treated as read-only once it is created.
Data	Contains a list of Platform Attestation Variable Data Structure defined in Table 41 Platform Attestation Variable Data Structure.
Security Property	Integrity after it is created. **NOTE**: Denial of service attacks are out of scope. An adversary may create more "PlatformRIM####" variables, but the adversary cannot modify the existing PlatformRIM#### variable. The adversary may create too many "PlatformRIM####" variables and trigger the OEM BIOS Variable Cleanup/Erase process. If that happens, all "PlatformRIM####" may be erased with confirmation from physical presence end user / IT administrator.

Table 39 Base/OEM Platform Certificate

Variable	Description
GUID	**{865DF6ED-29F8-439A-927A-7542CEB4A012}**
NAME	**L"PlatformCert"**
Attribute	**EFI_VARIABLE_BOOTSERVICE_ACCESS + EFI_VARIABLE_RUNTIME_ACCESS** The variable should be treated as read-only. NOTE: **EFI_VARIABLE_NON_VOLATILE** is not required if the OEM wants to create on boot every time.
Data	Contains a list of Platform Attestation Variable Data Structure defined in Table 41 Platform Attestation Variable Data Structure.
Security Property	Integrity

Table 40 Delta/VAR Platform Certificate

Variable	Description
GUID	**{865DF6ED-29F8-439A-927A-7542CEB4A012}**
NAME	**L"PlatformCert####"** *####* is replaced by a unique option number in printable hexadecimal representation using the digits 0–9, and the upper-case versions of the characters A–F (0000–FFFF).
Attribute	**EFI_VARIABLE_BOOTSERVICE_ACCESS + EFI_VARIABLE_RUNTIME_ACCESS + EFI_VARIABLE_NON_VOLATILE** The variable can be created at any time and should be treated as read-only once it is created.
Data	Contains a list of Platform Attestation Variable Data Structure defined in Table 41 Platform Attestation Variable Data Structure.
Security Property	Integrity after it is created. **NOTE**: Denial of service attacks are out of scope. An adversary might create more "PlatformCert####" variables, but the adversary cannot modify the existing PlatformCert#### variable. The adversary may create too many "PlatformCert####" variables and trigger the OEM BIOS Variable Cleanup/Erase process. If that happens, all "PlatformCert####" may be erased with confirmation from physical presence end user / IT administrator.

Table 41 Platform Attestation Variable Data Structure

Field	Byte Length	Byte Offset	Description
Type	4	0	The type of the data value. **0: Raw Data** – The data value is the platform certificate or the platform RIM. **1: Global Uniform Resource Identifier (URI)** – The data value is the URI that indicates the platform certificate or the platform RIM. Please refer to RFC2396 for the URI definition. **2: Local UEFI Device Path** – The data value is the UEFI device path that indicates the platform certificate or the platform RIM. Please refer to UEFI specification for the UEFI device path definition. **3: UEFI Variable** – The data value is the UEFI variable GUID/Name pair that indicates the platform certificate. The first 16 bytes are the UEFI variable GUID. The remaining bytes are the NULL terminated UEFI variable name. **4-0xFFFFFFFF:** Reserved by TCG
Length	4	4	The length in bytes of this data value.
Value	N	8	The type specific data value.

10.4.5.3 Startup Locality Event

Start of informative comment

The Startup Locality event is placed in the log before any event which extends PCR[0]. This allows a Verifier to initialize its internal PCR[0] state correctly. See the Method for Measurement subsection of Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers.

End of informative comment

The Startup Locality SHALL be recorded if the Locality issuing the TPM2_Startup command is Locality 3, or an H-CRTM event occurred. There SHALL be only one Startup Locality event recorded. The Startup Locality MAY be recorded if the Locality issuing the TPM2_Startup command is Locality 0. See Section 3.3.4.1 PCR[0] – SRTM, POST BIOS, and Embedded Drivers.

```
typedef struct tdTCG_EfiStartupLocalityEvent {
   BYTE       Signature[16];
   UINT8      StartupLocality;
} TCG_EfiStartupLocalityEvent;
```

Table 42 Startup Locality Event

Type	Name	Description
BYTE[16]	Signature	The NUL-terminated ASCII string “StartupLocality” SHALL be set to {0x53 0x74 0x61 0x72 0x74 0x75 0x70 0x4C 0x6F 0x63 0x61 0x6C 0x69 0x74 0x79 0x00}
UINT8	StartupLocality	SHALL be the value of the Startup Locality which sent the TPM2_Startup command. 0: Startup Locality is Locality 0 without an H-CRTM sequence. 3: Startup Locality is Locality 3 without an H-CRTM sequence. 4: Startup Locality is Locality 4 with an H-CRTM sequence initialized. Other values: reserved by TCG

10.4.5.4 SPDM Structures

Start of informative comment

There are SPDM artifacts that enable a Verifier to understand the process Platform Firmware utilizes to collect firmware measurements from a component that utilizes the SPDM protocol as defined by DMTF. Platform Firmware may ask for the certificate chain(s) for the component. This certificate chain might include instance specific information, e.g. device serial numbers or other unique identifiers, that should not be extended to a TPM PCR. Another option provided by the SPDM protocol to ensure freshness of measurements is the inclusion of a caller provided Nonce. The SPDM responder (the component) responds to a request for measurements with a signature over the transcript of the calls from the original request for measurements through the final block of measurements. In order for a Verifier to verify the Signature, the Verifier would need the entire Certificate chain and any Platform

Firmware provided Nonce. To accommodate this verification use case, PFP 1.06 adds additional unmeasured events as EV_NO_ACTION Event Types.

End of informative comment

The following structures SHALL NOT be present in the event log if SPDM measurements are not extended to PCR 2 or 3.

10.4.5.4.1 NV_INDEX_INSTANCE_EVENT_LOG_DATA

Start of informative comment

By default, Platform Firmware measures only class (non-identifying) information to the TPM PCRs, but not device instance or system unique information, such as asset, serial numbers, or DICE TCB information. Platform firmware may provision a NV index to record the measurement of instance information. This NV index can be used to verify the integrity of the instance information in the TCG event log.

The platform firmware uses TPM2_NV_Extend() to extend the digest. The verifier can use TPM2_NV_Certify() to certify the NV index. The process is similar to TPM2_Quote().

End of informative comment

```
typedef struct tdNV_INDEX_INSTANCE_EVENT_LOG_DATA {
   BYTE                          Signature[16];
   UINT16                        Version;
   UINT8                         Reserved[6];
   DEVICE_SECURITY_EVENT_DATA2   Data;
} NV_INDEX_INSTANCE_EVENT_LOG_DATA;
```

Table 43 NV Index Instance Event Log Data

Type	Name	Description
BYTE[16]	Signature	The NUL-terminated ASCII string “NvIndexInstance” SHALL be set to { 0x4e 0x76 0x49 0x6e 0x64 0x65 0x78 0x49 0x6e 0x73 0x74 0x61 0x6e 0x63 0x65 0x00 }
UINT16	Version	Version of this structure, SHALL be “1”
UINT8[6]	Reserved	SHALL be zero.
DEVICE_SECURITY_EVENT_DATA2	Data	SHALL be the DEVICE_SECURITY_EVENT_DATA2 structure, See Section 10.2.7 DEVICE_SECURITY_EVENT_DATA Structures

10.4.5.4.2 SPDM Cert Chain NV_EXTEND_INDEX Event Log Data

Start of informative comment

The SPDM Certificate chain may contain class and instance specific data. For example, the Root Certificate Authority public key is the class data, but the other certificates in the chain contain instance data, such as device identifiers. The instance specific data is extended into the NV_EXTEND index using the NV_INDEX_INSTANCE_EVENT_LOG_DATA structure.

End of informative comment

Table 44 SPDM Cert Chain NV Extend Event Log Data

Type	Name	Description
BYTE[16]	Signature	The NUL-terminated ASCII string “NvIndexInstance” SHALL be set to { 0x4e 0x76 0x49 0x6e 0x64 0x65 0x78 0x49 0x6e 0x73 0x74 0x61 0x6e 0x63 0x65 0x00 }
UINT16	Version	Version of this structure, SHALL be “1”
UINT8[6]	Reserved	SHALL be zero.
DEVICE_SECURITY_EVENT_DATA2	Data	SHALL be the DEVICE_SECURITY_EVENT_DATA2 structure. SubHeaderType: SPDM Cert Chain The digest extended to the NV Index SHALL be the hash of the entire SPDM Cert Chain. The SPDM Certificate chain is defined in the SPDM Specification version 1.1 section 10.6.1 Certificates and Certificate Chains. See Section 10.2.7.3 Authenticated SPDM Structures

10.4.5.4.3 NV_INDEX_DYNAMIC_EVENT_LOG_DATA

Start of informative comment

By default, the platform firmware SHALL NOT measure any dynamic / ephemeral / changeable / automatically updated data to the PCR. In SPDM measurement use case, the verifier may want to know the NONCE value in the challenge / response process and keep tracking if there is any duplication on the NONCE. Platform firmware may provision a NV index to record the measurement of this dynamic information. This NV index can be used to verify the integrity of the dynamic information in the TCG event log.

The platform firmware uses TPM2_NV_Extend() to extend the digest. The verifier can use TPM2_NV_Certify() to certify the NV index. The process is similar to TPM2_Quote().

End of informative comment

```
typedef struct tdNV_INDEX_DYNAMIC_EVENT_LOG_DATA {
   BYTE       Signature[16];
   UINT16     Version;
   UINT8      Reserved[6];
   UINT64     UID;
   UINT16     DescriptionSize;
   UINT8      Description[DescriptionSize];
```

```
    UINT16      DataSize;
    UINT8       Data[DataSize];
} NV_INDEX_DYNAMIC_EVENT_LOG_DATA;
```

Table 45 NV Extend Dynamic Event Log Data

Type	Name	Description
BYTE[16]	Signature	The NUL-terminated ASCII string “NvIndexDynamic ” SHALL be set to { 0x4e 0x76 0x49 0x6e 0x64 0x65 0x78 0x44 0x79 0x6e 0x61 0x6d 0x69 0x63 0x20 0x00 }
UINT16	Version	Version of this structure, SHALL be “1”
UINT8[6]	Reserved	SHALL be zero.
UINT64	UID	A universal ID assigned by the event log creator. The UID can be used to bind a dynamic event log entry to other data structure.
UINT16	DescriptionSize	The size in bytes of the description.
UINT8[DescriptionSize]	Description	The description for the dynamic data.
UINT16	DataSize	The size in bytes of the dynamic data.
UINT8[DataSize]	Data	The dynamic data.

10.4.5.4.4 SPDM Nonce Value NV_EXTEND_INDEX Event Log Data

Start of informative comment

SPDM Nonce value is a nonce or random data in the SPDM request or response command. The dynamic data is extended into the NV_EXTEND index using the NV_INDEX_DYNAMIC_EVENT_LOG_DATA structure.

End of informative comment

Table 46 SPDM Nonce Value NV Extend Event Log Data

Type	Name	Description
BYTE[16]	Signature	The NUL-terminated ASCII string “NvIndexDynamic ” SHALL be set to { 0x4e 0x76 0x49 0x6e 0x64 0x65 0x78 0x44 0x79 0x6e 0x61 0x6d 0x69 0x63 0x20 0x00 }
UINT16	Version	Version of this structure, SHALL be “1”
UINT8[6]	Reserved	SHALL be zero.

UINT64	UID	A universal ID assigned by the event log creator. The UID can be used to bind a dynamic event log entry to a DEVICE_SECURITY _EVENT_DATA2 structure.
UINT16	DescriptionSize	The size in bytes of the description.
UINT8[DescriptionSize]	Description	The description of the dynamic data: “SPDM CHALLENGE”, if the SPDM command is CHALLENGE request. “SPDM CHALLENGE_AUTH”, if the SPDM command is CHALLENGE_AUTH response. “SPDM GET_MEASUREMENTS”, if the SPDM command is GET_MEASUREMENTS request. “SPDM MEASUREMENTS”, if the SPDM command is MEASUREMENTS response. “SPDM KEY_EXCHANGE”, if the SPDM command is KEY_EXCHANGE request. “SPDM KEY_EXCHANGE_RSP”, if the SPDM command is KEY_EXCHANGE_RSP response. “SPDM PSK_EXCHANGE”, if the SPDM command is PSK_EXCHANGE request. “SPDM PSK_EXCHANGE_RSP”, if the SPDM command is PSK_EXCHANGE_RSP response. If the SPDM response does not correspond to one of the above responses, the Description SHALL be set to a NUL-terminated ASCII string.
UINT16	DataSize	The size in bytes of the dynamic data.
UINT8[DataSize]	Data	The dynamic data SHALL be: CHALLENGE.Nonce, if the SPDM command is CHALLENGE request. CHALLENGE_AUTH.Nonce, if the SPDM command is CHALLENGE_AUTH response. GET_MEASUREMENTS.Nonce, if the SPDM command is GET_MEASUREMENTS request. MEASUREMENTS.Nonce, if the SPDM command is MEASUREMENTS response. KEY_EXCHANGE.RandomData, if the SPDM command is KEY_EXCHANGE request.

		KEY_EXCHANGE_RSP.RandomData, if the SPDM command is KEY_EXCHANGE_RSP response. PSK_EXCHANGE.RequesterContext, if the SPDM command is PSK_EXCHANGE request. PSK_EXCHANGE_RSP.ResponderContext, if the SPDM command is PSK_EXCHANGE_RSP response.

10.4.5.5 H-CRTM Component Measurement Data

Start of informative comment

EV_EFI_HCRTM_EVENT may be used to report the digest or security version number (SVN) of the measured H-CRTM components within the H-CRTM sequence. The H-CRTM sequence only extends one event to PCR[0], but it is feasible to measure multiple components using _TPM_HASH_DATA. If that happens, it is recommended the platform firmware reports multiple EV_NO_ACTION TCG_HCRTMComponentEvent events, where each of the events records one of the measured components. As such, the Verifier can use TCG_HCRTMComponentEvent to reconstruct the digest in EV_EFI_HCRTM_EVENT and verify the individual measurements. The example below shows only two components for convenience, but there is no limitation imposed on the structure.

Assuming component_a and component_b have the corresponding hashes and SVNs - component_hash_a and component_svn_b, and assuming the H-CRTM has the ability to produce a hash, the H-CRTM may measure component_hash_a and component_svn_b in the H-CRTM sequence, as follows:

_TPM_Hash_Start

_TPM_Hash_Data (component_hash_a)

_TPM_Hash_Data (component_svn_b)

_TPM_Hash_End

NOTE: The TPM treats any data provided to _TPM_Hash_Data as raw data and, as a result, there is no requirement that component_hash_a use the same Hash algorithm as an active TPM PCR bank.

Then, the resulting PCR[0] value will be:

PCR[0] = Hash (0…04 || Hash (component_hash_a || component_svn_b))

The event records are constructed as follows:

EvType = EV_NO_ACTION

Digest = 0

Event = TCG_EfiStartupLocalityEvent (StartupLocality = 4)

EvType = EV_EFI_HCRTM_EVENT

Digest = Hash (component_hash_a || component_svn_b) // no change

Event = “HCRTM”

EvType = EV_NO_ACTION

Digest = 0

Event = TCG_HCRTMComponentEvent {

“H-CRTM CompMeas”, //Signature

sizeof(“component_a”), “component_a”, //ComponentDescriptionSize and ComponentDescription

Digest, //MeasurementFormatType

sizeof(TPMT_HA), TPMT_HA{ hashAlg = component_hash_alg, digest = component_hash_a}

} //ComponentMeasurementSize and ComponentMeasurement

EvType = EV_NO_ACTION

Digest = 0

Event = TCG_HCRTMComponentEvent {

“H-CRTM CompMeas”, //Signature

sizeof(“component_svn_b”), “component_svn_b”, // ComponentDescriptionSize and ComponentDescription

RawData, //MeasurementFormatType

Sizeof(component_svn_b), component_svn_b

} //ComponentMeasurementSize and ComponentMeasurement

End of informative comment

Within an H-CRTM sequence, an H-CRTM can measure the digest or raw data (such as SVN) of a component with H-CRTM sequence using _TPM_HASH_DATA. The _TPM_HASH_DATA can be issued multiple times to measure the digests or raw data of multiple components. If the H-CRTM measures the digests or raw data of the components, the platform firmware SHOULD use EV_NO_ACTION TCG_HCRTMComponentEvent to log the digests or raw data of the components. If logged, the number and order of TCG_HCRTMComponentEvent SHALL match the number and order of the H-CRTM measurements. All TCG_HCRTMComponentEvent SHALL be recorded immediately after the EV_EFI_HCRTM_EVENT in the event log.

```
Typedef struct tdTCG_HCRTMComponentEvent {
   BYTE       Signature[16];
   UINT8      ComponentDescriptionSize;
   BYTE       ComponentDescription[ComponentDescriptionSize];
   UINT8      MeasurementFormatType;
   UINT16     ComponentMeasurementSize;
   BYTE       ComponentMeasurement[ComponentMeasurementSize];
} TCG_HCRTMComponentEvent;
```

Table 47 H-CRTM Measured Component Events

Type	Name	Description
BYTE[16]	Signature	The NULL-terminated ASCII String “**H-CRTM CompMeas**”. SHALL be set to {0x48, 0x2d,

		0x43, 0x52, 0x54, 0x4d, 0x20, 0x43, 0x6f, 0x6d, 0x70, 0x4d, 0x65, 0x61, 0x73, 0x00}
BYTE	ComponentDescriptionSize	Size of the ComponentDescription.
BYTE [ComponentDescriptionSize]	ComponentDescription	The ASCII String that describes the component.
UINT8	MeasurementFormatType	BIT[7] – Format 0b – Digest (TPMT_HA) 1b – RawData BIT[6:0] – reserved by TCG
UINT16	ComponentMeasurementSize	Size of the ComponentMeasurement
BYTE [ComponentMeasurementSize]	ComponentMeasurement	If MeasurementFormatType is Digest, it is the tagged digest of the H-CRTM measured component. The format is TPMT_HA. H-CRTM only measures the digest of ComponentMeasurement. The hashAlg is not measured. NOTE: The hashAlg of ComponentMeasurement might be different from the hashAlg of TPM PCR bank. If MeasurementFormatType is RawData, it is the raw data of the H-CRTM measured component, such as an SVN.

10.4.6 Vendor Specific Events

Start of informative comment

PC OEM's may define their own events for PCR[6]. Some events may use the event type EV_NO_ACTION. PC OEM's may also define events that extend a PCR. Both the Vendor Name and Unique Identifier used to define a Vendor Specific Event are vendor defined 8-bit ASCII, non-NUL terminated strings. The event may be a description of the digests field or it may be the data used to produce the digests.

End of informative comment

Platform specific events that extend the PCR MUST use the event type EV_COMPACT_HASH. Each event that extends PCR[6] SHALL be logged with the following structure, where event data is a unique 8-bit ASCII, non-NUL terminated string pre-pended with the platform OEM name:

1. TCG_ PCR_EVENT2.pcrIndex = 6
2. TCG_ PCR_EVENT2.eventType = EV_COMPACT_HASH. See Section 10.4.1 Event Types.
3. TCG_ PCR_EVENT2.digests = digest which was extended to the PCR (for all PCR banks)
4. TCG_ PCR_EVENT2.event[] = "<Vendor Name> <Unique identifier>"

10.5 Event Log Integrity

10.5.1 EVENT_LOG_INTEGRITY_DATA

Start of informative comment

In PFP 1.06, with the addition of component specific identity information (SPDM instance information) to the event log, integrity protection of the event log becomes more important. A Verifier requires the SPDM instance information to assess the trustworthiness of the SPDM measurements made to TPM PCRs. A platform firmware vendor may provision NV Indexes to capture Event Log Integrity information. The objectAttributes of these NV Indexes are defined in the Platform TPM Profile Version 1.06. Platform firmware writes the EVENT_LOG_INTEGRITY_DATA to these indexes as defined in Section 3.3.6.1 NV Index for Event Log Integrity.

End of informative comment

```
typedef struct tdEVENT_LOG_INTEGRITY_DATA {
  UINT16          Version;
  BYTE            CurrentHash[Size of NV Index nameAlg];
  BYTE            PreviousHash[Size of NV Index nameAlg];
  } EVENT_LOG_INTEGRITY_DATA;
```

Table 48 EVENT_LOG_INTEGRITY_DATA definition

Type	Name	Description
UINT16	Version	Version of this structure. SHALL be set to 0x00 00
UINT8[Size of NV Index nameAlg]	CurrentHash	Hash of the event log in current boot. See Section **3.3.6.1 NV Index for Event Log Integrity**.
UINT8[Size of NV Index nameAlg]	PreviousHash	Hash of the event log from the previous boot. See Section **3.3.6.1 NV Index for Event Log Integrity**.

Table 49 EventLogIntegrityDesc EFI Variable definition

Variable	Description
GUID	**{7F89AD30-4B9B-418B-844E-2C24A5D3070E}**
NAME	**L"EventLogIntegrityDescExitPmAuth"** for the variable associated with **EVENT_LOG_INTEGRITY_NV_INDEX_EXIT_PM_AUTH**. **L"EventLogIntegrityDescReadyToBoot"** for the variable associated with **EVENT_LOG_INTEGRITY_NV_INDEX_READY_TO_BOOT**.
Attribute	Platform firmware SHALL create the **EventLogIntegrityDescExitPmAuth** variable at the **EFI_END_OF_DXE_EVENT_GUID** event and ensure the variable is read-only from an OS perspective. The **EventLogIntegrityDescExitPmAuth** attributes are **EFI_VARIABLE_BOOTSERVICE_ACCESS +** **EFI_VARIABLE_RUNTIME_ACCESS +** **EFI_VARIABLE_NON_VOLATILE +** **Read-Only** Platform firmware SHALL create the **EventLogIntegrityDescReadyToBoot** variable at the **EFI_EVENT_GROUP_READY_TO_BOOT** event. The **EventLogIntegrityDescReadyToBoot** attributes are **EFI_VARIABLE_BOOTSERVICE_ACCESS +** **EFI_VARIABLE_RUNTIME_ACCESS +** **EFI_VARIABLE_NON_VOLATILE**

Data	The NULL terminated ASCII description of the difference between current platform firmware configuration and last configuration. This variable SHALL only be present if EVENT_LOG_INTEGRITY_DATA.PreviousHash is not all zeros. Platform firmware security related configuration changes SHALL be recorded here. For example, Setup Password enable/disable, SPDM certifiate chain update. If there are multiple changes between the current boot and previous boot, the description SHALL be separated by a semicolon ";". For example, "UserPasswordPolicy;SPDMCertificateChain". If multiple changes occur and the record of these changes exceeds the length of maximum variable size, platform firmware SHALL record the string "TRUNCATED" as the last description. **NOTE:** These descriptions are vendor specific. A Verifier needs an OEM RIM to interpret this field.

11 Platform Hierarchy (Physical Presence)

Start of informative comment

TPM 2.0 augments the concept of Physical Presence with the Platform Hierarchy authorization. The majority of PC Client platforms implemented with TPM 1.2 utilized command-based Physical Presence, using the command TSC_PhysicalPresence, instead of a physical button. Because the platform hierarchy is the point of control for the state of the TPM, it is important that the platform hierarchy be properly protected.

End of informative comment

Platform Firmware MUST protect access to the Platform Hierarchy and prevent access to the platform hierarchy by non-manufacturer-controlled components.

1. Platform Firmware SHALL:
    a. Disable the platform Hierarchy, or
    b. If the Platform Hierarchy is enabled:
        i. If the TPM is hidden, Platform Firmware MUST change platformAuth to a high entropy value prior to disabling the TPM as defined in Section 7 TPM Discoverability.
        ii. If the TPM is visible, Platform Firmware MUST change platformAuth to a high entropy value prior to executing 3rd party code, e.g. non-manufacturer controlled UEFI applications.

12 Predictive Event Logs

Start of informative comment

With the advent of Hash algorithm agility within TPM 2.0, it is possible for a TPM to be configured with one (or more) Hash algorithm at the beginning of a platform's service life and have the Hash algorithm be changed at a later point in the platform's service life. If the OS present environment is using the TPM to protect secrets, it is desirable to have a way to convert TPM protected secrets from the Hash algorithm and PCR allocation with which they were instantiated to the new Hash algorithm and PCR configuration without have to unseal the secrets from the TPM. To support this, the TCG PC Client Physical Presence Interface Specification version 1.3 revision 52 introduced optional PPI operation 33 (LogAllDigests) to support predictive event logs. If the Platform Firmware supports predictive event logs, the requirements in this section must be implemented as defined. As an example, an OS that wants to change the PCR allocation and update its sealed objects will send a PPI operation 33 to Platform Firmware and reboot the system. After user confirmation, Platform Firmware will log digests for all supported hash algorithms and boot to the OS. The OS will receive all the digests for all events and migrate its objects to the desired hash algorithm. Following this, the OS will send a PPI operation to change the PCR allocation to the desired state. After the Platform Firmware performs this second PPI operation, it will produce the event log with the digests for the allocated PCRs.

It is likely, but not required, that Platform Firmware will have internal flags to track this state, as the requirement to calculate the predicative Event Log will only be necessary for one boot cycle (within which the OS present environment will recalculate and reseal its secrets).

End of informative comment

If predictive event logs are supported, Platform Firmware SHALL implement all of the following requirements:

1. Platform Firmware MUST support the TCG Physical Presence Operation 33 (LogAllDigests).
2. Platform Firmware MUST measure and record all mandatory events for the currently active PCR banks.
3. Platform Firmware MAY measure and record optional events for the currently active PCR banks.
4. On only the first reboot following receipt of PPI Operation 33 (LogAllDigests), Platform Firmware MUST:
    a. Calculate the tagged Hash for all supported algorithms for all events measured and recording in steps 1 and 2.
    b. Construct the correctly formatted Event Log and record all events with the tagged Hash from step 3a.

13 Supporting TCG Opal SSC Block SID enabled devices

Start of informative comment

The TCG Storage-Feature Set Block SID Authentication Specification defines a mechanism by which a host application can alert a storage device to block attempts to authenticate the SID authority until a subsequent device power cycle occurs.

This mechanism may be used by Platform Firmware to prevent malicious entities from taking ownership of a TCG Opal storage device which has not been owned.

The TCG PC Client Physical Presence Interface Specification version 1.3 revision 52 introduced PPI operations 96-101 as a convenience to provide a standard way for OS-present applications which can manage a TCG Opal storage device to interface with Platform Firmware through the PPI interface. These operations are optional, but if implemented require corresponding functionality within the Platform Firmware to respond to these requests.

End of informative comment

If a platform supports PPI operations 96-101, Platform Firmware MUST implement support for the TCG Storage-Feature Set Block SID Authentication Specification.

Annex A: TPM Interrupt ASL Example

Start of informative comment

This is an informative example of enabling TPM Interrupts.

```
//
    // Operational region for TPM access
    //
    OperationRegion (TPMR, SystemMemory, 0xfed40000, 0x5000)
    Field (TPMR, AnyAcc, NoLock, Preserve)
    {
      ACC0, 8,  // TPM_ACCESS_0
      Offset(0x8),
      INTE, 32, // TPM_INT_ENABLE_0
      INTV, 8,  // TPM_INT_VECTOR_0
      Offset(0x10),
      INTS, 32, // TPM_INT_STATUS_0
      INTF, 32, // TPM_INTF_CAPABILITY_0
      STS0, 32, // TPM_STS_0
      Offset(0x24),
      FIFO, 32, // TPM_DATA_FIFO_0
      Offset(0x30),
      TID0, 32, // TPM_INTERFACE_ID_0
              // ignore the rest
    }

    //
    // Operational region for TPM support, TPM Physical Presence and TPM Memory Clear
    // Region Offset 0xFFFF0000 and Length 0xF0 will be fixed in C code.
    //
    OperationRegion (TNVS, SystemMemory, 0xFFFF0000, 0xF0)
    Field (TNVS, AnyAcc, NoLock, Preserve)
    {
      IRQN,   32, //   IRQ Number for _CRS
      SFRB,   8   //   Is shortformed Pkglength for resource buffer
    }
```

```
    //
    // Possible resource settings returned by  _PRS method
    //   RESS : ResourceTemplate with PkgLength <=63
    //   RESL : ResourceTemplate with PkgLength > 63
    //
    // The format of the data has to follow the same format as
    // _CRS (according to ACPI spec).
    //
    Name (RESS, ResourceTemplate() {
      Memory32Fixed (ReadWrite, 0xfed40000, 0x5000)
      Interrupt(ResourceConsumer, Level, ActiveLow, Shared, , , ) {1,2,3,4,5,6,7,8,9,10}
    })

    Name (RESL, ResourceTemplate() {
      Memory32Fixed (ReadWrite, 0xfed40000, 0x5000)
      Interrupt(ResourceConsumer, Level, ActiveLow, Shared, , , )
{1,2,3,4,5,6,7,8,9,10,11,12,13,14,15}
    })

    //
    // Current resource settings for _CRS method
    //
    Name(RES0, ResourceTemplate () {
      Memory32Fixed (ReadWrite, 0xfed40000, 0x5000, REG0)
      Interrupt(ResourceConsumer, Level, ActiveLow, Shared, , , INTR) {12}
    })

    Name(RES1, ResourceTemplate () {
      Memory32Fixed (ReadWrite, 0xfed40000, 0x5000, REG1)
    })


    //
```

```
// Return the resource consumed by TPM device.
//
Method(_CRS,0,Serialized)
{
  //
  // IRQNum = 0 means disable IRQ support
  //
  If (LEqual(IRQN, 0)) {
    Return (RES1)
  }
  Else
  {
    CreateDWordField(RES0, ^INTR._INT, LIRQ)
    Store(IRQN, LIRQ)
    Return (RES0)
  }
}

//
// Set resources consumed by the TPM device. This is used to
// assign an interrupt number to the device. The input byte stream
// has to be the same as returned by _CRS (according to ACPI spec).
//
// Platform may choose to override this function with specific interrupt
// programing logic to replace FIFO/TIS SIRQ registers programing
//
Method(_SRS,1,Serialized)
{
  //
  // Do not configure Interrupt if IRQ Num is configured 0 by default
  //
  If (LEqual(IRQN, 0)) {
    Return (0)
  }
```

```
//
// Update resource descriptor
// Use the field name to identify the offsets in the argument
// buffer and RES0 buffer.
//
CreateDWordField(Arg0, ^INTR._INT, IRQ0)
CreateDWordField(RES0, ^INTR._INT, LIRQ)
Store(IRQ0, LIRQ)
Store(IRQ0, IRQN)

CreateBitField(Arg0, ^INTR._HE, ITRG)
CreateBitField(RES0, ^INTR._HE, LTRG)
Store(ITRG, LTRG)

CreateBitField(Arg0, ^INTR._LL, ILVL)
CreateBitField(RES0, ^INTR._LL, LLVL)
Store(ILVL, LLVL)

//
// Update TPM FIFO PTP/TIS interface only, identified by TPM_INTERFACE_ID_x lowest
// nibble.
// 0000 - FIFO interface as defined in PTP for TPM 2.0 is active
// 1111 - FIFO interface as defined in TIS1.3 is active
//
If (LOr(LEqual (And (TID0, 0x0F), 0x00), LEqual (And (TID0, 0x0F), 0x0F))) {
  //
  // If FIFO interface, interrupt vector register is
  // available. TCG PTP specification allows only
  // values 1..15 in this field. For other interrupts
  // the field should stay 0.
  //
  If (LLess (IRQ0, 16)) {
    Store (And(IRQ0, 0xF), INTV)
```

```
        }
        //
        // Interrupt enable register (TPM_INT_ENABLE_x) bits 3:4
        // contains settings for interrupt polarity.
        // The other bits of the byte enable individual interrupts.
        // They should be all be zero, but to avoid changing the
        // configuration, the other bits are be preserved.
        // 00 - high level
        // 01 - low level
        // 10 - rising edge
        // 11 - falling edge
        //
        // ACPI spec definitions:
        // _HE: '1' is Edge, '0' is Level
        // _LL: '1' is ActiveLow, '0' is ActiveHigh (for TPMs compliant to PTP 1.05 and earlier, not applicable to PTP 1.06)
        //
        If (LEqual (ITRG, 1)) {
          Or(INTE, 0x00000010, INTE)
        } Else {
          And(INTE, 0xFFFFFFEF, INTE)
        }
        if (LEqual (ILVL, 1)) {
          Or(INTE, 0x00000008, INTE)
        } Else {
          And(INTE, 0xFFFFFFF7, INTE)
        }
      }
    }

    Method(_PRS,0,Serialized)
    {
      //
      // IRQNum = 0 means disable IRQ support
```

```
      //
      If (LEqual(IRQN, 0)) {
        Return (RES1)
      } ElseIf(LEqual(SFRB, 0)) {
        //
        // Long format. Possible resources PkgLength > 63
        //
        Return (RESL)
      } Else {
        //
        // Short format. Possible resources PkgLength <=63
        //
        Return (RESS)
      }
    }
```

End of informative comment

Annex B: UEFI Variable Example

Start of Informative Comment

The following is an informative example of the data that would be populated in a measurement of the UEFI_IMAGE_SECURITY_DATABASE_GUID /EFI_IMAGE_SECURITY_DATABASE1 variable (the DBX) that contains a UEFI_VARIABLE_DATA structure. This example shows how the data retrieved from the GetVariable() API is populated in the TCG Event structure.

```
  Event:
    PCRIndex  - 7
    EventType - 0x80000001
    DigestCount: 0x00000001
    HashAlgo : 0x000B
    Digest(0): A044B4CE4A4DCA9AF312C897DC56EE1727C385EB88F7CFB9092B8265029D5B1E
    EventSize - 0x00000EB2
0000: CBB219D73A3D9645A3BCDAD00E67656F03000000000000008C0E000000000000
0020: 6400620078002616C4C14C509240ACA941F9369343288C0E0000000000003000
0040: 0000BD9AFA775903324DBD6028F4E78F784B80B4D96931BF0D02FD91A61E19D1
0060: 4F1DA452E66DB2408CA8604D411F92659F0ABD9AFA775903324DBD6028F4E78F
0080: 784BF52F83A3FA9CFBD6920F722824DBE4034534D25B8507246B3B957DAC6E1B
00A0: CE7ABD9AFA775903324DBD6028F4E78F784BC5D9D8A186E2C82D09AFAA2A6F7F
00C0: 2E73870D3E64F72C4E08EF67796A840F0FBDBD9AFA775903324DBD6028F4E78F
00E0: 784B363384D14D1F2E0B7815626484C459AD57A318EF4396266048D058C5A19B
0100: BF76BD9AFA775903324DBD6028F4E78F784B1AEC84B84B6C65A51220A9BE7181
0120: 965230210D62D6D33C48999C6B295A2B0A06BD9AFA775903324DBD6028F4E78F
0140: 784BE6CA68E94146629AF03F69C2F86E6BEF62F930B37C6FBCC878B78DF98C03
0160: 34E5BD9AFA775903324DBD6028F4E78F784BC3A99A460DA464A057C3586D83CE
0180: F5F4AE08B7103979ED8932742DF0ED530C66BD9AFA775903324DBD6028F4E78F
01A0: 784B58FB941AEF95A25943B3FB5F2510A0DF3FE44C58C95E0AB80487297568AB
01C0: 9771BD9AFA775903324DBD6028F4E78F784B5391C3A2FB112102A6AA1EDC25AE
01E0: 77E19F5D6F09CD09EEB2509922BFCD5992EABD9AFA775903324DBD6028F4E78F
0200: 784BD626157E1D6A718BC124AB8DA27CBB65072CA03A7B6B257DBDCBBD60F65E
0220: F3D1BD9AFA775903324DBD6028F4E78F784BD063EC28F67EBA53F1642DBF7DFF
0240: 33C6A32ADD869F6013FE162E2C32F1CBE56DBD9AFA775903324DBD6028F4E78F
0260: 784B29C6EB52B43C3AA18B2CD8ED6EA8607CEF3CFAE1BAFE1165755CF2E61484
0280: 4A44BD9AFA775903324DBD6028F4E78F784B90FBE70E69D633408D3E170C6832
02A0: DBB2D209E0272527DFB63D49D29572A6F44CBD9AFA775903324DBD6028F4E78F
02C0: 784B075EEA060589548BA060B2FEED10DA3C20C7FE9B17CD026B94E8A683B811
02E0: 5238BD9AFA775903324DBD6028F4E78F784B07E6C6A858646FB1EFC67903FE28
0300: B116011F2367FE92E6BE2B36999EFF39D09EBD9AFA775903324DBD6028F4E78F
0320: 784B09DF5F4E511208EC78B96D12D08125FDB603868DE39F6F72927852599B65
0340: 9C26BD9AFA775903324DBD6028F4E78F784B0BBB4392DAAC7AB89B30A4AC6575
0360: 31B97BFAAB04F90B0DAFE5F9B6EB90A06374BD9AFA775903324DBD6028F4E78F
0380: 784B0C189339762DF336AB3DD006A463DF715A39CFB0F492465C600E6C6BD7BD
03A0: 898CBD9AFA775903324DBD6028F4E78F784B0D0DBECA6F29ECA06F331A7D72E4
03C0: 884B12097FB348983A2A14A0D73F4F10140FBD9AFA775903324DBD6028F4E78F
03E0: 784B0DC9F3FB99962148C3CA833632758D3ED4FC8D0B0007B95B31E6528F2ACD
0400: 5BFCBD9AFA775903324DBD6028F4E78F784B106FACEACFECFD4E303B74F480A0
0420: 8098E2D0802B936F8EC774CE21F31686689CBD9AFA775903324DBD6028F4E78F
0440: 784B174E3A0B5B43C6A607BBD3404F05341E3DCF396267CE94F8B50E2E23A9DA
```

```
0460: 920CBD9AFA775903324DBD6028F4E78F784B18333429FF0562ED9F97033E1148
0480: DCEEE52DBE2E496D5410B5CFD6C864D2D10FBD9AFA775903324DBD6028F4E78F
04A0: 784B2B99CF26422E92FE365FBF4BC30D27086C9EE14B7A6FFF44FB2F6B900169
04C0: 9939BD9AFA775903324DBD6028F4E78F784B2BBF2CA7B8F1D91F27EE52B6FB2A
04E0: 5DD049B85A2B9B529C5D6662068104B055F8BD9AFA775903324DBD6028F4E78F
0500: 784B2C73D93325BA6DCBE589D4A4C63C5B935559EF92FBF050ED50C4E2085206
0520: F17DBD9AFA775903324DBD6028F4E78F784B2E70916786A6F773511FA7181FAB
0540: 0F1D70B557C6322EA923B2A8D3B92B51AF7DBD9AFA775903324DBD6028F4E78F
0560: 784B306628FA5477305728BA4A467DE7D0387A54F569D3769FCE5E75EC89D28D
0580: 1593BD9AFA775903324DBD6028F4E78F784B3608EDBAF5AD0F41A414A1777ABF
05A0: 2FAF5E670334675EC3995E6935829E0CAAD2BD9AFA775903324DBD6028F4E78F
05C0: 784B3841D221368D1583D75C0A02E62160394D6C4E0A6760B6F607B90362BC85
05E0: 5B02BD9AFA775903324DBD6028F4E78F784B3FCE9B9FDF3EF09D5452B0F95EE4
0600: 81C2B7F06D743A737971558E70136ACE3E73BD9AFA775903324DBD6028F4E78F
0620: 784B4397DACA839E7F63077CB50C92DF43BC2D2FB2A8F59F26FC7A0E4BD4D975
0640: 1692BD9AFA775903324DBD6028F4E78F784B47CC086127E2069A86E03A6BEF2C
0660: D410F8C55A6D6BDB362168C31B2CE32A5ADFBD9AFA775903324DBD6028F4E78F
0680: 784B518831FE7382B514D03E15C621228B8AB65479BD0CBFA3C5C1D0F48D9C30
06A0: 6135BD9AFA775903324DBD6028F4E78F784B5AE949EA8855EB93E439DBC65BDA
06C0: 2E42852C2FDF6789FA146736E3C3410F2B5CBD9AFA775903324DBD6028F4E78F
06E0: 784B6B1D138078E4418AA68DEB7BB35E066092CF479EEB8CE4CD12E7D072CCB4
0700: 2F66BD9AFA775903324DBD6028F4E78F784B6C8854478DD559E29351B826C06C
0720: B8BFEF2B94AD3538358772D193F82ED1CA11BD9AFA775903324DBD6028F4E78F
0740: 784B6F1428FF71C9DB0ED5AF1F2E7BBFCBAB647CC265DDF5B293CDB626F50A3A
0760: 785EBD9AFA775903324DBD6028F4E78F784B71F2906FD222497E54A34662AB24
0780: 97FCC81020770FF51368E9E3D9BFCBFD6375BD9AFA775903324DBD6028F4E78F
07A0: 784B726B3EB654046A30F3F83D9B96CE03F670E9A806D1708A0371E62DC49D2C
07C0: 23C1BD9AFA775903324DBD6028F4E78F784B72E0BD1867CF5D9D56AB158ADF3B
07E0: DDBC82BF32A8D8AA1D8C5E2F6DF29428D6D8BD9AFA775903324DBD6028F4E78F
0800: 784B7827AF99362CFAF0717DADE4B1BFE0438AD171C15ADDC248B75BF8CAA44B
0820: B2C5BD9AFA775903324DBD6028F4E78F784B81A8B965BB84D3876B9429A95481
0840: CC955318CFAA1412D808C8A33BFD33FFF0E4BD9AFA775903324DBD6028F4E78F
0860: 784B82DB3BCEB4F60843CE9D97C3D187CD9B5941CD3DE8100E586F2BDA563757
0880: 5F67BD9AFA775903324DBD6028F4E78F784B895A9785F617CA1D7ED44FC1A147
08A0: 0B71F3F1223862D9FF9DCC3AE2DF92163DAFBD9AFA775903324DBD6028F4E78F
08C0: 784B8AD64859F195B5F58DAFAA940B6A6167ACD67A886E8F469364177221C559
08E0: 45B9BD9AFA775903324DBD6028F4E78F784B8BF434B49E00CCF71502A2CD9008
0900: 65CB01EC3B3DA03C35BE505FDF7BD563F521BD9AFA775903324DBD6028F4E78F
0920: 784B8D8EA289CFE70A1C07AB7365CB28EE51EDD33CF2506DE888FBADD60EBF80
0940: 481CBD9AFA775903324DBD6028F4E78F784B9998D363C491BE16BD74BA10B94D
0960: 9291001611736FDCA643A36664BC0F315A42BD9AFA775903324DBD6028F4E78F
0980: 784B9E4A69173161682E55FDE8FEF560EB88EC1FFEDCAF04001F66C0CAF707B2
09A0: B734BD9AFA775903324DBD6028F4E78F784BA6B5151F3655D3A2AF0D47275979
09C0: 6BE4A4200E5495A7D869754C4848857408A7BD9AFA775903324DBD6028F4E78F
09E0: 784BA7F32F508D4EB0FEAD9A087EF94ED1BA0AEC5DE6F7EF6FF0A62B93BEDF5D
0A00: 458DBD9AFA775903324DBD6028F4E78F784BAD6826E1946D26D3EAF3685C88D9
0A20: 7D85DE3B4DCB3D0EE2AE81C70560D13C5720BD9AFA775903324DBD6028F4E78F
0A40: 784BAEEBAE3151271273ED95AA2E671139ED31A98567303A332298F83709A9D5
0A60: 5AA1BD9AFA775903324DBD6028F4E78F784BAFE2030AFB7D2CDA13F9FA333A02
0A80: E34F6751AFEC11B010DBCD441FDF4C4002B3BD9AFA775903324DBD6028F4E78F
0AA0: 784BB54F1EE636631FAD68058D3B0937031AC1B90CCB17062A391CCA68AFDBE4
```

```
0AC0: 0D55BD9AFA775903324DBD6028F4E78F784BB8F078D983A24AC4332163938835
0AE0: 14CD932C33AF18E7DD70884C8235F4275736BD9AFA775903324DBD6028F4E78F
0B00: 784BB97A0889059C035FF1D54B6DB53B11B9766668D9F955247C028B2837D7A0
0B20: 4CD9BD9AFA775903324DBD6028F4E78F784BBC87A668E81966489CB508EE8051
0B40: 83C19E6ACD24CF17799CA062D2E384DA0EA7BD9AFA775903324DBD6028F4E78F
0B60: 784BC409BDAC4775ADD8DB92AA22B5B718FB8C94A1462C1FE9A416B95D8A3388
0B80: C2FCBD9AFA775903324DBD6028F4E78F784BC617C1A8B1EE2A811C28B5A81B4C
0BA0: 83D7C98B5B0C27281D610207EBE692C2967FBD9AFA775903324DBD6028F4E78F
0BC0: 784BC90F336617B8E7F983975413C997F10B73EB267FD8A10CB9E3BDBFC667AB
0BE0: DB8BBD9AFA775903324DBD6028F4E78F784BCB6B858B40D3A098765815B592C1
0C00: 514A49604FAFD60819DA88D7A76E9778FEF7BD9AFA775903324DBD6028F4E78F
0C20: 784BCE3BFABE59D67CE8AC8DFD4A16F7C43EF9C224513FBC655957D735FA29F5
0C40: 40CEBD9AFA775903324DBD6028F4E78F784BD8CBEB9735F5672B367E4F96CDC7
0C60: 4969615D17074AE96C724D42CE0216F8F3FABD9AFA775903324DBD6028F4E78F
0C80: 784BE92C22EB3B5642D65C1EC2CAF247D2594738EEBB7FB3841A44956F59E2B0
0CA0: D1FABD9AFA775903324DBD6028F4E78F784BFDDD6E3D29EA84C7743DAD4A1BDB
0CC0: C700B5FEC1B391F932409086ACC71DD6DBD8BD9AFA775903324DBD6028F4E78F
0CE0: 784BFE63A84F782CC9D3FCF2CCF9FC11FBD03760878758D26285ED12669BDC6E
0D00: 6D01BD9AFA775903324DBD6028F4E78F784BFECFB232D12E994B6D485D2C7167
0D20: 728AA5525984AD5CA61E7516221F079A1436BD9AFA775903324DBD6028F4E78F
0D40: 784BCA171D614A8D7E121C93948CD0FE55D39981F9D11AA96E03450A415227C2
0D60: C65BBD9AFA775903324DBD6028F4E78F784B55B99B0DE53DBCFE485AA9C737CF
0D80: 3FB616EF3D91FAB599AA7CAB19EDA763B5BABD9AFA775903324DBD6028F4E78F
0DA0: 784B77DD190FA30D88FF5E3B011A0AE61E6209780C130B535ECB87E6F0888A0B
0DC0: 6B2FBD9AFA775903324DBD6028F4E78F784BC83CB13922AD99F560744675DD37
0DE0: CC94DCAD5A1FCBA6472FEE341171D939E884BD9AFA775903324DBD6028F4E78F
0E00: 784B3B0287533E0CC3D0EC1AA823CBF0A941AAD8721579D1C499802DD1C3A636
0E20: B8A9BD9AFA775903324DBD6028F4E78F784B939AEEF4F5FA51E23340C3F2E490
0E40: 48CE8872526AFDF752C3A7F3A3F2BC9F6049BD9AFA775903324DBD6028F4E78F
0E60: 784B64575BD912789A2E14AD56F6341F52AF6BF80CF94400785975E9F04E2D64
0E80: D745BD9AFA775903324DBD6028F4E78F784B45C7C8AE750ACFBB48FC37527D64
0EA0: 12DD644DAED8913CCD8A24C94D856967DF8E

    EventData - Type: EV_EFI_VARIABLE_DRIVER_CONFIG
      VariableName       - D719B2CB-3D3A-4596-A3BC-DAD00E67656F
      UnicodeNameLength  - 0x0000000000000003
      VariableDataLength - 0x0000000000000E8C
      UnicodeName        - dbx
      VariableData       - 26 16 C4 C1 4C 50 92 40 AC A9 41 F9 36 93 43 28
                           8C 0E 00 00 00 00 00 00 30 00 00 00 BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 80 B4 D9 69
                           31 BF 0D 02 FD 91 A6 1E 19 D1 4F 1D A4 52 E6 6D
                           B2 40 8C A8 60 4D 41 1F 92 65 9F 0A BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B F5 2F 83 A3
                           FA 9C FB D6 92 0F 72 28 24 DB E4 03 45 34 D2 5B
                           85 07 24 6B 3B 95 7D AC 6E 1B CE 7A BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B C5 D9 D8 A1
                           86 E2 C8 2D 09 AF AA 2A 6F 7F 2E 73 87 0D 3E 64
                           F7 2C 4E 08 EF 67 79 6A 84 0F 0F BD BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 36 33 84 D1
                           4D 1F 2E 0B 78 15 62 64 84 C4 59 AD 57 A3 18 EF
```

```
43 96 26 60 48 D0 58 C5 A1 9B BF 76 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 1A EC 84 B8
4B 6C 65 A5 12 20 A9 BE 71 81 96 52 30 21 0D 62
D6 D3 3C 48 99 9C 6B 29 5A 2B 0A 06 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B E6 CA 68 E9
41 46 62 9A F0 3F 69 C2 F8 6E 6B EF 62 F9 30 B3
7C 6F BC C8 78 B7 8D F9 8C 03 34 E5 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B C3 A9 9A 46
0D A4 64 A0 57 C3 58 6D 83 CE F5 F4 AE 08 B7 10
39 79 ED 89 32 74 2D F0 ED 53 0C 66 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 58 FB 94 1A
EF 95 A2 59 43 B3 FB 5F 25 10 A0 DF 3F E4 4C 58
C9 5E 0A B8 04 87 29 75 68 AB 97 71 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 53 91 C3 A2
FB 11 21 02 A6 AA 1E DC 25 AE 77 E1 9F 5D 6F 09
CD 09 EE B2 50 99 22 BF CD 59 92 EA BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B D6 26 15 7E
1D 6A 71 8B C1 24 AB 8D A2 7C BB 65 07 2C A0 3A
7B 6B 25 7D BD CB BD 60 F6 5E F3 D1 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B D0 63 EC 28
F6 7E BA 53 F1 64 2D BF 7D FF 33 C6 A3 2A DD 86
9F 60 13 FE 16 2E 2C 32 F1 CB E5 6D BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 29 C6 EB 52
B4 3C 3A A1 8B 2C D8 ED 6E A8 60 7C EF 3C FA E1
BA FE 11 65 75 5C F2 E6 14 84 4A 44 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 90 FB E7 0E
69 D6 33 40 8D 3E 17 0C 68 32 DB B2 D2 09 E0 27
25 27 DF B6 3D 49 D2 95 72 A6 F4 4C BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 07 5E EA 06
05 89 54 8B A0 60 B2 FE ED 10 DA 3C 20 C7 FE 9B
17 CD 02 6B 94 E8 A6 83 B8 11 52 38 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 07 E6 C6 A8
58 64 6F B1 EF C6 79 03 FE 28 B1 16 01 1F 23 67
FE 92 E6 BE 2B 36 99 9E FF 39 D0 9E BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 09 DF 5F 4E
51 12 08 EC 78 B9 6D 12 D0 81 25 FD B6 03 86 8D
E3 9F 6F 72 92 78 52 59 9B 65 9C 26 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 0B BB 43 92
DA AC 7A B8 9B 30 A4 AC 65 75 31 B9 7B FA AB 04
F9 0B 0D AF E5 F9 B6 EB 90 A0 63 74 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 0C 18 93 39
76 2D F3 36 AB 3D D0 06 A4 63 DF 71 5A 39 CF B0
F4 92 46 5C 60 0E 6C 6B D7 BD 89 8C BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 0D 0D BE CA
6F 29 EC A0 6F 33 1A 7D 72 E4 88 4B 12 09 7F B3
48 98 3A 2A 14 A0 D7 3F 4F 10 14 0F BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 0D C9 F3 FB
99 96 21 48 C3 CA 83 36 32 75 8D 3E D4 FC 8D 0B
00 07 B9 5B 31 E6 52 8F 2A CD 5B FC BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 10 6F AC EA
CF EC FD 4E 30 3B 74 F4 80 A0 80 98 E2 D0 80 2B
```

```
93 6F 8E C7 74 CE 21 F3 16 86 68 9C BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 17 4E 3A 0B
5B 43 C6 A6 07 BB D3 40 4F 05 34 1E 3D CF 39 62
67 CE 94 F8 B5 0E 2E 23 A9 DA 92 0C BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 18 33 34 29
FF 05 62 ED 9F 97 03 3E 11 48 DC EE E5 2D BE 2E
49 6D 54 10 B5 CF D6 C8 64 D2 D1 0F BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 2B 99 CF 26
42 2E 92 FE 36 5F BF 4B C3 0D 27 08 6C 9E E1 4B
7A 6F FF 44 FB 2F 6B 90 01 69 99 39 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 2B BF 2C A7
B8 F1 D9 1F 27 EE 52 B6 FB 2A 5D D0 49 B8 5A 2B
9B 52 9C 5D 66 62 06 81 04 B0 55 F8 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 2C 73 D9 33
25 BA 6D CB E5 89 D4 A4 C6 3C 5B 93 55 59 EF 92
FB F0 50 ED 50 C4 E2 08 52 06 F1 7D BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 2E 70 91 67
86 A6 F7 73 51 1F A7 18 1F AB 0F 1D 70 B5 57 C6
32 2E A9 23 B2 A8 D3 B9 2B 51 AF 7D BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 30 66 28 FA
54 77 30 57 28 BA 4A 46 7D E7 D0 38 7A 54 F5 69
D3 76 9F CE 5E 75 EC 89 D2 8D 15 93 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 36 08 ED BA
F5 AD 0F 41 A4 14 A1 77 7A BF 2F AF 5E 67 03 34
67 5E C3 99 5E 69 35 82 9E 0C AA D2 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 38 41 D2 21
36 8D 15 83 D7 5C 0A 02 E6 21 60 39 4D 6C 4E 0A
67 60 B6 F6 07 B9 03 62 BC 85 5B 02 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 3F CE 9B 9F
DF 3E F0 9D 54 52 B0 F9 5E E4 81 C2 B7 F0 6D 74
3A 73 79 71 55 8E 70 13 6A CE 3E 73 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 43 97 DA CA
83 9E 7F 63 07 7C B5 0C 92 DF 43 BC 2D 2F B2 A8
F5 9F 26 FC 7A 0E 4B D4 D9 75 16 92 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 47 CC 08 61
27 E2 06 9A 86 E0 3A 6B EF 2C D4 10 F8 C5 5A 6D
6B DB 36 21 68 C3 1B 2C E3 2A 5A DF BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 51 88 31 FE
73 82 B5 14 D0 3E 15 C6 21 22 8B 8A B6 54 79 BD
0C BF A3 C5 C1 D0 F4 8D 9C 30 61 35 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 5A E9 49 EA
88 55 EB 93 E4 39 DB C6 5B DA 2E 42 85 2C 2F DF
67 89 FA 14 67 36 E3 C3 41 0F 2B 5C BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 6B 1D 13 80
78 E4 41 8A A6 8D EB 7B B3 5E 06 60 92 CF 47 9E
EB 8C E4 CD 12 E7 D0 72 CC B4 2F 66 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 6C 88 54 47
8D D5 59 E2 93 51 B8 26 C0 6C B8 BF EF 2B 94 AD
35 38 35 87 72 D1 93 F8 2E D1 CA 11 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 6F 14 28 FF
71 C9 DB 0E D5 AF 1F 2E 7B BF CB AB 64 7C C2 65
```

```
DD F5 B2 93 CD B6 26 F5 0A 3A 78 5E BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 71 F2 90 6F
D2 22 49 7E 54 A3 46 62 AB 24 97 FC C8 10 20 77
0F F5 13 68 E9 E3 D9 BF CB FD 63 75 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 72 6B 3E B6
54 04 6A 30 F3 F8 3D 9B 96 CE 03 F6 70 E9 A8 06
D1 70 8A 03 71 E6 2D C4 9D 2C 23 C1 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 72 E0 BD 18
67 CF 5D 9D 56 AB 15 8A DF 3B DD BC 82 BF 32 A8
D8 AA 1D 8C 5E 2F 6D F2 94 28 D6 D8 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 78 27 AF 99
36 2C FA F0 71 7D AD E4 B1 BF E0 43 8A D1 71 C1
5A DD C2 48 B7 5B F8 CA A4 4B B2 C5 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 81 A8 B9 65
BB 84 D3 87 6B 94 29 A9 54 81 CC 95 53 18 CF AA
14 12 D8 08 C8 A3 3B FD 33 FF F0 E4 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 82 DB 3B CE
B4 F6 08 43 CE 9D 97 C3 D1 87 CD 9B 59 41 CD 3D
E8 10 0E 58 6F 2B DA 56 37 57 5F 67 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 89 5A 97 85
F6 17 CA 1D 7E D4 4F C1 A1 47 0B 71 F3 F1 22 38
62 D9 FF 9D CC 3A E2 DF 92 16 3D AF BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 8A D6 48 59
F1 95 B5 F5 8D AF AA 94 0B 6A 61 67 AC D6 7A 88
6E 8F 46 93 64 17 72 21 C5 59 45 B9 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 8B F4 34 B4
9E 00 CC F7 15 02 A2 CD 90 08 65 CB 01 EC 3B 3D
A0 3C 35 BE 50 5F DF 7B D5 63 F5 21 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 8D 8E A2 89
CF E7 0A 1C 07 AB 73 65 CB 28 EE 51 ED D3 3C F2
50 6D E8 88 FB AD D6 0E BF 80 48 1C BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 99 98 D3 63
C4 91 BE 16 BD 74 BA 10 B9 4D 92 91 00 16 11 73
6F DC A6 43 A3 66 64 BC 0F 31 5A 42 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 9E 4A 69 17
31 61 68 2E 55 FD E8 FE F5 60 EB 88 EC 1F FE DC
AF 04 00 1F 66 C0 CA F7 07 B2 B7 34 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B A6 B5 15 1F
36 55 D3 A2 AF 0D 47 27 59 79 6B E4 A4 20 0E 54
95 A7 D8 69 75 4C 48 48 85 74 08 A7 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B A7 F3 2F 50
8D 4E B0 FE AD 9A 08 7E F9 4E D1 BA 0A EC 5D E6
F7 EF 6F F0 A6 2B 93 BE DF 5D 45 8D BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B AD 68 26 E1
94 6D 26 D3 EA F3 68 5C 88 D9 7D 85 DE 3B 4D CB
3D 0E E2 AE 81 C7 05 60 D1 3C 57 20 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B AE EB AE 31
51 27 12 73 ED 95 AA 2E 67 11 39 ED 31 A9 85 67
30 3A 33 22 98 F8 37 09 A9 D5 5A A1 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B AF E2 03 0A
FB 7D 2C DA 13 F9 FA 33 3A 02 E3 4F 67 51 AF EC
```

```
11 B0 10 DB CD 44 1F DF 4C 40 02 B3 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B B5 4F 1E E6
36 63 1F AD 68 05 8D 3B 09 37 03 1A C1 B9 0C CB
17 06 2A 39 1C CA 68 AF DB E4 0D 55 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B B8 F0 78 D9
83 A2 4A C4 33 21 63 93 88 35 14 CD 93 2C 33 AF
18 E7 DD 70 88 4C 82 35 F4 27 57 36 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B B9 7A 08 89
05 9C 03 5F F1 D5 4B 6D B5 3B 11 B9 76 66 68 D9
F9 55 24 7C 02 8B 28 37 D7 A0 4C D9 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B BC 87 A6 68
E8 19 66 48 9C B5 08 EE 80 51 83 C1 9E 6A CD 24
CF 17 79 9C A0 62 D2 E3 84 DA 0E A7 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B C4 09 BD AC
47 75 AD D8 DB 92 AA 22 B5 B7 18 FB 8C 94 A1 46
2C 1F E9 A4 16 B9 5D 8A 33 88 C2 FC BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B C6 17 C1 A8
B1 EE 2A 81 1C 28 B5 A8 1B 4C 83 D7 C9 8B 5B 0C
27 28 1D 61 02 07 EB E6 92 C2 96 7F BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B C9 0F 33 66
17 B8 E7 F9 83 97 54 13 C9 97 F1 0B 73 EB 26 7F
D8 A1 0C B9 E3 BD BF C6 67 AB DB 8B BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B CB 6B 85 8B
40 D3 A0 98 76 58 15 B5 92 C1 51 4A 49 60 4F AF
D6 08 19 DA 88 D7 A7 6E 97 78 FE F7 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B CE 3B FA BE
59 D6 7C E8 AC 8D FD 4A 16 F7 C4 3E F9 C2 24 51
3F BC 65 59 57 D7 35 FA 29 F5 40 CE BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B D8 CB EB 97
35 F5 67 2B 36 7E 4F 96 CD C7 49 69 61 5D 17 07
4A E9 6C 72 4D 42 CE 02 16 F8 F3 FA BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B E9 2C 22 EB
3B 56 42 D6 5C 1E C2 CA F2 47 D2 59 47 38 EE BB
7F B3 84 1A 44 95 6F 59 E2 B0 D1 FA BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B FD DD 6E 3D
29 EA 84 C7 74 3D AD 4A 1B DB C7 00 B5 FE C1 B3
91 F9 32 40 90 86 AC C7 1D D6 DB D8 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B FE 63 A8 4F
78 2C C9 D3 FC F2 CC F9 FC 11 FB D0 37 60 87 87
58 D2 62 85 ED 12 66 9B DC 6E 6D 01 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B FE CF B2 32
D1 2E 99 4B 6D 48 5D 2C 71 67 72 8A A5 52 59 84
AD 5C A6 1E 75 16 22 1F 07 9A 14 36 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B CA 17 1D 61
4A 8D 7E 12 1C 93 94 8C D0 FE 55 D3 99 81 F9 D1
1A A9 6E 03 45 0A 41 52 27 C2 C6 5B BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 55 B9 9B 0D
E5 3D BC FE 48 5A A9 C7 37 CF 3F B6 16 EF 3D 91
FA B5 99 AA 7C AB 19 ED A7 63 B5 BA BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 77 DD 19 0F
A3 0D 88 FF 5E 3B 01 1A 0A E6 1E 62 09 78 0C 13
```

```
                          0B 53 5E CB 87 E6 F0 88 8A 0B 6B 2F BD 9A FA 77
                          59 03 32 4D BD 60 28 F4 E7 8F 78 4B C8 3C B1 39
                          22 AD 99 F5 60 74 46 75 DD 37 CC 94 DC AD 5A 1F
                          CB A6 47 2F EE 34 11 71 D9 39 E8 84 BD 9A FA 77
                          59 03 32 4D BD 60 28 F4 E7 8F 78 4B 3B 02 87 53
                          3E 0C C3 D0 EC 1A A8 23 CB F0 A9 41 AA D8 72 15
                          79 D1 C4 99 80 2D D1 C3 A6 36 B8 A9 BD 9A FA 77
                          59 03 32 4D BD 60 28 F4 E7 8F 78 4B 93 9A EE F4
                          F5 FA 51 E2 33 40 C3 F2 E4 90 48 CE 88 72 52 6A
                          FD F7 52 C3 A7 F3 A3 F2 BC 9F 60 49 BD 9A FA 77
                          59 03 32 4D BD 60 28 F4 E7 8F 78 4B 64 57 5B D9
                          12 78 9A 2E 14 AD 56 F6 34 1F 52 AF 6B F8 0C F9
                          44 00 78 59 75 E9 F0 4E 2D 64 D7 45 BD 9A FA 77
                          59 03 32 4D BD 60 28 F4 E7 8F 78 4B 45 C7 C8 AE
                          75 0A CF BB 48 FC 37 52 7D 64 12 DD 64 4D AE D8
                          91 3C CD 8A 24 C9 4D 85 69 67 DF 8E
=====================

To parse the variable data: (as defined in UEFI specification)
=====================
EFI_SIGNATURE_LIST :
    SignatureType       :  26 16 C4 C1 4C 50 92 40 AC A9 41 F9 36 93 43 28
(EFI_CERT_SHA256_GUID)
    SignatureListSize   :  8C 0E 00 00
    SignatureHeaderSize :  00 00 00 00
    SignatureSize       :  30 00 00 00
    EFI_SIGNATURE_DATA :
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
       SignatureData  :   80 B4 D9 69 31 BF 0D 02 FD 91 A6 1E 19 D1 4F 1D
                          A4 52 E6 6D B2 40 8C A8 60 4D 41 1F 92 65 9F 0A
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   F5 2F 83 A3 FA 9C FB D6 92 0F 72 28 24 DB E4 03
                          45 34 D2 5B 85 07 24 6B 3B 95 7D AC 6E 1B CE 7A
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   C5 D9 D8 A1 86 E2 C8 2D 09 AF AA 2A 6F 7F 2E 73
                          87 0D 3E 64 F7 2C 4E 08 EF 67 79 6A 84 0F 0F BD
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   36 33 84 D1 4D 1F 2E 0B 78 15 62 64 84 C4 59 AD
                          57 A3 18 EF 43 96 26 60 48 D0 58 C5 A1 9B BF 76
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   1A EC 84 B8 4B 6C 65 A5 12 20 A9 BE 71 81 96 52
                          30 21 0D 62 D6 D3 3C 48 99 9C 6B 29 5A 2B 0A 06
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   E6 CA 68 E9 41 46 62 9A F0 3F 69 C2 F8 6E 6B EF
                          62 F9 30 B3 7C 6F BC C8 78 B7 8D F9 8C 03 34 E5
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   C3 A9 9A 46 0D A4 64 A0 57 C3 58 6D 83 CE F5 F4
                          AE 08 B7 10 39 79 ED 89 32 74 2D F0 ED 53 0C 66
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   58 FB 94 1A EF 95 A2 59 43 B3 FB 5F 25 10 A0 DF
                          3F E4 4C 58 C9 5E 0A B8 04 87 29 75 68 AB 97 71
```

```
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   53 91 C3 A2 FB 11 21 02 A6 AA 1E DC 25 AE 77 E1
                           9F 5D 6F 09 CD 09 EE B2 50 99 22 BF CD 59 92 EA
                           BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
                           D6 26 15 7E 1D 6A 71 8B C1 24 AB 8D A2 7C BB 65
                           07 2C A0 3A 7B 6B 25 7D BD CB BD 60 F6 5E F3 D1
                           BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
                           D0 63 EC 28 F6 7E BA 53 F1 64 2D BF 7D FF 33 C6
                           A3 2A DD 86 9F 60 13 FE 16 2E 2C 32 F1 CB E5 6D
                           BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
                           29 C6 EB 52 B4 3C 3A A1 8B 2C D8 ED 6E A8 60 7C
                           EF 3C FA E1 BA FE 11 65 75 5C F2 E6 14 84 4A 44
                           BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
                           90 FB E7 0E 69 D6 33 40 8D 3E 17 0C 68 32 DB B2
                           D2 09 E0 27 25 27 DF B6 3D 49 D2 95 72 A6 F4 4C
                           BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
                           07 5E EA 06 05 89 54 8B A0 60 B2 FE ED 10 DA 3C
                           20 C7 FE 9B 17 CD 02 6B 94 E8 A6 83 B8 11 52 38
                           BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
                           07 E6 C6 A8 58 64 6F B1 EF C6 79 03 FE 28 B1 16
                           01 1F 23 67
                           FE 92 E6 BE 2B 36 99 9E FF 39 D0 9E BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 09 DF 5F 4E
                           51 12 08 EC 78 B9 6D 12 D0 81 25 FD B6 03 86 8D
                           E3 9F 6F 72 92 78 52 59 9B 65 9C 26 BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 0B BB 43 92
                           DA AC 7A B8 9B 30 A4 AC 65 75 31 B9 7B FA AB 04
                           F9 0B 0D AF E5 F9 B6 EB 90 A0 63 74 BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 0C 18 93 39
                           76 2D F3 36 AB 3D D0 06 A4 63 DF 71 5A 39 CF B0
                           F4 92 46 5C 60 0E 6C 6B D7 BD 89 8C BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 0D 0D BE CA
                           6F 29 EC A0 6F 33 1A 7D 72 E4 88 4B 12 09 7F B3
                           48 98 3A 2A 14 A0 D7 3F 4F 10 14 0F BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 0D C9 F3 FB
                           99 96 21 48 C3 CA 83 36 32 75 8D 3E D4 FC 8D 0B
                           00 07 B9 5B 31 E6 52 8F 2A CD 5B FC BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 10 6F AC EA
                           CF EC FD 4E 30 3B 74 F4 80 A0 80 98 E2 D0 80 2B
                           93 6F 8E C7 74 CE 21 F3 16 86 68 9C BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 17 4E 3A 0B
                           5B 43 C6 A6 07 BB D3 40 4F 05 34 1E 3D CF 39 62
                           67 CE 94 F8 B5 0E 2E 23 A9 DA 92 0C BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 18 33 34 29
                           FF 05 62 ED 9F 97 03 3E 11 48 DC EE E5 2D BE 2E
                           49 6D 54 10 B5 CF D6 C8 64 D2 D1 0F BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 2B 99 CF 26
                           42 2E 92 FE 36 5F BF 4B C3 0D 27 08 6C 9E E1 4B
                           7A 6F FF 44 FB 2F 6B 90 01 69 99 39 BD 9A FA 77
                           59 03 32 4D BD 60 28 F4 E7 8F 78 4B 2B BF 2C A7
                           B8 F1 D9 1F 27 EE 52 B6 FB 2A 5D D0 49 B8 5A 2B
```

```
9B 52 9C 5D 66 62 06 81 04 B0 55 F8 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 2C 73 D9 33
25 BA 6D CB E5 89 D4 A4 C6 3C 5B 93 55 59 EF 92
FB F0 50 ED 50 C4 E2 08 52 06 F1 7D BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 2E 70 91 67
86 A6 F7 73 51 1F A7 18 1F AB 0F 1D 70 B5 57 C6
32 2E A9 23 B2 A8 D3 B9 2B 51 AF 7D BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 30 66 28 FA
54 77 30 57 28 BA 4A 46 7D E7 D0 38 7A 54 F5 69
D3 76 9F CE 5E 75 EC 89 D2 8D 15 93 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 36 08 ED BA
F5 AD 0F 41 A4 14 A1 77 7A BF 2F AF 5E 67 03 34
67 5E C3 99 5E 69 35 82 9E 0C AA D2 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 38 41 D2 21
36 8D 15 83 D7 5C 0A 02 E6 21 60 39 4D 6C 4E 0A
67 60 B6 F6 07 B9 03 62 BC 85 5B 02 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 3F CE 9B 9F
DF 3E F0 9D 54 52 B0 F9 5E E4 81 C2 B7 F0 6D 74
3A 73 79 71 55 8E 70 13 6A CE 3E 73 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 43 97 DA CA
83 9E 7F 63 07 7C B5 0C 92 DF 43 BC 2D 2F B2 A8
F5 9F 26 FC 7A 0E 4B D4 D9 75 16 92 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 47 CC 08 61
27 E2 06 9A 86 E0 3A 6B EF 2C D4 10 F8 C5 5A 6D
6B DB 36 21 68 C3 1B 2C E3 2A 5A DF BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 51 88 31 FE
73 82 B5 14 D0 3E 15 C6 21 22 8B 8A B6 54 79 BD
0C BF A3 C5 C1 D0 F4 8D 9C 30 61 35 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 5A E9 49 EA
88 55 EB 93 E4 39 DB C6 5B DA 2E 42 85 2C 2F DF
67 89 FA 14 67 36 E3 C3 41 0F 2B 5C BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 6B 1D 13 80
78 E4 41 8A A6 8D EB 7B B3 5E 06 60 92 CF 47 9E
EB 8C E4 CD 12 E7 D0 72 CC B4 2F 66 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 6C 88 54 47
8D D5 59 E2 93 51 B8 26 C0 6C B8 BF EF 2B 94 AD
35 38 35 87 72 D1 93 F8 2E D1 CA 11 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 6F 14 28 FF
71 C9 DB 0E D5 AF 1F 2E 7B BF CB AB 64 7C C2 65
DD F5 B2 93 CD B6 26 F5 0A 3A 78 5E BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 71 F2 90 6F
D2 22 49 7E 54 A3 46 62 AB 24 97 FC C8 10 20 77
0F F5 13 68 E9 E3 D9 BF CB FD 63 75 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 72 6B 3E B6
54 04 6A 30 F3 F8 3D 9B 96 CE 03 F6 70 E9 A8 06
D1 70 8A 03 71 E6 2D C4 9D 2C 23 C1 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 72 E0 BD 18
67 CF 5D 9D 56 AB 15 8A DF 3B DD BC 82 BF 32 A8
D8 AA 1D 8C 5E 2F 6D F2 94 28 D6 D8 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 78 27 AF 99
36 2C FA F0 71 7D AD E4 B1 BF E0 43 8A D1 71 C1
```

```
5A DD C2 48 B7 5B F8 CA A4 4B B2 C5 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 81 A8 B9 65
BB 84 D3 87 6B 94 29 A9 54 81 CC 95 53 18 CF AA
14 12 D8 08 C8 A3 3B FD 33 FF F0 E4 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 82 DB 3B CE
B4 F6 08 43 CE 9D 97 C3 D1 87 CD 9B 59 41 CD 3D
E8 10 0E 58 6F 2B DA 56 37 57 5F 67 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 89 5A 97 85
F6 17 CA 1D 7E D4 4F C1 A1 47 0B 71 F3 F1 22 38
62 D9 FF 9D CC 3A E2 DF 92 16 3D AF BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 8A D6 48 59
F1 95 B5 F5 8D AF AA 94 0B 6A 61 67 AC D6 7A 88
6E 8F 46 93 64 17 72 21 C5 59 45 B9 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 8B F4 34 B4
9E 00 CC F7 15 02 A2 CD 90 08 65 CB 01 EC 3B 3D
A0 3C 35 BE 50 5F DF 7B D5 63 F5 21 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 8D 8E A2 89
CF E7 0A 1C 07 AB 73 65 CB 28 EE 51 ED D3 3C F2
50 6D E8 88 FB AD D6 0E BF 80 48 1C BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 99 98 D3 63
C4 91 BE 16 BD 74 BA 10 B9 4D 92 91 00 16 11 73
6F DC A6 43 A3 66 64 BC 0F 31 5A 42 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B 9E 4A 69 17
31 61 68 2E 55 FD E8 FE F5 60 EB 88 EC 1F FE DC
AF 04 00 1F 66 C0 CA F7 07 B2 B7 34 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B A6 B5 15 1F
36 55 D3 A2 AF 0D 47 27 59 79 6B E4 A4 20 0E 54
95 A7 D8 69 75 4C 48 48 85 74 08 A7 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B A7 F3 2F 50
8D 4E B0 FE AD 9A 08 7E F9 4E D1 BA 0A EC 5D E6
F7 EF 6F F0 A6 2B 93 BE DF 5D 45 8D BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B AD 68 26 E1
94 6D 26 D3 EA F3 68 5C 88 D9 7D 85 DE 3B 4D CB
3D 0E E2 AE 81 C7 05 60 D1 3C 57 20 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B AE EB AE 31
51 27 12 73 ED 95 AA 2E 67 11 39 ED 31 A9 85 67
30 3A 33 22 98 F8 37 09 A9 D5 5A A1 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B AF E2 03 0A
FB 7D 2C DA 13 F9 FA 33 3A 02 E3 4F 67 51 AF EC
11 B0 10 DB CD 44 1F DF 4C 40 02 B3 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B B5 4F 1E E6
36 63 1F AD 68 05 8D 3B 09 37 03 1A C1 B9 0C CB
17 06 2A 39 1C CA 68 AF DB E4 0D 55 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B B8 F0 78 D9
83 A2 4A C4 33 21 63 93 88 35 14 CD 93 2C 33 AF
18 E7 DD 70 88 4C 82 35 F4 27 57 36 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B B9 7A 08 89
05 9C 03 5F F1 D5 4B 6D B5 3B 11 B9 76 66 68 D9
F9 55 24 7C 02 8B 28 37 D7 A0 4C D9 BD 9A FA 77
59 03 32 4D BD 60 28 F4 E7 8F 78 4B BC 87 A6 68
E8 19 66 48 9C B5 08 EE 80 51 83 C1 9E 6A CD 24
```

```
                    CF 17 79 9C A0 62 D2 E3 84 DA 0E A7 BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B C4 09 BD AC
                    47 75 AD D8 DB 92 AA 22 B5 B7 18 FB 8C 94 A1 46
                    2C 1F E9 A4 16 B9 5D 8A 33 88 C2 FC BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B C6 17 C1 A8
                    B1 EE 2A 81 1C 28 B5 A8 1B 4C 83 D7 C9 8B 5B 0C
                    27 28 1D 61 02 07 EB E6 92 C2 96 7F BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B C9 0F 33 66
                    17 B8 E7 F9 83 97 54 13 C9 97 F1 0B 73 EB 26 7F
                    D8 A1 0C B9 E3 BD BF C6 67 AB DB 8B BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B CB 6B 85 8B
                    40 D3 A0 98 76 58 15 B5 92 C1 51 4A 49 60 4F AF
                    D6 08 19 DA 88 D7 A7 6E 97 78 FE F7 BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B CE 3B FA BE
                    59 D6 7C E8 AC 8D FD 4A 16 F7 C4 3E F9 C2 24 51
                    3F BC 65 59 57 D7 35 FA 29 F5 40 CE BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B D8 CB EB 97
                    35 F5 67 2B 36 7E 4F 96 CD C7 49 69 61 5D 17 07
                    4A E9 6C 72 4D 42 CE 02 16 F8 F3 FA BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B E9 2C 22 EB
                    3B 56 42 D6 5C 1E C2 CA F2 47 D2 59 47 38 EE BB
                    7F B3 84 1A 44 95 6F 59 E2 B0 D1 FA BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B FD DD 6E 3D
                    29 EA 84 C7 74 3D AD 4A 1B DB C7 00 B5 FE C1 B3
                    91 F9 32 40 90 86 AC C7 1D D6 DB D8 BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B FE 63 A8 4F
                    78 2C C9 D3 FC F2 CC F9 FC 11 FB D0 37 60 87 87
                    58 D2 62 85 ED 12 66 9B DC 6E 6D 01 BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B FE CF B2 32
                    D1 2E 99 4B 6D 48 5D 2C 71 67 72 8A A5 52 59 84
                    AD 5C A6 1E 75 16 22 1F 07 9A 14 36 BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B CA 17 1D 61
                    4A 8D 7E 12 1C 93 94 8C D0 FE 55 D3 99 81 F9 D1
                    1A A9 6E 03 45 0A 41 52 27 C2 C6 5B BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B 55 B9 9B 0D
                    E5 3D BC FE 48 5A A9 C7 37 CF 3F B6 16 EF 3D 91
                    FA B5 99 AA 7C AB 19 ED A7 63 B5 BA BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B 77 DD 19 0F
                    A3 0D 88 FF 5E 3B 01 1A 0A E6 1E 62 09 78 0C 13
                    0B 53 5E CB 87 E6 F0 88 8A 0B 6B 2F BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B C8 3C B1 39
                    22 AD 99 F5 60 74 46 75 DD 37 CC 94 DC AD 5A 1F
                    CB A6 47 2F EE 34 11 71 D9 39 E8 84 BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B 3B 02 87 53
                    3E 0C C3 D0 EC 1A A8 23 CB F0 A9 41 AA D8 72 15
                    79 D1 C4 99 80 2D D1 C3 A6 36 B8 A9 BD 9A FA 77
                    59 03 32 4D BD 60 28 F4 E7 8F 78 4B 93 9A EE F4
                    F5 FA 51 E2 33 40 C3 F2 E4 90 48 CE 88 72 52 6A
                    FD F7 52 C3 A7 F3 A3 F2 BC 9F 60 49
SignatureOwner :    BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
SignatureData  :    64 57 5B D9 12 78 9A 2E 14 AD 56 F6 34 1F 52 AF
```

```
                        6B F8 0C F9 44 00 78 59 75 E9 F0 4E 2D 64 D7 45
        SignatureOwner :   BD 9A FA 77 59 03 32 4D BD 60 28 F4 E7 8F 78 4B
        SignatureData  :   45 C7 C8 AE 75 0A CF BB 48 FC 37 52 7D 64 12 DD
                           64 4D AE D8 91 3C CD 8A 24 C9 4D 85 69 67 DF 8E
```

End of informative comment